ACSL by Example

Towards a Formally Verified Standard Library

Version 32.0.1
for Frama-C 32.0 (Germanium)
January 2026

Jens Gerlach

Former Authors

Malte Brodmann, Jochen Burghardt,
Andreas Carben Robert Clausecker,
Denis Efremov, Liangliang Gu
Kerstin Hartig, Timon Lapawczyk
Hans Werner Pohl, Tim Sikatzki
Juan Soto, Kim Völlinger

Fraunhofer
FOKUS

Acknowledgements and Publication Information

The research presented in this report was supported by the following projects:

- **VESSEDIA** — European Union Horizon 2020 research and innovation programme, Grant Agreement No 731453 (2017–2019), `https://vessedia.eu`.
- **STANCE** — European Union Seventh Framework Programme (FP7/2007–2013), Grant Agreement No 317753 (2012–2016), `http://www.stance-project.eu`.
- **DeviceSoft** — supported by the Programme Inter Carnot Fraunhofer from BMBF (Grant 01SF0804) and ANR (2009–2012).

Availability. This document is hosted at

`https://github.com/fraunhoferfokus/acsl-by-example`

The repository also contains the source code of all algorithms discussed in this report, their formal contracts, and the associated predicate definitions and lemmas. All examples are developed and proved using the Frama-C/WP [1] plugin.

Suggestions or error reports may be submitted via the GitHub issue tracker:

`https://github.com/fraunhoferfokus/acsl-by-example/issues`

Alternatively, correspondence may be addressed to:

`jens.gerlach@fokus.fraunhofer.de`

Except where otherwise noted, this work is licensed under the Creative Commons Attribution–NonCommercial–ShareAlike 4.0 International License:
`https://creativecommons.org/licenses/by-nc-sa/4.0`

1. Changes

For changes in previous versions we refer to Appendix B on Page 259.

1.1. New in Version 32.0.1 (January 2026, Germanium, 32.0)

This release is intended for Frama-C [2, v32.0] issued in December 2025. We are also using for this release the Why3 platform [3, v1.8.2] and the provers listed in the following table.

Prover	Type	Version	Reference
Alt-Ergo	automatic	2.6.2	[4]
CVC5	automatic	1.3.2	[5]
Z3	automatic	4.15.2	[6]
Coq	interactive	8.20.1	[7]

Table 1.1.: Information on automatic and interactive theorem provers

New examples

None.

Improvements

- Added `terminates` and `exits` clauses to all function contracts.
- Completed the `assigns` clauses of `stack_init` and `axiom_size_of_init` with explicit `\from` dependencies, clarifying the origin of assigned values and aligning the contracts with Frama-C/WP 's pedantic assigns checks.
- Replaced statement contracts in `pop_heap`, `partial_sort`, `selection_sort`, `insertion_sort`, and `merge` by assertions referring to an explicit ghost label, thereby making the proof obligations local to the respective loop bodies.
- Improved the description of non-mutating algorithms; removed `count2` and the inductively defined logic function `CountInd`.
- Improved the description of minimum and maximum algorithms.
- Improved the description of binary search algorithms and removed `binary_search2`.

List of Open Issues

10.7.1 Completeness of the postcondition of merge 219

Contents

1 Changes **3**
1.1 New in Version 32.0.1 (January 2026, Germanium, 32.0) . . . 3
List of Open Issues . . . 3

I Basics **11**

2 Introduction **13**
2.1 Frama-C . . . 14
2.2 Structure of this document . . . 14
2.3 Types, arrays, ranges and valid indices . . . 15

3 The Hoare calculus **17**
3.1 The assignment rule . . . 19
3.2 The sequence rule . . . 21
3.3 The implication rule . . . 21
3.4 The choice rule . . . 22
3.5 The loop rule . . . 23
3.6 Derived rules . . . 25

II Nonmutating and simple search algorithms **27**

4 Non-mutating algorithms **29**
4.1 The `find` algorithm . . . 30
4.2 The `find2` algorithm—reuse of specification elements . . . 33
4.3 The `find3` algorithm—using a logic function . . . 36
4.4 The `find_if_not` algorithm . . . 41
4.5 The `find_first_of` algorithm . . . 45
4.6 The `adjacent_find` algorithm . . . 48
4.7 The `equal` and `mismatch` algorithms . . . 51
4.8 The `search` algorithm . . . 54
4.9 The `search_n` algorithm . . . 57
4.10 The `find_end` algorithm . . . 60
4.11 The `count` algorithm . . . 63

5 Maximum and minimum algorithms **69**
5.1 A note on relational operators . . . 70
5.2 Predicates for bounds and extrema of arrays . . . 71
5.3 The auxiliary function `make_pair` . . . 73
5.4 The `clamp` algorithm . . . 74
5.5 The `max_element` algorithm . . . 76

5.6 The max_element2 algorithm (predicate-based specification) . . . 78
5.7 The max_seq algorithm . . . 80
5.8 The min_element algorithm . . . 81
5.9 The minmax_element algorithm . . . 83

6 Binary search algorithms 85
6.1 The predicate Increasing . . . 85
6.2 A weaker monotonicity predicate . . . 86
6.3 The lower_bound algorithm . . . 88
6.4 The upper_bound algorithm . . . 90
6.5 The equal_range algorithm . . . 92
6.6 The binary_search algorithm . . . 97

III Mutating and numeric algorithms 99

7 Mutating algorithms 101
7.1 The predicate Unchanged . . . 102
7.2 The fill algorithm . . . 104
7.3 The swap algorithm . . . 105
7.4 The swap_ranges algorithm . . . 107
7.5 The copy algorithm . . . 109
7.6 The copy_backward algorithm . . . 111
7.7 The reverse_copy algorithm . . . 113
7.8 The reverse algorithm . . . 115
7.9 The rotate_copy algorithm . . . 116
7.10 The rotate algorithm . . . 118
7.11 The replace_copy algorithm . . . 120
7.12 The replace algorithm . . . 122
7.13 The remove_copy algorithm (basic contract) . . . 123
7.14 The remove_copy2 algorithm (number of copied elements) . . . 125
7.15 The remove_copy3 algorithm (final contract) . . . 128
7.16 The remove algorithm . . . 135
7.17 The shuffle algorithm . . . 137
7.18 Verifying a random number generator . . . 142

8 Numeric algorithms 145
8.1 The iota algorithm . . . 146
8.2 The accumulate algorithm . . . 148
8.3 The inner_product algorithm . . . 152
8.4 The partial_sum algorithm . . . 156
8.5 The adjacent_difference algorithm . . . 159
8.6 Inverting partial_sum and adjacent_difference . . . 163

IV Sorting algorithms 167

9 Heap Algorithms 169
9.1 Basic heap concepts . . . 171
9.2 Representation of heap concepts in ACSL . . . 174
9.3 The auxiliary functions heap_parent and heap_child . . . 176
9.4 The is_heap_until algorithm . . . 177

9.5 The `is_heap` algorithm . . . 179
9.6 Reorderings and fluctuations . . . 180
9.7 The `push_heap` algorithm . . . 184
9.8 The `pop_heap` algorithm . . . 190
9.9 The `make_heap` algorithm . . . 196
9.10 The `sort_heap` algorithm . . . 198

10 Sorting Algorithms 201
10.1 The `is_sorted` algorithm . . . 202
10.2 The `partial_sort` algorithm . . . 204
10.3 The `bubble_sort` algorithm . . . 210
10.4 The `selection_sort` algorithm . . . 212
10.5 The `insertion_sort` algorithm . . . 214
10.6 The `heap_sort` algorithm . . . 217
10.7 The `merge` algorithm . . . 218

V Verification of data structures 223

11 The `stack` data type 225
11.1 Methodology overview . . . 226
11.2 Stack axioms . . . 227
11.3 The structure `stack` and its associated functions . . . 229
11.4 Stack invariants . . . 230
11.5 Equality of stacks . . . 231
11.6 Verification of stack functions . . . 234
11.7 Verification of stack axioms . . . 246

VI Appendices 251

A Results of formal verification with Frama-C 253
A.1 Verification settings . . . 253
A.2 Verification results (sequential) . . . 254

B Changes in previous releases 259
B.1 New in Version 32.0.0 (Germanium, December 2025) . . . 259
B.2 New in Version 22.0.0 (Titanium, November 2020) . . . 260
B.3 New in Version 21.1.1 (Scandium, September 2020) . . . 260
B.4 New in Version 21.1.0 (Scandium, July 2020) . . . 262
B.5 New in Version 20.0.2 (Calcium, April 2020) . . . 263
B.6 New in Version 20.0.1 (Calcium, March 2020) . . . 264
B.7 New in Version 20.0.0 (Calcium, December 2019) . . . 266
B.8 New in Version 19.1.0 (Potassium, October 2019) . . . 267
B.9 New in Version 19.0.0 (Potassium, June 2019) . . . 269
B.10 New in Version 18.0.0 (Argon, December 2018) . . . 269
B.11 New in Version 17.1.0 (Chlorine, July 2018) . . . 270
B.12 New in Version 16.1.1 (Sulfur, March 2018) . . . 270
B.13 New in Version 16.1.0 (Sulfur, December 2017) . . . 270
B.14 New in Version 15.1.2 (Phosphorus, October 2017) . . . 271
B.15 New in Version 15.1.1 (Phosphorus, September 2017) . . . 272
B.16 New in Version 15.1.0 (Phosphorus, June 2017) . . . 272

B.17 New in Version 14.1.1 (Silicon, April 2017) . . . 273
B.18 New in Version 14.1.0 (Silicon, January 2017) . . . 273
B.19 New in Version 13.1.1 (Aluminium, November 2016) . . . 274
B.20 New in Version 13.1.0 (Aluminium, August 2016) . . . 274
B.21 New in Version 12.1.0 (Magnesium, February 2016) . . . 275
B.22 New in Version 11.1.1 (Sodium, June 2015) . . . 277
B.23 New in Version 11.1.0 (Sodium, March 2015) . . . 277
B.24 New in Version 10.1.1 (Neon, January 2015) . . . 278
B.25 New in Version 10.1.0 (Neon, September 2014) . . . 279
B.26 New in Version 9.3.1 (Fluorine, not published) . . . 279
B.27 New in Version 9.3.0 (Fluorine, December 2013) . . . 279
B.28 New in Version 8.1.0 (Oxygen, not published) . . . 280
B.29 New in Version 7.1.1 (Nitrogen, August 2012) . . . 280
B.30 New in Version 7.1.0 (Nitrogen, December 2011) . . . 280
B.31 New in Version 6.1.0 (Carbon, not published) . . . 280
B.32 New in Version 5.1.1 (Boron, February 2011) . . . 280
B.33 New in Version 5.1.0 (Boron, May 2010) . . . 281
B.34 New in Version 4.2.2 (Beryllium, May 2010) . . . 281
B.35 New in Version 4.2.1 (Beryllium, April 2010) . . . 281
B.36 New in Version 4.2.0 (Beryllium, January 2010) . . . 282

Bibliography **283**

Index of ACSL definitions **285**

Index of examples **291**

List of Figures

4.1 Some simple examples for `find` 30
4.20 A simple example of `find_first_of` 45
4.24 A simple example of `adjacent_find` 48
4.33 A simple example of `search` 54
4.37 A simple example of `search_n` 57
4.41 A simple example of `find_end` 60

6.4 Some examples for `lower_bound` 88
6.7 Some examples for `upper_bound` 90
6.10 Some examples for `equal_range` 92
6.15 Some examples for `binary_search` 97

7.10 Effects of `copy` 109
7.11 Possible overlap of `copy` ranges 109
7.14 Possible overlap of `copy_backward` ranges 111
7.18 Sketch of predicate `Reverse` 113
7.23 Effects of `rotate_copy` 116
7.26 Effects of `rotate` 118
7.29 Effects of `replace` 120
7.36 Effects of `remove_copy` 123
7.43 Partitioning the input of `remove_copy` 128
7.51 Effects of `remove` 135
7.54 Effects of `shuffle` 137

9.1 Overview on heap algorithms 169
9.2 Tree representation of the multiset X 172
9.3 Underlying array of a heap 172
9.4 An alternative representation of the multiset X 173
9.5 Underlying array of the alternative representation 173
9.21 Example heap before the call of `push_heap` 185
9.22 Example heap after the call of `push_heap` 185
9.24 Heap after the prologue of `push_heap` 187
9.25 Heap after the main act of `push_heap` 188
9.26 Heap after the epilogue of `push_heap` 189
9.28 Heap before the call of `pop_heap` 191
9.29 Heap after the call of `pop_heap` 191
9.31 Heap after the prologue of `pop_heap` 193
9.32 Heap after the first iteration of of `pop_heap` 194
9.33 Heap after the second iteration of `pop_heap` 194
9.34 Heap after the third iteration of `pop_heap` 195
9.35 Heap after the epilogue of `pop_heap` 195
9.36 Array before the call of `make_heap` 196
9.39 Array after the call of `sort_heap` 198

10.3 Effects of `partial_sort` 204
10.8 An iteration of `partial_sort` 208
10.12 An iteration of `selection_sort` 212
10.15 An iteration of `insertion_sort` 214

11.1 Push and pop on a stack 225
11.2 Methodology Overview 226
11.4 Interpreting the data structure `stack` 229
11.9 Example of two equal stacks 232
11.10 Methodology for the verification of well-definition 234

A.1 Verification pipeline of automatic and interactive theorem provers 254

List of Tables

1.1 Information on automatic and interactive theorem provers 3

3.1 Some ACSL formula syntax 17

7.35 Properties of `remove_copy` 123

A.2 Results for non-mutating algorithms 254
A.3 Results for maximum and minimum algorithms 255
A.4 Results for binary search algorithms 255
A.5 Results for mutating algorithms 255
A.6 Results for numeric algorithms 256
A.7 Results for heap algorithms 256
A.8 Results for algorithms related to sorting 256
A.9 Results for stack functions 257
A.10 Results for the well-definition of stack functions 257
A.11 Results for stack axioms 257

Part I.

Basics

2. Introduction

This report provides various examples for the formal specification, implementation, and deductive verification of C programs using the ANSI/ISO-C Specification Language (ACSL [8]) and the Frama-C/WP plug-in [1] of Frama-C [2] (Framework for Modular Analysis of C programs).

We have chosen our examples from the C++ Standard Library whose initial version is still known as the *Standard Template Library* (STL). The C++ Standard Library contains a broad collection of *generic* algorithms that work not only on C arrays but also on more elaborate container data structures. For the purposes of this document we have selected representative algorithms, and converted their implementation from C++ function templates to C functions that work on arrays of type `int`.

We will continue to extend and refine this report by describing additional STL algorithms and data structures. Thus, step by step, this document will evolve from an ACSL tutorial to a report on a formally specified and deductively verified Standard Library for ANSI/ISO-C. Moreover, as ACSL is extended to a C++ specification language, our work may be extended to a deductively verified C++ Standard Library.

We encourage you to check vigilantly whether our formal specifications capture the essence of the informal description of the STL algorithms. We appreciate your feedback[1] and hope that this document helps foster the adoption of deductive verification techniques.

Acknowledgement

Many members from the Frama-C community provided valuable input and comments during the course of the development of this document. In particular, we wish to thank our project partners Patrick Baudin, Allan Blanchard, Loïc Correnson, Zaynah Dargaye, Florent Kirchner, Virgile Prevosto, and Armand Puccetti from CEA LIST[2] and Pascal Cuoq from TrustInSoft[3].

We also like to express our gratitude to Claude Marché (LRI/INRIA)[4] and Yannick Moy (AdaCore)[5] for their helpful comments and detailed suggestions for improvement. Finally, we would like to thank Aaron Rocha who sent us valuable improvement suggestions and error reports.

[1] We suggest GitHub's issue tracker: `https://github.com/fraunhoferfokus/acsl-by-example/issues`
[2] `http://www-list.cea.fr/en`
[3] `http://trust-in-soft.com`
[4] `https://www.lri.fr/index_en.php?lang=EN`
[5] `http://www.adacore.com`

2.1. Frama-C

The Framework for Modular Analyses of C, Frama-C [2], is a suite of software tools dedicated to the analysis of C source code. Its development efforts are conducted and coordinated at two French public institutions: CEA LIST [9], a laboratory of applied research on software-intensive technologies, and INRIA Saclay [10], the French National Institute for Research in Computer Science and Control in collaboration with LRI [11], the Laboratory for Computer Science at Université Paris-Sud.

ACSL (ANSI/ISO-C Specification Language) [8] is a formal language to express behavioral properties of C programs. This language can specify a wide range of functional properties by adding annotations to the code. It allows to create function contracts containing preconditions and postconditions. It is possible to define type and global invariants as well as logic specifications, such as predicates, lemmas, axioms or logic functions. Furthermore, ACSL allows statement annotations such as assertions or loop annotations.

Within Frama-C, the Frama-C/WP plug-in [1] enables deductive verification of C programs that have been annotated with ACSL. The Frama-C/WP plug-in uses Hoare-style weakest precondition computations to formally prove ACSL properties of C code. Verification conditions are generated and submitted to external automatic theorem provers or interactive proof assistants.

The Verification Group at Fraunhofer FOKUS [12] see the great potential for deductive verification using ACSL. However, we recognize that for a novice there are challenges to overcome in order to effectively use the Frama-C/WP plug-in for deductive verification. In order to help users gain confidence, we have written this tutorial that demonstrates how to write annotations for existing C programs. This document provides several examples featuring a variety of annotated functions using ACSL. For an in-depth understanding of ACSL, we strongly recommend users to read the official Frama-C introductory tutorial [13] first. The principles presented in this paper are also documented in the ACSL reference document [14].

2.2. Structure of this document

The functions presented in this document were selected from the C++ Standard Library. The original C++ implementation was stripped from its generic implementation and mapped to C arrays of type `value_type`.

Chapter 3 provides a short introduction into the Hoare Calculus. For a better understanding of Frama-C/WP and the theory behind it, we also recommend Allan Blanchard's ACSL tutorial [15].

We have grouped various standard algorithms in chapters as follows:

- non-mutating algorithms (Chapter 4)
- maximum/minimum algorithms (Chapter 5)
- binary search algorithms (Chapter 6)
- mutating algorithms (Chapter 7)
- numeric algorithms (Chapter 8)
- heap algorithms (Chapter 9)
- sorting algorithms and well-known classical implementations of sorting algorithms (Chapter 10)

The order of these chapters reflects their increasing complexity.

Using the example of a stack, we tackle in Chapter 11 the problem of how a data type and its associated C functions can be specified with ACSL and automatically verified with Frama-C.

Finally, Appendix A lists for each example the results of verification with Frama-C.

2.3. Types, arrays, ranges and valid indices

In order to keep algorithms and specifications as general as possible, we use abstract type names on almost all occasions. We currently defined the following types:

```
typedef int value_type;

typedef unsigned int size_type;

typedef int bool;
```

Programmers who know the types associated with C++ Standard Library containers will not be surprised that `value_type` refers to the type of values in an array whereas `size_type` will be used for the indices of an array.

This approach allows one to modify, say, an algorithm working on an **`int`** array to work on a **`char`** array by changing only one line of code, viz. the **`typedef`** of `value_type`. Moreover, we believe in better readability as it becomes clear whether a variable is used as an index or as a memory for a copy of an array element, just by looking at its type.

The latter reason also applies to the use of **`bool`**. To denote values of that type, we defined the identifiers **`false`** and **`true`** to be `0` and `1`, respectively. While any non-zero value is accepted to denote **`true`** in ACSL like in C the algorithms shown in this tutorial will always produce `1` for **`true`**. Due to the above definitions, the ACSL truth-value constant `\`**`false`** and `\`**`true`** can be used interchangeably with our **`false`** and **`true`**, respectively, in ACSL clauses, but not in C code.

2.3.1. Array and ranges

The C Standard describes an array as a "contiguously allocated nonempty set of objects" [16, §6.2.5.20]. If `n` is a constant integer expression with a value greater than zero, then

```
int a[n];
```

describes an array of type **`int`**. In particular, for each `i` that is greater than or equal to 0 and less than `n`, we can dereference the pointer `a+i`.

Let the following prototype represent a function, whose first argument is the address to a range and whose second argument is the length of this range.

```
void example(value_type* a, size_type n);
```

To be very precise, we have to use the term *range* instead of *array*. This is due to the fact, that functions may be called with empty ranges, i.e., with `n == 0`. Empty arrays, however, are not permitted according to the definition stated above. Nevertheless, we often use the term *array* and *range* interchangeably.

2.3.2. Specification of valid ranges in ACSL

The following ACSL fragment expresses the precondition that the function `example` expects that for each `i`, such that `0 <= i < n`, the pointer `a+i` may be safely dereferenced.

```
/*@
    requires 0 <= n;
    requires \valid(a + (0.. n-1));
*/
void example(value_type* a, size_type n);
```

In this case we refer to each index `i` with `0 <= i < n` as a *valid index* of `a`.

ACSL's built-in predicates `\valid(a + (0.. n))` and `\valid_read(a + (0.. n))` refer to all addresses `a+i` where `0 <= i <= n`. However, the array notation `int a[n]` of the C programming language refers only to the elements `a+i` where `i` satisfies `0 <= i < n`. Users of ACSL must therefore use the range notation `a+(0.. n-1)` in order to express a valid array of length `n`.

3. The Hoare calculus

In 1969, C.A.R. Hoare introduced a calculus for formal reasoning about properties of imperative programs [17], which became known as "Hoare Calculus".

The basic notion is

```
//@ assert P;
Q;
//@ assert R;
```

where `P` and `R` denote logical expressions and `Q` denotes a source-code fragment. Informally, this means

If `P` holds before the execution of `Q`, then `R` will hold after the execution.

Usually, `P` and `R` are called *precondition* and *postcondition* of `Q`, respectively. The syntax for logical expressions is described in [14, §2.2] in full detail. For the purposes of this tutorial, the notions shown in Table 3.1 are sufficient. Note that they closely resemble the logical and relational operators in C.

ACSL syntax	Name	Reading
`!P`	negation	`P` is not true
`P && Q`	conjunction	`P` is true and `Q` is true
`P || Q`	disjunction	`P` is true or `Q` is true
`P ==> Q`	implication	if `P` is true, then `Q` is true
`P <==> Q`	equivalence	if, and only if, `P` is true, then `Q` is true
`x < y == z`	relation chain	`x` is less than `y` and `y` is equal to `z`
`\forall int x; P(x)`	universal quantifier	`P(x)` is true for every `int` value of `x`
`\exists int x; P(x)`	existential quantifier	`P(x)` is true for some `int` value of `x`

Table 3.1.: Some ACSL formula syntax

Here we show three example source-code fragments and annotations.

`//@ assert x % 2 == 1;`
`++x;`
`//@ assert x % 2 == 0;`
	If `x` has an odd value before execution of the code `++x` then `x` has an even value thereafter.

`//@ assert 0 <= x <= y;`
`++x;`
`//@ assert 0 <= x <= y + 1;`
	If the value of `x` is in the range $\{0, \ldots, y\}$ before execution of the same code, then `x`'s value is in the range $\{0, \ldots, y + 1\}$ after execution.

`//@ assert true;`
`while (--x != 0)`
`  sum += a[x];`
`//@ assert x == 0;`
	Under any circumstances, the value of `x` is zero after execution of the loop code.

Any C programmer will confirm that these properties are valid.[6] The examples were chosen to demonstrate also the following issues:

- For a given code fragment, there does not exist one fixed pre- or postcondition. Rather, the choice of formulas depends on the actual property to be verified, which comes from the application context. The first two examples share the same code fragment, but have different pre- and postconditions.
- The postcondition need not be the most restricting possible formula that can be derived. In the second example, it is not an error that we stated only that `0 <= x` although we know that even `1 <= x`.
- In particular, pre- and postconditions need not contain all variables appearing in the code fragment. Neither `sum` nor `a[]` is referenced in the formulas of the loop example.
- We can use the predicate **`true`** to denote the absence of a properly restricting precondition, as we did before the **`while`** loop.
- It is not possible to express by pre- and postconditions that a given piece of code will always terminate. The loop example only states that *if* the loop terminates, then `x == 0` will hold. In fact, if `x` has a negative value on entry, the loop will run forever. However, if the loop terminates, `x == 0` will hold, and that is what the loop example claims.

  Usually, termination issues are dealt with separately from correctness issues. Termination proofs may, however, refer to properties stated (and verified) using the Hoare Calculus.

Hoare provided the rules shown in Listing 3.2 to 3.12 in order to reason about programs. We will comment on them in the following sections.

[6] We leave the important issues of overflow aside for a moment.

3.1. The assignment rule

We start with the rule that is probably the least intuitive of all Hoare-Calculus rules, viz. the assignment rule. It is depicted in Listing 3.2, where

$$P\{x \mapsto e\}$$

denotes the result of substituting each occurrence of the variable `x` in the predicate `P` by the expression `e`.

```
//@ assert P {x |--> e};
x = e;
//@ assert P;
```

Listing 3.2: The assignment rule

For example, if `P` is the predicate

```
x > 0 && a[2*x] == 0
```

then $P\{x \mapsto y + 1\}$ is the predicate

```
y+1 > 0 && a[2*(y+1)] == 0
```

Hence, we get Listing 3.3 as an example instance of the assignment rule. Note that parentheses are required in the index expression to get the correct `2*(y+1)` rather than the faulty `2*y+1`.

```
//@ assert y+1 > 0 && a[2*(y+1)] == 0;
x = y+1;
//@ assert x > 0 && a[2*x] == 0;
```

Listing 3.3: An assignment rule example instance

Note that after a substitution several different predicates `P` may result in the same predicate $P\{x \mapsto e\}$. For example, after applying the substitution $P\{x \mapsto y + 1\}$ each of the following four predicates

```
  x > 0 && a[2*x]     == 0
  x > 0 && a[2*(y+1)] == 0
y+1 > 0 && a[2*x]     == 0
y+1 > 0 && a[2*(y+1)] == 0
```

turns into

```
y+1 > 0 && a[2*(y+1)] == 0
```

For this reason, the same precondition and statement may result in several different postconditions (All four above expressions are valid postconditions in Listing 3.3, for example). However, given a postcondition and a statement, there is only one precondition that corresponds.

When first confronted with Hoare's assignment rule, most people are tempted to think of a simpler and more intuitive alternative, shown in Listing 3.4.

```
//@ assert P;
x = e;
//@ assert P && x == e;
```

Listing 3.4: Simpler, but *faulty* assignment rule

Listings 3.5–3.7 show some example instances of this faulty rule.

```
//@ assert y > 0;
x = y+1;
//@ assert y > 0 && x == y+1;
```

Listing 3.5: An example instance of the faulty rule from Listing 3.4

While Listing 3.5 happens to be ok, Listing 3.6 and 3.7 lead to postconditions that are obviously nonsensical formulas.

```
//@ assert true;
x = x+1;
//@ assert x == x+1;
```

Listing 3.6: An example instance of the faulty rule from Listing 3.4

The reason is that in the assignment in Listing 3.6 the left-hand side variable `x` also appears in the right-hand side expression `e`, while the assignment in Listing 3.7 just destroys the property from its precondition.

```
//@ assert x < 0;
x = 5;
//@ assert x < 0 && x == 5;
```

Listing 3.7: An example instance of the faulty rule from Listing 3.4

Note that the correct example Listing 3.5 can as well be obtained as an instance of the correct rule from Listing 3.2, since replacing `x` by `y+1` in its postcondition yields `y > 0 && y+1 == y+1` as precondition, which is logically equivalent to just `y > 0`.

3.2. The sequence rule

The sequence rule, shown in Listing 3.8, combines two code fragments `Q` and `S` into a single one `Q ; S`. Note that the postcondition for `Q` must be identical to the precondition of `S`. This just reflects the sequential execution ("first do `Q`, then do `S`") on a formal level. Thanks to this rule, we may "annotate" a program with interspersed formulas, as it is done in Frama-C.

```
//@ assert P;
Q;
//@ assert R;
```

and

```
//@ assert R;
S;
//@ assert T;
```

$\rightsquigarrow$

```
//@ assert P;
Q ; S;
//@ assert T;
```

Listing 3.8: The sequence rule

3.3. The implication rule

The implication rule, shown in Listing 3.9, allows us at any time to sharpen a precondition `P` and to weaken a postcondition `R`. More precisely, if we know that `P' ==> P` and `R ==> R'` then the we can replace the left contract in of Listing 3.9 by the right one.

```
//@ assert P;
Q;
//@ assert R;
```

$\rightsquigarrow$

```
//@ assert P';
Q;
//@ assert R';
```

Listing 3.9: The implication rule

3.4. The choice rule

The choice rule, depicted in Listing 3.10, is needed to verify conditional statements of the form

```
if (C) X;
else   Y;
```

Both the then and else branch must establish the same postcondition, viz. `S`. The implication rule can be used to weaken differing postconditions `S1` of a then-branch and `S2` of an else-branch into a unified postcondition `S1 || S2`, if necessary. In each branch, we may use what we know about the condition `C`. For example, in the else-branch, we may use that `C` is false. If the else-branch is missing, it can be considered as consisting of an empty sequence, having the postcondition `P && !C`.

```
//@ assert P && C;
X;
//@ assert S;
```

and

```
//@ assert P && !C;
Y;
//@ assert S;
```

$\rightsquigarrow$

```
//@ assert P;
if (C) X;
else   Y;
//@ assert S;
```

Listing 3.10: The choice rule

Listing 3.11 shows an example application of the choice rule.

```
//@ assert 0 <= i < n;           // given precondition
if (i < n-1) {
  //@ assert 0 <= i < n - 1;     // using that i < n-1 holds in this branch
  //@ assert 1 <= i+1 < n;       // by the implication rule
  i = i+1;
  //@ assert 1 <= i < n;         // by the assignment rule
  //@ assert 0 <= i < n;         // weakened by the implication rule
} else {
  //@ assert 0 <= i == n-1 < n; // using that !(i < n-1) holds in else part
  //@ assert 0 == 0 && 0 < n;    // weakened by the implication rule
  i = 0;
  //@ assert i == 0 && 0 < n;    // by the assignment rule
  //@ assert 0 <= i < n;         // weakened by the implication rule
}
//@ assert 0 <= i < n;           // by the choice rule from both branches
```

Listing 3.11: An example application of the choice rule

The variable `i` may be used as an index into a ring buffer `int a[n]`. The shown code fragment just advances the index `i` appropriately. We verified that `i` remains a valid index into `a[]` provided it was valid before. Note the use of the implication rule to establish preconditions for the assignment rule as needed, and to unify the postconditions of the then and else branches, as required by the choice rule.

3.5. The loop rule

The loop rule, shown in Listing 3.12, is used to verify a **while** loop. This requires to find an appropriate formula, P, which is preserved by each execution of the loop body. P is also called a loop invariant.

```
//@ assert P && B;
S;
//@ assert P;
```

$\rightsquigarrow$

```
//@ assert P;
while (B) {
  S;
}
//@ assert !B && P;
```

Listing 3.12: The loop rule

To find it requires some intuition in many cases; for this reason, automatic theorem provers usually have problems with this task.

As said above, the loop rule does not guarantee that the loop will always eventually terminate. It merely assures us that, if the loop has terminated, the postcondition holds. To emphasize this, the properties verifiable with the Hoare Calculus are usually called "partial correctness" properties, while properties that include program termination are called "total correctness" properties.

As an example application, let us consider an abstract ring-buffer. Listing 3.13 shows a verification proof for the index i lying always within the valid range [0..n-1] during, and after, the loop. It uses the proof from Listing 3.11 as a sub-part.

```
//@ assert  0 < n;                  // given precondition

int i = 0;
//@ assert  0 <= i < n;             // by the assignment rule

while (!done) {
  //@ assert 0 <= i < n && !done;   // may be assumed by the loop rule

  a[i] = getchar();
  //@ assert 0 <= i < n && !done;   // required property of getchar
  //@ assert 0 <= i < n;            // weakened by the implication rule

  i = (i < n-1) ? i+1 : 0;
  //@ assert 0 <= i < n;            // follows by the choice rule

  process(a, i, &done);
  //@ assert 0 <= i < n;            // required property of process
}
//@ assert 0 <= i < n;              // by the loop rule
```

Listing 3.13: An abstract ring buffer loop

To reuse the proof from Listing 3.11, we had to drop the conjunct `!done`, since we didn't consider it in Listing 3.11. In general, we may *not* infer

```
//@ assert P && S;
Q;
//@ assert R && S;
```

from

```
//@ assert P;
Q;
//@ assert R;
```

since the code fragment `Q` may just destroy the property `S`.

This is obvious for `Q` being the fragment from Listing 3.11, and `S` being e.g. `i != 0`.

Suppose for a moment that `process` had been implemented in a way such that it refuses to set `done` to **`true`** unless it is **`false`** at entry. In this case, we would really need that `!done` still holds after execution of Listing 3.11. We would have to do the proof again, looping-through an additional conjunct `!done`.

We have similar problems to carry the property `0 <= i < n && !done` and `0 <= i < n` over the statement `a[i] = getchar()` and `process(a, i, &done)`, respectively. We need to specify that neither `getchar` nor `process` is allowed to alter the value of `i` or `n`. In ACSL, there is a particular language construct `assigns` for that purpose, which is introduced in §7.3 on Page 105.

In our example, the loop invariant can be established between any two statements of the loop body. However, this need not be the case in general. The loop rule only requires the invariant holds before the loop and at the end of the loop body. For example, `process` could well change the value of `i`[7] and even `n` intermediately, as long as it re-establishes the property `0 <= i < n` immediately prior to returning.

The loop invariant, `0 <= i < n`, is established by the proof in Listing 3.11 also after termination of the loop. Thus, e.g., a final `a[i] = '\0'` after the loop would be guaranteed not to lead to a bounds violation.

Even if we would need the property `0 <= i < n` to hold only immediately before the assignment `a[i] = getchar()`, for example since `process`'s body didn't use `a` or `i`, we would still have to establish `0 <= i < n` as a loop invariant by the loop rule, since there is no other way to obtain any property inside a loop body. Apart from this formal reason it is obvious that `0 <= i < n` wouldn't hold during the second loop iteration unless we re-established it at the end of the first one, and that is just what the while rule requires.

[7] We would have to change the call to `process(a, &i, &done)` and the implementation of `process` appropriately. In this case we couldn't rely on the above-mentioned `assigns` clause for `process`.

3.6. Derived rules

The above rules do not cover all kinds of statements allowed in C. However, missing C-statements can be rewritten into a form that is semantically equivalent and covered by the Hoare rules.

For example, if the expression `E` doesn't have side-effects, then

```
switch (E) {
    case E1: Q1; break; ...
    case En: Qn; break;
    default: Q0; break;
}
```

is semantically equivalent to

```
if (E == E1) {
    Q1;
} else ... if (E == En) {
    Qn;
} else {
    Q0;
}
```

While the **`if-else`** form is usually slower in terms of execution speed on a real computer, this doesn't matter for verification purposes, which are separate from execution issues.

Similarly, a loop statement of the form

```
for (P; Q; R) {
  S;
}
```

can be re-expressed as

```
P;
while (Q) {
  S;
  R;
}
```

and so on.

It is then possible to derive a Hoare rule for each kind of statement not previously discussed, by applying the classical rules to the corresponding re-expressed code fragment. However, we do not present these derived rules here.

Although procedures cannot be re-expressed in the above way if they are (directly or mutually) recursive, it is still possible to derive Hoare rules for them. This requires the finding of appropriate "procedure invariants" similar to loop invariants. Non-recursive procedures can, of course, just be inlined to make the classical Hoare rules applicable.

Note that **`goto`** cannot be rewritten in the above way; in fact, programs containing **`goto`** statements cannot be verified with the Hoare Calculus. See [18] for a similar calculus that can deal with arbitrary flowcharts, and hence arbitrary jumps. In fact, Hoare's work was based on that calculus. Later calculi inspired from Hoare's work have been designed to re-integrate support for arbitrary jumps. However, in this tutorial, we will not discuss example programs containing a **`goto`**.

Part II.

Nonmutating and simple search algorithms

4. Non-mutating algorithms

In this chapter, we consider *non-mutating* algorithms of the C++ Standard Library [19, §28.5]. These algorithms neither modify their arguments nor any other memory locations. This requirement can be formally expressed by the following *assigns clause*:

```
assigns \nothing;
```

This assigns clause is therefore used uniformly in the specification of all algorithms presented in this chapter.

The specifications of these algorithms are relatively simple, but they nevertheless illustrate a range of specification techniques. The examples are arranged so that earlier algorithms are simpler than the later ones. All algorithms considered here operate on one-dimensional arrays.

- `find` in §4.1 provides a *sequential* or *linear search* and returns the smallest index at which a given value occurs in a given range. In §4.2, a user-defined ACSL predicate is introduced to simplify the reuse of common specification elements. This simplified version is referred to as `find2`. In §4.3, we provide a third specification of `find` (called `find3`) that relies on a user-defined ACSL logic function expressing the idea of linear search at the logic level.
- `find_if_not` in §4.4 is a small variation of `find` that searches for the first position at which a given value does *not* occur.
- `find_first_of` in §4.5 provides, like `find`, a *sequential search*. Unlike `find`, however, it does not search for a particular value, but for an arbitrary element of a given set.
- `adjacent_find` in §4.6 can be used to find equal neighboring elements in an array.
- `equal` and `mismatch` in §4.7 are used to compare two ranges element-by-element and to identify the first position at which they differ.
- `search` and `search_n` in §4.8 and §4.9, respectively, search a subsequence that is identical to a given sequence when compared element-by-element and return the position of the first occurrence.
- `find_end` in §4.10 searches for a subsequence that matches a given sequence element-by-element and returns the *largest* index at which such a match starts. Thus, `find_end` can be seen as the counterpart of `search` that identifies the *last* occurrence rather than the first.
- `count` in §4.11 returns the number of occurrences of a given value in a range. Here we explicitly define a logic function for counting elements and show that the implementation complies with it.

4.1. The `find` algorithm

The `find` algorithm in the C++ Standard Library [19, §28.5.5] implements *sequential search* for general sequences. We have modified the generic implementation, which relies heavily on C++ templates, to that of a range of type `value_type`. The signature now reads:

```
size_type find(const value_type* a, size_type n, value_type v);
```

The function `find` returns the least *valid* index `i` of `a` where the condition `a[i] == v` holds. If no such index exists then `find` returns the length `n` of the array.

As an example, we consider in Figure 4.1 an array. The arrows indicate which indices will be returned by `find` for a given value. Note that the index 9 points *one past end* of the array. Values that are not contained in the array are colored in gray.

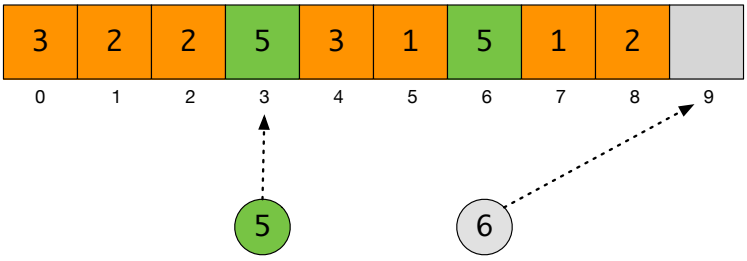

Figure 4.1.: Some simple examples for `find`

4.1.1. Formal specification of `find`

Listing 4.2 presents our first formal specification of `find`. It illustrates the basic structure of an ACSL function contract and introduces the most common specification clauses that will be used throughout this document.

The *preconditions* of the function are expressed by the `requires`-clauses. They state that the length `n` of the array is non-negative and that the pointer `a` denotes a contiguous block of `n` objects of type `value_type`. This pattern of specifying valid memory ranges occurs frequently and will be reused in many subsequent specifications (see §2.3).

The `assigns`-clause specifies the memory locations that may be modified by the function. Since `find` is a non-mutating algorithm, it does not modify any memory location visible to its caller; this is expressed by the clause `assigns \nothing` (see Page 29).

The clause `terminates \true;` states that the function is guaranteed to terminate for all inputs satisfying the preconditions. In particular, the implementation contains neither infinite loops nor unbounded recursion.

The clause `exits \false;` specifies that the function never terminates by an abnormal exit. That is, it cannot call `exit()`, trigger a runtime abort, or otherwise leave its execution context without returning normally to its caller.

```
/*@
  requires   \valid_read(a + (0..n-1));

  terminates \true;
  exits      \false;
  assigns    \nothing;

  ensures    0 <= \result <= n;

  behavior some:
    assumes  \exists integer i; 0 <= i < n && a[i] == v;
    assigns  \nothing;
    ensures  0 <= \result < n;
    ensures  a[\result] == v;
    ensures  \forall integer i; 0 <= i < \result ==> a[i] != v;

  behavior none:
    assumes  \forall integer i; 0 <= i < n ==> a[i] != v;
    assigns  \nothing;
    ensures  \result == n;

  complete behaviors;
  disjoint behaviors;
*/
size_type
find(const value_type* a, size_type n, value_type v);
```

Listing 4.2: Formal specification of `find`

At this point, the contract only guarantees that `find` returns a non-negative index that is less than or equal to the array length `n`. More precise statements about the return value can be made once additional assumptions about the contents of the array are introduced. For this reason, the specification of `find` is subdivided into two *behaviors*, named `some` and `none`.

- The behavior `some` applies when the sought-after value is contained in the array. This situation is characterized by the `assumes`-clause of the behavior. Under this assumption, the contract ensures that `find` returns a valid index `i` such that `a[i] == v`. Moreover, the final postcondition of this behavior states that the returned index is the *smallest* index with this property, thereby capturing the fact that `find` returns the first occurrence of the value in the array.
- The behavior `none` covers the complementary case in which the sought-after value does not occur in the array, as expressed by its `assumes`-clause. In this case, the contract requires `find` to return the value `n`, which indicates that the search was unsuccessful.

The formulas used in the `assumes`-clauses of the behaviors `some` and `none` are logical negations of each other. Consequently, the two behaviors are *disjoint* and *complete*: for every input satisfying the preconditions, exactly one of the two behaviors applies.

4.1.2. Implementation of `find`

The implementation of `find` [4.3] follows a simple linear search scheme.

```
size_type
find(const value_type* a, size_type n, value_type v)
{
  /*@
    loop invariant 0 <= i <= n;
    loop invariant \forall integer k; 0 <= k < i ==> a[k] != v;
    loop assigns i;
    loop variant n-i;
   */
  for (size_type i = 0u; i < n; i++) {
    if (a[i] == v) {
      return i;
    }
  }

  return n;
}
```

Listing 4.3: Implementation of `find`

The array `a` is traversed from left to right until either an occurrence of the sought-after value is found or the end of the array is reached. If the value is found, the function returns its index; otherwise it indicates failure in accordance with the contract of `find` [4.2]. The algorithm performs no modification of the input array and executes at most `n` iterations.

The noteworthy elements of the implementation are the *loop annotations*. The first loop *invariant* establishes that all accesses to `a` are performed with valid indices. The second loop *invariant* supports the proof of the postconditions of the behavior `some` in the contract of `find` [4.2] by expressing that, at each iteration step, the sought-after value has not been encountered in the already inspected prefix of the array. Finally, the loop *variant* `n-i` is required to generate the verification conditions that establish termination of the loop.

4.2. The `find2` algorithm—reuse of specification elements

In this section we specify `find` in a slightly different way. The approach is motivated by the observation that a considerable number of closely related ACSL formulas occur both in the contract of `find` [4.2] and in the implementation `find` [4.3]. Typical examples are the following formulas:

```
\exists integer i; 0 <= i < n        &&   a[i] == v;

\forall integer i; 0 <= i < \result  ==>  a[i] != v;

\forall integer i; 0 <= i < n        ==>  a[i] != v;

\forall integer k; 0 <= k < i        ==>  a[k] != v;
```

Observe that the first formula is the logical negation of the third one. More generally, these formulas express two complementary situations: either some element of the array equals the searched value, or no such element exists in a given range.

4.2.1. The predicates `SomeEqual` and `NoneEqual`

To make these common patterns explicit and reusable, we introduce in the following listing two predicates, `SomeEqual` [4.4] and `NoneEqual` [4.4].

```
/*@
  axiomatic SomeNone
  {
    predicate
    SomeEqual{A}(value_type* a, integer m, integer n, value_type v) =
      \exists integer i; m <= i < n && a[i] == v;

    predicate
    SomeEqual{A}(value_type* a, integer n, value_type v) =
      SomeEqual(a, 0, n, v);

    predicate
    NoneEqual(value_type* a, integer m, integer n, value_type v) =
      \forall integer i; m <= i < n  ==>  a[i] != v;

    predicate
    NoneEqual(value_type* a, integer n, value_type v) =
      NoneEqual(a, 0, n, v);

    lemma NotSomeEqual_NoneEqual:
      \forall value_type *a, v, integer m, n;
        !SomeEqual(a, m, n, v)  ==>  NoneEqual(a, m, n, v);

    lemma NoneEqual_NotSomeEqual:
      \forall value_type *a, v, integer m, n;
       NoneEqual(a, m, n, v)   ==>  !SomeEqual(a, m, n, v);
  }
*/
```

Listing 4.4: The logic definition(s) `SomeNone`

Intuitively, `SomeEqual` expresses that there exists a valid index `i` such that the array element `a[i]` equals the value `v`, whereas `NoneEqual` states that no such index exists. These predicates abstract recurring

existential and universal quantification into named logical concepts that can be used uniformly in both specifications and proofs.

The predicates `SomeEqual` and `NoneEqual`, as well as the lemmas `NotSomeEqual_NoneEqual` and `NoneEqual_NotSomeEqual`, are grouped into the axiomatic block `SomeNone` [4.4]. This provides a modest form of modularization for related predicates, logic functions, and lemmas. Note that both predicates are provided in overloaded variants. One variant applies to array sections, while the other applies to complete arrays. This avoids the need for separate predicate names while still covering both use cases.

Finally, note that the predicate `SomeEqual` is equipped with an explicit label, namely `A`. This label indicates that the evaluation of the predicate depends on a specific memory state, namely the contents of `a[0..n-1]` at label `A`. Formally, the defining formula corresponds to

```
\exists integer i; 0 <= i < n && \at(a[i],A) == v;
```

In accordance with ACSL conventions, the explicit use of `\at` can be omitted whenever the label `A` is the only label in scope. For this reason, the abbreviated form `a[i]` is used in the definition of `SomeEqual` and `NoneEqual`, and the label is omitted entirely in the overloaded versions for complete arrays.

4.2.2. Formal specification of `find2`

With the predicates `SomeEqual` [4.4] and `NoneEqual` [4.4] we encapsulate all occurrences of existential and universal quantification in both the specification and the implementation of `find2` [4.5].

```
/*@
  requires   valid:   \valid_read(a + (0..n-1));

  terminates          \true;
  exits               \false;
  assigns             \nothing;

  ensures    result:  0 <= \result <= n;

  behavior some:
    assumes           SomeEqual(a, n, v);
    assigns           \nothing;
    ensures  bound:   0 <= \result < n;
    ensures  result:  a[\result] == v;
    ensures  first:   NoneEqual(a, \result, v);

  behavior none:
    assumes           NoneEqual(a, n, v);
    assigns           \nothing;
    ensures  result:  \result == n;

  complete behaviors;
  disjoint behaviors;
*/
size_type
find2(const value_type* a, size_type n, value_type v);
```

Listing 4.5: Formal specification of `find2`

As a result, the revised contract is more concise and more readable than that of `find` [4.2]. In particular, the assumptions of the two behaviors `some` and `none` are manifestly mutually exclusive, since one is the literal negation of the other. This mutual exclusivity, which was previously implicit in the structure of quantified formulas, is now expressed explicitly through predicate usage.

Moreover, the requirement that `find` returns the smallest index at which the searched value occurs can be expressed naturally using the `NoneEqual` [4.4] predicate. This is reflected in the final postcondition of behavior `some`, which states that no occurrence of the value precedes the returned index.

Finally, the specification of `find2` makes systematic use of user-defined names for `requires` and `ensures` clauses (sometimes also called *labels*, the distinction from program-state identifiers being clear from the context). We strongly recommend this practice, especially for more complex annotations. Among other benefits, Frama-C can be instructed to verify only clauses with a given name, which significantly improves proof control and diagnosability.

4.2.3. Implementation of `find2`

The predicate `NoneEqual` is also used in the loop annotations of the implementation `find2` [4.6].

```
size_type
find2(const value_type* a, size_type n, value_type v)
{
  /*@
    loop invariant bound:     0 <= i <= n;
    loop invariant not_found: NoneEqual(a, i, v);
    loop assigns i;
    loop variant n-i;
   */
  for (size_type i = 0u; i < n; i++) {
    if (a[i] == v) {
      return i;
    }
  }

  return n;
}
```

Listing 4.6: Implementation of `find2`

As in the function contract, this allows us to avoid explicit universal quantification in the annotations and to express the relevant properties in a uniform and declarative way. In particular, `NoneEqual` is used to state that all elements examined so far differ from the searched value. This closely mirrors the informal loop invariant of the algorithm and aligns the structure of the implementation annotations with that of the formal specification.

As in the specification, we systematically use labels to name individual annotations. This improves readability and enables selective verification of specific invariants or assertions.

4.3. The `find3` algorithm—using a logic function

In this section we present a third specification of linear search. While this approach requires additional preparatory work, it results in a significantly more concise and transparent function contract.

4.3.1. The logic function `Find`

We begin with a *recursive* definition of the ACSL logic function `Find`. Due to the number of lemmas associated with `Find`, its definition is distributed over several listings. The function is provided in two *overloaded* variants: the first operates on *array sections*, whereas the second is intended for *complete arrays*.

The first group of lemmas captures elementary properties of `Find` that are directly related to extending the search range from `a[0..n-1]` to `a[0..n]`. The remaining lemmas are of a more structural nature and will later be used in the verification of `find3`. At the end of this section we will discuss in what sense the contracts of `find2` and `find3` are equivalent. For orientation, we briefly summarize the role of the most important lemmas associated with `Find`: Lemmas of the first group characterize how the value of `Find` changes when the search interval is extended by one element.

```
/*@
  axiomatic Find
  {
    logic integer
    Find(value_type* a, integer m, integer n, value_type v) =
      (n <= m) ?
       0 : ((0 <= Find(a, m, n-1, v) < n-m-1) ?
         Find(a, m, n-1, v) : ((a[n-1] == v) ? n-m-1 : n-m));

    logic integer
    Find(value_type* a, integer n, value_type v) = Find(a, 0, n, v);

    lemma Find_Empty:
      \forall value_type *a, v, integer m, n;
        n <= m  ==>  Find(a, m, n, v) == 0;

    lemma Find_Hit:
      \forall value_type *a, v, integer m, n;
        m <= n                  ==>
        Find(a, m, n, v) < n-m  ==>
        Find(a, m, n+1, v) == Find(a, m, n, v);

    lemma Find_MissHit:
      \forall value_type *a, v, integer m, n;
        m <= n                      ==>
        a[n] == v                   ==>
        Find(a, m, n, v)    == n-m  ==>
        Find(a, m, n+1, v) == n-m;

    lemma Find_MissMiss:
      \forall value_type *a, v, integer m, n;
        m <= n                      ==>
        a[n] != v                   ==>
        Find(a, m, n, v)    == n-m  ==>
        Find(a, m, n+1, v) == (n+1)-m;
```

Listing 4.7: The logic function `Find` (1)

Lemmas of the second group relate `Find` to the predicates `SomeEqual` [4.4] and `NoneEqual` [4.4], thereby connecting the functional view of search with its predicate-based formulation. Further lemmas establish equivalences between the two overloaded versions of `Find` and enable its use in function contracts.

```
    lemma Find_Lower:
      \forall value_type *a, v, integer m, n;
        0 <= Find(a, m, n, v);

    lemma Find_Upper:
      \forall value_type *a, v, integer m, n;
        m <= n  ==>  Find(a, m, n, v) <= n-m;

    lemma Find_WeaklyIncreasing:
      \forall value_type *a, v, integer m, n;
        m <= n  ==>  Find(a, m, n, v) <= Find(a, m, n+1, v);

    lemma Find_Increasing:
      \forall value_type *a, v, integer k, m, n;
        m <= k <= n  ==>
        Find(a, m, k, v) <= Find(a, m, n, v);

    lemma Find_Limit:
      \forall value_type *a, v, integer k, m, n;
        m <= k < n  ==>
        a[k] == v   ==>
        Find(a, m, n, v) <= k-m;

    lemma Find_NoneEqual:
      \forall value_type *a, v, integer m, n;
        m <= n                 ==>
        NoneEqual(a, m, n, v)  ==>
        Find(a, m, n, v) == n-m;

    lemma Find_SomeEqual:
      \forall value_type *a, v, integer k, m, n;
        m <= k < n             ==>
        a[k] == v              ==>
        NoneEqual(a, m, k, v)  ==>
        Find(a, m, n, v) == k-m;

    lemma Find_ResultNoneEqual:
      \forall value_type *a, v, integer m, n;
        m <= n  ==>  NoneEqual(a, m, m + Find(a, m, n, v), v);

    lemma Find_ResultEqual:
      \forall value_type *a, v, integer m, n;
        0 <= Find(a, m, n, v) < n-m  ==>
        a[m + Find(a, m, n, v)] == v;
  }
*/
```

Listing 4.8: The logic function `Find` (2)

4.3.2. Formal specification of `find3`

Using the logic function `Find` we can now give a third formal specification of linear search. In contrast to `find2` [4.5], all quantification over the input array is encapsulated in the logic function itself. As a result, the contract of `find3` [4.9] is significantly more compact and its logical structure is easier to recognize at a glance.

```
/*@
  requires   valid:   \valid_read(a + (0..n-1));

  terminates          \true;
  exits               \false;
  assigns             \nothing;

  ensures    result:  0 <= \result <= n;
  ensures    result:  \result == Find(a, n, v);
*/
size_type
find3(const value_type* a, size_type n, value_type v);
```

Listing 4.9: Formal specification of `find3`

This conciseness is achieved at the cost of a more elaborate logical infrastructure: the burden of reasoning about existence, minimality, and absence of matching elements is shifted from the function contract to the definition of the ACSL logic function `Find` [4.7] and its associated lemmas.

4.3.3. Implementation of `find3`

The following listing shows the implementation of `find3` [4.10]. Operationally, the code is similar to the previous implementations of linear search. However, the verification now relies on relating the operational behavior of the loop to the abstract specification given by `Find` [4.7].

```
size_type
find3(const value_type* a, size_type n, value_type v)
{
  /*@
    loop invariant bound:     0 <= i <= n;
    loop invariant not_found: Find(a, i, v) == i;
    loop assigns i;
    loop variant n-i;
   */
  for (size_type i = 0u; i < n; i++) {
    if (a[i] == v) {
      //@ assert found: Find(a, n, v) == i;
      return i;
    }
  }

  return n;
}
```

Listing 4.10: Implementation of `find3`

To this end, we introduce the assertion `found`, which makes explicit the connection between the current loop index and the value returned by `Find`. This assertion serves as a bridge between the concrete control

flow of the implementation and the functional characterization used in the contract.

4.3.4. The equivalence of `find2` and `find3`

A natural question at this point is in what precise sense the contracts of `find2` [4.5] and `find3` [4.9] can be regarded as equivalent. In this section we make this notion of equivalence explicit and show how it can be established by verification.

We call two function contracts *equivalent* if each of them is sufficiently strong to verify an implementation that is specified using the other. In other words, either contract can be used as a drop-in replacement for the other in a verified context.

To demonstrate this mutual sufficiency, we introduce two auxiliary examples, `find4` and `find5`. Both functions are trivial wrappers whose implementations consist of a single call to an already verified linear search routine, while their contracts are taken from the alternative specification.

The implementation of `find4` [4.11] consists solely of a call to `find3`:

```
size_type
find4(const value_type* a, size_type n, value_type v)
{
  return find3(a, n, v);
}
```

Listing 4.11: Implementation of `find4`

The contract of `find4` [4.12], however, is identical to that of `find2` [4.5]:

```
/*@
  requires   valid:   \valid_read(a + (0..n-1));

  terminates          \true;
  exits               \false;
  assigns             \nothing;

  ensures    result:  0 <= \result <= n;

  behavior some:
    assumes           SomeEqual(a, n, v);
    assigns           \nothing;
    ensures  bound:   0 <= \result < n;
    ensures  result:  a[\result] == v;
    ensures  first:   NoneEqual(a, \result, v);

  behavior none:
    assumes           NoneEqual(a, n, v);
    assigns           \nothing;
    ensures  result:  \result == n;

  complete behaviors;
  disjoint behaviors;
*/
size_type
find4(const value_type* a, size_type n, value_type v);
```

Listing 4.12: Formal specification of `find4`

Analogously, the implementation of `find5` [4.13] is a direct call to `find2`:

```
size_type
find5(const value_type* a, size_type n, value_type v)
{
  return find2(a, n, v);
}
```

Listing 4.13: Implementation of `find5`

The contract of `find5` [4.14], on the other hand, coincides with that of `find3` [4.9]. Thus, `find5` reverses the roles of implementation and specification compared to `find4`.

```
/*@
  requires   valid:   \valid_read(a + (0..n-1));

  terminates          \true;
  exits               \false;
  assigns             \nothing;

  ensures    result:  0 <= \result <= n;
  ensures    result:  \result == Find(a, n, v);
*/
size_type
find5(const value_type* a, size_type n, value_type v);
```

Listing 4.14: Formal specification of `find5`

The successful verification of both wrapper functions (cf. Table A.2) shows that each contract is sufficient to justify an implementation written against the other. This establishes the equivalence of the contracts of `find2` [4.5] and `find3` [4.9].

The notion of equivalence used here is inherently *contextual*. It is defined relative to the underlying specification language, the available logical infrastructure, and the verification framework. Two contracts are considered equivalent not in an abstract semantic sense, but insofar as each can be used to verify implementations specified with the other within the same formal setting.

4.4. The `find_if_not` algorithm

Many algorithms in the C++ standard library are parameterized not only by the type of the input sequence but also by so-called *function objects*. A representative example is the `find_if_not` algorithm, which takes a *predicate function object* P and returns the first position i in the input sequence for which $P(i)$ does *not* hold.

Although function objects could in principle be emulated in C by means of *function pointers*, we deliberately do not pursue this approach here. At present, function pointers are only partially supported by Frama-C, and there is currently no support for parameterized ACSL predicates. For these reasons, our implementation of `find_if_not` is restricted to the special case of locating the first position in an array at which a given value does *not* occur. The resulting function signature is therefore

```
size_type find_if_not(const value_type* a,
                      size_type n,
                      value_type v);
```

From an algorithmic point of view, this is not a particularly exciting addition to our collection of verified algorithms. Its value lies instead in providing a clean setting for introducing the predicates `AllEqual` [4.15] and `SomeNotEqual` [4.15], and, more importantly, the logic function `FindNotEqual` [4.16]. These definitions will later play a central role in the specification of the algorithm `remove_copy`, and in particular of its variant `remove_copy3` [7.48].

```
/*@
  axiomatic AllSomeNot
  {
    predicate
    AllEqual(value_type* a, integer m, integer n, value_type v) =
      \forall integer i; m <= i < n  ==>  a[i] == v;

    predicate
    AllEqual(value_type* a, integer m, integer n) =
      AllEqual(a, m, n, a[m]);

    predicate
    AllEqual(value_type* a, integer n, value_type v) =
      AllEqual(a, 0, n, v);

    predicate
    SomeNotEqual{A}(value_type* a, integer m, integer n, value_type v) =
      \exists integer i; m <= i < n && a[i] != v;

    predicate
    SomeNotEqual{A}(value_type* a, integer n, value_type v) =
      SomeNotEqual(a, 0, n, v);

    lemma NotAllEqual_SomeNotEqual:
      \forall value_type *a, v, integer m, n;
        !AllEqual(a, m, n, v)  ==>  SomeNotEqual(a, m, n, v);

    lemma SomeNotEqual_NotAllEqual:
      \forall value_type *a, v, integer m, n;
       SomeNotEqual(a, m, n, v)   ==>  !AllEqual(a, m, n, v);
  }
*/
```

Listing 4.15: The logic definition(s) `AllSomeNot`

The predicate `AllEqual` expresses that every element of the array segment `a[m..n-1]` is equal to the value `v`. The predicate `SomeNotEqual` is defined as its logical negation. Together, these predicates naturally complement the previously introduced predicates `SomeEqual` [4.4] and `NoneEqual` [4.4].

In addition, two overloaded variants of `AllEqual` are provided. The first variant uses the value `a[m]` itself as the reference value `v`. The second variant is a convenient abbreviation for the common case where the lower bound `m` is equal to 0.

4.4.1. The logic function `FindNotEqual`

The definition of the overloaded logic function `FindNotEqual` is given in Listings 4.16. This function is closely related to the logic function `Find` [4.7]. While `Find` [4.7] returns the first position in a sequence at which a given value occurs, `FindNotEqual` returns the first position at which the array element *differs* from a given reference value.

```
/*@
  axiomatic FindNotEqual
  {
    logic integer
    FindNotEqual(value_type* a, integer m, integer n, value_type v) =
      (n <= m) ?
       0 : ((0 <= FindNotEqual(a, m, n-1, v) < n-m-1) ?
         FindNotEqual(a, m, n-1, v) : ((a[n-1] != v) ? n-m-1 : n-m));

    logic integer
    FindNotEqual(value_type* a, integer n, value_type v) =
      FindNotEqual(a, 0, n, v);

    lemma FindNotEqual_Empty:
      \forall value_type *a, v, integer m, n;
        n <= m  ==>  FindNotEqual(a, m, n, v) == 0;

    lemma FindNotEqual_Hit:
      \forall value_type *a, v, integer m, n;
        m <= n                          ==>
        FindNotEqual(a, m, n, v) < n-m  ==>
        FindNotEqual(a, m, n+1, v) == FindNotEqual(a, m, n, v);

    lemma FindNotEqual_MissHit:
      \forall value_type *a, v, integer m, n;
        m <= n                              ==>
        a[n] != v                           ==>
        FindNotEqual(a, m, n, v)    == n-m  ==>
        FindNotEqual(a, m, n+1, v) == n-m;

    lemma FindNotEqual_MissMiss:
      \forall value_type *a, v, integer m, n;
        m <= n                              ==>
        a[n] == v                           ==>
        FindNotEqual(a, m, n, v)    == n-m  ==>
        FindNotEqual(a, m, n+1, v) == (n+1)-m;
```

Listing 4.16: The logic function `FindNotEqual` (1)

More precisely, `FindNotEqual` characterizes the smallest index i in the range under consideration such that `a[i] != v` holds. If no such position exists, the function yields the upper bound of the considered range.

In addition to the core definition, several lemmas establish basic properties of `FindNotEqual`. Among them is the lemma `FindNotEqual_Unchanged`, which expresses the stability of the function under modifications of array segments that are known to be unchanged. This lemma relies on the predicate `Unchanged` [7.1], which will be introduced formally in a later chapter. The forward reference is intentional, as the lemma already reflects a recurring verification pattern that will be used extensively in subsequent algorithm specifications.

```
    lemma FindNotEqual_Lower:
      \forall value_type *a, v, integer m, n;
        0 <= FindNotEqual(a, m, n, v);

    lemma FindNotEqual_Upper:
      \forall value_type *a, v, integer m, n;
        m <= n  ==>  FindNotEqual(a, m, n, v) <= n-m;

    lemma FindNotEqual_Unchanged{K,L}:
      \forall value_type *a, v, integer m, n;
        Unchanged{K,L}(a, m, n)  ==>
        FindNotEqual{K}(a, m, n, v) == FindNotEqual{L}(a, m, n, v);

    lemma FindNotEqual_WeaklyIncreasing:
      \forall value_type *a, v, integer m, n;
        m <= n  ==>  FindNotEqual(a, m, n, v) <= FindNotEqual(a, m, n+1, v);

    lemma FindNotEqual_Increasing:
      \forall value_type *a, v, integer k, m, n;
        m <= k <= n  ==>  FindNotEqual(a, m, k, v) <= FindNotEqual(a, m, n, v);

    lemma FindNotEqual_Limit:
      \forall value_type *a, v, integer k, m, n;
        m <= k < n  ==>
        a[k] != v   ==>
        FindNotEqual(a, m, n, v) <= k-m;

    lemma FindNotEqual_AllEqual:
      \forall value_type *a, v, integer m, n;
        m <= n                ==>
        AllEqual(a, m, n, v)  ==>
        FindNotEqual(a, m, n, v) == n-m;

    lemma FindNotEqual_SomeNotEqual:
      \forall value_type *a, v, integer k, m, n;
        m <= k < n            ==>
        a[k] != v             ==>
        AllEqual(a, m, k, v)  ==>
        FindNotEqual(a, m, n, v) == k-m;

    lemma FindNotEqual_ResultAllEqual:
      \forall value_type *a, v, integer m, n;
        m <= n  ==>  AllEqual(a, m, m + FindNotEqual(a, m, n, v), v);

    lemma FindNotEqual_ResultNotEqual:
      \forall value_type *a, v, integer m, n;
        0 <= FindNotEqual(a, m, n, v) < n-m  ==>
        a[m + FindNotEqual(a, m, n, v)] != v;
  }
*/
```

Listing 4.17: The logic function `FindNotEqual` (2)

4.4.2. Formal specification of `find_if_not`

The contract of `find_if_not` [4.18] is, unsurprisingly, very similar to that of `find3` [4.9]. Both specifications follow the same overall structure and rely on a logic function that characterizes the first position in the array at which a distinguishing property holds.

```
/*@
  requires   valid:   \valid_read(a + (0..n-1));

  terminates          \true;
  exits               \false;
  assigns             \nothing;

  ensures    result:  0 <= \result <= n;
  ensures    result:  \result == FindNotEqual(a, n, v);
*/
size_type
find_if_not(const value_type* a, size_type n, value_type v);
```

Listing 4.18: Formal specification of `find_if_not`

The only substantive difference is that the logic function `Find` [4.7] is replaced by `FindNotEqual` [4.16]. As a consequence, the postcondition no longer characterizes the first occurrence of a given value, but instead the first position at which the array element differs from the reference value. All other aspects of the contract — in particular the treatment of the case where no such position exists — remain unchanged.

4.4.3. Implementation of `find_if_not`

The implementation of `find_if_not` [4.19] closely mirrors that of `find3` [4.10]. The control structure, loop invariants, and termination argument are identical. As in the specification, the only essential change is the systematic replacement of `Find` by `FindNotEqual`.

```
size_type
find_if_not(const value_type* a, size_type n, value_type v)
{
  /*@
    loop invariant bound:     0 <= i <= n;
    loop invariant not_found: FindNotEqual(a, i, v) == i;
    loop assigns i;
    loop variant n-i;
   */
  for (size_type i = 0u; i < n; i++) {
    if (a[i] != v) {
      //@ assert found: FindNotEqual(a, n, v) == i;
      return i;
    }
  }

  return n;
}
```

Listing 4.19: Implementation of `find_if_not`

In the loop body, this change manifests itself in the condition that is checked at each iteration: instead of testing whether `a[i]` equals the given value, we now test that it *differs* from `v`. The proof obligations gen-

erated by this implementation are discharged in the same way as for `find3` [4.10], using the corresponding lemmas for `FindNotEqual`.

4.5. The `find_first_of` algorithm

The algorithm `find_first_of` [19, §28.5.7] can be viewed as a natural generalization of `find` (see §4.1 and §4.2). Whereas `find` performs a sequential search for a single, fixed value, `find_first_of` searches for the first occurrence of *any* value drawn from a given set.

```
size_type
find_first_of(const value_type* a, size_type m,
              const value_type* b, size_type n);
```

Operationally, `find_first_of` performs a linear scan of the array `a[0..m-1]` and returns the least index `i` such that the element `a[i]` is equal to at least one element of the array `b[0..n-1]`. If no such index exists, the function indicates failure in the same way as `find`. The algorithm thus combines an outer sequential search over `a` with an implicit membership test against the array `b`.

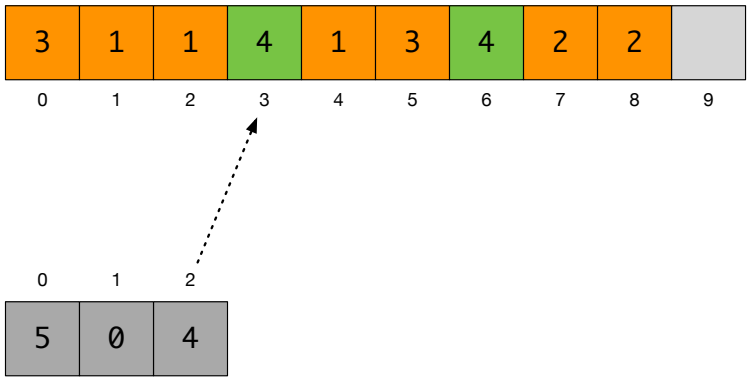

Figure 4.20.: A simple example of `find_first_of`

Figure 4.20 illustrates this behavior. Among the elements of the left-hand array, the arrow marks the smallest index at which a value occurs that is also present in the three-element array on the right.

4.5.1. The predicate `HasValueOf`

In order to express this membership test at the level of specifications, we introduce the predicate `HasValueOf` [4.21]. Intuitively, `HasValueOf` formalizes the fact that the two arrays `a` and `b` share at least one common value at valid indices. More precisely, it states the existence of indices `i` and `j` within bounds such that `a[i] == b[j]` holds.

```
/*@
  axiomatic HasValueOf
  {
    predicate
    HasValueOf{A}(value_type* a, integer m, value_type* b, integer n) =
      \exists integer i; 0 <= i < m && SomeEqual{A}(b, n, a[i]);
  }
*/
```

Listing 4.21: The logic definition(s) `HasValueOf`

As in our treatment of `find2` (§4.2), we deliberately encapsulate the underlying existential quantification in a named predicate. This serves two purposes. First, it keeps the function contract of `find_first_of` free of explicit quantifiers, thereby improving readability. Second, it makes the logical structure of the specification explicit: the algorithm succeeds precisely when `HasValueOf` holds, and fails otherwise.

Rather than introducing a new primitive notion of equality, `HasValueOf` is defined in terms of the already established predicate `SomeEqual` [4.4]. This reuse emphasizes the close conceptual relationship between `find_first_of` and the earlier variants of `find`, while avoiding unnecessary duplication at the specification level.

4.5.2. Formal specification of `find_first_of`

The following listing presents the formal specification of `find_first_of`. As in the specification of `find2` [4.5], the function contract is formulated in terms of named predicates rather than explicit quantification. In particular, it relies on the predicates `HasValueOf` [4.21] and `SomeEqual` [4.4] to characterize the existence of matching elements between the two input arrays.

```
/*@
  requires   valid:  \valid_read(a + (0..m-1));
  requires   valid:  \valid_read(b + (0..n-1));

  terminates        \true;
  exits             \false;
  assigns           \nothing;

  ensures    result:   0 <= \result <= m;

  behavior found:
    assumes          HasValueOf(a, m, b, n);
    assigns          \nothing;
    ensures bound:   0 <= \result < m;
    ensures result:  SomeEqual(b, n, a[\result]);
    ensures first:   !HasValueOf(a, \result, b, n);

  behavior not_found:
    assumes          !HasValueOf(a, m, b, n);
    assigns          \nothing;
    ensures result:  \result == m;

  complete behaviors;
  disjoint behaviors;
*/
size_type
find_first_of(const value_type* a, size_type m,
              const value_type* b, size_type n);
```

Listing 4.22: Formal specification of `find_first_of`

This choice makes the logical structure of the contract immediately apparent. The successful behavior corresponds to the case where `HasValueOf` holds, that is, where at least one element of `a` occurs in `b`. Conversely, the failure behavior captures the absence of any such common value. By reusing `SomeEqual` [4.4], the specification emphasizes the close relationship between `find_first_of` and the earlier variants of `find`, while avoiding a proliferation of ad hoc predicates.

4.5.3. Implementation of `find_first_of`

The implementation of `find_first_of` [4.23] is deliberately structured as a thin wrapper around the previously verified function `find2` [4.5]. Rather than performing a nested search explicitly, the algorithm delegates the inner membership test to `find2`, thereby making reuse an explicit design choice.

```
size_type
find_first_of (const value_type* a, size_type m,
               const value_type* b, size_type n)
{
  /*@
    loop invariant bound:      0 <= i <= m;
    loop invariant not_found: !HasValueOf(a, i, b, n);
    loop assigns i;
    loop variant m-i;
  */
  for (size_type i = 0u; i < m; i++) {
    if (find2(b, n, a[i]) < n) {
      return i;
    }
  }

  return m;
}
```

Listing 4.23: Implementation of `find_first_of`

This approach has two immediate benefits. First, it leads to a concise and readable implementation whose control flow closely mirrors the informal description of the algorithm. Second, and more importantly from a verification perspective, it significantly reduces the annotation burden: no additional loop invariants are required beyond those already established for `find2`. As a result, the correctness of `find_first_of` follows largely from the existing specification and proof of `find2`, illustrating how verified components can be composed to obtain new, verified functionality with minimal additional effort.

From an algorithmic point of view, this implementation realizes `find_first_of` as a linear scan over the array `a`. At each position, the current element `a[i]` is tested for membership in the array `b` by invoking `find2`. The function thus combines an outer sequential traversal with a logically specified membership test, closely reflecting the structure captured by the predicate `HasValueOf` [4.21] at the specification level.

4.6. The `adjacent_find` algorithm

The algorithm `adjacent_find`, as specified in the C++ Standard Library [19, §28.5.8], searches an array for the first occurrence of two equal adjacent elements.

```
size_type
adjacent_find(const value_type* a, size_type n);
```

More precisely, `adjacent_find` returns the smallest index `i` such that both `i` and `i+1` are valid indices of the array and

```
a[i] == a[i+1]
```

holds. If no such index exists, the algorithm returns `n`.

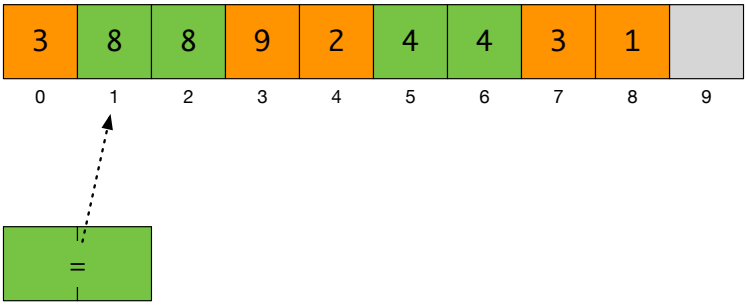

Figure 4.24.: A simple example of `adjacent_find`

Figure 4.24 illustrates this behavior. The arrow marks the smallest index at which two adjacent elements of the array are equal.

4.6.1. The predicate `HasEqualNeighbors`

As in the case of the other search algorithms considered so far, we first introduce a predicate that captures the essential logical property underlying the algorithm. The predicate `HasEqualNeighbors` [4.25] formalizes the existence of two adjacent valid indices at which the array holds equal values.

```
/*@
  axiomatic HasEqualNeighbors
  {
    predicate
    HasEqualNeighbors{L}(value_type* a, integer n) =
      \exists integer i; 0 <= i < n-1 && a[i] == a[i+1];
  }
*/
```

Listing 4.25: The predicate `HasEqualNeighbors`

By encapsulating this property in a named predicate, we avoid explicit quantification in the function contract and make the specification more readable and structured.

4.6.2. Formal specification of `adjacent_find`

Our specification of `adjacent_find` [4.26] relies on the predicate `HasEqualNeighbors` [4.25]. This predicate captures the essential property underlying the algorithm, namely the existence of two adjacent valid indices at which the array contains equal values.

```
/*@
  requires   valid:     \valid_read(a + (0..n-1));

  terminates            \true;
  exits                 \false;
  assigns               \nothing;

  ensures   result:     0 <= \result <= n;

  behavior some:
    assumes             HasEqualNeighbors(a, n);
    assigns             \nothing;
    ensures  result:    0 <= \result < n-1;
    ensures  adjacent:  a[\result] == a[\result+1];
    ensures  first:     !HasEqualNeighbors(a, \result);

  behavior none:
    assumes             !HasEqualNeighbors(a, n);
    assigns             \nothing;
    ensures  result:    \result == n;

  complete behaviors;
  disjoint behaviors;
*/
size_type
adjacent_find(const value_type* a, size_type n);
```

Listing 4.26: Formal specification of `adjacent_find`

As in the specifications of the other search algorithms, the function contract is structured into distinct behaviors. The successful behavior corresponds exactly to the case where `HasEqualNeighbors` holds, while the failure behavior captures its negation. By formulating the contract in terms of this predicate, the specification avoids explicit quantification and makes the intended meaning of the postcondition immediately apparent.

4.6.3. Implementation of `adjacent_find`

The implementation of `adjacent_find` [4.27] follows closely the informal description of the algorithm. As a preliminary step, it checks whether the array contains at least two elements. If this is not the case, no pair of adjacent elements can exist, and the function can immediately return failure.

```
size_type
adjacent_find(const value_type* a, size_type n)
{
  if (1u < n) {
    /*@
      loop invariant bound:  0 <= i < n;
      loop invariant none:   !HasEqualNeighbors(a, i+1);
      loop assigns i;
      loop variant n-i;
    */
    for (size_type i = 0u; i + 1u < n; ++i) {
      if (a[i] == a[i + 1u]) {
        return  i;
      }
    }
  }

  return n;
}
```

Listing 4.27: Implementation of `adjacent_find`

For the general case, the algorithm performs a linear scan of the array, comparing each element with its immediate successor. The loop invariant makes explicit use of the predicate `HasEqualNeighbors` [4.25] to relate the progress of the scan to the postcondition of the successful behavior. In particular, it ensures that, at each iteration, either a matching pair has already been encountered or no such pair exists in the portion of the array examined so far.

This close correspondence between the loop invariant and the function postcondition allows the correctness of the implementation to be established in a direct and transparent manner.

4.7. The `equal` and `mismatch` algorithms

The algorithms `equal` [19, §28.5.11] and `mismatch` [19, §28.5.10] of the C++ Standard Library compare two sequences element by element. For our purposes, we restrict attention to arrays of a fixed value type `value_type` and adapt the generic interfaces accordingly. The resulting function signatures are

```
bool       equal(const value_type* a, size_type n, const value_type* b);

size_type  mismatch(const value_type* a, size_type n, const value_type* b);
```

The function `equal` returns **`true`** if and only if `a[i] == b[i]` holds for every index `0 <= i < n`. If this condition fails for at least one index, the function returns **`false`**.

The algorithm `mismatch` is slightly more informative. Rather than merely reporting whether the two ranges are equal, it returns the smallest index at which the arrays `a` and `b` differ. If no such index exists—that is, if the arrays agree at all positions `0 <= i < n`—then `mismatch` returns the common length `n` of the two ranges.

4.7.1. The `Equal` predicate

The fact that two arrays `a[0]..a[n-1]` and `b[0]..b[n-1]` coincide element by element is a basic property that arises naturally in many specifications. Rather than restating this condition repeatedly, it is preferable to capture it once in a dedicated predicate.

To this end, we define several *overloaded* versions of the predicate `Equal` [4.28], shown in the following listing.

```
/*@
  axiomatic Equal
  {
    predicate
    Equal{K,L}(value_type* a, integer m, integer n, value_type* b) =
      \forall integer i; m <= i < n  ==>  \at(a[i],K) == \at(b[i],L);

    predicate
    Equal{K,L}(value_type* a, integer n, value_type* b) =
      Equal{K,L}(a, 0, n, b);

    predicate
    Equal{K,L}(value_type* a, integer m, integer n,
               value_type* b, integer p) = Equal{K,L}(a+m, n-m, b+p);

    predicate
    Equal{K,L}(value_type* a, integer m, integer n, integer p) =
      Equal{K,L}(a, m, n, a, p);
  }
*/
```

Listing 4.28: The logic definition(s) `Equal`

The parameters `K` and `L` in the definition of `Equal` are so-called *labels*. They denote the program states in which the arrays `a[..]` and `b[..]` are to be evaluated. The Frama-C tool chain provides several standard labels, such as `Old` and `Post`, which refer to the pre-state and post-state of a function, respectively. User-defined labels may be introduced to name intermediate program points.

For a detailed discussion of labels and their semantics, we refer to the ACSL specification [14, §2.6.9].

4.7.2. Formal specification of `equal` and `mismatch`

Using the predicate `Equal` [4.28], we can give a concise and precise specification of `equal` [4.29]. In the `ensures` clause, `Equal` is instantiated with the predefined label `Here`, which refers to the post-state of the function. This states that, upon termination, the two input ranges are equal element by element. Note that the postcondition is formulated as a logical equivalence. An equality would not be well-formed in ACSL, since `Equal` is a predicate, not a logic function, and therefore cannot appear as the argument of the equality operator.

```
/*@
  requires   valid:   \valid_read(a + (0..n-1));
  requires   valid:   \valid_read(b + (0..n-1));

  terminates          \true;
  exits               \false;
  assigns             \nothing;

  ensures    result:  \result <==> Equal{Here,Here}(a, n, b);
*/
bool
equal(const value_type* a, size_type n, const value_type* b);
```

Listing 4.29: Formal specification of `equal`

The formal specification of `mismatch` [4.30] is necessarily more elaborate. Unlike `equal`, which merely reports whether the two arrays coincide, `mismatch` returns additional information in the form of an index.

```
/*@
  requires   valid:   \valid_read(a + (0..n-1));
  requires   valid:   \valid_read(b + (0..n-1));

  terminates          \true;
  exits               \false;
  assigns             \nothing;

  ensures    result:  0 <= \result <= n;

  behavior all_equal:
    assumes           Equal{Here,Here}(a, n, b);
    assigns           \nothing;
    ensures result:   \result == n;

  behavior some_not_equal:
    assumes           !Equal{Here,Here}(a, n, b);
    assigns           \nothing;
    ensures bound:    0 <= \result < n;
    ensures result:   a[\result] != b[\result];
    ensures first:    Equal{Here,Here}(a, \result, b);

  complete behaviors;
  disjoint behaviors;
*/
size_type
mismatch(const value_type* a, size_type n, const value_type* b);
```

Listing 4.30: Formal specification of `mismatch`

Conceptually, the specification of `mismatch` closely parallels that of `find2` [4.5]. While `find2` returns

the smallest index `i` such that `a[i] == v` holds, `mismatch` returns the smallest index `i` such that `a[i] != b[i]`. In both cases, the returned index identifies the first position at which a distinguishing condition becomes true.

As in the specification of `find2`, the behaviors of `mismatch` are both complete and mutually exclusive. This is apparent from the `assumes` clauses of the behaviors `all_equal` and `some_not_equal`, which are exact negations of one another. The predicate `Equal` [4.28] plays a central role here, both in expressing the case where no mismatch exists and in delimiting the prefix of equal elements in the case where a mismatch is found.

4.7.3. Implementation of `equal` and `mismatch`

The implementation of `equal` [4.31] is deliberately minimal: it consists solely of a call to `mismatch` and a comparison of the returned index with the array length `n`. This directly reflects the relationship between the two algorithms established at the level of their specifications.

```
bool
equal(const value_type* a, size_type n, const value_type* b)
{
  return mismatch(a, n, b) == n;
}
```

Listing 4.31: Implementation of `equal`

The implementation of `mismatch` [4.32] follows the expected linear scan of the two arrays, but is enriched with loop annotations to support deductive verification.

```
size_type
mismatch(const value_type* a, size_type n, const value_type* b)
{
  /*@
    loop invariant bound:  0 <= i <= n;
    loop invariant equal:  Equal{Here,Here}(a, i, b);
    loop assigns i;
    loop variant n-i;
  */
  for (size_type i = 0u; i < n; i++) {
    if (a[i] != b[i]) {
      return i;
    }
  }

  return n;
}
```

Listing 4.32: Implementation of `mismatch`

As in the specification, we employ the predicate `Equal` [4.28] in the loop invariant to express that all indices `k` strictly smaller than the current index `i` satisfy `a[k] == b[k]`. This invariant captures the essential progress property of the algorithm: at each iteration, the examined prefix of the arrays is known to be equal.

The presence of this invariant is crucial for proving that the value returned by `mismatch` is indeed the smallest index at which the two ranges differ, or `n` if no such index exists.

4.8. The `search` algorithm

The `search` algorithm of the C++ Standard Library [19, §28.5.13] searches for the first occurrence of a sequence as a contiguous subsequence of another sequence, using element-by-element comparison. For our purposes, we adapt the generic formulation to arrays of type `value_type`. The resulting function signature is as follows:

```
size_type search(const value_type* a, size_type n,
                 const value_type* b, size_type p);
```

The function `search` returns the smallest index `s` such that the subrange `a[s..s+p-1]` is element-wise equal to the range `b[0..p-1]`, that is, `a[s+k] == b[k]` holds for every index `k` with `0 <= k < p` (see Figure 4.33). If no such index exists, the function returns the end of the searched range, represented here by the length `n` of the array `a`.

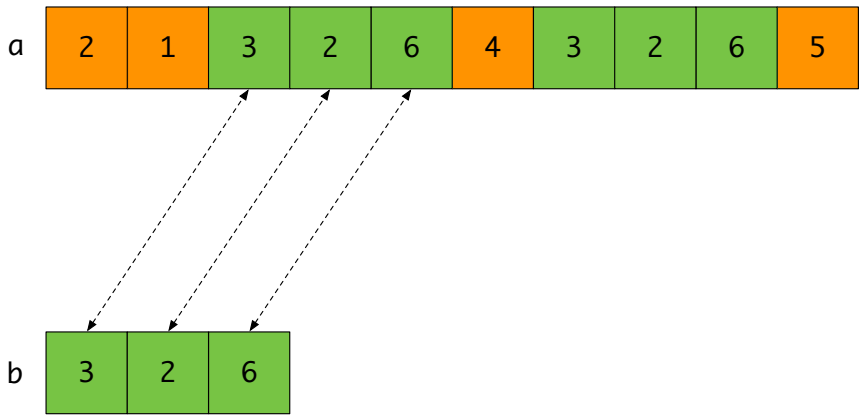

Figure 4.33.: A simple example of `search`

4.8.1. The predicate `HasSubRange`

The formal specification of `search` begins with the introduction of the predicate `HasSubRange` [4.34].

```
/*@
  axiomatic HasSubRange
  {
    predicate
    HasSubRange{L}(value_type* a, integer m, integer n, value_type* b, integer p) =
      \exists integer k; (m <= k <= n-p) && Equal{L,L}(a+k, p, b);

    predicate
    HasSubRange{L}(value_type* a, integer n, value_type* b, integer p) =
      HasSubRange{L}(a, 0, n, b, p);

    lemma HasSubRange_Sizes:
      \forall value_type *a, *b, integer m, n, p;
        HasSubRange(a, m, n, b, p)  ==>  p <= n-m;
  }
*/
```

Listing 4.34: The logic definition(s) `HasSubRange`

This predicate expresses that the array `a` contains `b` as a *contiguous* subrange. Using the predicate `Equal` [4.28], it formalizes the existence of an index `s` such that the ranges `a[s..s+p-1]` and `b[0..p-1]` are equal element by element.

For such a subrange of length `p` to exist, the array `a` must be long enough to accommodate it. This necessary size constraint is captured by the lemma `HasSubRange_Sizes`.

4.8.2. Formal specification of `search`

The following listing shows the formal specification of `search` [4.35].

```
/*@
  requires   valid:   \valid_read(a + (0..n-1));
  requires   valid:   \valid_read(b + (0..p-1));

  terminates          \true;
  exits               \false;
  assigns             \nothing;

  ensures    result:  0 <= \result <= n;

  behavior has_match:
    assumes          HasSubRange(a, n, b, p);
    assigns          \nothing;
    ensures bound:   0 <= \result <= n-p;
    ensures result:  Equal{Here,Here}(a+\result, p, b);
    ensures first:   !HasSubRange(a, \result+p-1, b, p);

  behavior no_match:
    assumes          !HasSubRange(a, n, b, p);
    assigns          \nothing;
    ensures result:  \result == n;

  complete behaviors;
  disjoint behaviors;
*/
size_type
search(const value_type* a, size_type n,
       const value_type* b, size_type p);
```

Listing 4.35: Formal specification of `search`

At a conceptual level, the specification of `search` closely resembles that of `find` [4.2]. As before, we distinguish two behaviors in order to capture the essential cases.

- The behavior `has_match` applies when the array `a` contains the sequence `b` as a contiguous subrange. This situation is characterized in the `assumes` clause by the predicate `HasSubRange` [4.34].

  The ensures clause `bound` of behavior `has_match` states that the returned index lies in the range `[0..n-p]`. Clause `result` expresses that `search` returns an index at which a copy of `b` occurs in `a`. Finally, clause `first` enforces minimality: the returned index is the least one with this property, i.e. the sequence `b` does not occur in the prefix `a[0..\result+p-2]`.

- The behavior `no_match` covers the complementary case in which the array `a` does not contain `b` as a subrange. In this situation, `search` is required to return the length `n` of the array `a`. If either of the ranges `a` or `b` is empty, the return value is `0`, in accordance with the definition of the algorithm.

The `assumes` clause of behavior `has_match` is the logical negation of the `assumes` clause of behavior `no_match`. Consequently, the two behaviors are explicitly specified as *complete* and *disjoint*.

4.8.3. Implementation of `search`

The implementation of `search` [4.36] is simple and easy to follow: it checks every possible starting position in `a` until it finds a match for `b` or reaches the end of `a`. Because it may compare up to `p` elements at each of the `n-p+1` starting positions, its worst-case number of comparisons grows proportionally to $n \cdot p$.

More advanced string search algorithms, such as the classic method of Knuth, Morris, and Pratt [20], avoid this quadratic behavior and guarantee at most $n + p$ comparisons. [8]

```
size_type
search(const value_type* a, size_type n,
       const value_type* b, size_type p)
{
  if (p <= n) {
    /*@
      loop invariant bound:     i <= n-p+1;
      loop invariant not_found: !HasSubRange(a, p+i-1, b, p);
      loop assigns i;
      loop variant n-i;
    */
    for (size_type i = 0u; i <= n - p; ++i) {
      if (equal(a + i, p, b)) {
        //@ assert has_match: HasSubRange(a, n, b, p);
        return i;
      }
    }
  }

  //@ assert no_match: !HasSubRange(a, n, b, p);
  return n;
}
```

Listing 4.36: Implementation of `search`

Depending on the version of Frama-C/WP, the loop invariant `not_found` may be essential for establishing the postconditions of the behavior `has_match`. It expresses that no occurrence of the sequence `b` has been found in `a` up to the current iteration.

Neither the case `p == 0` nor the case `n == 0` requires special treatment, not even for efficiency reasons. In the former case, the call `equal(a+i, p, b)` succeeds immediately, while in the latter case the condition `p > n` applies.

[8] The efficiency of `search` has also been discussed by the C++ standardization committee; see `http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n3905.html`.

4.9. The `search_n` algorithm

The `search_n` algorithm of the C++ Standard Library [19, §28.5.13] locates the first position at which a given value occurs a prescribed number of consecutive times in a sequence. For our purposes, we adapt the generic interface to operate on an array of elements of type `value_type`. The resulting function signature is

```
size_type
search_n(const value_type* a, size_type n,
         size_type p, value_type v);
```

This interface closely resembles that of `search` (Section 4.8). The only difference is that `search_n` takes a single value `v` instead of a pattern array.

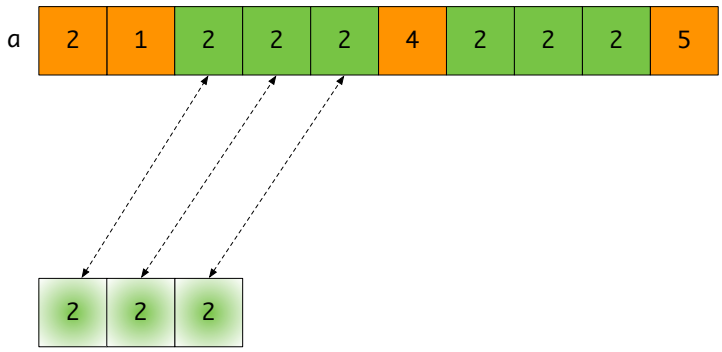

Figure 4.37.: A simple example of `search_n`

The function `search_n` returns the smallest index `s` such that `a[s+k] == v` holds for all indices `k` with `0 <= k < p` (see Figure 4.37). If no such index exists, the function returns the array length `n`.

4.9.1. The predicate `HasConstantSubRange`

The specification of `search_n` is based on the predicate `HasConstantSubRange` [4.38].

```
/*@
  axiomatic HasConstantSubRange
  {
    predicate
    HasConstantSubRange{L}(value_type* a, integer m, integer n, value_type v,
        integer p) =
      \exists integer k; m <= k <= n-p && AllEqual(a, k, k+p, v);

    predicate
    HasConstantSubRange{L}(value_type* a, integer n, value_type v, integer p) =
      HasConstantSubRange(a, 0, n, v, p);

    lemma HasConstantSubRange_Sizes:
      \forall value_type *a, v, integer n, p;
        HasConstantSubRange(a, n, v, p)  ==>  p <= n;
  }
*/
```

Listing 4.38: The logic definition(s) `HasConstantSubRange`

This predicate expresses that the array a of length n contains a contiguous subsequence consisting of p occurrences of the value v. Its definition reuses the predicate AllEqual [4.15] to characterize constant segments. As with HasSubRange [4.34], the existence of such a subsequence requires that the array be sufficiently long. This size constraint is made explicit by the lemma HasConstantSubRange_Sizes [4.38].

4.9.2. Formal specification of search_n

As for search [4.35], the specification of search_n [4.39] closely resembles that of find2 [4.5].

```
/*@
  requires   valid:    \valid_read(a + (0..n-1));

  terminates           \true;
  exits                \false;
  assigns              \nothing;

  ensures    result:   0 <= \result <= n;

  behavior has_match:
    assumes            HasConstantSubRange(a, n, v, p);
    assigns            \nothing;
    ensures result:    0 <= \result <= n-p;
    ensures match:     AllEqual(a, \result, \result+p, v);
    ensures first:     !HasConstantSubRange(a, \result+p-1, v, p);

  behavior no_match:
    assumes            !HasConstantSubRange(a, n, v, p);
    assigns            \nothing;
    ensures result:    \result == n;

  complete behaviors;
  disjoint behaviors;
*/
size_type
search_n(const value_type* a, size_type n, value_type v, size_type p);
```

Listing 4.39: Formal specification of search_n

We again distinguish two behaviors capturing the essential cases.

- The behavior has_match applies if the array a contains a p-fold repetition of the value v. This condition is expressed in the assumes clause using the predicate HasConstantSubRange [4.38]. The ensures clauses state that the return value lies in [0..n-p], that p or more consecutive occurrences of v start at this position, and that this position is minimal.
- The behavior no_match covers the complementary case in which no such subsequence exists. Here, search_n returns the length n of the range a.

4.9.3. Implementation of search_n

Although the specification of search_n [4.39] closely mirrors that of search [4.35], their implementations differ substantially. While search [4.36] uses a simple but non-optimal $O(n \cdot p)$ algorithm, the implementation of search_n [4.40] runs in linear time $O(n)$.

```
size_type
search_n(const value_type* a, size_type n, value_type v, size_type p)
{
  if (0u < p) {
    if (p <= n) {
      size_type start = 0u;

      /*@
        loop invariant match:     AllEqual(a, start, i, v);
        loop invariant start:     0 < start ==> a[start-1] != v;
        loop invariant bound:     start <= i + 1 <= start + p;
        loop invariant not_found: !HasConstantSubRange(a, i, v, p);
        loop assigns i, start;
        loop variant n - i;
      */
      for (size_type i = 0u; i < n; ++i) {
        if (a[i] != v) {
          start = i + 1u;
          //@ assert not_found: !HasConstantSubRange(a, i+1, v, p);
        }
        else {
          //@ assert match: a[i] == v;
          //@ assert match: AllEqual(a, start, i+1, v);
          if (p == i + 1u - start) {
            //@ assert bound: start + p == i + 1;
            //@ assert match: AllEqual(a, start, start+p, v);
            //@ assert match: \exists integer k; 0 <= k <= n-p && AllEqual(a, k, k+p
                , v);
            //@ assert match: HasConstantSubRange(a, n, v, p);
            return start;
          }
          else {
            //@ assert bound: i + 1 < start + p;
            continue;
          }
        }

        //@ assert not_found: !HasConstantSubRange(a, i+1, v, p);
      }

      //@ assert not_found: !HasConstantSubRange(a, n, v, p);
      return n;
    }
    else {
      //@ assert not_found: n < p;
      //@ assert not_found: !HasConstantSubRange(a, n, v, p);
      return n;
    }
  }
  else {
    //@ assert bound: p == 0;
    //@ assert match: AllEqual(a, 0, 0, v);
    //@ assert match: HasConstantSubRange(a, n, v, 0);
    return 0u;
  }
}
```

Listing 4.40: Implementation of `search_n`

The implementation tracks, in the variable `start`, the beginning of the current maximal block of consecutive values `v`. The loop invariant `not_found` expresses that no `p`-fold repetition has been encountered so far; once such a repetition is found, the loop terminates and `start` is returned.

The boundary cases `n < p` and `p == 0` are handled explicitly in separate branches, which simplifies the verification by automatic provers.

4.10. The `find_end` algorithm

The algorithm `find_end`, as specified in the C++ Standard Library [19, §28.5.6], searches for the *last* occurrence of a given sequence as a contiguous subsequence of another sequence, using element-by-element comparison. For our purposes, we specialize the generic formulation to arrays of type `value_type`. The resulting function signature is as follows:

```
size_type
find_end(const value_type* a, size_type n,
         const value_type* b, size_type p);
```

The function `find_end` returns the greatest index `s` such that `a[s+k] == b[k]` holds for all indices `k` with `0 <= k < p`. In other words, `s` denotes the starting position of the last occurrence of the subarray `b[0..p-1]` within `a[0..n-1]` (see Figure 4.41). If no such index exists, the function returns the value `n`.

A special case arises when `p == 0`. In this situation, the empty sequence is considered to occur at the end of the array, and the function therefore returns `n`.

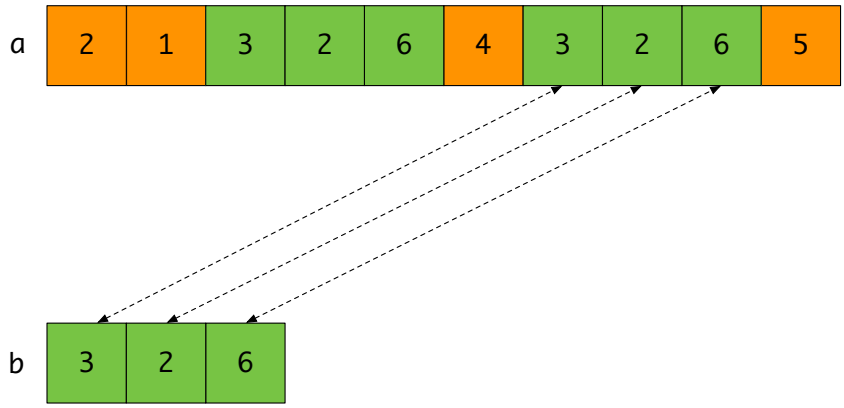

Figure 4.41.: A simple example of `find_end`

4.10.1. Formal specification of `find_end`

Listing 4.42 shows the formal specification of `find_end` [4.42]. From a conceptual point of view, the contract of `find_end` is very similar to that of `find2` [4.5]. As before, we use behaviors to capture the essential cases of the algorithm. These behaviors are both *complete* and *disjoint*.

```
/*@
  requires   valid:   \valid_read(a + (0..n-1));
  requires   valid:   \valid_read(b + (0..p-1));

  terminates          \true;
  exits               \false;
  assigns             \nothing;

  ensures    result:  0 <= \result <= n;

  behavior has_match:
    assumes          HasSubRange(a, n, b, p);
    assigns          \nothing;
    ensures bound:   0 <= \result <= n-p;
    ensures result:  Equal{Here,Here}(a + \result, p, b);
    ensures last:    !HasSubRange(a, \result + 1, n, b, p);

  behavior no_match:
    assumes          !HasSubRange(a, n, b, p);
    assigns          \nothing;
    ensures result:  \result == n;

  complete behaviors;
  disjoint behaviors;
*/
size_type
find_end(const value_type* a, size_type n,
         const value_type* b, size_type p);
```

Listing 4.42: Formal specification of `find_end`

The behavior `has_match` applies when the array `a` contains a contiguous subsequence that is identical to `b`. This condition is expressed in the corresponding `assumes` clause using the predicate `HasSubRange` [4.34]. The `ensures` clause `bound` states that the return value lies in the range `0..n-p`. The `result` clause of behavior `has_match` expresses that the returned index indeed denotes a position in `a` where the sequence `b` occurs. Finally, the clause `last` formalizes the defining property of `find_end`: it ensures that there is no occurrence of `b` in `a` starting at any index strictly greater than `\result`.

The behavior `no_match` covers the complementary case in which no subsequence of `a` is equal to `b`. In this situation, `find_end` must return the value `n`, that is, the length of the array `a`.

4.10.2. Implementation of `find_end`

The implementation of `find_end` [4.43] closely follows that of `search` [4.36], with the essential difference that the last matching position is recorded rather than the first.

```
size_type
find_end(const value_type* a, size_type n,
         const value_type* b, size_type p)
{
  size_type r = n;

  if ((0u < p) && (p <= n)) {
    /*@
      loop invariant bound   :  r <= n - p || r == n;
      loop invariant not_found: r == n ==> !HasSubRange(a, p+i-1, b, p);
      loop invariant found:     r < n  ==> Equal{Here,Here}(a+r, p, b);
      loop invariant last:      r < n  ==> !HasSubRange(a, r+1, i+p-1, b, p);
      loop assigns i, r;
      loop variant n - i;
    */
    for (size_type i = 0u; i <= n - p; ++i) {
      if (equal(a + i, p, b)) {
        r = i;
      }
    }
  }

  return r;
}
```

Listing 4.43: Implementation of `find_end`

The variable `r` stores the prospective return value according to the current state of the search. It is initially set to `n`, indicating that no occurrence of `b` has been found so far. Whenever an occurrence is detected, `r` is updated to the corresponding starting index.

The invariant `bound` states that `r` either retains the value `n` or lies in the range `0..n-p`. In the former case, the invariant `not_found` expresses that no occurrence of `b` has been identified. In the latter case, the invariant `found` states that the subsequence `b[0..p-1]` occurs in `a` starting at index `r`. Finally, the invariant `last` ensures that no occurrence of `b` has been found at any index strictly greater than `r`.

4.11. The `count` algorithm

The `count` algorithm of the C++ Standard Library [19, §28.5.9] determines how often a given value occurs in a sequence. For the purposes of this chapter, we restrict attention to arrays of elements of type `value_type` and adapt the generic interface accordingly. The resulting function signature is:

```
size_type
count(const value_type* a, size_type n, value_type v);
```

Given an array `a` of length `n`, the function returns the number of indices `i` with `0 <= i < n` for which `a[i] == v` holds.

4.11.1. The logic function `Count`

In order to specify `count`, we first need a formal notion of counting occurrences of a value in an array segment. The ACSL specification language does not provide such a notion as a primitive.[9] We therefore introduce an axiomatic definition of the *logic function* `Count` [4.44], which captures the standard intuitive behavior of counting a given value in a contiguous array section.

The expression `Count(a,m,n,v)` denotes the number of occurrences of `v` in the range `a[m..n-1]`.

- The ACSL keyword `axiomatic` is used to group the definition of the logic function `Count` together with the lemmas that characterize its behavior. The bounds `m` and `n`, as well as the return value of `Count`, are of type `integer`.
- The logic function `Count` is defined recursively. The definition distinguishes the empty range from the non-empty case and, in the latter, inspects the value of the current array element. The recursion decreases the length of the range. An overloaded variant of `Count` is provided that takes only the array length, corresponding to the common case `m = 0`.
- Lemma `Count_Empty` [4.44] characterizes the value of `Count` on empty ranges.
- Lemmas `Count_Hit` [4.44] and `Count_Miss` express how counting over a range of length `n-m` reduces to counting over a range of length `n-m-1`, depending on whether the last element matches the value `v`.
- Lemmas `Count_One` [4.44] and `Count_Single` build on `Count_Hit` and `Count_Miss`. They are convenient derived results that simplify several Coq proofs by shifting the induction scheme from `n-1 -> n` to `n -> n+1`.

[9] Strictly speaking, the ACSL documentation mentions the operator `numof` as part of a collection of so-called *higher-order logic constructions* [14, §2.6.7]. These *extended quantifiers*, however, have originally been described as experimental features and are therefore not used here.

```
/*@
  axiomatic Count
  {
    logic integer
    Count(value_type* a, integer m, integer n, value_type v) =
      n <= m ? 0 : Count(a, m, n-1, v) + (a[n-1] == v ? 1 : 0);

    logic integer
    Count(value_type* a, integer n, value_type v) = Count(a, 0, n, v);

    lemma Count_Empty:
      \forall value_type *a, v, integer m, n;
        n <= m  ==>  Count(a, m, n, v) == 0;

    lemma Count_Hit:
      \forall value_type *a, v, integer n, m;
        m < n         ==>
        a[n-1] == v   ==>
        Count(a, m, n, v) == Count(a, m, n-1, v) + 1;

    lemma Count_Miss:
      \forall value_type *a, v, integer n, m;
        m < n         ==>
        a[n-1] != v   ==>
        Count(a, m, n, v) == Count(a, m, n-1, v);

    lemma Count_One:
      \forall value_type *a, v, integer m, n;
        m <= n  ==>  Count(a, m, n+1, v) == Count(a, m, n, v) + Count(a, n, n+1, v);

    lemma Count_Single{K,L}:
      \forall value_type *a, *b, v, integer m, n;
        \at(a[m],K) == \at(b[n],L)  ==>
        Count{K}(a, m, m+1, v) == Count{L}(b, n, n+1, v);
```

Listing 4.44: The logic function `Count` (1)

Listing 4.45 shows another group of `Count` lemmas from the axiomatic block.

- The logic function `Count` depends only on the values in the memory locations `a[m..n-1]`. Lemma `Count_Unchanged` makes this explicit by stating that `Count` yields the same result in two program states `K` and `L`, provided that the values of `a[m..n-1]` are unchanged. The predicate `Unchanged` is used to express this premise.
- Lemma `Count_Equal` generalizes `Count_Unchanged` to the comparison of two arrays.
- Lemmas `Count_Union` and `Count_Cut` support reasoning about partitions of array ranges.

```
    lemma Count_Equal{K,L}:
      \forall value_type *a, v, integer m, n, p;
        0 <= m <= n              ==>
        Equal{K,L}(a, m, n, p)   ==>
        Count{K}(a, m, n, v) == Count{L}(a, p, p + (n-m), v);

    lemma Count_Unchanged{K,L}:
      \forall value_type *a, v, integer m, n;
        Unchanged{K,L}(a, m, n)  ==>  Count{K}(a, m, n, v) == Count{L}(a, m, n, v);

    lemma Count_Union:
      \forall value_type *a, v, integer k, m, n;
        0 <= k <= m <= n  ==>
        Count(a, k, n, v) == Count(a, k, m, v) + Count(a, m, n, v);

    lemma Count_Cut:
      \forall value_type *a, v, integer k, m, n;
        0 <= k <= m < n  ==> Count(a, k, n, v) ==
          Count(a, k, m, v) + Count(a, m, m+1, v) + Count(a, m+1, n, v);
```

Listing 4.45: The logic function `Count` (2)

The final group of initial `Count` lemmas are shown in Listing 4.46.

- Lemmas `Count_Single_Bounds` and `Count_Bounds` establish lower and upper bounds for the value of `Count`. Lemma `Count_Increasing` states that `Count` is monotonic with respect to range extension.
- Finally, lemmas `Count_Single_Shift` and `Count_Shift` express that `Count` is invariant under shifts of the array index.

```
    lemma Count_Single_Bounds:
      \forall value_type *a, v, integer n;
        0 <= Count(a, n, n+1, v) <= 1;

    lemma Count_Bounds:
      \forall value_type *a, v, integer m, n;
        0 <= m <= n  ==>  0 <= Count(a, m, n, v) <= n-m;

    lemma Count_Increasing:
      \forall value_type *a, v, integer m, n, p;
        m <= n <= p  ==>  Count(a, m, n, v) <= Count(a, m, p, v);

    lemma Count_Single_Shift:
      \forall value_type *a, v, integer n;
        0 <= n  ==>  Count(a+n, 0, 1, v) == Count(a, n, n+1, v);

    lemma Count_Shift:
      \forall value_type *a, v, integer m, n;
        0 <= m  ==>  0 <= n  ==>  Count(a+m, 0, n, v) == Count(a, m, m+n, v);
  }
*/
```

Listing 4.46: The logic function `Count` (3)

4.11.2. Relating `Count` and `Find`

So far, the logic function `Count` has been treated independently of the predicates used in the specifications of searching algorithms. In those specifications, existential properties are expressed using the predicate `SomeEqual` [4.4], introduced earlier in the discussion of the various `find` variants.

Lemma `Count_SomeEqual` [4.47], shown below, establishes a direct connection between these two notions. It states that, for a non-empty array range `[m,n)`, a strictly positive value of `Count` implies that `SomeEqual` holds on the same range. In other words, if the count of occurrences of a value `v` in `a[m..n-1]` is positive, then `v` occurs at least once in that range.

```
/*@
  axiomatic CountFind
  {
    lemma Count_SomeEqual:
      \forall value_type *a, v, integer m, n;
        0 <= m < n              ==>
        0 < Count(a, m, n, v)   ==>
        SomeEqual(a, m, n, v);
  }
*/
```

Listing 4.47: The logic definition(s) `CountFind`

This lemma makes explicit a basic consistency property between counting and existence. It is stated only in this direction; the converse implication, although expected from the definition of `Count`, is not required here and is therefore not included.

4.11.3. Formal specification of `count`

The contract of `count` [4.48] is formulated using the logic function `Count` [4.44], which expresses the number of occurrences of a given value in an array range. In addition, the specification explicitly states that the result returned by `count` is non-negative and does not exceed the length of the array.

```
/*@
  requires   valid:  \valid_read(a + (0..n-1));

  terminates         \true;
  exits              \false;
  assigns            \nothing;

  ensures    bound:  0 <= \result <= n;
  ensures    count:  \result == Count(a, n, v);
*/
size_type
count(const value_type* a, size_type n, value_type v);
```

Listing 4.48: Formal specification of `count`

4.11.4. Implementation of `count`

The following listing shows a straightforward implementation of `count` [4.49]. One of the loop invariants refers to the logic function `Count`, thereby establishing a direct connection between the operational behavior of the loop and the abstract counting property used in the specification.

```
size_type
count(const value_type* a, size_type n, value_type v)
{
  size_type counted = 0u;

  /*@
    loop invariant bound: 0 <= i <= n;
    loop invariant bound: 0 <= counted <= i;
    loop invariant count: counted == Count(a, i, v);
    loop assigns i, counted;
    loop variant n-i;
  */
  for (size_type i = 0u; i < n; ++i) {
    if (a[i] == v) {
      counted++;
    }
  }

  return counted;
}
```

Listing 4.49: Implementation of `count`

5. Maximum and minimum algorithms

In this chapter we study the formal specification of algorithms in the C++ Standard Library [19, §28.7.8] that compute maximum or minimum values of their arguments. Like the algorithms discussed in Chapter 4, these algorithms do not modify any memory locations outside their scope.

A distinctive feature of the algorithms in this chapter is that they rely essentially on comparisons between values. More precisely, they are specified in terms of binary relational operators such as <, whose mathematical properties play a central role in the correctness arguments.

The chapter is therefore organized in two parts. We first introduce several auxiliary notions that are required for precise and reusable specifications. We then turn to the actual maximum and minimum algorithms provided by the library.

More specifically, the chapter contains the following material.

- In §5.1 we discuss basic properties that relational operators must satisfy in order to be suitable for use in maximum and minimum algorithms.
- In §5.2 we introduce a collection of logical predicates that describe order-related properties of arrays whose elements are of type `value_type`. These predicates will be used throughout the remainder of the chapter.
- Section 5.3 introduces the type `size_type_pair` together with the function `make_pair`. These definitions provide a uniform way of representing and constructing pairs of indices in specifications. They are used in particular for the specification of `minmax_element` [5.17], but are otherwise independent of any specific maximum or minimum algorithm.
- The algorithm `clamp`, discussed in §5.4, is a simple example that restricts a value to lie within a pair of boundary values.
- The algorithm `max_element`, presented in §5.5, returns an index to a maximum element in a given range. As with `find`, it returns the smallest index among all valid candidates. In §5.6 we give an alternative specification, `max_element2`, based on user-defined predicates.
- The algorithm `max_seq` in §5.7 is closely related to `max_element` and serves as an example of *modular verification*. Unlike `max_element`, it returns the maximum value itself rather than an index.
- The algorithm `min_element`, discussed in §5.8, computes an index to a smallest element in a range.
- Finally, the algorithm `minmax_element` in §5.9 computes both the smallest and the largest element of a range simultaneously. Its specification and implementation rely on the auxiliary function `make_pair` introduced in §5.3.

We begin with a discussion of relational operators in §5.1, which provides the foundation for all subsequent specifications.

5.1. A note on relational operators

Note that in order to compare values, algorithms in the C++ Standard Library [19, §28.7.8] usually rely solely on the *less than* operator < or on special function objects. To be precise, the operator < must induce a *strict partial order*, which means that the following rules must hold.[10]

$$\begin{array}{llll} \text{irreflexivity} & \forall x & : \neg(x < x) & \\ \text{asymmetry} & \forall x, y & : x < y & \implies \neg(y < x) \\ \text{transitivity} & \forall x, y, z & : x < y \wedge y < z & \implies x < z \end{array}$$

If one wishes to check that the operator < of our `value_type` (see §2.3). satisfies these properties, the corresponding lemmas of `Less` [5.1] can be formulated and verified with Frama-C.

```
/*@
  axiomatic Less
  {
    lemma Less_Irreflexivity:
      \forall value_type a; !(a < a);

    lemma Less_Antisymmetry:
      \forall value_type a, b; (a < b)  ==>  !(b < a);

    lemma Less_Transitivity:
      \forall value_type a, b, c; (a < b) && (b < c)  ==>  (a < c);

    lemma Greater_Less:
      \forall value_type a, b; (a > b)  <==>  (b < a);

    lemma LessOrEqual_Less:
      \forall value_type a, b; (a <= b)  <==>  !(b < a);

    lemma GreaterOrEqual_Less:
    \forall value_type a, b; (a >= b)  <==> !(a < b);
  }
*/
```

Listing 5.1: The logic definition(s) `Less`

It is possible to specify and implement the algorithms of this chapter by using only operator <. For example, `a <= b` can be written as `a < b || a == b`, or, for our ordering on `value_type`, as `!(b < a)`. Listing `Less` [5.1] therefore also contains lemmas on representing the operators >, <=, and >= in terms of operator <.

Remark on `StrictWeakOrdering`. The C++ Standard formally requires comparison predicates used by most algorithms to satisfy the concept of a *strict weak ordering*. This notion slightly relaxes that of a strict partial order by allowing distinct elements to be considered *equivalent* with respect to the ordering, that is, neither $x < y$ nor $y < x$ holds. In this chapter, we restrict attention to orderings that are strict partial orders. This simplification is sufficient for the algorithms considered here and avoids the additional technical overhead associated with equivalence classes induced by weak orderings.

[10] See `http://en.wikipedia.org/wiki/Partially_ordered_set`

5.2. Predicates for bounds and extrema of arrays

We begin by introducing predicates that express how all elements of an array segment compare to a given value. The predicates `LowerBound` [5.2] and `UpperBound`, together with their strict variants `StrictLowerBound` and `StrictUpperBound`, are shown in the following listing.

```
/*@
  axiomatic ArrayBounds
  {
    predicate
    LowerBound{L}(value_type* a, integer m, integer n, value_type v) =
      \forall integer i; m <= i < n  ==>  v <= a[i];

    predicate
    LowerBound{L}(value_type* a, integer n, value_type v) =
      LowerBound{L}(a, 0, n, v);

    predicate
    StrictLowerBound{L}(value_type* a, integer m, integer n, value_type v) =
      \forall integer i; m <= i < n  ==>  v < a[i];

    predicate
    StrictLowerBound{L}(value_type* a, integer n, value_type v) =
      StrictLowerBound{L}(a, 0, n, v);

    predicate
    UpperBound{L}(value_type* a, integer m, integer n, value_type v) =
      \forall integer i; m <= i < n  ==>  a[i] <= v;

    predicate
    UpperBound{L}(value_type* a, integer n, value_type v) =
      UpperBound{L}(a, 0, n, v);

    predicate
    StrictUpperBound{L}(value_type* a, integer m, integer n, value_type v) =
      \forall integer i; m <= i < n  ==>  a[i] < v;

    predicate
    StrictUpperBound{L}(value_type* a, integer n, value_type v) =
      StrictUpperBound{L}(a, 0, n, v);
  }
*/
```

Listing 5.2: The logic definition(s) `ArrayBounds`

These predicates provide a concise and uniform way to state global ordering properties of array segments. They will be used extensively throughout this chapter and again in Chapter 6.

Using these bounds predicates as building blocks, we now define predicates that characterize extrema of array segments. An element is a maximum (respectively minimum) if it constitutes an upper (respectively lower) bound for all elements in the range. This idea is captured by the predicates `MaxElement` [5.3] and `MinElement` [5.3] shown below.

```
/*@
  axiomatic ArrayExtrema
  {
    predicate
    MaxElement{L}(value_type* a, integer n, integer max) =
      0 <= max < n && UpperBound(a, n, a[max]);

    predicate
    MinElement{L}(value_type* a, integer n, integer min) =
      0 <= min < n && LowerBound(a, n, a[min]);
  }
*/
```

Listing 5.3: The logic definition(s) `ArrayExtrema`

We will discuss these extrema predicates in more detail in §5.6 and §5.8.

5.3. The auxiliary function `make_pair`

In several specifications we need to reason about *pairs of indices* rather than individual positions. For this purpose, and purely as a technical convenience, we introduce a small auxiliary data type. This construction is independent of any particular algorithm and, in particular, is *not* conceptually related to algorithms computing maximum or minimum elements.

The following listing introduces the type `size_type_pair`, which simply bundles two values of type `size_type` into a single object. It serves as a lightweight vehicle for passing index pairs through specifications and implementations.

```
struct size_type_pair {
  size_type first;
  size_type second;
};

typedef struct size_type_pair size_type_pair;
```

Listing 5.4: The auxiliary type `size_type_pair`

To construct values of type `size_type_pair`, we provide the auxiliary function `make_pair`, which combines two indices `first` and `second` into a single object. The function serves purely as a technical convenience.

The formal specification of `make_pair` [5.5] is shown in the following listing.

```
/*@
    terminates          \true;
    exits               \false;
    assigns             \nothing;

    ensures   result:  \result.first  == first;
    ensures   result:  \result.second == second;
*/
size_type_pair
make_pair(size_type first, size_type second);
```

Listing 5.5: Formal specification of `make_pair`

The next listing gives a straightforward implementation that directly reflects the specification.

```
size_type_pair
make_pair(size_type first, size_type second)
{
  size_type_pair pair;

  pair.first  = first;
  pair.second = second;

  return pair;
}
```

Listing 5.6: Implementation of `make_pair`

5.4. The `clamp` algorithm

The `clamp` algorithm of the C++ Standard Library [19, §28.7.9] restricts a value to lie within a given closed interval. Given a value `v` and boundary values `lower` and `upper`, the algorithm returns the nearest value to `v` that lies within the interval `[lower, upper]`.

The signature of our version of `clamp` is as follows:

```
value_type clamp(value_type v, value_type lower, value_type upper);
```

- If `v` lies between `lower` and `upper`, it is returned unchanged.
- If `v` is smaller than `lower`, the function returns `lower`.
- If `v` exceeds `upper`, the function returns `upper`.

5.4.1. Formal specification of `clamp`

The following listing shows the formal specification of `clamp` [5.7]. We require as a precondition that the lower bound does not exceed the upper bound, that is, `lower <= upper`.

```
/*@
  requires   bound:  lower < upper;

  terminates         \true;
  exits              \false;
  assigns            \nothing;

  ensures    bound:  lower <= \result <= upper;

  behavior lower_bound:
    assumes          v < lower;
    assigns          \nothing;
    ensures result:  \result == lower;

  behavior between:
    assumes          lower <= v <= upper;
    assigns          \nothing;
    ensures result:  \result == v;

  behavior upper_bound:
    assumes          upper < v;
    assigns          \nothing;
    ensures result:  \result == upper;

  complete behaviors;
  disjoint behaviors;
*/
value_type
clamp(value_type v, value_type lower, value_type upper);
```

Listing 5.7: Formal specification of `clamp`

5.4.2. Implementation of `clamp`

The implementation of `clamp` [5.8] is straightforward. Due to its simple control flow, the correctness of the implementation can be verified without introducing any additional annotations.

```
value_type
clamp(value_type v, value_type lower, value_type upper)
{
  return (v < lower) ? lower : (upper < v) ? upper : v;
}
```

Listing 5.8: Implementation of `clamp`

5.5. The `max_element` algorithm

The `max_element` algorithm of the C++ Standard Library [19, §28.7.8] determines the position of a maximal element in a finite sequence. The signature of our version of `max_element` is

```
size_type max_element(const value_type* a, size_type n);
```

The algorithm distinguishes two conceptually simple cases. If the range `a[0..n-1]` is empty (`n == 0`), no maximum exists and the function returns the sentinel value `n`. Otherwise (`n > 0`), the algorithm scans the array from left to right and determines the first occurrence of a maximum value.

5.5.1. Formal specification of `max_element`

The following listing shows the formal ACSL specification of `max_element`.

```
/*@
  requires   valid:  \valid_read(a + (0..n-1));

  terminates         \true;
  exits              \false;
  assigns            \nothing;

  ensures    result:  0 <= \result <= n;

  behavior empty:
    assumes          n == 0;
    assigns          \nothing;
    ensures result:  \result == 0;

  behavior not_empty:
    assumes          0 < n;
    assigns          \nothing;
    ensures result:  0 <= \result < n;
    ensures upper:   \forall integer i; 0 <= i < n       ==> a[i] <= a[\result];
    ensures first:   \forall integer i; 0 <= i < \result ==> a[i] <  a[\result];

  complete behaviors;
  disjoint behaviors;
*/
size_type
max_element(const value_type* a, size_type n);
```

Listing 5.9: Formal specification of `max_element`

The specification distinguishes two behaviors, `empty` and `not_empty`, corresponding to the cases `n == 0` and `n > 0`, respectively.

In behavior `not_empty`, the ensures clauses `max` and `first` state that the returned index refers to a maximal element and denotes its first occurrence.

5.5.2. Implementation of `max_element`

The following listing shows a straightforward implementation of `max_element` [5.10] together with its loop annotations.

```
size_type
max_element(const value_type* a, size_type n)
{
  if (0u < n) {
    size_type max = 0u;

    /*@
      loop invariant bound:  0 <= i <= n;
      loop invariant max:    0 <= max <  n;
      loop invariant upper:  \forall integer k; 0 <= k < i   ==> a[k] <= a[max];
      loop invariant first:  \forall integer k; 0 <= k < max ==> a[k] <  a[max];
      loop assigns max, i;
      loop variant n-i;
    */
    for (size_type i = 1u; i < n; i++) {
      if (a[max] < a[i]) {
        max = i;
      }
    }

    return max;
  }

  return n;
}
```

Listing 5.10: Implementation of `max_element`

The implementation reflects the same two cases as the specification. The case `n == 0` is handled explicitly before the loop. For `n > 0`, the loop traverses the array and maintains the index of a candidate maximum.

The loop invariant `max` anticipates the postcondition `max` of behavior `not_empty` by stating that the current candidate index refers to a maximal element of the prefix inspected so far. The invariant `upper` ensures that the candidate index remains within bounds. Finally, the invariant `first` records that the current candidate index refers to the first occurrence of the maximal value seen so far, which allows the postcondition `first` to be proved at loop termination.

5.6. The `max_element2` algorithm (predicate-based specification)

In this section we present an alternative specification of the `max_element` algorithm. Instead of expressing maximality directly through quantified conditions, this variant relies on previously introduced predicates.

In particular, we make use of the predicate `UpperBound` [5.2], which expresses that a given value is greater than or equal to all elements of a given array segment. Its strict counterpart, `StrictUpperBound` [5.2], is used to capture the notion of a *first* maximum.

In addition, we employ the predicate `MaxElement` [5.3]. This predicate characterizes an index `max` whose associated element is an upper bound of the range `a[0..n-1]` and, at the same time, belongs to that range. Together, these predicates allow a more declarative formulation of the specification.

5.6.1. Formal specification of `max_element2`

The following listing shows the formal specification of `max_element2` [5.11].

```
/*@
  requires   valid:   \valid_read(a + (0..n-1));

  terminates          \true;
  exits               \false;
  assigns             \nothing;

  ensures    result:  0 <= \result <= n;

  behavior empty:
    assumes          n == 0;
    assigns          \nothing;
    ensures result:  \result == 0;

  behavior not_empty:
    assumes          0 < n;
    assigns          \nothing;
    ensures result:  0 <= \result < n;
    ensures max:     MaxElement(a, n, \result);
    ensures first:   StrictUpperBound(a, \result, a[\result]);

  complete behaviors;
  disjoint behaviors;
*/
size_type
max_element2(const value_type* a, size_type n);
```

Listing 5.11: Formal specification of `max_element2`

The predicate `MaxElement` [5.3] is used to express that the returned index refers to a maximum element of the array. The predicate `StrictUpperBound` [5.2] is employed to specify that this index denotes the first occurrence of such a maximum.

5.6.2. Implementation of `max_element2`

The implementation of `max_element2` [5.12] is structurally identical to that of `max_element` [5.10]. The difference lies solely in the loop annotations, which now make use of the predicate-based characterizations introduced above.

```
size_type
max_element2(const value_type* a, size_type n)
{
  if (0u < n) {
    size_type max = 0u;

    /*@
      loop invariant bound:   0 <= i <= n;
      loop invariant max:     0 <= max < n;
      loop invariant upper:   UpperBound(a, i, a[max]);
      loop invariant first:   StrictUpperBound(a, max, a[max]);
      loop assigns max, i;
      loop variant n-i;
    */
    for (size_type i = 0u; i < n; i++) {
      if (a[max] < a[i]) {
        max = i;
      }
    }

    return max;
  }

  return n;
}
```

Listing 5.12: Implementation of `max_element2`

5.7. The max_seq algorithm

In this section we consider the function max_seq [13, Ch. 3], which is closely related to max_element. The essential difference is that max_seq returns the maximum value itself, rather than its position. Consequently, max_seq requires a *non-empty* input range.

The function max_seq can easily be implemented using max_element2 [5.12]. Moreover, relying solely on the formal specification of max_element2 [5.11], we can deductively verify the correctness of this implementation. This provides a simple example of *modular verification* in the following sense:

> Any implementation of max_element2 that is separately proven to satisfy the contract max_element2 [5.11] guarantees the correctness of max_seq. Once the contracts have been defined, the function max_element2 may be implemented independently of max_seq, without affecting its verification.

5.7.1. Formal specification of max_seq

The following listing shows the formal specification of max_seq [5.13].

```
/*@
  requires    0 < n;
  requires    \valid_read(p + (0..n-1));

  terminates  \true;
  exits       \false;
  assigns     \nothing;

  ensures     \forall integer i; 0 <= i <= n-1 ==> \result >= p[i];
  ensures     \exists integer e; 0 <= e <= n-1 &&  \result == p[e];
*/
value_type
max_seq(const value_type* p, size_type n);
```

Listing 5.13: Formal specification of max_seq

The first requires-clause expresses that max_seq expects a *non-empty* range. The postconditions formalize that the returned value is indeed a maximum of the range.

5.7.2. Implementation of max_seq

The implementation of max_seq [5.14] consists of a single call to max_element2 [5.12]. Since max_seq requires a non-empty range, this call returns an index of a maximum element. The fact that max_element2 returns the smallest such index is irrelevant in this context.

```
value_type
max_seq(const value_type* p, size_type n)
{
  return p[max_element2(p, n)];
}
```

Listing 5.14: Implementation of max_seq

5.8. The `min_element` algorithm

The `min_element` algorithm of the C++ Standard Library [19, §28.7.8] determines the position of a minimal element in a finite sequence. The signature of our version of `min_element` is

```
size_type min_element(const value_type* a, size_type n);
```

The function operates on the range `a[0..n-1]`. If the range is non-empty, it returns the unique index `i` such that `a[i]` is minimal among all elements of the range and `i` denotes the first position with this property. If the range is empty (`n == 0`), the function returns `n`.

Conceptually, `min_element` is the natural dual of the `max_element` algorithm. Accordingly, its specification and implementation closely follow the predicate-based variant of `max_element` discussed in §5.6.

5.8.1. Formal specification of `min_element`

The following listing shows the formal specification of `min_element` [5.15].

```
/*@
  requires   valid:  \valid_read(a + (0..n-1));

  terminates          \true;
  exits               \false;
  assigns             \nothing;

  ensures    result:  0 <= \result <= n;

  behavior  empty:
    assumes          n == 0;
    assigns          \nothing;
    ensures  result: \result == 0;

  behavior  not_empty:
    assumes          0 < n;
    assigns          \nothing;
    ensures result:  0 <= \result < n;
    ensures min:     MinElement(a, n, \result);
    ensures first:   StrictLowerBound(a, \result, a[\result]);

  complete behaviors;
  disjoint behaviors;
*/
size_type
min_element(const value_type* a, size_type n);
```

Listing 5.15: Formal specification of `min_element`

As in the predicate-based specification of `max_element`, the contract is formulated using previously introduced predicates. The predicate `LowerBound` [5.2] expresses that a given value is less than or equal to all elements of a range, while `StrictLowerBound` [5.2] is used to characterize the first occurrence of a minimum. The predicate `MinElement` [5.3] combines these properties with the requirement that the value occurs in the range.

5.8.2. Implementation of `min_element`

The implementation of `min_element` [5.16] closely follows that of `max_element` [5.10]. Its loop annotations make systematic use of the predicates `LowerBound` [5.2] and `StrictLowerBound` [5.2] to establish the postconditions of the specification.

```
size_type
min_element(const value_type* a, size_type n)
{
  if (0u < n) {
    size_type min = 0u;

    /*@
      loop invariant bound:  0 <= i   <= n;
      loop invariant min:    0 <= min <  n;
      loop invariant lower:  LowerBound(a, i, a[min]);
      loop invariant first:  StrictLowerBound(a, min, a[min]);
      loop assigns min, i;
      loop variant n-i;
    */
    for (size_type i = 0u; i < n; i++) {
      if (a[i] < a[min]) {
        min = i;
      }
    }

    return min;
  }

  return n;
}
```

Listing 5.16: Implementation of `min_element`

5.9. The `minmax_element` algorithm

The `minmax_element` algorithm of the C++ Standard Library [19, §28.7.8] determines both a minimum and a maximum element in a finite sequence. The signature of our version of `minmax_element` is

```
size_type_pair minmax_element(const value_type* a, size_type n);
```

The function returns a pair of indices (see §5.3). The first component denotes the first position at which a minimum value occurs in the range `a[0..n-1]`, while the second component denotes the last position at which a maximum value occurs.

The properties of the index identifying the minimum are identical to those of `min_element` [5.15]. By contrast, the properties of the index identifying the maximum differ slightly from those specified for `max_element2` [5.11]. While `max_element` returns the first occurrence of a maximum value, `minmax_element` returns the last occurrence of such a value.

5.9.1. Formal specification of `minmax_element`

The following listing shows the formal ACSL specification of `minmax_element` [5.17].

```
/*@
  requires   valid:  \valid_read(a + (0..n-1));

  terminates         \true;
  exits              \false;
  assigns            \nothing;

  ensures    result:  0 <= \result.first  <= n;
  ensures    result:  0 <= \result.second <= n;

  behavior empty:
    assumes          0 == n;
    assigns          \nothing;
    ensures result:  \result.first == 0;
    ensures result:  \result.second == 0;

  behavior not_empty:
    assumes          0 < n;
    assigns          \nothing;
    ensures result:  0 <= \result.first < n;
    ensures result:  0 <= \result.second < n;

    ensures min:     MinElement(a, n, \result.first);
    ensures first:   StrictLowerBound(a, \result.first, a[\result.first]);
    ensures max:     MaxElement(a, n, \result.second);
    ensures last:    StrictUpperBound(a, \result.second+1, n, a[\result.second]);
*/
size_type_pair
minmax_element(const value_type* a, size_type n);
```

Listing 5.17: Formal specification of `minmax_element`

The specification combines the ideas used for `min_element` and `max_element`. The predicates `MinElement` [5.3] and `MaxElement` [5.3] are used to characterize minimality and maximality, respectively. The predicate `StrictLowerBound` [5.2] specifies that the minimum index refers to the first occurrence of a min-

imum value, while `StrictUpperBound` [5.2] is adapted to express that the returned maximum index denotes the last occurrence of a maximum value.

Accordingly, the postcondition `last` differs from the corresponding postcondition of `max_element`: it states that no element at or after the returned index is greater than or equal to the maximum value.

5.9.2. Implementation of `minmax_element`

The implementation of `minmax_element` [5.18] uses the auxiliary function `make_pair` [5.5] to construct the result. Apart from this, it closely follows the structure of the implementations of `min_element` and `max_element`.

```
size_type_pair
minmax_element(const value_type* a, size_type n)
{
  if (0u < n) {
    size_type min = 0u;
    size_type max = 0u;

    /*@
      loop invariant bound: 0 <= i    <= n;
      loop invariant min:   0 <= min  <  n;
      loop invariant max:   0 <= max  <  n;
      loop invariant lower: LowerBound(a, i, a[min]);
      loop invariant upper: UpperBound(a, i, a[max]);
      loop invariant first: StrictLowerBound(a, min, a[min]);
      loop invariant last:  StrictUpperBound(a, max+1, i, a[max]);
      loop assigns min, max, i;

      loop variant n-i;
    */
    for (size_type i = 0u; i < n; i++) {
      if (a[i] >= a[max]) {
        max = i;
      }

      if (a[i] < a[min]) {
        min = i;
      }
    }

    return make_pair(min, max);
  }

  return make_pair(n, n);
}
```

Listing 5.18: Implementation of `minmax_element`

We focus on the loop invariant `last`, which is the only invariant that differs from the earlier implementations. It reflects the modified use of the predicate `StrictUpperBound` [5.2], whose range is adjusted to capture the fact that the algorithm returns the last occurrence of a maximum value.

6. Binary search algorithms

In this chapter, we consider the following *binary search* algorithms of the C++ Standard Library [19, §28.7.3], namely

- `lower_bound` in §6.3,
- `upper_bound` in §6.4,
- two variants of the implementation of `equal_range` in §6.5, and
- `binary_search` in §6.6.

As in the case of the maximum and minimum algorithms discussed in Chapter 5, the binary search algorithms rely primarily on the less-than operator < (and the derived operators <=, > and >=) to reason about the relative order of values. In particular, they determine whether a given value is contained in an increasing (that is, ordered) range by comparing it to selected elements of that range.

Consequently, in contrast to the `find` algorithm introduced in §4.1, the equality operator == plays only a supporting role in the specification of binary search algorithms. For this reason, the correctness of binary search is fundamentally tied to formal assumptions about the monotonicity of the input range, made explicit in §6.1 and §6.2.

6.1. The predicate `Increasing`

We begin by introducing a logic predicate that expresses an ordering property of array segments. This predicate abstracts a recurring universal quantification pattern over array indices and allows us to state ordering assumptions in a concise and uniform manner. It serves as a foundational precondition for the formal specification of all binary search algorithms discussed in this chapter.

The following listing introduces the predicate `Increasing` [6.1]. This predicate, which is provided in overloaded form, characterizes array segments or entire arrays whose elements are *increasing*, in the sense that they are ordered with respect to the non-strict order <=. By convention, order properties based on the strict relation < are referred to as *strictly increasing*.

```
/*@
  axiomatic Increasing
  {
    predicate
    Increasing{L}(value_type* a, integer m, integer n) =
      \forall integer i, j; m <= i < j < n  ==>  a[i] <= a[j];

    predicate
    Increasing{L}(value_type* a, integer n) = Increasing{L}(a, 0, n);
  }
*/
```

Listing 6.1: The logic definition(s) `Increasing`

More precisely, the predicate `Increasing` requires that, for all indices within the specified segment, each element is smaller than or equal to any element at a greater index. This captures the notion of an increasing range at the logic level. In this chapter, the predicate `Increasing` is used primarily as a *precondition* that formalizes the ordering assumptions under which binary search algorithms operate. In later chapters, for example in the specification of `sort_heap` [9.40], the same predicate appears as an essential *postcondition*, expressing that an algorithm has produced an increasing array (see in particular Chapter 10).

6.2. A weaker monotonicity predicate

In the verification of algorithms that construct or transform arrays, it is often inconvenient to maintain global ordering properties of the form expressed by the predicate `Increasing` as loop invariants. Instead, such algorithms typically establish ordering properties through local comparisons between neighbouring elements.

For this reason, we introduce the predicate `WeaklyIncreasing` [6.2], which provides a weaker, neighbour-based formulation of monotonicity. This predicate is particularly well suited for use in assertions and loop invariants, where it allows us to express intermediate ordering properties that arise naturally during algorithm execution.

```
/*@
  axiomatic WeaklyIncreasing
  {
    predicate
    WeaklyIncreasing{L}(value_type* a, integer m, integer n) =
      \forall integer i; m <= i < n-1  ==>  a[i] <= a[i+1];

    predicate
    WeaklyIncreasing{L}(value_type* a, integer n) = WeaklyIncreasing{L}(a, 0, n);
  }
*/
```

Listing 6.2: The logic definition(s) `WeaklyIncreasing`

Users inexperienced in formal verification often underestimate the difference between `Increasing` [6.1] and `WeaklyIncreasing` [6.2]. Although both predicates characterize non-decreasing order of array segments, they differ in the scope of the comparisons they express: `WeaklyIncreasing` only compares direct neighbours, whereas `Increasing` relates all pairs of elements at increasing indices.

As a consequence, `Increasing` immediately implies `WeaklyIncreasing`, while the converse implication typically requires an induction over the length of the array segment. Such reasoning relies on the transitivity of the operator <= and therefore exceeds the capabilities of automated provers. For this reason, both directions of the relationship between `Increasing` and `WeaklyIncreasing` are made explicit by dedicated auxiliary lemmas (see, in particular, `Increasing_WeaklyIncreasing` [6.3] and `WeaklyIncreasing_Increasing` [6.3]).

In addition to relating the predicates `Increasing` and `WeaklyIncreasing`, the lemma collection also contains auxiliary results that support the reuse of `Increasing` under common transformations. The lemma `Increasing_Shift` [6.3] expresses that the increasing property is preserved under shifts of the index range, which is essential when reasoning about subranges addressed via pointer arithmetic. The lemma `Increasing_Equal` [6.3] captures the fact that equality of array elements at corresponding positions preserves the increasing property, and is used to transfer ordering assumptions between logically

equivalent ranges. Both lemmas also play an essential role in the verification of sorting algorithms, where ordering properties must be maintained and propagated through successive transformations of the array.

```
/*@
  axiomatic IncreasingLemmas
  {
    lemma Increasing_Shift{L}:
      \forall value_type *a, integer l, r;
        0 <= l <= r              ==>
        Increasing{L}(a, l, r)   ==>
        Increasing{L}(a+l, r-l);

    lemma Increasing_Equal{K,L}:
      \forall value_type* a, integer m, n, p;
        Increasing{K}(a, m, n)     ==>
        Equal{K,L}(a, m, n, m+p)   ==>
        Increasing{L}(a, m+p, n+p);

    lemma WeaklyIncreasing_Increasing{L}:
      \forall value_type* a, integer m, n;
        0 <= m <= n                 ==>
        WeaklyIncreasing(a, m, n)   ==>
        Increasing(a, m, n);

    lemma Increasing_WeaklyIncreasing{L}:
      \forall value_type* a, integer m, n;
        0 <= m <= n                  ==>
        Increasing(a, m, n)          ==>
        WeaklyIncreasing(a, m, n);
  }
*/
```

Listing 6.3: The logic definition(s) `IncreasingLemmas`

The auxiliary lemmas provided above make the necessary reasoning steps explicit at the logic level. They are reused throughout the verification of binary search algorithms and, as discussed earlier, also play a central role in the verification of sorting algorithms.

In practice, we exploit the relationship between the predicates `Increasing` and `WeaklyIncreasing` in a systematic manner:

- The predicate `Increasing` [6.1] is used in preconditions and postconditions of function contracts, where global ordering assumptions on array segments are required.
- The predicate `WeaklyIncreasing` [6.2] is employed in assertions and loop invariants whenever we need to express that an algorithm (typically a sorting algorithm) maintains or produces a weakly increasing array through local comparisons.
- When it is necessary to conclude that a *weakly increasing* array is in fact *increasing*, we rely on the lemma `WeaklyIncreasing_Increasing` [6.3].

6.3. The `lower_bound` algorithm

The `lower_bound` algorithm is one of the four binary search algorithms of the C++ Standard Library [19, §28.7.3.1]. For our purposes we have modified the generic implementation to that of an array of type `value_type`. The signature now reads:

```
size_type
lower_bound(const value_type* a, size_type n, value_type v);
```

As with the other binary search algorithms `lower_bound` requires that its input array is in increasing order. The index `lb`, that `lower_bound` returns satisfies the inequality

$$0 \leq lb \leq n \tag{6.1}$$

and has the following properties for any valid index `k` of the array.

$$0 \leq k < lb \quad \Longrightarrow \quad a[k] < v \tag{6.2}$$

$$lb \leq k < n \quad \Longrightarrow \quad v \leq a[k] \tag{6.3}$$

Conditions (6.2) and (6.3) imply that `v` can only occur in the array section `a[lb..n-1]`. In this sense `lower_bound` returns a *lower bound* for the potential indices.

As an example, we consider in Figure 6.4 an increasingly ordered array. The arrows indicate which indices are returned by `lower_bound` for a given value. Note that the index 9 points *one past the end* of the array. Values that are not contained in the array are colored in gray.

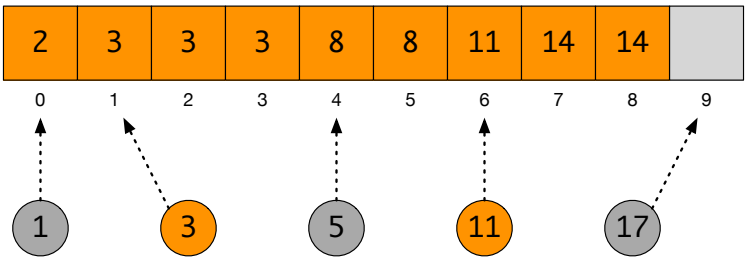

Figure 6.4.: Some examples for `lower_bound`

Figure 6.4 also clarifies that care must be taken when interpreting the return value of `lower_bound`. An important difference to the algorithms in Chapter 4 is that a return value of `lower_bound` that is less than n does not necessarily imply `a[lb] == v`. We can only be sure that `v <= a[lb]` holds.

6.3.1. Formal specification of `lower_bound`

The specification of `lower_bound` [6.5] is shown in the following listing. The precondition `increasing` formalizes the requirement that the array values are in increasing order. The postconditions reflect the conditions listed above and are expressed using the predicates `LowerBound` [5.2] and `StrictUpperBound` [5.2].

- Condition (6.1) is reflected by the postcondition `result`.
- Condition (6.2) is reflected by the postcondition `left`.
- Condition (6.3) is reflected by the postcondition `right`.

```
/*@
  requires   valid:       \valid_read(a + (0..n-1));
  requires   increasing:  Increasing(a, n);

  terminates              \true;
  exits                   \false;
  assigns                 \nothing;

  ensures    result:      0 <= \result <= n;
  ensures    left:        StrictUpperBound(a, 0, \result, v);
  ensures    right:       LowerBound(a, \result, n, v);
*/
size_type
lower_bound(const value_type* a, size_type n, value_type v);
```

Listing 6.5: Formal specification of `lower_bound`

6.3.2. Implementation of `lower_bound`

Our implementation of `lower_bound` [6.6] follows the standard binary search scheme and maintains a shrinking search interval `[left,right)` that is known to contain the result.

```
size_type
lower_bound(const value_type* a, size_type n, value_type v)
{
  size_type left  = 0u;
  size_type right = n;

  /*@
    loop invariant bound:  0 <= left <= right <= n;
    loop invariant left:   StrictUpperBound(a, 0,  left, v);
    loop invariant right:  LowerBound(a, right, n, v);

    loop assigns left, right;
    loop variant right - left;
  */
  while (left < right) {
    const size_type middle = left + (right - left) / 2u;

    if (a[middle] < v) {
      left = middle + 1u;
    }
    else {
      right = middle;
    }
  }

  return left;
}
```

Listing 6.6: Implementation of `lower_bound`

Each iteration step narrows down the range that contains the sought-after result. The loop invariants express that in each iteration step all indices less than the temporary left bound `left` contain values that are less than `v` and all indices not less than the temporary right bound `right` contain values that are greater or equal than `v`. The expression to compute `middle` is slightly more complex than the naïve `(left+right)/2`, but it avoids potential overflows.

6.4. The upper_bound algorithm

The upper_bound algorithm of the C++ Standard Library [19, §28.7.3.2] is a variant of binary search and is closely related to lower_bound [6.5]. The signature reads:

```
size_type
upper_bound(const value_type* a, size_type n, value_type v);
```

As with the other binary search algorithms, upper_bound requires that its input array is in increasing order. The index ub returned by upper_bound satisfies the inequality

$$0 \leq ub \leq n \tag{6.4}$$

and is involved in the following implications for any valid index k of the array under consideration

$$0 \leq k < ub \quad \Longrightarrow \quad a[k] \leq v \tag{6.5}$$

$$ub \leq k < n \quad \Longrightarrow \quad v < a[k] \tag{6.6}$$

Conditions (6.5) and (6.6) imply that v can only occur in the array section a[0..ub-1]. In this sense, upper_bound returns an *upper bound* for the potential indices at which v can occur. It also follows that the searched-for value v can *never* be located at the index ub.

Figure 6.7 is a variant of Figure 6.4 for the upper_bound algorithm, using the same example array. The arrows indicate which indices are returned by upper_bound for a given value. Compared to Figure 6.4, only the arrows corresponding to values that *are present* in the array change their target index.

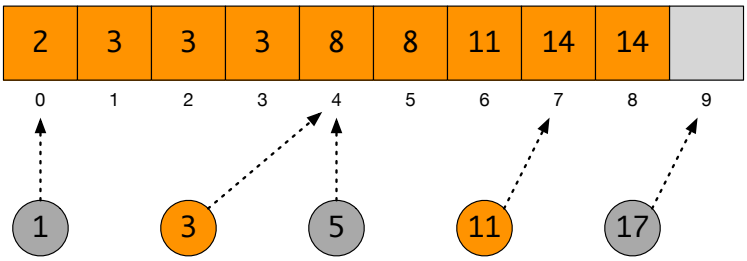

Figure 6.7.: Some examples for upper_bound

6.4.1. Formal specification of upper_bound

The following listing shows the specification of upper_bound [6.8], which is closely related to the specification of lower_bound [6.5]. The precondition increasing formalizes the requirement that the array values are in increasing order.

The postconditions reflect the conditions listed above and are expressed using the predicates UpperBound [5.2] and StrictLowerBound [5.2], namely,

- Condition (6.4) is reflected by the postcondition result.
- Condition (6.5) is reflected by the postcondition left.
- Condition (6.6) is reflected by the postcondition right.

```
/*@
  requires   valid:       \valid_read(a + (0..n-1));
  requires   increasing:  Increasing(a, n);

  terminates              \true;
  exits                   \false;
  assigns                 \nothing;

  ensures    result:      0 <= \result <= n;
  ensures    left:        UpperBound(a, 0, \result, v);
  ensures    right:       StrictLowerBound(a, \result, n, v);
*/
size_type
upper_bound(const value_type* a, size_type n, value_type v);
```

Listing 6.8: Formal specification of `upper_bound`

6.4.2. Implementation of `upper_bound`

Our implementation of `upper_bound` [6.9] follows the standard binary search scheme and maintains a shrinking search interval `[left,right)` that is known to contain the sought-after result.

```
size_type
upper_bound(const value_type* a, size_type n, value_type v)
{
  size_type left  = 0u;
  size_type right = n;

  /*@
    loop invariant bound:  0 <= left <= right <= n;
    loop invariant left:   UpperBound(a, 0, left, v);
    loop invariant right:  StrictLowerBound(a, right, n, v);

    loop assigns left, right;
    loop variant right - left;
  */
  while (left < right) {
    const size_type middle = left + (right - left) / 2u;

    if (a[middle] <= v) {
      left = middle + 1u;
    }
    else {
      right = middle;
    }
  }

  return right;
}
```

Listing 6.9: Implementation of `upper_bound`

The loop invariants express that, throughout the loop, all indices less than the temporary left bound `left` contain values not greater than `v`, and all indices not less than the temporary right bound `right` contain values greater than `v`.

6.5. The `equal_range` algorithm

The `equal_range` algorithm is one of the four binary search algorithms provided by the C++ Standard Library [19, §28.7.3.3]. As with the other binary search algorithms, `equal_range` requires that its input range is sorted in increasing order.

The specification of this algorithm in the C++ standard is particularly simple, since it explicitly states that `equal_range` combines the results of `lower_bound` [6.5] and `upper_bound` [6.8]. In C++, `equal_range` therefore returns a pair of iterators that delimit the subrange of elements equal to a given value.

As usual for our purposes, we adapt `equal_range` to operate on an array of type `value_type`. Instead of returning a pair of iterators, our version returns a pair of indices. The return type of `equal_range` is the struct `size_type_pair` shown in Listing 5.4, and its signature reads as follows:

```
size_type_pair
equal_range(const value_type* a, size_type n, value_type v);
```

In accordance with the preceding discussion, Figure 6.10 combines Figures 6.4 and 6.7 in order to visualize the behavior of `equal_range` for selected test cases. The two types of arrows, → and ⇢, represent the respective fields `first` and `second` of the return value. For values that are not contained in the array, the two arrows point to the same index.

More generally, if `equal_range` returns the pair (lb, ub), then the difference $ub - lb$ is equal to the number of occurrences of the argument `v` in the array.

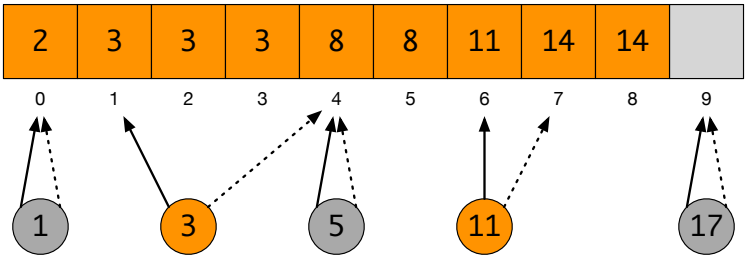

Figure 6.10.: Some examples for `equal_range`

We present two different implementations of `equal_range` and verify both of them against the *same formal specification*. Although the implementations differ substantially in structure, they are intended to be observationally equivalent.

The first implementation, `equal_range` [6.12], is deliberately straightforward: it invokes `lower_bound` and `upper_bound` separately and returns the pair consisting of their respective results. The second implementation, `equal_range2` [6.13], is more elaborate and follows an efficiency-oriented strategy by attempting to minimize duplicate computations.

6.5.1. Formal specification of `equal_range`

Let (lb, ub) be the return value of `equal_range`. Then the conditions (6.1)–(6.6) can be combined into the inequality

$$0 \leq lb \leq ub \leq n. \tag{6.7}$$

Moreover, for any valid index k of the array under consideration, the following three implications hold:

$$0 \leq k < lb \quad \Longrightarrow \quad a[k] < v, \tag{6.8}$$
$$lb \leq k < ub \quad \Longrightarrow \quad a[k] = v, \tag{6.9}$$
$$ub \leq k < n \quad \Longrightarrow \quad a[k] > v. \tag{6.10}$$

Here are some justifications for these conditions.

- Conditions (6.8) and (6.10) are exactly Conditions (6.2) and (6.6), respectively.
- The inequality (6.7) follows from (6.1) and (6.4) together with the following observation. If ub were smaller than lb, then by (6.8) we would have $a[ub] < v$. On the other hand, (6.10) implies $v < a[ub]$. This contradiction shows that $lb \leq ub$ must hold.
- Condition (6.9) follows from the combination of (6.3) and (6.5), together with the fact that $\leq$ is a total order on the integers.

The following listing shows the shared ACSL specification of `equal_range` [6.11] and `equal_range2`. The precondition `increasing` expresses that the array values are required to be in increasing order.

```
/*@
  requires   valid:       \valid_read(a + (0..n-1));
  requires   increasing:  Increasing(a, n);

  terminates              \true;
  exits                   \false;
  assigns                 \nothing;

  ensures    result:      0 <= \result.first <= \result.second <= n;
  ensures    left:        StrictUpperBound(a, 0, \result.first, v);
  ensures    middle:      AllEqual(a, \result.first, \result.second, v);
  ensures    right:       StrictLowerBound(a, \result.second, n, v);
 */
size_type_pair
equal_range(const value_type* a, size_type n, value_type v);
```

Listing 6.11: Formal specification of `equal_range`

The postconditions reflect the conditions listed above and can be expressed using the already introduced predicates `AllEqual` [4.15], `StrictUpperBound` [5.2], and `StrictLowerBound`. In particular, we observe:

- Condition (6.7) becomes postcondition `result`.
- Condition (6.8) becomes postcondition `left`.
- Condition (6.9) becomes postcondition `middle`.
- Condition (6.10) becomes postcondition `right`.

6.5.2. Implementation of `equal_range`

Our first implementation of `equal_range` [6.12] is shown in the following listing. It serves as a baseline implementation and closely mirrors the structure suggested by the specification: it invokes `lower_bound` [6.5] and `upper_bound` [6.8] independently and returns the resulting pair.

```
size_type_pair
equal_range(const value_type* a, size_type n, value_type v)
{
  size_type first  = lower_bound(a, n, v);
  size_type second = upper_bound(a, n, v);
  //@ assert aux: second < n  ==>  v < a[second];
  return make_pair(first, second);
}
```

Listing 6.12: Implementation of `equal_range`

In an early version of this document, the relation `first <= second` was established using the interactive theorem prover Coq. After revisiting this proof, we reformulated the argument by introducing the auxiliary assertion `aux`, which exploits a property already guaranteed by the postcondition of `upper_bound` [6.8]. As a result, both the assertion `aux` and the postcondition `first <= second` can now be verified fully automatically.

6.5.3. Implementation of `equal_range2`

The first implementation of `equal_range` [6.12] performs more work than necessary, since it computes the lower and upper bounds independently. The second implementation, `equal_range2` [6.13], follows a more typical and efficient C++ approach.

The algorithm first performs a binary-search–style range reduction that narrows the search interval as much as possible with respect to the argument `v`. Only once this reduced range has been determined are `upper_bound` and `lower_bound` applied to the corresponding subranges. In this way, duplicate comparisons are avoided while preserving the same observable behavior and the same formal specification.

Due to the higher code complexity of the second implementation, additional assertions had to be inserted in order to ensure that Frama-C/WP is able to verify the correctness of the code. In contrast to the first implementation, these assertions do not concern the algorithmic behavior itself, but rather properties of pointer arithmetic and shifting base pointers.

```
size_type_pair
equal_range2(const value_type* a, size_type n, value_type v)
{
  size_type first  = 0u;
  size_type middle = 0u;
  size_type last   = n;

  /*@
    loop invariant bounds: 0 <= first <= last <= n;
    loop invariant left:   StrictUpperBound(a, 0, first, v);
    loop invariant right:  StrictLowerBound(a, last, n, v);
    loop assigns first, last, middle;
    loop variant last - first;
   */
  while (last > first) {
    middle = first + (last - first) / 2u;

    if (a[middle] < v) {
      first = middle + 1u;
    }
    else if (v < a[middle]) {
      last = middle;
    }
    else {
      break;
    }
  }

  if (first < last) {
    //@ assert increasing: Increasing(a, first, middle);
    size_type left = first + lower_bound(a + first, middle - first, v);
    //@ assert middle: LowerBound(a, left, middle, v);
    //@ assert left:   StrictUpperBound(a, first, left, v);
    ++middle;
    //@ assert increasing: Increasing(a, middle, last);
    size_type right = middle + upper_bound(a + middle, last - middle, v);
    //@ assert middle: UpperBound(a, middle, right, v);
    //@ assert right:  StrictLowerBound(a, right, last, v);
    //@ assert middle: AllEqual(a, left, right, v);
    return make_pair(left, right);
  }
  else {
    return make_pair(first, first);
  }
}
```

Listing 6.13: Implementation of `equal_range2`

All of these assertions fall into three closely related groups, which are briefly discussed below. To enable the automatic verification of the corresponding proof obligations, we introduce a small collection of auxiliary lemmas, summarized under the name `ArrayBoundsShift` [6.14]. These lemmas capture general facts about array bounds under pointer shifts and can be reused independently of the present algorithm.

The `increasing` properties. Both `lower_bound` and `upper_bound` require that they operate on increasingly ordered arrays. This requirement also applies to `equal_range`. However, in the second implementation a more specific formulation is needed for the subarrays on which these functions are invoked. In particular, using the predicate `Increasing` to write

```
        Increasing(a, middle, last)
```

```
/*@
  axiomatic ArrayBoundsShift
  {
    lemma LowerBound_Shift{L}:
      \forall value_type *a, val, integer b, c, d;
        LowerBound{L}(a+b, c,   d,   val)  ==>
        LowerBound{L}(a,   c+b, d+b, val);

    lemma StrictLowerBound_Shift{L}:
      \forall value_type *a, val, integer b, c, d;
        StrictLowerBound{L}(a+b, c,   d,   val)  ==>
        StrictLowerBound{L}(a,   c+b, d+b, val);

    lemma UpperBound_Shift{L}:
      \forall value_type *a, val, integer b, c;
        UpperBound{L}(a+b,  0, c-b, val)  ==>
        UpperBound{L}(a,    b, c,   val);

    lemma StrictUpperBound_Shift{L}:
      \forall value_type *a, val, integer b, c;
        StrictUpperBound{L}(a+b, 0, c-b, val)  ==>
        StrictUpperBound{L}(a,   b, c,   val);
  }
*/
```

Listing 6.14: The logic definition(s) `ArrayBoundsShift`

expresses the required ordering property for an intermediate subarray. Making this property explicit allows the automatic provers to discharge the preconditions of the call to `lower_bound`. A corresponding assertion is used before the call to `upper_bound`.

The `strict` and `constant` properties. Part of the postconditions of `equal_range` [6.11] is that the argument `v` is both a strict upper bound and a strict lower bound for the corresponding array segments. However, the calls to `lower_bound` and `upper_bound` only establish the following properties:

```
        StrictUpperBound(a + first, 0, left - first, v)
        StrictLowerBound(a + middle, right - middle, last - middle, v)
```

Taken in isolation, these properties are not sufficient to derive the required postconditions automatically. In each case, a single intermediate assertion is introduced to bridge this gap and to guide the prover towards the desired result.

Closely related to the `strict` properties are the `constant` properties, which capture bounds that are invariant with respect to the current subarray. They guide the prover towards the following assertions:

```
        LowerBound(a, left, n, v)
        UpperBound(a, 0, right, v)
```

Together, the `strict` and `constant` properties allow the postcondition `middle` to be derived automatically.

6.6. The `binary_search` algorithm

The `binary_search` algorithm is one of the four binary search algorithms of the C++ Standard Library [19, §28.7.3.4]. For the purposes of this chapter, we adapt the generic, iterator-based interface of the standard algorithm to operate on a contiguous array of elements of type `value_type`. Its signature therefore reads:

```
bool binary_search(const value_type* a, size_type n, value_type v);
```

The function `binary_search` requires that the input array `a[0..n-1]` is sorted in increasing order. It returns **`true`** if and only if there exists an index `i` such that `a[i] == v`. More precisely, the C++ Standard Library specifies this condition in terms of the ordering relation, requiring that both `a[i] <= v` and `v <= a[i]` hold. For our definition of `value_type` (see §2.3), this is equivalent to the equality `a[i] == v`.

The following figure illustrates the behavior of `binary_search` on several example arrays.

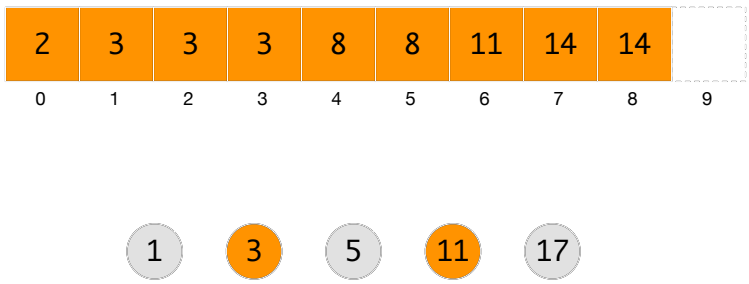

Figure 6.15.: Some examples for `binary_search`

In contrast to the other binary search algorithms discussed in this chapter, `binary_search` does not return a specific array position or range, but only a boolean result. For this reason, no arrows are required in Figure 6.15. Instead, the colors orange and grey of the sought-after values indicate whether the algorithm returns **`true`** or **`false`**, respectively.

6.6.1. Formal specification of `binary_search`

The ACSL specification of `binary_search` [6.16] is shown in the following listing. Conceptually, the result of `binary_search` must express the existence of an index at which the searched value occurs in the input array. At the level of ACSL formulas, this condition can be written as

```
\exists integer i; 0 <= i < n && a[i] == v;
```

While this formulation is logically correct, repeating such existential expressions verbatim across multiple specifications is undesirable. As discussed earlier in this report, careful *specification and proof engineering* aims at factoring out recurring logical patterns into named predicates. This improves readability, promotes uniformity across specifications, and reduces the effort required to maintain and adapt proofs.

For this reason, we reuse the predicate `SomeEqual` [4.4], which captures the above existential condition once and for all. Using this predicate leads to a more concise and robust specification of `binary_search` and serves as a further illustration of the benefits of systematic abstraction at the specification level.

```
/*@
  requires   valid:       \valid_read(a + (0..n-1));
  requires   increasing:  Increasing(a, n);

  terminates              \true;
  exits                   \false;
  assigns                 \nothing;

  ensures    result:      \result <==> SomeEqual(a, n, v);
*/
bool
binary_search(const value_type* a, size_type n, value_type v);
```

Listing 6.16: Formal specification of `binary_search`

It is instructive to compare the specification of `binary_search` [6.16] with that of `find2` [4.5]. At the level of functional behavior, both algorithms serve the same purpose: they determine whether a given value is contained in an array. Consequently, their ACSL specifications are closely related.

By contrast, the essential difference between the two algorithms lies in their complexity guarantees. The C++ Standard Library requires `find` to have *linear* complexity, whereas `binary_search` must have *logarithmic* complexity. Such asymptotic complexity requirements, however, are beyond the *current expressive power* of ACSL and therefore cannot be captured directly in the specification.

6.6.2. Implementation of `binary_search`

Our implementation of `binary_search` [6.17] is deliberately simple and closely follows the structure suggested by the specification. It relies on a single call to `lower_bound` [6.5] to identify a candidate position at which the searched value may occur.

```
bool
binary_search(const value_type* a, size_type n, value_type v)
{
  const size_type i = lower_bound(a, n, v);
  return (i < n) && (a[i] <= v);
}
```

Listing 6.17: Implementation of `binary_search`

Recall that `lower_bound` returns the smallest index `i` such that `v <= a[i]` holds, or `n` if no such index exists. Consequently, if `lower_bound` returns an index `0 <= i < n` and, in addition, `a[i] <= v` holds, then both ordering relations `a[i] <= v` and `v <= a[i]` are satisfied, which implies that `a[i] == v`.

The implementation therefore checks whether the index returned by `lower_bound` lies within the array bounds and whether the corresponding array element is equal to the searched value. Based on this test, it returns either **`true`** or **`false`**, in accordance with the specification.

Part III.

Mutating and numeric algorithms

7. Mutating algorithms

Let us now turn our attention to another class of algorithms, viz. *mutating* algorithms of the C++ Standard Library [19, §28.6], i.e., algorithms that change one or more ranges. In Frama-C, you can explicitly specify that, e.g., entries in an array `a` may be modified by a function `f`, by including the following *assigns clause* into the `f`'s specification:

```
assigns a[0..length-1];
```

The expression `length-1` refers to the value of `length` when `f` is entered, see [14, §2.3.2]. Below are the algorithms we will discuss in this chapter.

- In order to allow for a finer control of which parts of an array, we introduce in §7.1 the auxiliary predicate `Unchanged`.
- `fill` in §7.2 initializes each element of an array by a given fixed value.
- `swap` in §7.3 exchanges two values.
- `swap_ranges` in §7.4 exchanges the contents of the arrays of equal length, element by element. We use this example to present "modular verification", as `swap_ranges` reuses the verified properties of `swap`.
- `copy` in §7.5 copies a source array to a destination array.
- `copy_backward` in §7.6 also copies a source array to a destination array. This version, however, uses another separation condition than `copy`.
- `reverse_copy` and `reverse` in §7.7 and §7.8, respectively, reverse an array. Whereas `reverse_copy` copies the result to a separate destination array, the `reverse` algorithm works in place.
- `rotate_copy` in §7.9 rotates a source array by `m` positions and copies the results to a destination array.
- `rotate` in §7.10 rotates *in place* a source array by `m` positions.
- `replace_copy` and `replace` in §7.11 and §7.12, respectively, substitute each occurrence of a value by a given new value. Whereas `replace_copy` copies the result to a separate array, the `replace` algorithm works in place.
- `remove_copy` and `remove` in §7.13–§7.16 *filter* all occurrences of a given value from an array. Whereas `remove_copy` copies the result to a separate array, the `remove` algorithm works in place. Note that we provide altogether three versions of how to specify `remove_copy`. This shall help the reader to understand that finding appropriate contracts is an iterative process and that it is usually a good idea to *not* strive for a "complete" contract right from the beginning.
- `shuffle` in §7.17 randomly reorders the elements of an array thereby relying on the simple random number generator `random_number` in §7.18.

7.1. The predicate `Unchanged`

Many of the algorithms in this section iterate sequentially over one or several sequences. For the verification of such algorithms it is often important to express that a section of an array, or the complete array, have remained *unchanged*. As this cannot always be expressed by an `assigns` clause, we introduce in the following listing the overloaded predicate `Unchanged` [7.1]. The expression `Unchanged{K,L}(a,m,n)` is true if the range `a[m..n-1]` in state `K` is element-wise equal to that range in state `L`.

```
/*@
  axiomatic Unchanged
  {
    predicate
    Unchanged{K,L}(value_type* a, integer m, integer n) =
      \forall integer i; m <= i < n ==>  \at(a[i],K) == \at(a[i],L);

    predicate
    Unchanged{K,L}(value_type* a, integer n) = Unchanged{K,L}(a, 0, n);
  }
*/
```

Listing 7.1: The logic definition(s) `Unchanged`

In some situations we use the predicate `ArrayUpdate`, which relies on the predicate `Unchanged` and the the logic function `At` [7.49], to concisely describe which parts of an array have changed or remained unchanged when updating an individual array element.

```
/*@
  axiomatic ArrayUpdate
  {
    predicate
      ArrayUpdate{K,L}(value_type* a, integer n, integer i, value_type v) =
        0 <= i < n                  &&
        Unchanged{K,L}(a, 0, i)     &&
        Unchanged{K,L}(a, i+1, n)   &&
        At{K}(a, i) != v            &&
        At{L}(a, i) == v;

    lemma ArrayUpdate_Shrink{K,L}:
      \forall value_type *a, v, integer n, i;
        0 <= i < n-1                    ==>
        ArrayUpdate{K,L}(a, n, i, v)    ==>
        ArrayUpdate{K,L}(a, n-1, i, v);

    lemma ArrayUpdate_UpperBound{K,L}:
      \forall value_type *a, v, w, integer n, i;
        ArrayUpdate{K,L}(a, n, i, v)    ==>
        v <= w                          ==>
        UpperBound{K}(a, n, w)          ==>
        UpperBound{L}(a, n, w);
  }
*/
```

Listing 7.2: The logic definition(s) `ArrayUpdate`

In the following listing we show a few lemmas for `Unchanged` [7.1] that we need for the verification of various algorithms.

```
/*@
  axiomatic UnchangedLemmas
  {
    lemma Unchanged_Shrink{K,L}:
      \forall value_type *a, integer m, n, p, q;
         m <= p <= q <= n          ==>
         Unchanged{K,L}(a, m, n)   ==>
         Unchanged{K,L}(a, p, q);

    lemma Unchanged_Extend{K,L}:
      \forall value_type *a, integer n;
        Unchanged{K,L}(a, n)         ==>
        \at(a[n],K) == \at(a[n],L)   ==>
        Unchanged{K,L}(a, n+1);

    lemma Unchanged_Shift{K,L}:
      \forall value_type *a, integer p, q, r;
        Unchanged{K,L}(a+p, q, r)  ==>  Unchanged{K,L}(a, p+q, p+r);

    lemma Unchanged_Symmetric{K,L}:
      \forall value_type *a, integer m, n;
        Unchanged{K,L}(a, m, n)  ==>
        Unchanged{L,K}(a, m, n);

    lemma Unchanged_Transitive{K,L,M}:
      \forall value_type *a, integer m, n;
        Unchanged{K,L}(a, m, n)  ==>
        Unchanged{L,M}(a, m, n)  ==>
        Unchanged{K,M}(a, m, n);
  }
*/
```

Listing 7.3: The logic definition(s) `UnchangedLemmas`

- Lemma `Unchanged_Shrink` [7.3] states that if the range `a[m..n-1]` does not change when going from state `K` to state `L`, then `a[p..q-1]` does not change either, provided the latter is a subrange of the former, i.e. provided $0 \leq m \leq p \leq q \leq n$ holds.
- Lemma `Unchanged_Extend` [7.3] expresses the simple fact that "unchangedness" is an inductive property.
- Lemma `Unchanged_Shift` [7.3] states how `Unchanged` behaves under pointer additions.
- Lemmas `Unchanged_Symmetric` [7.3] and `Unchanged_Transitive` [7.3] express respectively the symmetry and transitivity of `Unchanged` with respect to program states.

7.2. The `fill` algorithm

The `fill` algorithm in the C++ Standard Library [19, §28.6.6] initializes general sequences with a particular value. The signature of our modified variant reads:

```
void fill(value_type* a, size_type n, value_type v);
```

7.2.1. Formal specification of `fill`

The following listing shows the formal specification of `fill` [7.4]. We can express the postcondition of `fill` simply by using the overloaded predicate `AllEqual` [4.15].

```
/*@
  requires   valid:   \valid(a + (0..n-1));

  terminates           \true;
  exits                \false;
  assigns              a[0..n-1];

  ensures    constant: AllEqual(a, n, v);
*/
void
fill(value_type* a, size_type n, value_type v);
```

Listing 7.4: Formal specification of `fill`

The `assigns`-clauses formalize that `fill` modifies only the entries of the range `a[0..n-1]`. In general, when more than one *assigns clause* appears in a function's specification, it is permitted to modify any of the referenced memory locations. However, if no *assigns clause* appears at all, the function is free to modify any memory location, see [14, §2.3.2]. To forbid a function to do any modifications outside its scope, a clause `assigns \nothing;` must be used, as we practised in the example specifications in Chapter 4.

7.2.2. Implementation of `fill`

The implementation of `fill` [7.5] comes with the loop invariant `constant` expresses that for each iteration the array is *filled* with the value of `v` up to the index `i` of the iteration. Note that we use here again the predicate `AllEqual` [4.15].

```
void
fill(value_type* a, size_type n,  value_type v)
{
  /*@
    loop invariant bound:    0 <= i <= n;
    loop invariant constant: AllEqual(a, i, v);
    loop assigns i, a[0..n-1];
    loop variant n-i;
  */
  for (size_type i = 0u; i < n; ++i) {
    a[i] = v;
  }
}
```

Listing 7.5: Implementation of `fill`

7.3. The `swap` algorithm

The `swap` algorithm [19, §28.6.3] in the C++ Standard Library exchanges the contents of two variables. Similarly, the `iter_swap` algorithm [19, §28.6.3] exchanges the contents referenced by two pointers. Since C and hence ACSL, does not support an `&` type constructor ("declarator"), we will present an algorithm that processes pointers and refer to it as `swap`.

7.3.1. Formal specification of `swap`

The contract of `swap` [7.6] is shown in the following listing. The preconditions state that both pointer arguments of `swap` must be dereferenceable.

```
/*@
  requires   valid:     \valid(p);
  requires   valid:     \valid(q);

  terminates            \true;
  exits                 \false;
  assigns               *p, *q;

  ensures    exchange:  *p == \old(*q);
  ensures    exchange:  *q == \old(*p);
*/
void
swap(value_type* p, value_type* q);
```

Listing 7.6: Formal specification of `swap`

Upon termination of `swap` the entries must be mutually exchanged. The expression `\old(*p)` refers to the value of `*p` before `swap` has be called. By default, a postcondition refers the values after the functions has been terminated.

7.3.2. Implementation of `swap`

The following listing shows the straight-forward implementation of `swap` [7.7]. No interspersed ACSL annotations are needed achieve a verification by Frama-C/WP.

```
void
swap(value_type* p, value_type* q)
{
  value_type save = *p;
  *p = *q;
  *q = save;
}
```

Listing 7.7: Implementation of `swap`

7.4. The swap_ranges algorithm

The swap_ranges algorithm in the C++ Standard Library [19, §28.6.3] exchanges the contents of two expressed ranges element-wise. After translating C++ reference types and iterators to C, our version of the original signature reads:

```
void swap_ranges(value_type* a, size_type n, value_type* b);
```

We do not return a value since it would equal n, anyway.

7.4.1. Formal specification of swap_ranges

The following listing shows a specification for the swap_ranges [7.8] algorithm.

```
/*@
  requires   valid:  \valid(a + (0..n-1));
  requires   valid:  \valid(b + (0..n-1));
  requires   sep:    \separated(a+(0..n-1), b+(0..n-1));

  terminates         \true;
  exits              \false;
  assigns            a[0..n-1];
  assigns            b[0..n-1];

  ensures    equal:  Equal{Old,Here}(a, n, b);
  ensures    equal:  Equal{Old,Here}(b, n, a);
*/
void
swap_ranges(value_type* a, size_type n, value_type* b);
```

Listing 7.8: Formal specification of swap_ranges

The swap_ranges algorithm works correctly only if a and b do not overlap. Because of that fact we use the clause sep to tell Frama-C that a and b must not overlap.

With the assigns-clause we postulate that the swap_ranges algorithm alters the elements contained in two distinct ranges, modifying the corresponding elements and nothing else.

The postconditions of swap_ranges specify that the content of each element in its post-state must equal the pre-state of its counterpart. We can use the predicate Equal [4.28] together with the label Old and Here to express the postcondition of swap_ranges. In our specification, for example, we specify that the array a in the memory state that corresponds to the label Here is equal to the array b at the label Old. Since we are specifying a postcondition Here refers to the post-state of swap_ranges whereas Old refers to the pre-state.

7.4.2. Implementation of `swap_ranges`

The implementation of `swap_ranges` [7.9] together with the necessary loop annotations is shown in the following listing. Unsurprisingly, we are repeatedly calling `swap` [7.6].

```
void
swap_ranges(value_type* a, size_type n, value_type* b)
{
  /*@
    loop invariant bound:  0 <= i <= n;
    loop invariant equal:  Equal{Pre,Here}(a, i, b);
    loop invariant equal:  Equal{Pre,Here}(b, i, a);

    loop invariant unchanged:  Unchanged{Pre,Here}(a, i, n);
    loop invariant unchanged:  Unchanged{Pre,Here}(b, i, n);

    loop assigns i, a[0..n-1], b[0..n-1];
    loop variant n-i;
  */
  for (size_type i = 0u; i < n; ++i) {
    swap(a + i, b + i);
  }
}
```

Listing 7.9: Implementation of `swap_ranges`

For the postcondition `swap_ranges` [7.8] to hold, our loop invariants must ensure that at each iteration all of the corresponding elements that have already been visited are swapped.

Note that there are two additional loop invariants which claim that all the elements that have not visited yet equal their original values. This annotation allows us to prove the postconditions of `swap_ranges` despite the fact that the loop assigns is coarser than it should be. The predicate `Unchanged` [7.1] is used to express this property.

7.5. The copy algorithm

The copy algorithm in the C++ Standard Library [19, §28.6.1] implements a duplication algorithm for general sequences. For our purposes we have modified the generic implementation to that of a range of type value_type. The signature now reads:

```
void copy(const value_type* a, size_type n, value_type* b);
```

Informally, the function copies every element from the source range a[0..n-1] to the destination range b[0..n-1], as shown in Figure 7.10.

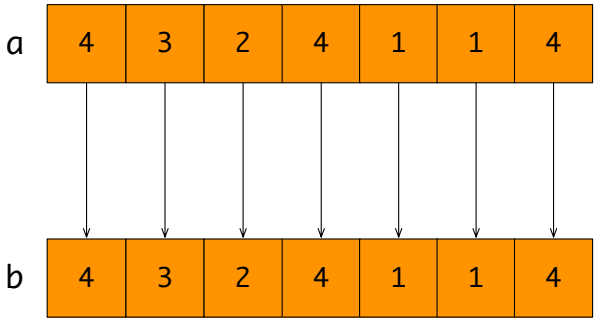

Figure 7.10.: Effects of copy

7.5.1. Formal specification of copy

Figure 7.10 might suggest that the ranges a[0..n-1] and b[0..n-1] must not overlap. However, since the informal specification requires that elements are copied in the order of increasing indices only a weaker condition is necessary. To be more specific, it is required that the pointer b does not refer to elements of a[0..n-1] as shown in the example in Figure 7.11.

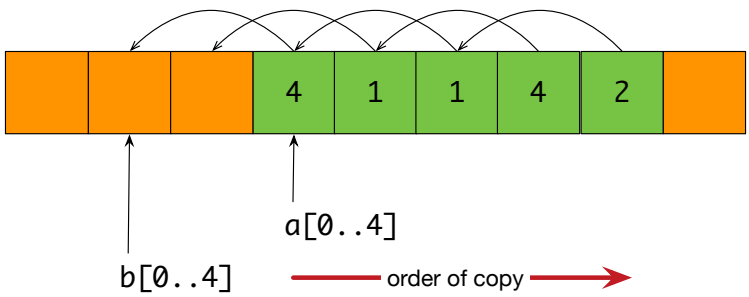

Figure 7.11.: Possible overlap of copy ranges

The specification of copy is shown in the following listing. The copy algorithm expects that the ranges a and b are valid for reading and writing, respectively. Note the precondition sep that expresses the previously discussed non-overlapping property.

```
/*@
  requires   valid:  \valid_read(a + (0..n-1));
  requires   valid:  \valid(b + (0..n-1));
  requires   sep:    \separated(a + (0..n-1), b);

  terminates         \true;
  exits              \false;
  assigns            b[0..n-1];

  ensures    equal:  Equal{Old,Here}(a, n, b);
*/
void
copy(const value_type* a, const size_type n, value_type* b);
```

Listing 7.12: Formal specification of `copy`

Again, we can use the `Equal` [4.28] predicate to express that the array `a` equals `b` after `copy` has been called. Nothing else must be altered. To state this we use the `assigns`-clause.

7.5.2. Implementation of `copy`

The following listing shows an implementation of the `copy` function.

```
void
copy(const value_type* a, size_type n, value_type* b)
{
  /*@
    loop invariant bound:     0 <= i <= n;
    loop invariant equal:     Equal{Pre,Here}(a, i, b);
    loop invariant unchanged: Unchanged{Pre,Here}(a, i, n);
    loop assigns   i, b[0..n-1];
    loop variant n-i;
  */
  for (size_type i = 0u; i < n; ++i) {
    b[i] = a[i];
  }
}
```

Listing 7.13: Implementation of `copy`

For the postcondition `equal` to be true, we must ensure that for every index `i`, the value `a[i]` must not yet have been changed before it is copied to `b[i]`. We express this by using the `Unchanged` predicate.[11]

The `assigns` clause ensures that nothing but the range `b[0..n-1]` and the loop variable `i` is modified. Keep in mind, however, that parts of the source range `a[0..n-1]` might change due to its potential overlap with the destination range.

[11] Alternatively, this could also be expressed by changing the `loop assigns` clause to `i, b[0..i-1]`; however, Frama-C doesn't yet support `loop assigns` clauses containing the loop variable.

7.6. The copy_backward algorithm

The copy_backward algorithm in the C++ Standard Library [19, §28.6.1] implements another duplication algorithm for general sequences. For our purposes we have modified the generic implementation to that of a range of type value_type. The signature now reads:

```
void copy_backward(const value_type* a, size_type n, value_type* b);
```

The main reason for the existence of copy_backward is to allow copying when the start of the destination range a[0..n-1] is contained in the source range b[0..n-1]. In this case, copy can't be employed since its precondition sep is violated, as can be seen in the contract of copy [7.12].

The informal specification of copy_backward states that copying starts at the end of the source range. For this to work, however, the pointer b+n must not be contained in the source range. Note that the order of operation (or procedure) calls cannot be specified in ACSL.[12] A similar remark about order of operations tacitly applied to earlier functions as well, e.g. to copy, where the C++ order was prescribed by confining the signature to a ForwardIterator.

Figure 7.14 gives an example where copy_backward, but *not* copy, can be applied.

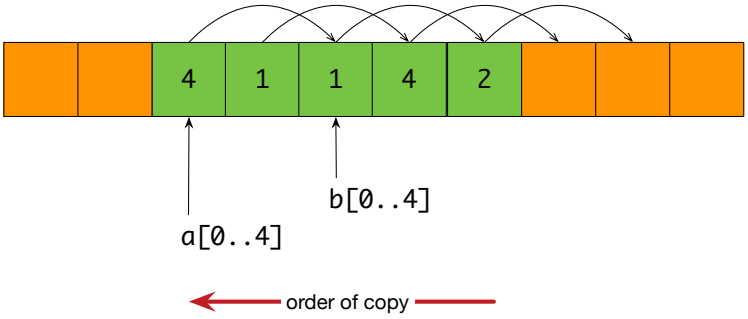

Figure 7.14.: Possible overlap of copy_backward ranges

Note that in the original signature the argument b refers to one past the end of the destination range. Here, however, it refers to its start. The reason for this change is that in C++ copy_backward is defined for *bidirectional iterators* which do not provide random access operations such as adding or subtracting an index. Since our C version works on pointers we do not consider it as necessary to use the one past the end pointer.

7.6.1. Formal specification of copy_backward

The specification of copy_backward is shown in the following listing. The copy_backward algorithm expects that the ranges a[0..n-1] and b[0..n-1] are valid for reading and writing, respectively. Precondition sep formalizes the constraints on the overlap of the source and destination ranges as discussed at the beginning of this section.

[12] The Aoraï specification language and the corresponding Frama-C plugin are provided to specify and verify temporal properties of code; however, they are beyond the scope of this tutorial.

```
/*@
  requires   valid:  \valid_read(a + (0..n-1));
  requires   valid:  \valid(b + (0..n-1));
  requires   sep:    \separated(a + (0..n-1), b + n);

  terminates         \true;
  exits              \false;
  assigns            b[0..n-1];

  ensures    equal:  Equal{Old,Here}(a, n, b);
*/
void
copy_backward(const value_type* a, size_type n, value_type* b);
```

Listing 7.15: Formal specification of `copy_backward`

The function `copy_backward` assigns the elements from the source range `a` to the destination range `b`, modifying the memory of the elements pointed to by `b`. Again, we can use the `Equal` [4.28] predicate to express that the array `a` equals `b` after `copy_backward` has been called.

7.6.2. Implementation of `copy_backward`

The following listing shows an implementation of the `copy_backward` function.

```
void
copy_backward(const value_type* a, size_type n, value_type* b)
{
  /*@
    loop invariant bound:     0 <= i <= n;
    loop invariant equal:     Equal{Pre,Here}(a, i, n, b);
    loop invariant unchanged: Unchanged{Pre,Here}(a, i);
    loop assigns i, b[0..n-1];
    loop variant i;
   */
  for (size_type i = n; i > 0u; --i) {
    b[i - 1u] = a[i - 1u];
  }
}
```

Listing 7.16: Implementation of `copy_backward`

We have loop invariants similar to `copy`, stating the loop variable's range (`bound`) and the area that has already been copied in each cycle (`equal`).

7.7. The `reverse_copy` algorithm

The `reverse_copy` algorithm of the C++ Standard Library [19, §28.6.10] inverts the order of elements in a sequence. `reverse_copy` does not change the input sequence, and copies its result to the output sequence. For our purposes we have modified the generic implementation to that of a range of type `value_type`. The signature now reads:

```
void reverse_copy(const value_type* a, size_type n, value_type* b);
```

Informally, `reverse_copy` copies the elements from the array `a` into array `b` such that the copy is a reverse of the original array. In order to concisely formalize these conditions we define in the following listing the predicate `Reverse` [7.17] (see also Figure 7.18).

```
/*@
  axiomatic Reverse
  {
    predicate
    Reverse{K,L}(value_type* a, integer n, value_type* b) =
      \forall integer i; 0 <= i < n  ==>  \at(a[i],K) == \at(b[n-1-i], L);

    predicate
    Reverse{K,L}(value_type* a, integer m, integer n,
                 value_type* b, integer p) = Reverse{K,L}(a+m, n-m, b+p);

    predicate
    Reverse{K,L}(value_type* a, integer m, integer n, value_type* b) =
      Reverse{K,L}(a, m, n, b, m);

    predicate
    Reverse{K,L}(value_type* a, integer m, integer n, integer p) =
      Reverse{K,L}(a, m, n, a, p);

    predicate
    Reverse{K,L}(value_type* a, integer m, integer n) =
      Reverse{K,L}(a, m, n, m);

    predicate
    Reverse{K,L}(value_type* a, integer n) = Reverse{K,L}(a, 0, n);
  }
*/
```

Listing 7.17: The logic definition(s) `Reverse`

We also define several overloaded variants of `Reverse` that provide default values for some of the parameters. These overloaded versions enable us to write later more concise ACSL annotations.

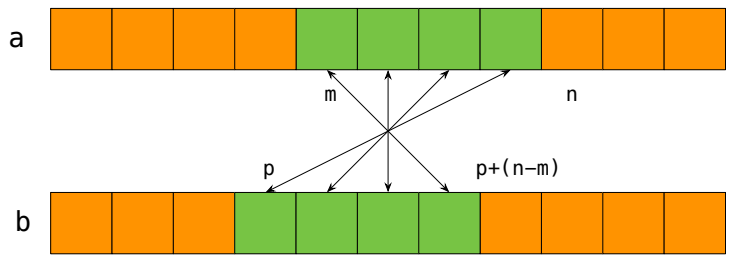

Figure 7.18.: Sketch of predicate `Reverse`

7.7.1. Formal specification of `reverse_copy`

The specification of `reverse_copy` [7.19] is shown in the following listing We use the second version of predicate `Reverse` [7.17] in order to formulate the postcondition of `reverse_copy`.

```
/*@
  requires   valid:      \valid_read(a + (0..n-1));
  requires   valid:      \valid(b + (0..n-1));
  requires   sep:        \separated(a + (0..n-1), b + (0..n-1));

  terminates             \true;
  exits                  \false;
  assigns                b[0..(n-1)];

  ensures    reverse:    Reverse{Old,Here}(a, n, b);
  ensures    unchanged:  Unchanged{Old,Here}(a, n);
*/
void
reverse_copy(const value_type* a, size_type n, value_type* b);
```

Listing 7.19: Formal specification of `reverse_copy`

7.7.2. Implementation of `reverse_copy`

The implementation of `reverse_copy` [7.20] is shown in the following listing. For the postcondition to be true, we must ensure that for every element `i`, the comparison `b[i] == a[n-1-i]` holds. This is formalized by the loop invariant `reverse` where we employ the first version of `Reverse` [7.17].

```
void
reverse_copy(const value_type* a, size_type n, value_type* b)
{
  /*@
    loop invariant bound:   0 <= i <= n;
    loop invariant reverse: Reverse{Here,Pre}(b, 0, i, a, n-i);
    loop assigns i, b[0..n-1];
    loop variant n-i;
  */
  for (size_type i = 0u; i < n; ++i) {
    b[i] = a[n - 1u - i];
  }
}
```

Listing 7.20: Implementation of `reverse_copy`

7.8. The `reverse` algorithm

The `reverse` algorithm of the C++ Standard Library [19, §28.6.10] inverts the order of elements *within* a sequence. The signature of our version of `reverse` reads.

```
void reverse(value_type* a, size_type n);
```

7.8.1. Formal specification of `reverse`

The specification for the `reverse` [7.21] function is shown in the following listing.

```
/*@
  requires   valid:    \valid(a + (0..n-1));

  terminates           \true;
  exits                \false;
  assigns              a[0..n-1];

  ensures    reverse:  Reverse{Old,Here}(a, n);
*/
void
reverse(value_type* a, size_type n);
```

Listing 7.21: Formal specification of `reverse`

7.8.2. Implementation of `reverse`

Since the implementation of `reverse` [7.22] operates *in place* we use `swap` [7.6] in order to exchange the elements of the first half of the array with the corresponding elements of the second half. We reuse the predicates `Reverse` [7.17] and `Unchanged` [7.1] in order to write concise loop invariants.

```
void
reverse(value_type* a, size_type n)
{
  const size_type half = n / 2u;

  //@ assert half: half <= n - half;
  //@ assert half: 2*half <= n <= 2*half + 1;
  /*@
    loop invariant bound:    0 <= i <= half <= n-i;
    loop invariant left:     Reverse{Pre,Here}(a, 0, i, n-i);
    loop invariant middle: Unchanged{Pre,Here}(a, i, n-i);
    loop invariant right:    Reverse{Pre,Here}(a, n-i, n, 0);
    loop assigns i, a[0..n-1];
    loop variant half - i;
  */
  for (size_type i = 0u; i < half; ++i) {
    swap(&a[i], &a[n - 1u - i]);
  }
}
```

Listing 7.22: Implementation of `reverse`

7.9. The `rotate_copy` algorithm

The `rotate_copy` algorithm of the C++ Standard Library [19, §28.6.11] copies, in a particular way, the elements of one sequence of length n into a separate sequence. More precisely,

- the first m elements of the first sequence become the last m elements of the second sequence, and
- the last $n - m$ elements of the first sequence become the first $n - m$ elements of the second sequence.

Figure 7.23 illustrates the effects of `rotate_copy` by highlighting how the initial and final segments of the array `a[0..n-1]` are mapped to corresponding subranges of the array `b[0..n-1]`.

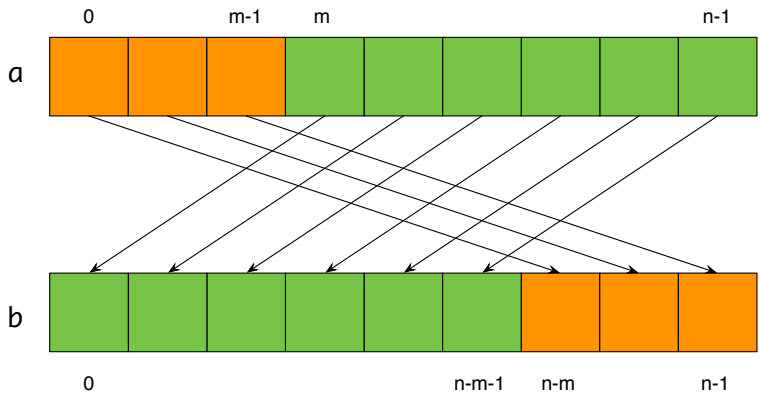

Figure 7.23.: Effects of `rotate_copy`

For our purposes we have modified the generic implementation to that of a range of type `value_type`. The signature now reads:

```
void rotate_copy(const value_type* a, size_type m, size_type n, value_type* b);
```

7.9.1. Formal specification of `rotate_copy`

The specification of `rotate_copy` is shown in the following listing. Note that we require explicitly that both ranges do not overlap and that we are only able to *read* from the range `a[0..n-1]`.

```
/*@
  requires   bound:      0 <= m <= n;
  requires   valid:      \valid_read(a + (0..n-1));
  requires   valid:      \valid(b + (0..n-1));
  requires   sep:        \separated(a + (0..n-1), b + (0..n-1));

  terminates             \true;
  exits                  \false;
  assigns                b[0..(n-1)];

  ensures    left:       Equal{Old,Here}(a, 0, m,   b, n-m);
  ensures    right:      Equal{Old,Here}(a, m, n-m, b, 0);
  ensures    unchanged:  Unchanged{Old,Here}(a, n);
*/
void
rotate_copy(const value_type* a, size_type m, size_type n, value_type* b);
```

Listing 7.24: Formal specification of `rotate_copy`

7.9.2. Implementation of `rotate_copy`

The following listing shows an implementation of the `rotate_copy` function. The implementation simply calls the function `copy` twice.

```
void
rotate_copy(const value_type* a, size_type m, size_type n, value_type* b)
{
  copy(a,  m, b + (n - m));
  copy(a + m, n - m, b);
}
```

Listing 7.25: Implementation of `rotate_copy`

7.10. The `rotate` algorithm

The algorithm `rotate` is an *in-place* variant of the algorithm `rotate_copy` [7.24]. We have modified the generic specification of `rotate` [19, §28.6.11] such that it refers to a range of objects of `value_type`. The signature now reads:

```
size_type rotate(const value_type* a, size_type m, size_type n);
```

7.10.1. Formal specification of `rotate`

Figure 7.26 shows informally the behavior of `rotate`. The figure is of course very similar to the one for `rotate_copy` (see Figure 7.23). The notable difference is that `rotate` operates *in place* of the array `a[0..n-1]`.

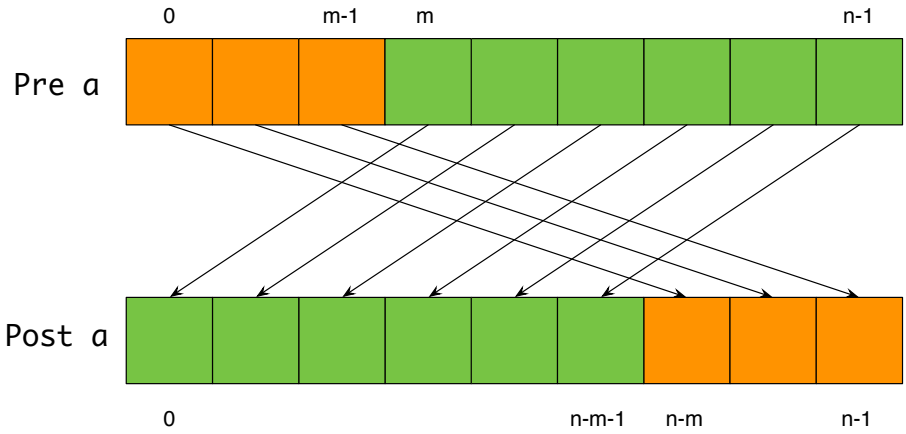

Figure 7.26.: Effects of `rotate`

The specification of `rotate` is shown in the following listing.

```
/*@
  requires   valid:   \valid(a + (0..n-1));
  requires   bound:   m <= n;

  terminates          \true;
  exits               \false;
  assigns             a[0..n-1];

  ensures    result:  \result == n-m;
  ensures    left:    Equal{Old,Here}(a, 0, m, n-m);
  ensures    right:   Equal{Old,Here}(a, m, n, 0);
*/
size_type
rotate(value_type* a, size_type m, size_type n);
```

Listing 7.27: Formal specification of `rotate`

7.10.2. Implementation of `rotate`

The following listing shows an implementation of the `rotate` function together with several ACSL annotations. Actually, there are several ways to implement `rotate`. We have chosen a particularly simple one that is derived from an implementation of `std::rotate` for *bidirectional iterators* [19, §27.2.6] and which essentially consists of several calls to the algorithm `reverse` [7.21].

Note the statement contract of the final call of `reverse` [7.21]. Here we use both the labels `Pre` and `Old` which refer to the pre-states of `reverse` and the function `rotate` itself, respectively.

```
size_type
rotate(value_type* a, size_type m, size_type n)
{
  // if one subrange is empty, then nothings needs to be done
  if ((0u < m) && (m < n)) {
    reverse(a,  m);
    reverse(a + m, n - m);
    /*@
      requires left:   Reverse{Pre,Here}(a, 0, m, 0);
      requires right:  Reverse{Pre,Here}(a, m, n, m);

      assigns          a[0..n-1];

      ensures left:    Reverse{Old,Here}(a, 0, m, n-m);
      ensures right:   Reverse{Old,Here}(a, m, n, 0);
    */
    reverse(a, n);
    //@ assert left:   Equal{Pre,Here}(a, 0, m, n-m);
    //@ assert right:  Equal{Pre,Here}(a, m, n, 0);
  }

  return n - m;
}
```

Listing 7.28: Implementation of `rotate`

7.11. The `replace_copy` algorithm

The `replace_copy` algorithm of the C++ Standard Library [19, §28.6.5] substitutes specific elements from general sequences. Here, the general implementation has been altered to process `value_type` ranges. The new signature reads:

```
size_type replace_copy(const value_type* a, size_type n, value_type* b,
                       value_type v, value_type w);
```

The `replace_copy` algorithm copies the elements from the range `a[0..n]` to range `b[0..n]`, substituting every occurrence of `v` by `w`. The return value is the length of the range. As the length of the range is already a parameter of the function this return value does not contain new information.

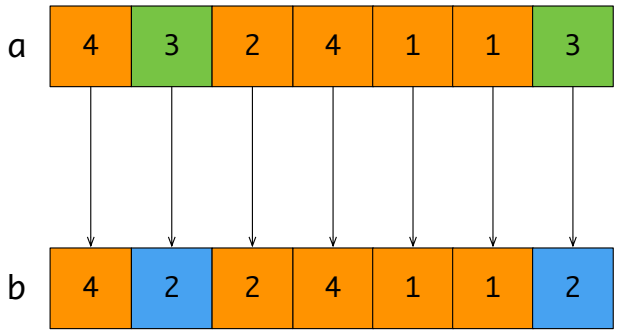

Figure 7.29.: Effects of `replace`

Figure 7.29 shows the behavior of `replace_copy` at hand of an example where all occurrences of the value 3 in `a[0..n-1]` are replaced with the value 2 in `b[0..n-1]`.

7.11.1. The predicate `Replace`

We start with defining in the following listing the predicate `Replace` [7.30] that describes the intended relationship between the input array `a[0..n-1]` and the output array `b[0..n-1]`. Note the introduction of *local bindings* `\let ai = ...` and `\let bi = ...` in the definition of `Replace` (see [14, §2.2]).

```
/*@
  axiomatic Replace
  {
    predicate
    Replace{K,L}(value_type* a, integer n, value_type* b,
                 value_type v, value_type w) =
      \forall integer i; 0 <= i < n  ==>
        \let ai = \at(a[i],K);
        \let bi = \at(b[i],L);
        (ai == v  ==>  bi == w) && (ai != v  ==>  bi == ai) ;

    predicate
    Replace{K,L}(value_type* a, integer n, value_type v, value_type w) =
      Replace{K,L}(a, n, a, v, w);
  }
*/
```

Listing 7.30: The logic definition(s) `Replace`

This listing also contains a second, overloaded version of `Replace` which we will use for the specification of the related in-place algorithm `replace` [7.33].

7.11.2. Formal specification of `replace_copy`

Using predicate `Replace` the specification of `replace_copy` [7.31] is as simple as shown in the following listing. Note that we also require that the input range `a[0..n-1]` and output range `b[0..n-1]` do not overlap.

```
/*@
  requires   valid:     \valid_read(a + (0..n-1));
  requires   valid:      \valid(b + (0..n-1));
  requires   sep:        \separated(a + (0..n-1), b + (0..n-1));

  terminates             \true;
  exits                  \false;
  assigns                b[0..n-1];

  ensures    result:     \result == n;
  ensures    replace:    Replace{Old,Here}(a, n, b, v, w);
  ensures    unchanged:  Unchanged{Old,Here}(a, n);
*/
size_type
replace_copy(const value_type* a, size_type n, value_type* b,
             value_type v, value_type w);
```

Listing 7.31: Formal specification of `replace_copy`

7.11.3. Implementation of `replace_copy`

The implementation (including loop annotations) of `replace_copy` [7.32] is shown in the following listing. Note how the structure of the loop annotations resembles the specification of `replace_copy` [7.31].

```
size_type
replace_copy(const value_type* a, size_type n, value_type* b, value_type v,
             value_type w)
{
  /*@
    loop invariant bounds:    0 <= i <= n;
    loop invariant replace:   Replace{Pre,Here}(a, i, b, v, w);
    loop assigns i, b[0..n-1];
    loop variant n-i;
  */
  for (size_type i = 0u; i < n; ++i) {
    b[i] = (a[i] == v ? w : a[i]);
  }

  return n;
}
```

Listing 7.32: Implementation of `replace_copy`

7.12. The `replace` algorithm

The `replace` algorithm of the C++ Standard Library [19, §28.6.5] substitutes specific values in a general sequence. Here, the general implementation has been altered to process `value_type` ranges. The new signature reads

```
void replace(value_type* a, size_type n, value_type v, value_type w);
```

The `replace` algorithm substitutes all elements from the range `a[0..n-1]` that equal `v` by `w`.

7.12.1. Formal specification of `replace`

Using the second predicate `Replace` [7.30] the specification of `replace` [7.33] can be expressed as in the following listing.

```
/*@
  requires   valid:    \valid(a + (0..n-1));

  terminates           \true;
  exits                \false;
  assigns              a[0..n-1];

  ensures    replace:  Replace{Old,Here}(a, n, v, w);
*/
void
replace(value_type* a, size_type n, value_type v, value_type w);
```

Listing 7.33: Formal specification of `replace`

7.12.2. Implementation of `replace`

The implementation of `replace` [7.34] is shown in the following listing. The loop invariant `unchanged` expresses that when entering iteration `i` the elements `a[i..n-1]` have not yet changed.

```
void
replace(value_type* a, size_type n, value_type v, value_type w)
{
  /*@
    loop invariant bounds:    0 <= i <= n;
    loop invariant replace:   Replace{Pre,Here}(a, i, v, w);
    loop invariant unchanged: Unchanged{Pre,Here}(a, i, n);
    loop assigns i, a[0..n-1];
    loop variant n-i;
  */
  for (size_type i = 0u; i < n; ++i) {
    if (a[i] == v) {
      a[i] = w;
    }
  }
}
```

Listing 7.34: Implementation of `replace`

7.13. The `remove_copy` algorithm (basic contract)

The `remove_copy` algorithm of the C++ Standard Library [19, §28.6.8] copies all elements of a sequence other than a given value. Here, the general implementation has been altered to process `value_type` ranges. The new signature reads:

```
size_type
remove_copy(const value_type* a, size_type n, value_type* b, value_type v);
```

The requirements of `remove_copy` are:

Requirements	Description
Remove Copy Size	The output range has to fit in all the elements of the input range, except the ones that equal the value `v` by `remove_copy`.
Remove Copy Separated	The input range and the output range do not overlap
Remove Copy Elements	The `remove_copy` algorithm copies elements that are not equal to `v` from range `a[0..n-1]` to the range `b[0..\result-1]`.
Remove Copy Stability	The algorithm is stable, that is, the relative order of the elements in `b` is the same as in `a`.
Remove Copy Return	The return value is the length of the resulting range.
Remove Copy Complexity	The algorithm takes *n* comparisons in every case.

Table 7.35.: Properties of `remove_copy`

Figure 7.36 shows an example of how `remove_copy` is supposed to copy elements that differ from `4` from the input range to the output range.

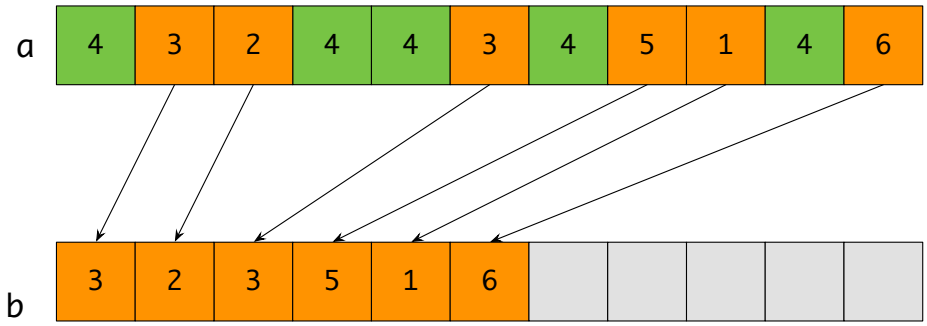

Figure 7.36.: Effects of `remove_copy`

7.13.1. Formal specification of `remove_copy`

The following listing shows our first attempt to specify `remove_copy`. In postcondition `discard` we use of the predicate `NoneEqual` [4.4] to show that the value `v` does not occur in the range `b[0..\result]`.

```
/*@
  requires   valid:      \valid_read(a + (0..n-1));
  requires   valid:      \valid(b + (0..n-1));
  requires   sep:        \separated(a + (0..n-1), b + (0..n-1));

  terminates             \true;
  exits                  \false;
  assigns                b[0..n-1];

  ensures    bound:      0 <= \result <= n;
  ensures    discard:    NoneEqual(b, \result, v);
  ensures    unchanged:  Unchanged{Old,Here}(a, n);
  ensures    unchanged:  Unchanged{Old,Here}(b, \result, n);
*/
size_type
remove_copy(const value_type *a, size_type n, value_type *b, value_type v);
```

Listing 7.37: Formal specification of `remove_copy`

One shortcoming of this specification is that the postcondition `bound` only makes very general and not very precise statements about the number of copied elements. We will address this problem in the contract of `remove_copy2` [7.41]. A more serious shortcoming is, however, that we haven't specified what the relationship between the elements of the input range `a[0..n-1]` and the output range `b[0..\result -1]` looks like. This problem will be tackled in the contract of `remove_copy3` [7.48].

7.13.2. Implementation of `remove_copy`

An implementation of `remove_copy` is shown in the following listing.

```
size_type
remove_copy(const value_type *a, size_type n, value_type *b, value_type v)
{
  size_type k = 0u;

  /*@
    loop invariant bound:     0 <= k <= i <= n;
    loop invariant discard:   NoneEqual(b, k, v);
    loop invariant unchanged: Unchanged{Pre,Here}(b, k, n);
    loop assigns   k, i, b[0..n-1];
    loop variant   n-i;
  */
  for (size_type i = 0u; i < n; ++i) {
    if (a[i] != v) {
      b[k++] = a[i];
    }
  }

  return k;
}
```

Listing 7.38: Implementation of `remove_copy`

Here we also need to add another loop invariant `discard` which basically checks if `v` occurs in `b[0..k]` for each iteration of the loop.

7.14. The `remove_copy2` algorithm (number of copied elements)

In this section we improve the contract of `remove_copy` [7.37] by formally specifying the number `\result` of elements copied by `remove_copy`.

The number of copied elements equals of course the number of elements in the input range `a[0..n-1]` that are different from `v`. One can formally describe this number by relying on the logic function `Count` [4.44].

```
logic integer
CountNotEqual(value_type* a, integer n, value_type v) =  n - Count(a, n, v);
```

In fact, we have used this kind of definition in earlier version of this document. We have found it, however, worthwhile to provide a separate definition of `CountNotEqual` and express the relationship with `Count` as a lemma. This definition is shown in the Listings 7.39 and 7.40.

```
/*@
  axiomatic CountNotEqual
  {
    logic integer
    CountNotEqual(value_type* a, integer m, integer n, value_type v) =
      n <= m ? 0 : CountNotEqual(a, m, n-1, v) + (a[n-1] == v ? 0 : 1);

    logic integer
    CountNotEqual(value_type* a, integer n, value_type v) =
      CountNotEqual(a, 0, n, v);

    lemma CountNotEqual_Empty:
      \forall value_type *a, v, integer m, n;
        n <= m  ==>  CountNotEqual(a, m, n, v) == 0;

    lemma CountNotEqual_Hit:
      \forall value_type *a, v, integer m, n;
        m <= n     ==>
        a[n] != v  ==>
        CountNotEqual(a, m, n+1, v) == CountNotEqual(a, m, n, v) + 1;

    lemma CountNotEqual_Miss:
      \forall value_type *a, v, integer m, n;
        m <= n     ==>
        a[n] == v  ==>
        CountNotEqual(a, m, n+1, v) == CountNotEqual(a, m, n, v);

    lemma CountNotEqual_Lower:
      \forall value_type *a, v, integer m, n;
        m <= n  ==>  0 <= CountNotEqual(a, m, n, v);

    lemma CountNotEqual_Upper:
      \forall value_type *a, v, integer m, n;
        m <= n  ==>  CountNotEqual(a, m, n, v) <= n-m;
```

Listing 7.39: The logic function `CountNotEqual` (1)

The above mentioned relationship with `Count` [4.44] is expressed as lemma `CountNotEqual_Count` [7.39] in the following listing.

```
    lemma CountNotEqual_WeaklyIncreasing:
      \forall value_type *a, v, integer m, n;
        m <= n  ==>  CountNotEqual(a, m, n, v) <= CountNotEqual(a, m, n+1, v);

    lemma CountNotEqual_Increasing:
      \forall value_type *a, v, integer k, m, n;
        m <= k <= n  ==>  CountNotEqual(a, m, k, v) <= CountNotEqual(a, m, n, v);

    lemma CountNotEqual_Unchanged{K,L}:
      \forall value_type *a, v, integer m, n;
        Unchanged{K,L}(a, m, n)  ==>
        CountNotEqual{K}(a, m, n, v) == CountNotEqual{L}(a, m, n, v);

    lemma CountNotEqual_Count:
      \forall value_type *a, v, integer m, n;
        m <= n  ==>  CountNotEqual(a, m, n, v) == n - m - Count(a, m, n, v);

    lemma CountNotEqual_Union:
      \forall value_type *a, v, integer k, m, n;
        0 <= k <= m <= n  ==>
        CountNotEqual(a, k, n, v) ==
        CountNotEqual(a, k, m, v) + CountNotEqual(a, m, n, v);
  }
*/
```

Listing 7.40: The logic function `CountNotEqual` (2)

7.14.1. Formal specification of `remove_copy2`

We extend our formal specification by using `CountNotEqual` [7.39] and add the new postcondition `size`, which states that the returning value of `remove_copy2` equals `CountNotEqual`. The following listing shows the formal specification of `remove_copy2` [7.41].

```
/*@
  requires   valid:      \valid_read(a + (0..n-1));
  requires   valid:      \valid(b + (0..n-1));
  requires   sep:        \separated(a + (0..n-1), b + (0..n-1));

  terminates             \true;
  exits                  \false;
  assigns                b[0..n-1];

  ensures    size:       \result == CountNotEqual(a, n, v);
  ensures    bound:      0 <= \result <= n;
  ensures    discard:    NoneEqual(b, \result, v);
  ensures    unchanged:  Unchanged{Old,Here}(a, n);
  ensures    unchanged:  Unchanged{Old,Here}(b, \result, n);
*/
size_type
remove_copy2(const value_type* a, size_type n, value_type* b, value_type v);
```

Listing 7.41: Formal specification of `remove_copy2`

7.14.2. Implementation of `remove_copy2`

The following listing shows the implementation of our extended of `remove_copy2`. Here we added the loop invariant `size` which corresponds to the postcondition in `remove_copy2` [7.41]. In order to ensure that the loop invariant `size` can be verified we have added the assertions `size` and `unchanged`.

```
size_type
remove_copy2(const value_type* a, size_type n, value_type* b, value_type v)
{
  size_type k = 0u;

  /*@
    loop invariant size:      k == CountNotEqual(a, i, v);
    loop invariant bound:     0 <= k <= i <= n;
    loop invariant discard:   NoneEqual(b, k, v);
    loop invariant unchanged: Unchanged{Pre,Here}(b, k, n);
    loop assigns   k, i, b[0..n-1];
    loop variant   n-i;
  */
  for (size_type i = 0u; i < n; ++i) {
    if (a[i] != v) {
      b[k++] = a[i];
      //@ assert unchanged: Unchanged{LoopCurrent,Here}(a, n);
      //@ assert size:      k == CountNotEqual(a, 0, i+1, v);
    }
  }

  return k;
}
```

Listing 7.42: Implementation of `remove_copy2`

While we now can precisely speak of the number of copied elements, it is still not possible to say something about the exact relationship between the elements of range `a[0..n-1]` and range `b[0..n-1]`. We will address this question the contract of `remove_copy3` [7.48].

7.15. The `remove_copy3` algorithm (final contract)

In this section we extend the contracts of `remove_copy` [7.37] and `remove_copy2` [7.41] by introducing a logic function, which describes the relationship between the elements of input range `a[0..n-1]` and the output range `b[0..\result-1]`. Note that we have shown in the previous section that `\result` equals `CountNotEqual(a, n, v)`.

7.15.1. A closer look on the properties of `remove_copy`

Figure 7.43 shows a modified version of the Figure 7.36. We left out the indices of values that were not copied into the target array. Furthermore we have added a dashed arrow which points to the index that corresponds to the *one past the end* location of the input and output range.

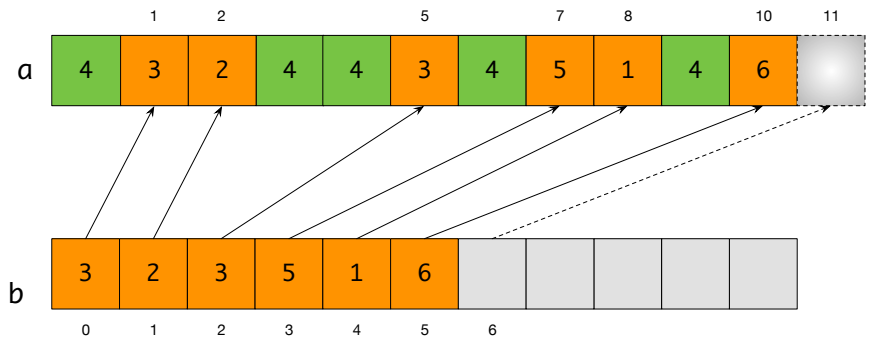

Figure 7.43.: Partitioning the input of `remove_copy`

These arrows between the indices of the array `b` and array `a` define the following sequence p of seven indices. The index of the *one past the end* is underlined. $p = (1, 2, 5, 7, 8, 10, \underline{11})$

More generally, we refer to the sequence p as the *partitioning sequence* associated with `remove_copy` applied to the array `a[0..n-1]`. Let m denote the length of the partitioning sequence. Then we obtain the following **strictly monotone increasing** sequence:

$$0 \leq p_0 < \cdots < p_m = n \tag{7.1}$$

The open index intervals (p_i, p_{i+1}) marks for all i with $0 \leq i < m$ **consecutive ranges** of the value `v` in the source array, that is,

$$a[k] = v \qquad \forall k : p_i < k < p_{i+1} \tag{7.2}$$

Additionally, the half-open interval $[0, p_0)$ also marks a **consecutive range** of the value `v` in the source array:

$$a[k] = v \qquad \forall k : 0 \leq k < p_0 \tag{7.3}$$

Another observation is that

$$a[p_i] \neq v \qquad \forall i : 0 \leq i < m \tag{7.4}$$

Finally, we have

$$a[p_i] = b[i] \qquad \forall i : 0 \leq i < m \tag{7.5}$$

which, together with (7.4), states that the target array does not contain the value `v`:

$$b[i] \neq v \qquad \forall i : 0 \leq i < m \tag{7.6}$$

7.15.2. More lemmas on `CountNotEqual`

Our formalization the properties of §7.15.1 relies on the logic function `CountNotEqual` [7.39]. We also rely on the logic function `FindNotEqual` [4.16] and the lemmas of `CountFindNotEqual` [7.44] in the following listing that provide more facts about `CountNotEqual` and `FindNotEqual`.

```
/*@
  axiomatic CountFindNotEqual
  {
    lemma CountNotEqual_AllEqual:
      \forall value_type *a, v, integer m, n;
        0 <= m <= n             ==>
        AllEqual(a, m, n, v)    ==>
        CountNotEqual(a, m, n, v) == 0;

    lemma CountNotEqual_SomeNotEqual:
      \forall value_type *a, v, integer m, n;
        0 <= m < n                      ==>
        0 < CountNotEqual(a, m, n, v)   ==>
        SomeNotEqual(a, m, n, v);

    lemma CountNotEqual_FindNotEqual:
      \forall value_type *a, v, integer m, n;
        0 <= m < n                      ==>
        0 < CountNotEqual(a, m, n, v)   ==>
        FindNotEqual(a, m, n, v) < n-m;

    lemma CountNotEqual_Zero:
      \forall value_type *a, v, integer m, n;
        0 <= m < n   ==>
        CountNotEqual(a, m, m + FindNotEqual(a, m, n, v), v) == 0;

    lemma CountNotEqual_Decrement:
      \forall value_type *a, v, integer m, n;
        0 <= m < n  ==>
        CountNotEqual(a, m + FindNotEqual(a, m, n, v), n, v) ==
        CountNotEqual(a, 0, n, v) - CountNotEqual(a, 0, m, v);
  }
*/
```

Listing 7.44: The logic definition(s) `CountFindNotEqual`

7.15.3. Formalizing the properties of the partitions

The function `IndexOfNotEqual`, whose axiomatic definition is given in Listings 7.45 and 7.46, defines the partitioning sequence p from §7.15.1. Intuitively, `IndexOfNotEqual (a, n, v, k)` returns the array index associated with the k-th element of $a[0..n-1]$ that is not equal to v.

- Negative values of k correspond to invalid requests and yield -1.
- Out-of-range requests (that is, k greater than or equal to the total number of elements different from v) yield n, the first index past the end of the array.
- For intermediate values, the function is defined inductively: the index for rank k is computed as the successor of the index for rank $k-1$, advanced to the next element that differs from v.

We remind the reader that logic functions (and predicates) must be total, that is, they must be defined for

all possible argument values.

```
/*@
  axiomatic IndexOfNotEqual
  {
    logic integer
    IndexOfNotEqual(value_type* a, integer n, value_type v, integer k) =
      \let count = CountNotEqual(a, n, v);
      \let start = IndexOfNotEqual(a, n, v, k-1) + 1;
      // Negative rank: invalid request
      k < 0 ? -1 :
        // Now we know 0 <= k
        (n <= 0 ? 0 :
          // Now we know 0 < n
          k < count ? start + FindNotEqual(a, start, n, v) : n
        );

    lemma IndexOfNotEqual_Left:
      \forall value_type *a, v, integer n, k;
        k < 0  ==>  IndexOfNotEqual(a, n, v, k) == -1;

    lemma IndexOfNotEqual_Empty:
      \forall value_type *a, v, integer n, k;
        n <= 0 <= k  ==>
        IndexOfNotEqual(a, n, v, k) == 0;

    lemma IndexOfNotEqual_Right:
      \forall value_type *a, v, integer n, k;
        0 <= n                        ==>
        CountNotEqual(a, n, v) <= k  ==>  IndexOfNotEqual(a, n, v, k) == n;

    lemma IndexOfNotEqual_Next:
      \forall value_type *a, v, integer n, k;
        \let x = IndexOfNotEqual(a, n, v, k-1) + 1;
        0 <= n                            ==>
        0 <= k < CountNotEqual(a, n, v)  ==>
        IndexOfNotEqual(a, n, v, k) == x + FindNotEqual(a, x, n, v);

    lemma IndexOfNotEqual_Lower:
      \forall value_type *a, v, integer n, k;
        0 < n                             ==>
        0 <= k < CountNotEqual(a, n, v)  ==>
        0 <= IndexOfNotEqual(a, n, v, k);

    lemma IndexOfNotEqual_Core:
      \forall value_type *a, v, integer n, k;
        \let R = IndexOfNotEqual(a, n, v, k);
        0 < n                             ==>
        0 <= k < CountNotEqual(a, n, v)  ==>
        (R < n      &&
         a[R] != v  &&
         CountNotEqual(a, R, n, v) == CountNotEqual(a, 0, n, v) - k);
```

Listing 7.45: The logic function `IndexOfNotEqual` (1)

```
    lemma IndexOfNotEqual_Upper:
      \forall value_type *a, v, integer n, k;
        0 < n                              ==>
        0 <= k < CountNotEqual(a, n, v)  ==>
        IndexOfNotEqual(a, n, v, k) < n;

    lemma IndexOfNotEqual_NotEqual:
      \forall value_type *a, v, integer n, k;
        0 < n  ==>
        0 <= k < CountNotEqual(a, n, v)  ==>
        a[IndexOfNotEqual(a, n, v, k)] != v;

    lemma IndexOfNotEqual_Count:
      \forall value_type *a, v, integer n, k;
        0 < n                              ==>
        0 <= k < CountNotEqual(a, n, v)  ==>
        CountNotEqual(a, IndexOfNotEqual(a, n, v, k), n, v) ==
        CountNotEqual(a, 0, n, v) - k;

    lemma IndexOfNotEqual_StrictlyWeakIncreasing:
      \forall value_type *a, v, integer n, k;
        0 < n                             ==>
        0 < k < CountNotEqual(a, n, v)  ==>
        IndexOfNotEqual(a, n, v, k-1) < IndexOfNotEqual(a, n, v, k);

    lemma IndexOfNotEqual_Segment:
      \forall value_type *a, v, integer n, k;
        0 < n                              ==>
        0 <= k                             ==>
        k + 1 < CountNotEqual(a, n, v)   ==>
          AllEqual(a, IndexOfNotEqual(a, n, v, k) + 1,
                      IndexOfNotEqual(a, n, v, k+1), v);

    lemma IndexOfNotEqual_Unchanged{K,L}:
      \forall value_type *a, v, integer n, k;
        Unchanged{K,L}(a, n)  ==>
        IndexOfNotEqual{K}(a, n, v, k) == IndexOfNotEqual{L}(a, n, v, k);
  }
*/
```

Listing 7.46: The logic function `IndexOfNotEqual` (2)

The lemmas for `IndexOfNotEqual` correspond to the properties of §7.15.1 in the following way.

- Property (7.1) is expressed by the lemmas
  - `IndexOfNotEqual_Empty`,
  - `IndexOfNotEqual_Left`,
  - `IndexOfNotEqual_Right`, and
  - `IndexOfNotEqual_StrictlyWeakIncreasing`,

  which together capture boundary cases and monotonicity of the partitioning sequence.
- Properties (7.2) and (7.3) are formalized by the lemma `IndexOfNotEqual_Segment`, which characterizes the segments of consecutive occurrences of the value `v`.
- Property (7.4) is expressed by the lemma `IndexOfNotEqual_NotEqual`.

- Property (7.5) is formulated using the predicate `Remove` [7.47], relating the partitioning sequence to the construction of the target array.

We single out the lemma `IndexOfNotEqual_Core`, which subsumes the statements of the subsequent lemmas

- `IndexOfNotEqual_Upper`,
- `IndexOfNotEqual_NotEqual`, and
- `IndexOfNotEqual_Count`.

Although these three lemmas do not add any new information, we retain them because each corresponds directly to an individual property discussed in §7.15.1, and thus serves as a convenient reference point.

The presence of the lemma `IndexOfNotEqual_Core` itself may nevertheless invite the question why such a combined statement is needed at all. The reason is methodological: during verification, the individual properties turned out to be so tightly interdependent that they could not be established in isolation. Only their joint formulation admitted a successful proof, of which `IndexOfNotEqual_Core` is the formal embodiment.

7.15.4. The predicate `Remove`

The predicate `Remove` [7.47] primarily serves in order to improve the readability of our specification `remove_copy3` [7.48].

```
/*@
  axiomatic Remove
  {
    predicate
    Remove{K,L}(value_type* a, integer n, integer i, value_type* b, value_type v) =
      \forall integer k; 0 <= k < CountNotEqual{K}(a, i, v)  ==>
        \let j = IndexOfNotEqual{K}(a, n, v, k);
        \at(b[k],L) == \at(a[j],K);

    predicate
    Remove{K,L}(value_type* a, integer n, value_type* b, value_type v) =
      Remove{K,L}(a, n, n, b, v);

    predicate
    Remove{K,L}(value_type* a, integer n, integer i, value_type v) =
      \forall integer k; 0 <= k < CountNotEqual{K}(a, i, v)  ==>
        \let j = IndexOfNotEqual{K}(a, n, v, k);
        \at(a[k],L) == \at(a[j],K);

    predicate
    Remove{K,L}(value_type* a, integer n, value_type v) =
      Remove{K,L}(a, n, n, v);
  }
*/
```

Listing 7.47: The logic definition(s) `Remove`

As mentioned before this predicate encapsulates the Property (7.5) from §7.15.1. Note that `Remove` [7.47] also contains an overloaded version of `Remove` which will be used for the specification of the *in-place* variant `remove` [7.52] of `remove_copy`.

7.15.5. Formal specification of `remove_copy3`

The following listing shows the formal specification of `remove_copy` [7.37]. The additional postcondition `remove` makes use of the predicate `Remove` [7.47] which we have just described. Furthermore, we have again the postcondition `unchanged` which states that the source array `a[0..n-1]` does not change.

```
/*@
  requires   valid:      \valid_read(a + (0..n-1));
  requires   valid:      \valid(b + (0..n-1));
  requires   sep:        \separated(a + (0..n-1), b + (0..n-1));

  terminates             \true;
  exits                  \false;
  assigns                b[0..n-1];

  ensures    size:       \result == CountNotEqual{Old}(a, n, v);
  ensures    bound:      0 <= \result <= n;
  ensures    remove:     Remove{Old,Here}(a, n, b, v);
  ensures    discard:    NoneEqual(b, \result, v);
  ensures    unchanged:  Unchanged{Old,Here}(a, n);
  ensures    unchanged:  Unchanged{Old,Here}(b, \result, n);
*/
size_type
remove_copy3(const value_type* a, size_type n, value_type* b, value_type v);
```

Listing 7.48: Formal specification of `remove_copy3`

7.15.6. Implementation of `remove_copy3`

We discuss now some aspects of the implementation of `remove_copy3` [7.50]. We introduce the loop invariant `mapping`. This invariant states that the variable `i` will always be smaller or equal to the result of `IndexOfNotEqual(a, n, v, k)`. We also add the assertion `mapping` to our implementation as stepping stone for the provers to verify the correctness of this loop invariant.

Somewhat surprisingly, in order to reduce excessive verification times we had to add an else-branch to our implementation that besides the assertion `unchanged` is empty.

Regarding the assertion `update`, one might wonder why we do not simply write `\at(a[i], Pre)`. However, this expression would be wrong because the index `i` would then be interpreted as `\at(i,Pre)` which doesn't makes sense for a local variable. Frama-C/WP consequently rejects this expression with the following error message.

```
Warning: unbound logic variable i. Ignoring code annotation
```

We could explicitly refer to the current value of `i` by using the subexpression `\at(i,Here)` inside the assertion `update`. We felt, however, tow introduce the predicate `At` [7.49] to simplify the comparison of array elements in programme states where the particular index variable isn't visible.

```
/*@
  axiomatic At
  {
    logic value_type At{L}(value_type* x, integer i) = \at(x[i],L);
  }
*/
```

Listing 7.49: The logic definition(s) `At`

The second argument `At` is interpreted at the programme point here it appears, that is, `Here`. Using this auxiliary logic function the assertion `update` is arguably more readable.

```
size_type
remove_copy3(const value_type* a, size_type n, value_type* b, value_type v)
{
  size_type k = 0u;

  /*@
    loop invariant size:       k == CountNotEqual{Pre}(a,i,v);
    loop invariant bound:      0 <= k <= i <= n;
    loop invariant remove:     Remove{Pre,Here}(a, n, i, b, v);
    loop invariant discard:    NoneEqual(b, k, v);
    loop invariant interval:   IndexOfNotEqual{Pre}(a, n, v, k-1) <= i;
    loop invariant interval:   i <= IndexOfNotEqual{Pre}(a, n, v, k);
    loop invariant unchanged:  Unchanged{Pre,Here}(a, n);
    loop invariant unchanged:  Unchanged{Pre,Here}(b, k, n);
    loop assigns   k, i, b[0..n-1];
    loop variant   n-i;
  */
  for (size_type i = 0u; i < n; ++i) {
    if (a[i] != v) {
      b[k++] = a[i];
      //@ assert size:      k == CountNotEqual{Pre}(a, i+1, v);
      //@ assert update:    b[k-1] == At{Pre}(a, i);
      //@ assert interval:  i == IndexOfNotEqual{Pre}(a, n, v, k-1);
      //@ assert remove:    Remove{Pre,Here}(a, n, i, b, v);
      //@ assert remove:    Remove{Pre,Here}(a, n, i+1, b, v);
      //@ assert unchanged: Unchanged{Pre,Here}(a, n);
      //@ assert unchanged: Unchanged{Pre,Here}(b, k, n);
    }
    else {
      //@ assert unchanged: Unchanged{Pre,Here}(a, n);
    }

    //@ assert remove:    Remove{Pre,Here}(a, n, i, b, v);
    //@ assert unchanged: Unchanged{Pre,Here}(a, n);
  }

  return k;
}
```

Listing 7.50: Implementation of `remove_copy3`

7.16. The remove algorithm

The C++ Standard Library also contains a function remove [19, 28.6.8] performing the same operation as remove_copy as an in-place algorithm. Its signature is very similar to that of remove_copy, except that there is no need for an output array.

```
size_type remove(value_type* a, size_type n, value_type v);
```

Figure 7.51 shows how remove is supposed to remove all occurrences of the given value 4 from a range.

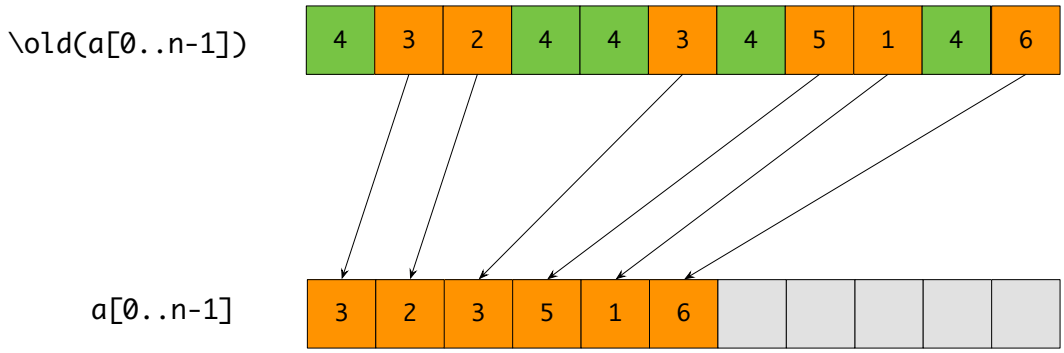

Figure 7.51.: Effects of remove

7.16.1. Formal specification of remove

The following listing shows a formal specification of the function remove [7.52]. Our specification is very similar to the one of remove_copy3 [7.48] except that we using a version of Remove [7.47] that takes only one pointer argument.

```
/*@
  requires   valid:      \valid(a + (0..n-1));

  terminates             \true;
  exits                  \false;
  assigns                a[0..n-1];

  ensures    size:       \result == CountNotEqual{Old}(a, n, v);
  ensures    bound:      0 <= \result <= n;
  ensures    remove:     Remove{Old,Here}(a, n, v);
  ensures    discard:    NoneEqual(a, \result, v);
  ensures    unchanged:  Unchanged{Old,Here}(a, \result, n);
*/
size_type
remove(value_type* a, size_type n, value_type v);
```

Listing 7.52: Formal specification of remove

7.16.2. Implementation of `remove`

In the following listing we show our implementation of `remove` [7.53] together with the additional loop annotations. Again, the annotations are very similar to those of the implementation of `remove_copy3` [7.50].

```
size_type
remove(value_type* a, size_type n, value_type v)
{
  size_type k = 0u;

  /*@
    loop invariant size:       k == CountNotEqual{Pre}(a,i,v);
    loop invariant bound:      0 <= k <= i <= n;
    loop invariant remove:     Remove{Pre,Here}(a, n, i, v);
    loop invariant discard:    NoneEqual(a, k, v);
    loop invariant interval:   IndexOfNotEqual{Pre}(a, n, v, k-1) <= i;
    loop invariant interval:   i <= IndexOfNotEqual{Pre}(a, n, v, k);
    loop invariant unchanged:  Unchanged{Pre,Here}(a, k, n);
    loop invariant unchanged:  a[k] == At{Pre}(a, k);
    loop assigns   k, i, a[0..n-1];
    loop variant   n-i;
  */
  for (size_type i = 0u; i < n; ++i ) {
    if (a[i] != v) {
      a[k++] = a[i];
      //@ assert size:      k == CountNotEqual{Pre}(a, 0, i+1, v);
      //@ assert update:    a[k-1] == At{Pre}(a, i);
      //@ assert interval:  i == IndexOfNotEqual{Pre}(a, n, v, k-1);
      //@ assert remove:    Remove{Pre,Here}(a, n, i,   v);
      //@ assert remove:    Remove{Pre,Here}(a, n, i+1, v);
    }
  }

  return k;
}
```

Listing 7.53: Implementation of `remove`

Also note the use of the predicate `At` [7.49] in the loop invariant `unchanged` and the assertion `update`.

7.17. The `shuffle` algorithm

The `shuffle` algorithm in the C++ Standard Library [19, §28.6.13] randomly rearranges the elements of a given range, that is, it randomly picks one of its possible orderings. For our purposes we have modified the generic implementation to that of a range of type `value_type`. The signature now reads:

```
void shuffle(value_type* a, size_type n, unsigned short* rand);
```

The argument `rand` holds the state of a simple random number generator that is used in the implementation of `shuffle`.

Figure 7.54 illustrates an example run of `shuffle`. In this figure, the values 1, 2, 3, and 4 occur twice, once, once, and three times, respectively, both before and after the `shuffle` run. This expresses that the range has been reordered.

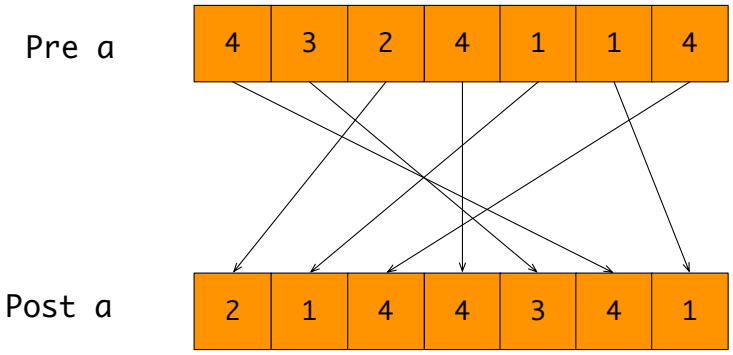

Figure 7.54.: Effects of `shuffle`

7.17.1. The predicate `MultisetReorder`

The `shuffle` algorithm is the first example in this document where we have to specify a *rearrangement* or *reordering* of the elements of a given range. We say that an array has been reordered between two states if the number of each element in the array remains unchanged. In other words, reordering leaves the *multiset*[13] of elements in the range unchanged.

We use the predicate `MultisetReorder` [7.55] to formally describe this property. This predicate, which is given in two overloaded versions, relies on the logic function `Count` [4.44]. We list here several lemma with basic properties of `MultisetReorder`. We will use these lemmas during the verification of various algorithms.

```
/*@
  axiomatic MultisetReorder
  {
    predicate
    MultisetReorder{K,L}(value_type* a, integer m, integer n) =
      \forall value_type v;
        Count{K}(a, m, n, v) == Count{L}(a, m, n, v);

    predicate
    MultisetReorder{K,L}(value_type* a, integer n) =
      MultisetReorder{K,L}(a, 0, n);

    lemma Unchanged_MultisetReorder{K,L}:
      \forall value_type *a, integer k, n;
        Unchanged{K,L}(a, k, n)  ==>  MultisetReorder{K,L}(a, k, n);

    lemma MultisetReorder_DisjointUnion{K,L}:
      \forall value_type *a, integer i, k, n;
        0 <= i <= k <= n              ==>
        MultisetReorder{K,L}(a, i, k)  ==>
        MultisetReorder{K,L}(a, k, n)  ==>
        MultisetReorder{K,L}(a, i, n);

    lemma MultisetReorder_Symmetric{K,L}:
      \forall value_type *a, integer m, n;
        MultisetReorder{K,L}(a, m, n)  ==>  MultisetReorder{L,K}(a, m, n);

    lemma MultisetReorder_Transitive{K,L,M}:
      \forall value_type *a, integer m, n;
        MultisetReorder{K,L}(a, m, n)  ==>
        MultisetReorder{L,M}(a, m, n)  ==>
        MultisetReorder{K,M}(a, m, n);
  }
*/
```

Listing 7.55: The logic definition(s) `MultisetReorder`

[13] See `http://en.wikipedia.org/wiki/Multiset`

7.17.2. Formal specification of `shuffle`

In the specification of the `shuffle` [7.56] algorithm we demand that the range `a[0..n-1]` is valid for reading and writing. We use the predicate `MultisetReorder` [7.55] to express that the contents of `a[0..n-1]` is just permuted, i.e., the number of occurrences of each of its members remains unchanged. The array `rand` contains a seed for the random number generator used to randomize the shuffle. By specifying that the function assigns to `rand` we capture that the function may return a different permutation every time.

Note that our specification only states that the resulting range is a reordering of the input range; nothing more and nothing less. Ideally, we would also specify that sequence of reorderings obtained by repeated calls of `shuffle` is required to be random. The informal specification [19, §28.6.13] of `shuffle` states that *that each possible permutation of those elements has equal probability of appearance*. However, ACSL does currently not support the specification of temporal properties related to repeated call results.

```
/*@
  requires   valid:     \valid(a + (0..n-1));
  requires   valid:     \valid(seed + (0..2));
  requires   sep:       \separated(a + (0..n-1), seed + (0..2));

  terminates            \true;
  exits                 \false;
  assigns               a[0..n-1];
  assigns               seed[0..2];

  ensures     reorder:  MultisetReorder{Old,Here}(a,n);
*/
void
shuffle(value_type* a, size_type n, unsigned short* seed);
```

Listing 7.56: Formal specification of `shuffle`

More generally speaking, it is not trivial to capture the notion of randomness in a mathematically precise way. As a typical example, we refer to a paper [21, p.6–8], which just gives four statistical tests indicating the randomness of the permutations computed with their algorithm. From a theoretical point of view, a sequence of permutations can be called “random” if its Kolmogorov complexity exceeds a certain measure, however, this property is undecidable [22].

7.17.3. Implementation of `shuffle`

The following listing shows our implementation of the function `shuffle` [7.57]. It repeatedly calls the function `swap` [7.6] to *transpose* (randomly) selected elements. For details of out source of randomness we refer to the function `random_number` [7.60].

```
void
shuffle(value_type* a, size_type n, unsigned short* seed)
{
  if (0u < n) {
    /*@
      loop invariant bounds:    1 <= i <= n;
      loop invariant reorder:   MultisetReorder{Pre,Here}(a, 0, i);
      loop invariant unchanged: Unchanged{Pre,Here}(a, i, n);
      loop assigns   i, a[0..n-1], seed[0..2];
      loop variant   n - i;
    */
    for (size_type i = 1u; i < n; ++i) {
      size_type k = random_number(seed, i) + 1u;

      //@ assert less: 0 <= k <= i;
      if (k < i) {
        swap(&a[k], &a[i]);
        //@ assert swapped: ArraySwap{LoopCurrent,Here}(a, k, i, n);
        //@ assert reorder: MultisetReorder{LoopCurrent,Here}(a, i+1);
        //@ assert reorder: MultisetReorder{Pre,Here}(a, i+1);
      }
      else {
        //@ assert unchanged: Unchanged{LoopCurrent,Here}(a, i+1);
        //@ assert reorder:   MultisetReorder{Pre,Here}(a, i+1);
      }

      //@ assert reorder: MultisetReorder{Pre,Here}(a, i+1);
    }
  }
}
```

Listing 7.57: Implementation of `shuffle`

The loop invariants `reorder` and `unchanged` of `shuffle` are necessary for the verification of the postcondition `reorder`: in the `i`th loop cycle, the subrange `a[0..i-1]` has been reordered, while the remaining subrange `a[i..n-1]` is yet unchanged. We also formulate several auxiliary assertions `reorder` which use the the predefined label `LoopCurrent`, to guide the automatic verification the loop invariant `reorder`. Please not the empty **`else`**-branch hat only contains an assertion `reorder`. We introduced this assertion to support the verification of the `reorder` property.

In addition, we rely on the predicate `ArraySwap` [7.58] rather than the literal postcondition of `swap` [7.6], since this leads to to more concise annotations and better a performance of the automatic provers.

```
/*@
  axiomatic ArraySwap
  {
    predicate
    ArraySwap{K,L}(value_type* a, integer i, integer k, integer n) =
      0 <= i < k < n               &&
      At{K}(a, i) == At{L}(a, k)   &&
      At{K}(a, k) == At{L}(a, i)   &&
      Unchanged{K,L}(a, 0,   i)    &&
      Unchanged{K,L}(a, i+1, k)    &&
      Unchanged{K,L}(a, k+1, n);
  }
*/
```

Listing 7.58: The logic definition(s) `ArraySwap`

The lemma `MultisetSwap_Middle` [7.59] states that swapping the elements `a[i]` and `a[k]` is a particular kind of reordering on the range `a[i..k]`.

```
/*@
  axiomatic MultisetSwap
  {
    lemma MultisetSwap_Middle{K,L}:
      \forall value_type* a, integer i, k, n;
        ArraySwap{K,L}(a, i, k, n)  ==>  MultisetReorder{K,L}(a, i, k+1);

    lemma MultisetSwap_FrontMiddle{K,L}:
      \forall value_type* a, integer i, k, n;
        ArraySwap{K,L}(a, i, k, n)  ==>  MultisetReorder{K,L}(a, 0, k+1);
  }
*/
```

Listing 7.59: The logic definition(s) `MultisetSwap`

7.18. Verifying a random number generator

We describe in this section `random_number` [7.60] which implements a simple random-number generator. As in the case of `shuffle` [7.56] itself, we do not formulate precise properties of randomness and only require its result to be in the specified range `[0..n-1]`. Again, the `assigns` clause to the array `state` models the dependency on an additional state.

Note that in the following listing, we also provide the rather simple specification of the function `random_init` that is called to initialize the state of the random generator.

```
/*@
  requires   pos:     0 < n;
  requires   valid:   \valid(state + (0..2));

  terminates          \true;
  exits               \false;
  assigns             state[0..2];

  ensures    result:  0 <= \result < n;
*/
size_type
random_number(unsigned short* state, size_type n);


/*@
  requires   valid:  \valid(state + (0..2));

  terminates         \true;
  exits              \false;
  assigns            state[0..2];
*/
void
random_init(unsigned short* state);
```

Listing 7.60: Formal specification of `random_number`

The implementations of `random_number` and `random_init` are shown in the following listing. Internally, we rely on a custom implementation of the POSIX.1 random number generator `lrand48()`[14] This random number generator is a linear congruence generator with a 48 bit state and the iteration procedure

$$x_{n+1} = ax_n + c \bmod 2^{48} \tag{7.7}$$

where $a = 25214903917$ and $c = 11$ are relatively prime integers.

As a part of the iteration procedure in Equation (7.7) an unsigned overflow may occur. This does not affect the result as we are only interested in its lowest 48 bits. However, as one of the options we use, `-warn-unsigned-overflow`, causes Frama-C/WP assert the absence of unsigned overflow this algorithm does not verify under the same options used for the other algorithms. As an exception, we have therefore decided to disable `-warn-unsigned-overflow` for this function as the unsigned overflow is both benign and well-defined (cf. [16, §6.2.5, 9]).

[14] See `http://pubs.opengroup.org/onlinepubs/9699919799/functions/lrand48.html`

```
// see IEEE 1003.1-2008, 2016 Edition for specification
/*@
  requires valid: \valid(seed + (0..2));
  assigns seed[0..2];
  ensures lower:  0 <= \result;
  ensures upper:  \result <= 0x7fffffff;
*/
static long
my_lrand48(unsigned short* seed)
{
  unsigned long long state = (unsigned long long)seed[0] << 32
                           | (unsigned long long)seed[1] << 16
                           | (unsigned long long)seed[2];
  state = (0x5deece66dull * state + 0xbull) % (1ull << 48);
  //@ assert lower: state < (1ull << 48);
  long result = state / (1ull << 17);
  //@ assert lower: 0 <= result;
  seed[0u] = state >> 32 & 0xffff;
  seed[1u] = state >> 16 & 0xffff;
  seed[2u] = state >>  8 & 0xffff;
  return result;
}

size_type
random_number(unsigned short* state, size_type n)
{
  return my_lrand48(state) % n;
}

void
random_init(unsigned short* state)
{
  state[0] = 0x243f;
  state[1] = 0x6a88;
  state[2] = 0x85a3;
}
```

Listing 7.61: Implementation of `random_number`

Note that we use the custom acsl lemma `RandomNumberModulo` [7.62] from the following listing to support the verification of some assertions.

```
/*@
  axiomatic C_Bit
  {
    lemma RandomNumberModulo:
      \forall unsigned long long a;
        (a % (1ull << 48)) < (1ull << 48);
  }
*/
```

Listing 7.62: The logic definition(s) `C_Bit`

8. Numeric algorithms

The algorithms that we considered so far only *compared*, *read* or *copied* values in sequences. In this chapter, we consider so-called *numeric* algorithms of the C++ Standard Library [19, §29.8] that use arithmetic operations on `value_type` to combine the elements of sequences.

```
#define VALUE_TYPE_MAX  INT_MAX
#define VALUE_TYPE_MIN  INT_MIN
```

Listing 8.1: Limits of `value_type`

In order to refer to potential arithmetic overflows we introduce the two constants shown in Listing 8.1 which refer to the numeric limits of `value_type` (see also §2.3).

We consider the following algorithms.

- `iota` writes sequentially increasing values into a range (§8.1)
- `accumulate` computes the sum of the elements in a range (§8.2)
- `inner_product` computes the inner product of two ranges (§8.3)
- `partial_sum` computes the sequence of partial sums of a range (§8.4)
- `adjacent_difference` computes the differences of adjacent elements in a range (§8.5)
- Finally, in §8.6 we show that under appropriate preconditions the algorithms `partial_sum` and `adjacent_difference` are inverse to each other.

The formal specifications of these algorithms raise new questions. In particular, we now have to deal with arithmetic overflows in `value_type`.

8.1. The iota algorithm

The iota algorithm in the C++ Standard Library [19, §29.8.12] assigns sequentially increasing values to a range, where the initial value is user-defined. Our version of the original signature reads:

```
void iota(value_type* a, size_type n, value_type v);
```

Starting at v, the function assigns consecutive integers to the elements of the range a. When specifying iota we must be careful to deal with possible overflows of the argument v.

8.1.1. Formal specification of iota

The specification of iota relies on the logic function IotaGenerate [8.2] that is defined in the following listing.

```
/*@
  axiomatic IotaGenerate
  {
    predicate
    IotaGenerate(value_type* a, integer n, value_type v) =
      \forall integer i; 0 <= i < n  ==>  a[i] == v+i;
  }
*/
```

Listing 8.2: The logic definition(s) IotaGenerate

The specification of iota is shown in the following listing. It uses the logic function IotaGenerate [8.2] in order to express the postcondition increment.

```
/*@
  requires   valid:      \valid(a + (0..n-1));
  requires   limit:      v + n <= VALUE_TYPE_MAX;

  terminates             \true;
  exits                  \false;
  assigns                a[0..n-1];

  ensures    increment:  IotaGenerate(a, n, v);
*/
void
iota(value_type* a, size_type n, value_type v);
```

Listing 8.3: Formal specification of iota

The specification of iota refers to VALUE_TYPE_MAX which is the maximum value of the underlying integer type (see Listing 8.1). In order to avoid integer overflows the sum v+n must not be greater than the constant VALUE_TYPE_MAX.

8.1.2. Implementation of `iota`

The following listing shows an implementation of the `iota` function.

```
void
iota(value_type* a, size_type n, value_type v)
{
  /*@
    loop invariant bound:     0 <= i <= n;
    loop invariant limit:     v == \at(v, Pre) + i;
    loop invariant increment: IotaGenerate(a, i, \at(v, Pre));

    loop assigns i, v, a[0..n-1];
    loop variant n-i;
  */
  for (size_type i = 0u; i < n; ++i) {
    a[i] = v++;
  }
}
```

Listing 8.4: Implementation of `iota`

The loop invariant `increment` describes that in each iteration of the loop the current value `v` is equal to the sum of the value `v` in state of function entry and the loop index `i`. We have to refer here to `\at(v,Pre)` which is the value on entering `iota`.

8.2. The `accumulate` algorithm

The `accumulate` algorithm in the C++ Standard Library [19, §29.8.2] computes the sum of an given initial value and the elements in a range. Our version of the original signature reads:

```
value_type
accumulate(const value_type* a, size_type n, value_type init);
```

The result of `accumulate` shall equal the value $\texttt{init} + \sum_{i=0}^{\texttt{n}-1} \texttt{a}[i]$. This implies that `accumulate` will return `init` for an empty range.

8.2.1. The logic function `Accumulate`

As in the case of `count` [4.48] we specify `accumulate` by first defining the *logic function* `Accumulate` [8.5] that formally defines the summation of elements in an array.

```
/*@
  axiomatic Accumulate
  {
    logic integer
    Accumulate{L}(value_type* a, integer n, integer init) =
      n <= 0 ? init : Accumulate(a, n-1, init) + a[n-1];

    predicate
    AccumulateBounds{L}(value_type* a, integer n, value_type init) =
      \forall integer i; 0 <= i <= n  ==>
        VALUE_TYPE_MIN <= Accumulate(a, i, init) <= VALUE_TYPE_MAX;

    lemma Accumulate_Init:
      \forall value_type *a, init, integer n;
         n <= 0  ==>  Accumulate(a, n, init) == init;

    lemma Accumulate_Next{L}:
      \forall value_type* a, init, integer n;
        0 <= n  ==>
        Accumulate(a, n+1, init) == Accumulate(a, n, init) + a[n];

    lemma Accumulate_Unchanged{K,L}:
      \forall value_type *a, init, integer n;
        Unchanged{K,L}(a, n)  ==>
        Accumulate{K}(a, n, init) == Accumulate{L}(a, n, init);

    lemma Accumulate_Unchanged_Shrink{K,L}:
      \forall value_type *a, init, integer m, n;
        0 <= m <= n            ==>
        Unchanged{K, L}(a, n)  ==>
        Accumulate{K}(a, m, init) == Accumulate{L}(a, m, init);

    lemma AccumulateBounds_Unchanged{K,L}:
      \forall value_type *a, init, integer n;
        Unchanged{K, L}(a, n)           ==>
        AccumulateBounds{K}(a, n, init) ==>
        AccumulateBounds{L}(a, n, init);
  }
*/
```

Listing 8.5: The logic definition(s) `Accumulate`

With this definition the following equation holds for $n \geq 0$

$$\texttt{Accumulate}(\mathsf{a}, \mathsf{n}, \texttt{init}) = \texttt{init} + \sum_{i=0}^{\mathsf{n}-1} \mathsf{a}[i] \tag{8.1}$$

The predicate `AccumulateBounds` [8.5] that we will subsequently use in order to compactly express requirements that exclude numeric overflows while accumulating value. This predicate states that for $0 \leq i < n$ the *partial sums*

$$\texttt{init} + \sum_{k=0}^{\mathsf{i}} \mathsf{a}[k] \tag{8.2}$$

do not overflow. If one of them did, one couldn't guarantee that the result of C implementation of `accumulate` equals the mathematical description of `Accumulate`.

8.2.2. `AccumulateDefault`—a variant of `Accumulate`

The following listing shows another version of `Accumulate` [8.5], called `AccumulateDefault` [8.6].

```
/*@
  axiomatic AccumulateDefault
  {
    logic integer
    AccumulateDefault{L}(value_type* a, integer n) =
      Accumulate(a, n, (value_type)(0));

    predicate
    AccumulateDefaultBounds{L}(value_type* a, integer n) =
      AccumulateBounds(a, n, (value_type)(0));

    lemma AccumulateDefault_Unchanged{K,L}:
      \forall value_type *a, integer n;
        0 <= n                ==>
        Unchanged{K,L}(a, n)  ==>
        AccumulateDefault{K}(a, n) == AccumulateDefault{L}(a, n);

    lemma AccumulateDefault_Zero{L}:
      \forall value_type* a; AccumulateDefault(a, 0) == 0;

    lemma AccumulateDefault_One{L}:
      \forall value_type* a; AccumulateDefault(a, 1) == a[0];

    lemma AccumulateDefault_Next{L}:
      \forall value_type* a, integer n;
        0 <= n  ==>
        AccumulateDefault(a, n+1) == AccumulateDefault(a, n) + a[n];

    lemma AccumulateDefaultBounds_Shrink{L}:
      \forall value_type* a, integer m, n;
        0 <= m <= n                    ==>
        AccumulateDefaultBounds(a, n)  ==>  AccumulateDefaultBounds(a, m);
  }
*/
```

Listing 8.6: The logic definition(s) `AccumulateDefault`

The function `AccumulateDefault` uses `a[0]` as default value of `init`. Thus, for `AccumulateDefault` we have

$$\texttt{AccumulateDefault}(a, n) = \sum_{i=0}^{n-1} a[i] \tag{8.3}$$

We will use this version for the specification of the algorithm `partial_sum` [8.13].

This listing also includes additional properties of observable `AccumulateDefault` behavior, here given as a lemmas. It also contains the predicate `AccumulateDefaultBounds` [8.6] with corresponding numeric limits for the predicate `AccumulateDefault`.

8.2.3. Formal specification of `accumulate`

Using the logic function `Accumulate` and the predicate `AccumulateBounds`, the specification of `accumulate` is then as simple as shown in the following listing.

```
/*@
  requires   valid:   \valid_read(a + (0..n-1));
  requires   bounds:  AccumulateBounds(a, n, init);

  terminates          \true;
  exits               \false;
  assigns             \nothing;

  ensures    result:  \result == Accumulate(a, n, init);
*/
value_type
accumulate(const value_type* a, size_type n, value_type init);
```

Listing 8.7: Formal specification of `accumulate`

8.2.4. Implementation of `accumulate`

The following listing shows an implementation of the `accumulate` function with corresponding loop annotations.

```
value_type
accumulate(const value_type* a, size_type n, value_type init)
{
  /*@
    loop invariant index:    0 <= i <= n;
    loop invariant partial:  init == Accumulate(a, i, \at(init,Pre));
    loop assigns i, init;
    loop variant n-i;
  */
  for (size_type i = 0u; i < n; ++i) {
    //@ assert rte_help: init + a[i] == Accumulate(a, i+1, \at(init,Pre));
    init = init + a[i];
  }

  return init;
}
```

Listing 8.8: Implementation of `accumulate`

Note that loop invariant `partial` claims that in the i-th iteration step `result` equals the accumulated value of Equation (8.2). This depends on the property `bounds` of `accumulate` [8.7] which expresses that there is no numeric overflow when updating the variable `init`.

8.3. The `inner_product` algorithm

The `inner_product` algorithm in the C++ Standard Library [19, §29.8.4] computes the *inner product*[15] of two ranges. Our version of the original signature reads:

```
value_type
inner_product(const value_type* a, const value_type* b,
              size_type n, value_type init);
```

The result of `inner_product` equals the value

$$\texttt{init} + \sum_{i=0}^{\texttt{n}-1} \texttt{a}[i] \cdot \texttt{b}[i]$$

thus, `inner_product` will return `init` for empty ranges.

8.3.1. The logic function `InnerProduct`

As in the case of `accumulate` [8.7] we specify `inner_product` by defining in the following listing the logic function `InnerProduct` that formally expresses the summation of the element-wise product of two arrays.

Predicate `ProductBounds` [8.9] expresses that for $0 \leq i < n$ the products

$$\texttt{a}[i] \cdot \texttt{b}[i] \tag{8.4}$$

do not overflow. Predicate `InnerProductBounds` [8.9], on the other hand, states that for $0 \leq i < n$ the following sums do not overflow. cc

$$\texttt{init} + \sum_{k=0}^{\texttt{i}} \texttt{a}[k] \cdot \texttt{b}[k] \tag{8.5}$$

Otherwise, one cannot guarantee that the result of our implementation of `inner_product` [8.11] equals the mathematical description of `InnerProduct`. Finally, Lemma `InnerProduct_Unchanged` [8.9] states that the result of the `InnerProduct` only depends on the values of `a[0..n-1]` and `b[0..n-1]`.

[15] Also referred to as *dot product*, see `http://en.wikipedia.org/wiki/Dot_product`

```
/*@
  axiomatic InnerProduct
  {
    logic integer
    InnerProduct{L}(value_type* a, value_type* b, integer n,
                    value_type init) =
      n <= 0 ? init : InnerProduct(a, b, n-1, init) + (a[n-1] * b[n-1]);

    predicate
    ProductBounds(value_type* a, value_type* b, integer n) =
      \forall integer i; 0 <= i < n  ==>
        VALUE_TYPE_MIN <= a[i] * b[i] <= VALUE_TYPE_MAX;

    predicate
    InnerProductBounds(value_type* a, value_type* b, integer n,
                       value_type init) =
      \forall integer i; 0 <= i <= n  ==>
        VALUE_TYPE_MIN <= InnerProduct(a, b, i, init) <= VALUE_TYPE_MAX;

    lemma InnerProduct_Init:
      \forall value_type *a, *b, init, integer n;
         n <= 0  ==>  InnerProduct(a, b, n, init) == init;

    lemma InnerProduct_Next{L}:
      \forall value_type* a, *b, init, integer n;
        0 <= n  ==>
        InnerProduct(a, b, n+1, init) == InnerProduct(a, b, n, init) + a[n] * b[n];

    lemma InnerProduct_Unchanged{K,L}:
      \forall value_type *a, *b, init, integer n;
        Unchanged{K,L}(a, n)  ==>
        Unchanged{K,L}(b, n)  ==>
        InnerProduct{K}(a, b, n, init) == InnerProduct{L}(a, b, n, init);
  }
*/
```

Listing 8.9: The logic definition(s) `InnerProduct`

8.3.2. Formal specification of `inner_product`

Using the logic function `InnerProduct` [8.9], we specify `inner_product` as shown in the following listing. Note that we needn't require that `a` and `b` are separated.

```
/*@
  requires   valid:      \valid_read(a + (0..n-1));
  requires   valid:      \valid_read(b + (0..n-1));
  requires   bounds:     ProductBounds(a, b, n);
  requires   bounds:     InnerProductBounds(a, b, n, init);

  terminates             \true;
  exits                  \false;
  assigns                \nothing;

  ensures    result:     \result == InnerProduct(a, b, n, init);
  ensures    unchanged:  Unchanged{Old,Here}(a, n);
  ensures    unchanged:  Unchanged{Old,Here}(b, n);
*/
value_type
inner_product(const value_type* a, const value_type* b, size_type n,
              value_type init);
```

Listing 8.10: Formal specification of `inner_product`

8.3.3. Implementation of `inner_product`

The following listing shows an implementation of `inner_product` with corresponding loop annotations.

```
value_type
inner_product(const value_type* a, const value_type* b, size_type n,
              value_type init)
{
  /*@
     loop invariant index: 0 <= i <= n;
     loop invariant inner: init == InnerProduct(a, b, i, \at(init,Pre));
     loop assigns i, init;
     loop variant n-i;
  */
  for (size_type i = 0u; i < n; ++i) {
    /*@
      assert rte_help: init + a[i] * b[i] ==
                       InnerProduct(a, b, i+1, \at(init,Pre));
    */
    init = init + a[i] * b[i];
  }

  return init;
}
```

Listing 8.11: Implementation of `inner_product`

Note that the loop invariant `inner` claims that in the i-th iteration step the current value of `init` equals the accumulated value of Equation (8.5). This depends of course on the properties `bounds` in the contract of `inner_product` [8.10], which express that there is no arithmetic overflow when computing the updates of the variable `init`.

8.4. The `partial_sum` algorithm

The `partial_sum` algorithm in the C++ Standard Library [19, §29.8.6] computes the sum of a given initial value and the elements in a range. Our version of the original signature reads:

```
size_type
partial_sum(const value_type* a, size_type n, value_type* b);
```

After executing the function `partial_sum` the array `b[0..n-1]` holds the following values

$$\mathsf{b}[i] = \sum_{k=0}^{\mathsf{i}} \mathsf{a}[k] \tag{8.6}$$

for $0 \leq i < n$. Equations (8.6) and (8.3) suggest that we define in the following listing the ACSL predicate `PartialSum` by using the logic function `AccumulateDefault` [8.6].

```
/*@
  axiomatic PartialSum
  {
    predicate
    PartialSum{L}(value_type* a, integer n, value_type* b) =
      \forall integer i; 0 <= i < n  ==>  b[i] == AccumulateDefault(a, i+1);

    lemma PartialSum_Section{K}:
      \forall value_type *a, *b, integer m, n;
      0 <= m <= n              ==>
      PartialSum{K}(a, n, b)   ==>
      PartialSum{K}(a, m, b);

    lemma PartialSum_Step{L}:
      \forall value_type *a, *b, integer n;
        0 <= n                              ==>
        PartialSum(a, n, b)                 ==>
        b[n] == AccumulateDefault(a, n+1)   ==>
        PartialSum(a, n+1, b);

    lemma PartialSum_Unchanged{K,L}:
      \forall value_type *a, *b, integer n;
        0 <= n  ==>
        PartialSum{K}(a, n, b)   ==>
        Unchanged{K, L}(a, n)    ==>
        Unchanged{K, L}(b, n)    ==>
        PartialSum{L}(a, n, b);

    lemma PartialSum_One{L}:
      \forall value_type *a, *b, integer n;
        b[0] == AccumulateDefault(a, 1)  ==>  PartialSum(a, 1, b);
  }
*/
```

Listing 8.12: The logic definition(s) `PartialSum`

8.4.1. Formal specification of `partial_sum`

The specification of `partial_sum` [8.13] demands that the arrays `a[0..n-1]` and `b[0..n-1]` are separated, that is, they do not overlap. Note that is a stricter requirement than in the case of the original C++ version of `partial_sum`, which allows that `a` equals `b`, thus allowing the computation of partial sums *in place*.

```
/*@
  requires   valid:       \valid_read(a + (0..n-1));
  requires   valid:       \valid(b + (0..n-1));
  requires   sep:         \separated(a + (0..n-1), b + (0..n-1));
  requires   bounds:      AccumulateDefaultBounds(a, n);

  terminates              \true;
  exits                   \false;
  assigns                 b[0..n-1];

  ensures    result:      \result == n;
  ensures    partialsum:  PartialSum(a, n, b);
  ensures    unchanged:   Unchanged{Old,Here}(a, n);
*/
size_type
partial_sum(const value_type* a, size_type n, value_type* b);
```

Listing 8.13: Formal specification of `partial_sum`

8.4.2. Implementation of `partial_sum`

The following listing shows an implementation of `partial_sum` with corresponding loop annotations.

```
size_type
partial_sum(const value_type* a, size_type n, value_type* b)
{
  if (0u < n) {
    //@ assert limits:      AccumulateDefaultBounds(a, n);
    b[0u] = a[0u];
    //@ assert unchanged:   Unchanged{Pre,Here}(a, n);
    //@ assert limits:      AccumulateDefaultBounds(a, n);
    //@ assert accumulate:  b[0] == AccumulateDefault(a, 1);
    //@ assert partialsum:  PartialSum(a, 1, b);

    /*@
       loop invariant bound:      1 <= i <= n;
       loop invariant unchanged:  Unchanged{Pre,Here}(a, n);
       loop invariant accumulate: b[i-1] == AccumulateDefault(a, i);
       loop invariant limits:     AccumulateDefaultBounds(a, n);
       loop invariant partialsum: PartialSum(a, i, b);
       loop assigns i, b[1..n-1];
       loop variant n - i;
    */
    for (size_type i = 1u; i < n; ++i) {
      b[i] = b[i - 1u] + a[i];
      //@ assert unchanged:  Unchanged{LoopCurrent,Here}(b, i);
      //@ assert unchanged:  Unchanged{LoopCurrent,Here}(a, n);
      //@ assert partialsum: b[i] == AccumulateDefault(a, i+1);
      //@ assert limits:     AccumulateDefaultBounds(a, n);
    }
  }

  return n;
}
```

Listing 8.14: Implementation of `partial_sum`

8.5. The `adjacent_difference` algorithm

The `adjacent_difference` algorithm in the C++ Standard Library [19, §29.8.11] computes the differences of adjacent elements in a range. Our version of the original signature reads:

```
size_type
adjacent_difference(const value_type* a, size_type n, value_type* b);
```

After executing the function `adjacent_difference` the array `b[0..n-1]` holds the following values

$$\begin{aligned} \mathsf{b}[0] &= \mathsf{a}[0] \\ \mathsf{b}[1] &= \mathsf{a}[1] - \mathsf{a}[0] \\ &\vdots \\ \mathsf{b}[n-1] &= \mathsf{a}[n-1] - \mathsf{a}[n-2] \end{aligned} \tag{8.7}$$

8.5.1. The predicate `AdjacentDifference`

We start with the definition of the logic function `Difference` whose definition is shown in the following listing.

```
/*@
  axiomatic Difference
  {
    logic integer
    Difference{L}(value_type* a, integer n) =
      n <= 0 ? a[0] : a[n] - a[n-1];

    lemma Difference_Zero{L}:
      \forall value_type *a; Difference(a, 0) == a[0];

    lemma Difference_Next{L}:
      \forall value_type *a, integer n;
        0 < n  ==>  Difference(a, n) == a[n] - a[n-1];

    lemma Difference_Unchanged{K,L}:
      \forall value_type *a, integer n;
        0 <= n  ==>  Unchanged{K,L}(a, n+1)  ==>
        Difference{K}(a, n) == Difference{L}(a, n);
  }
*/
```

Listing 8.15: The logic definition(s) `Difference`

Building on top of `Difference` we now introduce the predicate `AdjacentDifference`. We also provide the predicate `AdjacentDifferenceBounds` that captures conditions that prevent numeric overflows while computing differences of the form `a[i] - a[i-1]`.

```
/*@
  axiomatic AdjacentDifference
  {
    predicate
    AdjacentDifference{L}(value_type* a, integer n, value_type* b) =
      \forall integer i; 0 <= i < n  ==>  b[i] == Difference(a, i);

    predicate
    AdjacentDifferenceBounds(value_type* a, integer n) =
      \forall integer i; 1 <= i < n  ==>
        VALUE_TYPE_MIN <= Difference(a, i) <= VALUE_TYPE_MAX;

    lemma AdjacentDifference_Step{K,L}:
      \forall value_type *a, *b, integer n;
        AdjacentDifference{K}(a, n, b)      ==>
        Unchanged{K,L}(b, n)                ==>
        Unchanged{K,L}(a, n+1)              ==>
        \at(b[n],L) == Difference{L}(a, n)  ==>
        AdjacentDifference{L}(a, n+1, b);

    lemma AdjacentDifference_Section{K}:
      \forall value_type *a, *b, integer m, n;
        0 <= m <= n                     ==>
        AdjacentDifference{K}(a, n, b)  ==>
        AdjacentDifference{K}(a, m, b);
  }
*/
```

Listing 8.16: The logic definition(s) `AdjacentDifference`

Lemmas `AdjacentDifference_Step` [8.16] and `AdjacentDifference_Section` [8.16] will help us later in the verification of `adjacent_difference_inv` [8.22].

8.5.2. Formal specification of `adjacent_difference`

Using the predicates `AdjacentDifference` [8.16] and `AdjacentDifferenceBounds` [8.16] we can provide in the following listing a concise formal specification of `adjacent_difference`. As in the case of the specification of `partial_sum` [8.13] we require that the arrays `a[0..n-1]` and `b[0..n-1]` are separated.

```
/*@
   requires   valid:       \valid_read(a + (0..n-1));
   requires   valid:       \valid(b + (0..n-1));
   requires   sep:         \separated(a + (0..n-1), b + (0..n-1));
   requires   bounds:      AdjacentDifferenceBounds(a, n);

   terminates              \true;
   exits                   \false;
   assigns                 b[0..n-1];

   ensures    result:      \result == n;
   ensures    difference:  AdjacentDifference(a, n, b);
   ensures    unchanged:   Unchanged{Old,Here}(a, n);
*/
size_type
adjacent_difference(const value_type* a, size_type n, value_type* b);
```

Listing 8.17: Formal specification of `adjacent_difference`

8.5.3. Implementation of `adjacent_difference`

The following listing shows an implementation of `adjacent_difference` with corresponding loop annotations. In order to achieve the verification of the loop invariant `difference` we rely on

- the assertions `bound` and `difference`
- the lemmas `AdjacentDifference_Step` [8.16] and `AdjacentDifference_Section` [8.16]
- a statement contract with the two postconditions labeled as `step`

```
size_type
adjacent_difference(const value_type* a, size_type n, value_type* b)
{
  if (0u < n) {
    b[0u] = a[0u];

    /*@
       loop invariant index:      1 <= i <= n;
       loop invariant unchanged:  Unchanged{Pre,Here}(a, n);
       loop invariant difference: AdjacentDifference(a, i, b);
       loop assigns i, b[1..n-1];
       loop variant n - i;
    */
    for (size_type i = 1u; i < n; ++i) {
      //@ assert bound: VALUE_TYPE_MIN <= Difference(a, i) <= VALUE_TYPE_MAX;
      /*@
        assigns b[i];
        ensures step: Unchanged{Old,Here}(b, i);
        ensures step: b[i] == Difference(a, i);
      */
      b[i] = a[i] - a[i - 1u];
      //@ assert difference: AdjacentDifference(a, i+1, b);
    }
  }

  return n;
}
```

Listing 8.18: Implementation of `adjacent_difference`

8.6. Inverting `partial_sum` and `adjacent_difference`

In this section we show that under appropriate preconditions the algorithms `partial_sum` and `adjacent_difference` are inverse to each other.

8.6.1. Inverting `partial_sum`

Let `a[0..n-1]` and `b[0..n-1]` be the respective input and output of `partial_sum`. We have in other words

$$\begin{aligned} \mathsf{b}[0] &= \mathsf{a}[0] \\ \mathsf{b}[1] &= \mathsf{a}[0] + \mathsf{a}[1] \\ &\vdots \\ \mathsf{b}[n-1] &= \mathsf{a}[0] + \mathsf{a}[1] + \ldots + \mathsf{a}[n-1] \end{aligned}$$

If we apply now the algorithm `adjacent_difference` to `b[0..n-1]`, then we find for its output `a'[0..n-1]`

$$\begin{aligned} \mathsf{a}'[0] = \mathsf{b}[0] &= \mathsf{a}[0] \\ \mathsf{a}'[1] = \mathsf{b}[1] - \mathsf{b}[0] &= \mathsf{a}[1] \\ &\vdots \\ \mathsf{a}'[n-1] = \mathsf{b}[n-1] - \mathsf{b}[n-2] &= \mathsf{a}[n-1] \end{aligned}$$

Before we start show the ACSL lemmas of our claim, we present the predicate `DefaultBounds` [8.19] in order to express that the values in the input (and output!) array `a[0..n-1]` do not overflow.

```
/*@
  axiomatic DefaultBounds
  {
    predicate
    DefaultBounds{L}(value_type* a, integer n) =
      \forall integer i; 0 <= i < n  ==>
        VALUE_TYPE_MIN <= a[i] <= VALUE_TYPE_MAX;
  }
*/
```

Listing 8.19: The logic definition(s) `DefaultBounds`

Lemma `PartialSum_Inverse` from the following listing expresses as ACSL lemmas that the algorithms `partial_sum` and `adjacent_difference` are inverse to each other.

```
/*@
  axiomatic NumericInverse
  {
    lemma PartialSum_Inverse:
      \forall value_type *a, *b, integer n;
        0 <= n                  ==>
        PartialSum(a, n, b)   ==>
        AdjacentDifference(b, n, a);

    lemma AdjacentDifference_Inverse:
      \forall value_type *a, *b, integer n;
        0 <= n                          ==>
        AdjacentDifference(a, n, b)   ==>
        PartialSum(b, n, a);

    lemma AdjacentDifference_InverseBounds:
      \forall value_type *a, *b, integer n;
        0 <= n                          ==>
        DefaultBounds(a, n)             ==>
        AdjacentDifference(a, n, b)   ==>
        AccumulateDefaultBounds(b, n);
  }
*/
```

Listing 8.20: The logic definition(s) `NumericInverse`

The following listing now shows C function `partial_sum_inv` (both the contract and the implementation). This function calls first `partial_sum` and then `adjacent_difference`.

```
/*@
  requires   valid:      \valid(a + (0..n-1));
  requires   valid:      \valid(b + (0..n-1));
  requires   sep:        \separated(a + (0..n-1), b + (0..n-1));
  requires   bounds:     AccumulateDefaultBounds(a, n);
  requires   bounds:     DefaultBounds(a, n);

  terminates             \true;
  exits                  \false;
  assigns                a[0..n-1], b[0..n-1];

  ensures    unchanged:  Unchanged{Pre,Here}(a, n);
*/
void
partial_sum_inv(value_type* a, size_type n, value_type* b)
{
  partial_sum(a, n, b);
  adjacent_difference(b, n, a);
}
```

Listing 8.21: Implementation of `partial_sum_inv`

The contract of `partial_sum_inv` formulates preconditions that shall guarantee that during the computation neither arithmetic overflows (property `bounds`) nor unintended aliasing of arrays (property `sep`) occur. Under these precondition, Frama-C shall verify that the final call to `adjacent_difference` [8.17] just restores the original contents of `a[0..n-1]` that we supplied for the initial call to `partial_sum` [8.13].

8.6.2. Inverting `adjacent_difference`

After executing the function `adjacent_difference` [8.17] on the input array `a[0..n-1]` the output array `b[0..n-1]` holds the following values

$$\begin{aligned} \mathsf{b}[0] &= \mathsf{a}[0] \\ \mathsf{b}[1] &= \mathsf{a}[1] - \mathsf{a}[0] \\ &\vdots \\ \mathsf{b}[n-1] &= \mathsf{a}[n-1] - \mathsf{a}[n-2] \end{aligned}$$

If we call now `partial_sum` with the array `b[0..n-1]` as input, then we obtain for its output `a'[0..n-1]`

$$\begin{aligned} \mathsf{a}'[0] &= \mathsf{b}[0] & &= \mathsf{a}[0] \\ \mathsf{a}'[1] &= \mathsf{b}[0] + \mathsf{b}[1] & &= \mathsf{a}[1] \\ &\vdots \\ \mathsf{a}'[n-1] &= \mathsf{b}[0] + \mathsf{b}[1] + \ldots + \mathsf{b}[n-1] & &= \mathsf{a}[n-1] \end{aligned}$$

which means that applying `partial_sum` [8.13] on the output of `adjacent_difference` produces the original input. Lemma `AdjacentDifference_Inverse` [8.20] expresses this property as a lemma.

The function `adjacent_difference_inv` [8.22] first calls `adjacent_difference` and then `partial_sum`. The contract of this function formulates preconditions that shall guarantee that during the computation neither arithmetic overflows (property `bound`) nor unintended aliasing of arrays (property `sep`) occur. In order to improve the automatic verification of `adjacent_difference_inv` we also use lemma `Unchanged_Transitive` [7.3]. Lemma `AdjacentDifference_InverseBounds` [8.20] simplifies the verification of the precondition `bounds` of `partial_sum`.

```
/*@
  requires   size:       0 <= n;
  requires   valid:      \valid(a + (0..n-1));
  requires   valid:      \valid(b + (0..n-1));
  requires   sep:        \separated(a + (0..n-1), b + (0..n-1));
  requires   bounds:     DefaultBounds(a, n);
  requires   bounds:     AdjacentDifferenceBounds(a, n);

  terminates             \true;
  exits                  \false;
  assigns                a[0..n-1], b[0..n-1];

  ensures    unchanged:  Unchanged{Old,Here}(a, n);
*/
void
adjacent_difference_inv(value_type* a, size_type n, value_type* b)
{
  adjacent_difference(a, n, b);
  partial_sum(b, n, a);
}
```

Listing 8.22: Implementation of `adjacent_difference_inv`

Part IV.

Sorting algorithms

9. Heap Algorithms

The heap algorithms of the C++ Standard Library [19, 28.7.7] were already part of *ACSL by Example* from 2010–2012. In this chapter we re-introduce them and discuss—based on the bachelor thesis of one of the authors—the verification efforts in some detail [23].

The C++ standard[16] introduces the concept of a *heap* as follows:

1. A *heap* is a particular organization of elements in a range between two random access iterators `[a,b)`. Its two key properties are:
   a) There is no element greater than `*a` in the range and
   b) `*a` may be removed by `pop_heap()`, or a new element added by `push_heap()`, in $O(\log(N))$ time.
2. These properties make heaps useful as priority queues.
3. `make_heap()` converts a range into a heap and `sort_heap()` turns a heap into an increasing sequence.

Figure 9.1 gives an overview on the five heap algorithms by means of an example. Algorithms, which in a typical implementation are in a caller-callee relation, have the same color.

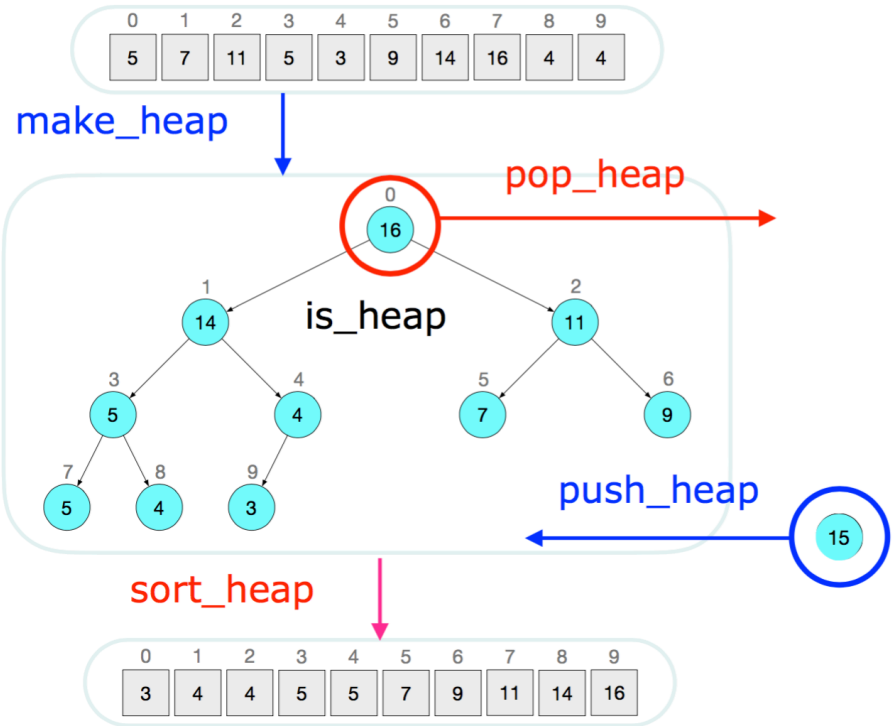

Figure 9.1.: Overview on heap algorithms

[16] See `http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2011/n3242.pdf`

Roughly speaking, the algorithms from Figure 9.1 have the following behavior.

- In §9.1 we briefly recapitulate basic heap concepts.
- In §9.2 we show how these heap concepts can be described in ACSL.
- In §9.3 we verify two auxiliary heap functions.
- The algorithms `is_heap_until` and `is_heap` from §9.4 and §9.5 allow to test at run time whether a given array is arranged as a heap
- The algorithm `push_heap` from §9.7 *adds* an element to a given heap in such a way that resulting array is again a heap
- The algorithm `pop_heap` from §9.8, on the other hand, *removes* an element from a given heap in such a way that the resulting array is again a heap
- The algorithm `make_heap` from §9.9 rearranges a given array into a heap.
- Finally, the algorithm `sort_heap` from §9.10 sorts a heap into an increasing range.

In §9.1 we present in more detail how heaps are defined. The ACSL logic functions and predicate that formalize the basic heap properties of heaps are introduced in §9.2.

9.1. Basic heap concepts

The description of heaps at the beginning of this chapter is of course fairly vague. It outlines only the most important properties of various operations but does not clearly state what specific and verifiable properties a range must satisfy such that it may be called a heap.

A more detailed description can be found in the Apache C++ Standard Library User's Guide:[17]

> A heap is a binary tree in which every node is larger than the values associated with either child. A heap and a binary tree, for that matter, can be very efficiently stored in a vector, by placing the children of node i at positions $2i + 1$ and $2i + 2$.

We have, in other words, the following basic relations between indices of a heap:

$$\text{left child for index } i \qquad \text{child}_\text{l} : i \mapsto 2i + 1 \tag{9.1}$$

$$\text{right child for index } i \qquad \text{child}_\text{r} : i \mapsto 2i + 2 \tag{9.2}$$

and

$$\text{parent index for index } i \qquad \text{parent} : i \mapsto \frac{i-1}{2} \tag{9.3}$$

These function are related through the following two equations that hold for all integers i. Note that in ACSL integer division rounds towards zero (cf. [14, §2.2.4]).

$$\text{parent}(\text{child}_\text{l}(i)) = i \tag{9.4}$$

$$\text{parent}(\text{child}_\text{r}(i)) = i \tag{9.5}$$

In order to given an example for the usefulness of heaps we consider the following multiset of integers X.

$$X = \{2, 3, 3, 3, 6, 7, 8, 8, 9, 11, 13, 14\} \tag{9.6}$$

[17] See `http://stdcxx.apache.org/doc/stdlibug/14-7.html`

Figure 9.2 shows how the multiset from Equation (9.6) can, according to the parent-child relations of a heap, be represented as a tree.

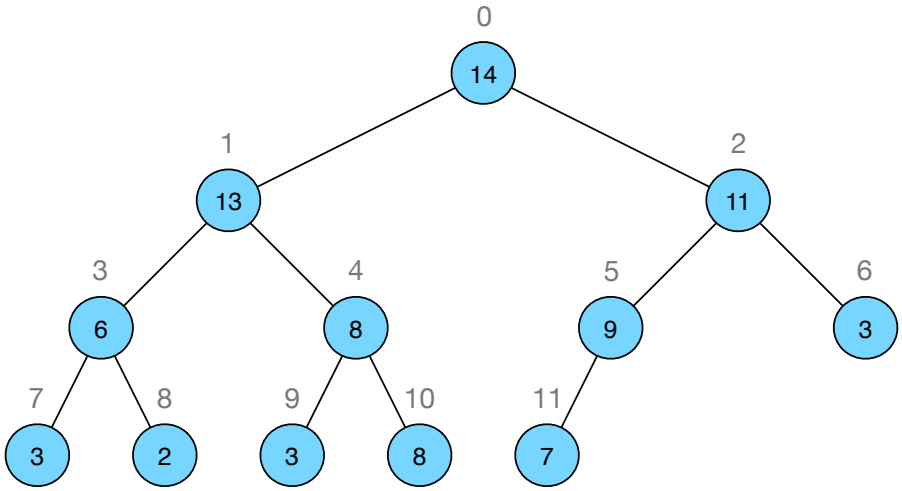

Figure 9.2.: Tree representation of the multiset X

The numbers outside the nodes in Figure 9.2 are the indices at which the respective node value is stored in the underlying array of a heap (cf. Figure 9.3).

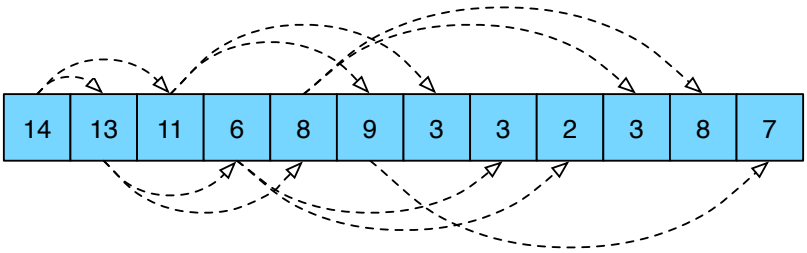

Figure 9.3.: Underlying array of a heap

It is important to understand that there can be various representations of a multiset as a heap. Figure 9.4, for example, arranges the elements of the multiset X as a heap in a different tree.

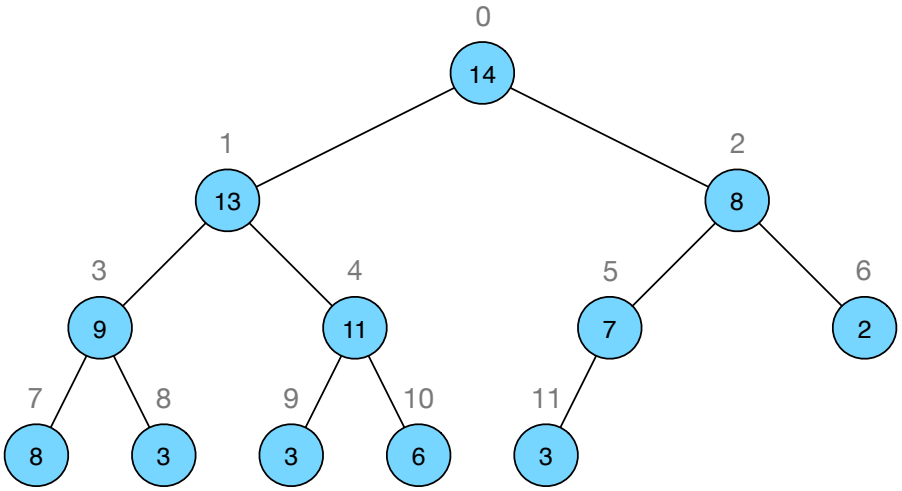

Figure 9.4.: An alternative representation of the multiset X

Figure 9.5 then shows the underlying array that corresponds to the tree in Figure 9.4.

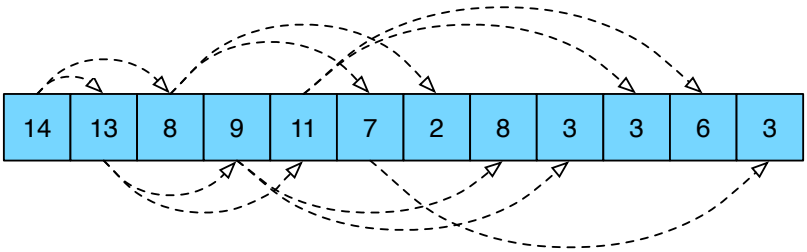

Figure 9.5.: Underlying array of the alternative representation

9.2. Representation of heap concepts in ACSL

The following listing shows three logic functions `HeapLeft`, `HeapRight` and `HeapParent` that correspond to the definitions (9.1), (9.2) and (9.3), respectively. This listing also contains a number of ACSL lemma that state among other things that

- the `HeapParent` function satisfies the equations (9.4) and (9.5) and
- the function `HeapParent` is the *left inverse* to the `HeapLeft` and `HeapRight` functions.[18]

```
/*@
  axiomatic HeapNodes
  {
    logic integer HeapLeft(integer i) = 2*i + 1;

    logic integer HeapRight(integer i) = 2*i + 2;

    logic integer HeapParent(integer i) = (i-1) / 2;

    lemma HeapParent_Zero{L}: HeapParent(0) == 0;

    lemma Heap_ParentLeft:
      \forall integer p; 0 <= p  ==>  HeapParent(HeapLeft(p)) == p;

    lemma Heap_ParentRight:
      \forall integer p; 0 <= p  ==>  HeapParent(HeapRight(p)) == p;

    lemma Heap_ParentChild:
      \forall integer c, p;
        0 < c  ==>  HeapParent(c) == p  ==>
        (c == HeapLeft(p) || c == HeapRight(p));

    lemma Heap_Childs:
      \forall integer a, b;
        0 < a  ==>  0 < b                ==>
        HeapParent(a) == HeapParent(b)   ==>
        (a == b || a+1 == b || a == b+1);

    lemma Heap_ParentBounds:
      \forall integer c; 0 < c  ==>  0 <= HeapParent(c) < c;

    lemma Heap_ChildBounds:
      \forall integer p; 0 <= p  ==>  p < HeapLeft(p) < HeapRight(p);
  }
*/
```

Listing 9.6: The logic definition(s) `HeapNodes`

[18] See Section *Left and right inverses* at `http://en.wikipedia.org/wiki/Inverse_function`

On top of these basic definitions we introduce the predicate `Heap` [9.7]. The fact that element at index 0 of a (maximum) heap, is always the largest element of the heap is express by Lemma `Heap_Maximum` [9.7] using the predicate `MaxElement` [5.3].

```
/*@
  axiomatic Heap
  {
    predicate
    Heap{L}(value_type* a, integer n) =
      \forall integer i; 0 < i < n  ==>  a[i] <= a[HeapParent(i)];

    lemma Heap_Maximum{L} :
      \forall value_type* a, integer n;
        0 < n  ==>  Heap(a, n)  ==>  MaxElement(a, n, 0);

   lemma Heap_Shrink{L}:
     \forall value_type *a, integer m, n;
       0 <= m <= n  ==>  Heap(a, n)   ==>  Heap(a, m);

   lemma Heap_Unchanged{K,L}:
     \forall value_type *a, integer n;
       0 <= n  ==>  Unchanged{K,L}(a, n)  ==>  Heap{K}(a, n)  ==>  Heap{L}(a, n);

   predicate
   HeapCompatible{L}(value_type* a, integer n, integer m, value_type v) =
     (0 <= m < n)                                        &&
     (0 <= HeapParent(m)  ==>  v <= a[HeapParent(m)])    &&
     (HeapLeft(m)  <  n   ==>  a[HeapLeft(m)]  <= v)     &&
     (HeapRight(m) <  n   ==>  a[HeapRight(m)] <= v);

    lemma HeapCompatible_Update{K,L}:
      \forall value_type *a, v, integer m, n;
         0 <= m < n                        ==>
         Heap{K}(a, n)                     ==>
         HeapCompatible{K}(a, n, m, v)     ==>
         ArrayUpdate{K,L}(a, n, m, v)      ==>
         Heap{L}(a, n);

  }
*/
```

Listing 9.7: The logic definition(s) `Heap`

The lemmas `Heap_Shrink` and `Heap_Unchanged` formulate simple rules to “transfer” the heap property from an array to a related (sub-)array.

The predicate `HeapCompatible` expresses under which conditions the changing of an individual heap element does maintain the heap property. This predicate together with lemma `HeapCompatible_Update` will be useful in the verification of the algorithms `push_heap` [9.23] and `pop_heap` [9.30].

9.3. The auxiliary functions heap_parent and heap_child

This section features the two auxiliary heap functions We start with the function heap_parent [9.8] which is in principle the C counterpart of the ACSL function HeapParent [9.6]. We say *in principle* because our definition avoids the border case of the parent node of 0.

```
/*@
   terminates          \true;
   exits               \false;
   assigns             \nothing;

   ensures    parent:  \result == HeapParent(child);
 */
size_type
heap_parent(size_type child);
```

Listing 9.8: Formal specification of heap_parent

```
size_type
heap_parent(size_type child)
{
  return (0u < child) ?  (child - 1u) / 2u : 0u;
}
```

Listing 9.9: Implementation of heap_parent

Neither do we provide exact C-counterparts for the logic functions HeapLeft [9.6] and HeapRight [9.6]. In fact, we have encountered only one situation (in the implementation of pop_heap [9.30]), where such functions would have been useful. However, what we really need in pop_heap is to determine for a given index p a child index c where the maximum of the respective values a[HeapLeft(p)] and a[HeapRight(p)] resides. This computation is performed by the function heap_child [9.10].

```
/*@
   requires   bounds:  0 <= p < n;
   requires   valid:   \valid(a + (0..n-1));

   terminates          \true;
   exits               \false;
   assigns             \nothing;

   ensures    bounds:  p < \result <= n;
   ensures    parent:  \result < n       ==>  p == HeapParent(\result);
   ensures    parent:  \result < n-1     ==>  HeapLeft(p)  < n-1;
   ensures    parent:  \result < n-1     ==>  HeapRight(p) < n;
   ensures    left:    HeapLeft(p)  < n  ==>  \result < n;
   ensures    right:   HeapRight(p) < n  ==>  \result < n;
   ensures    max:     HeapLeft(p)  < n  ==>  a[HeapLeft(p)]  <= a[\result];
   ensures    max:     HeapRight(p) < n  ==>  a[HeapRight(p)] <= a[\result];
   ensures    none:    \result == n      ==>  n <= HeapLeft(p);
   ensures    none:    \result == n      ==>  n <= HeapRight(p);
*/
size_type
heap_child(const value_type* a, size_type n, size_type p);
```

Listing 9.10: Formal specification of heap_child

Note that in the implementation of `heap_child` [9.11] we explicitly handle the case that the computation of child indices could overflow. If this occurs, the function `heap_child` returns `n`.

```
size_type
heap_child(const value_type* a, size_type n, size_type p)
{
  if (p + 1u <= n - p - 1u) {
    const size_type left  = 2u * p + 1u;
    const size_type right = left + 1u;

    if (right < n) {
      // case of two children: select child with maximum value
      return a[right] <= a[left] ? left : right;
    }
    else {
      // at most one child that comes before n-1 can exist
      return left;
    }
  }
  else {
    return n;
  }
}
```

Listing 9.11: Implementation of `heap_child`

9.4. The `is_heap_until` algorithm

The `is_heap_until` algorithm of the C++ Standard Library [19, §28.7.7.5] works on generic sequences. For our purposes we have modified the generic implementation to that of an array of type `value_type`. The signature now reads:

```
size_type is_heap_until(const value_type* a, int n);
```

The algorithm `is_heap_until` returns the largest range of an array, beginning at the first position, where it still satisfies the heap properties we have semi-formally described in the beginning of this chapter. In particular, `is_heap_until` will return the size of the array, called with the array argument from Figure 9.3.

9.4.1. Formal specification of `is_heap_until`

The specification of `is_heap_until` is shown in the following listing. The index `\result` returned by `is_heap_until` indicates that the array `a[0..\result-1]` is a heap. In addition the postcondition `last` states, that for all indices greater than or equal to `i` the predicate `Heap` [9.7] is not satisfied.

```
/*@
   requires   valid:  \valid_read(a + (0..n-1));

   terminates         \true;
   exits              \false;
   assigns            \nothing;

   ensures    bound:  0 <= \result <= n;
   ensures    heap:   Heap(a, \result);
   ensures    last:   \forall integer i; \result < i <= n ==> !Heap(a, i);
*/
size_type
is_heap_until(const value_type* a, size_type n);
```

Listing 9.12: Formal specification of `is_heap_until`

9.4.2. Implementation of `is_heap_until`

The following listing shows one way to implement the function `is_heap_until`.

```
size_type
is_heap_until(const value_type* a, size_type n)
{
  size_type parent = 0u;

  /*@
    loop invariant bound:    0 <= parent < child <= n+1;
    loop invariant parent:   parent == HeapParent(child);
    loop invariant heap:     Heap(a, child);
    loop invariant not_heap: a[parent] < a[child] ==> \forall integer i; child < i
        <= n ==> !Heap(a, i);

    loop assigns child, parent;
    loop variant n - child;
  */
  for (size_type child = 1u; child < n; ++child) {
    if (a[parent] < a[child]) {
      return child;
    }

    if ((child % 2u) == 0u) {
      ++parent;
    }
  }

  return n;
}
```

Listing 9.13: Implementation of `is_heap_until`

The algorithms starts at the index 1, which is the smallest index, where a child node of the heap might reside. The algorithms checks for each (child) index whether the value at the corresponding parent index is greater than or equal to the value at the child index. If the value at a parent index is smaller than the value at a (child) index, `is_heap_until` returns the (child) index. Otherwise, if the algorithm iterates through the whole array, the size of the array is returned.

9.5. The `is_heap` algorithm

The `is_heap` algorithm of the C++ Standard Library [19, §28.7.7.5] works on generic sequences. For our purposes we have modified the generic implementation to that of an array of type `value_type`. The signature now reads:

```
bool is_heap(const value_type* a, int n);
```

The algorithm `is_heap` checks whether a given array satisfies the heap properties we have semi-formally described in the beginning of this chapter. In particular, `is_heap` will return **`true`** called with the array argument from Figure 9.3.

9.5.1. Formal specification of `is_heap`

The specification of `is_heap` is shown in the following listing. The function returns **`true`** if and only if the input array satisfies the predicate `Heap` [9.7].

```
/*@
   requires   valid:  \valid_read(a +(0..n-1));

   terminates         \true;
   exits              \false;
   assigns            \nothing;

   ensures    heap:   \result <==> Heap(a, n);
*/
bool
is_heap(const value_type* a, size_type n);
```

Listing 9.14: Formal specification of `is_heap`

9.5.2. Implementation of `is_heap`

Our implementation of `is_heap` in the following listing utilizes the function `is_heap_until` [9.12].

```
bool
is_heap(const value_type* a, size_type n)
{
  return is_heap_until(a, n) == n;
}
```

Listing 9.15: Implementation of `is_heap`

9.6. Reorderings and fluctuations

One particular challenge posed by heap algorithms is that while temporarily causing *small fluctuations* in the number of values within an array they essentially only *reorder* it, that is they leave the multiset of its values unchanged. In this section we will introduce various predicates that will help us mastering this challenge.

9.6.1. Formalizing small fluctuations

The predicate `MultisetAdd` in the following listing expresses that the number of occurrences of a specific element in an array has increased by one between two program points `K` and `L`.

```
/*@
  axiomatic MultisetOperations
  {
    predicate
    MultisetAdd{K,L}(value_type* a, integer n, value_type v) =
      Count{L}(a, 0, n, v) == Count{K}(a, 0, n, v) + 1;

    predicate
    MultisetMinus{K,L}(value_type* a, integer n, value_type v) =
      MultisetAdd{L,K}(a, n, v);

    predicate
    MultisetRetain{K,L}(value_type* a, integer n, value_type v) =
      Count{K}(a, 0, n, v) == Count{L}(a, 0, n, v);

    lemma MultisetAdd_Distinct{K,L}:
      \forall value_type *a, v, integer m, n;
        0 <= m < n                        ==>
        At{K}(a, m) != v                  ==>
        At{L}(a, m) == v                  ==>
        MultisetReorder{K,L}(a, 0, m)     ==>
        MultisetReorder{K,L}(a, m+1, n)   ==>
        MultisetAdd{K,L}(a, n, v);

    lemma MultisetMinus_Distinct{K,L}:
      \forall value_type *a, v, integer m, n;
        0 <= m < n                        ==>
        At{K}(a, m) == v                  ==>
        At{L}(a, m) != v                  ==>
        MultisetReorder{K,L}(a, 0, m)     ==>
        MultisetReorder{K,L}(a, m+1, n)   ==>
        MultisetMinus{K,L}(a, n, v);

    lemma MultisetRetain_Distinct{K,L}:
      \forall value_type *a, v, integer m, n;
        0 <= m < n                        ==>
        At{K}(a, m) != v                  ==>
        At{L}(a, m) != v                  ==>
        MultisetReorder{K,L}(a, 0, m)     ==>
        MultisetReorder{K,L}(a, m+1, n)   ==>
        MultisetRetain{K,L}(a, n, v);
  }
*/
```

Listing 9.16: The logic definition(s) `MultisetOperations`

The predicate `MultisetMinus`, on the other hand, expresses that the number of occurrences of a specific element in an array has decreased by one between two program points `K` and `L`. Note that we have defined `MultisetMinus` by calling `MultisetAdd` with the labels reversed. Finally, the predicate `MultisetRetain` expresses that a the number of occurrences of a given value does not change between two program points. In order to guide the automatic provers, we also provide some lemmas that formalize conditions under which the respective predicates hold.

Using the predicate `MultisetReorder` [7.55] and the logic function `At` [7.49] we also formulate a few simple lemmas that describe when the predicates from Listing `MultisetOperations` [9.16] hold.

9.6.2. Simple properties of fluctuations

The predicate `MultisetRetainRest` [9.17] uses `MultisetRetain` [9.16] in order to express that all values of an array, except the two given values `u` and `v`, occur as often in program point `K` and program point `L`.

The lemmas in this listing express conditions under which small fluctuations—expressed by the predicates `MultisetAdd` [9.16] and `MultisetMinus` [9.16]—in the number of occurrences between three program points even with each other.

```
/*@
  axiomatic MultisetRetainRest
  {
    predicate
    MultisetRetainRest{K,L}(value_type* a, integer n, value_type  v, value_type w) =
      \forall value_type x;
        x != v  ==>  x != w  ==>  MultisetRetain{K,L}(a, n, x);

    lemma Multiset_AddMinusRetain{K,L,M}:
      \forall value_type *a, u, integer n;
        MultisetAdd{K,L}(a, n, u)    ==>
        MultisetMinus{L,M}(a, n, u)  ==>
        MultisetRetain{K,M}(a, n, u);

    lemma Multiset_MinusAddRetain{K,L,M}:
      \forall value_type *a, u, integer n;
        MultisetMinus{K,L}(a, n, u) ==>
        MultisetAdd{L,M}(a, n, u)   ==>
        MultisetRetain{K,M}(a, n, u);

    lemma Multiset_AddMinusRetainReorder{K,L,M}:
      \forall value_type *a, u, v, integer n;
        u != v                                ==>
        MultisetAdd{K,L}(a, n, u)             ==>
        MultisetMinus{K,L}(a, n, v)           ==>
        MultisetRetainRest{K,L}(a, n, u, v)   ==>
        MultisetAdd{L,M}(a, n, v)             ==>
        MultisetMinus{L,M}(a, n, u)           ==>
        MultisetRetainRest{L,M}(a, n, v, u)   ==>
        MultisetReorder{K,M}(a, n);
  }
*/
```

Listing 9.17: The logic definition(s) `MultisetRetainRest`

9.6.3. Combining fluctuations

Small fluctuations are so prevalent in the central heap algorithms `push_heap` [9.23] and `pop_heap` [9.30] that it is worthwhile to introduce another predicate to concisely capture this feature. We refer to this predicate as `MultisetParity` [9.18] because it describes the situation where the number of occurrences

- of the first of two given values increases by one
- while that of the second value decreases by one
- and the remaining values retain their respective number of occurrences.

With this predicate we can formulate several lemmas that describe useful combinations of reorderings and fluctuations. For example, lemma `MultisetParity_MultisetReorder` [9.18] describes the situation where two fluctuation cancel each other and consequently establish a reordering of an array.

```
/*@
  axiomatic MultisetParity
  {
    predicate
      MultisetParity{K,L}(value_type* a, integer n, value_type u, value_type v) =
        MultisetAdd{K,L}(a, n, u)     &&
        MultisetMinus{K,L}(a, n, v)   &&
        MultisetRetainRest{K,L}(a, n, u, v);


    lemma MultisetParity_UnchangedFirst{K,L,M}:
      \forall value_type *a, u, v, integer n;
        u != v                          ==>
        Unchanged{K,L}(a, n)            ==>
        MultisetParity{L,M}(a, n, u, v)  ==>
        MultisetParity{K,M}(a, n, u, v);

    lemma MultisetParity_UnchangedSecond{K,L,M}:
      \forall value_type *a, u, v, integer n;
        u != v                          ==>
        MultisetParity{K,L}(a, n, u, v)  ==>
        Unchanged{L,M}(a, n)            ==>
        MultisetParity{K,M}(a, n, u, v);

    lemma MultisetParity_MultisetReorder{K,L,M}:
      \forall value_type *a, u, v, integer n;
        u != v                          ==>
        MultisetParity{K,L}(a, n, u, v)  ==>
        MultisetParity{L,M}(a, n, v, u)  ==>
        MultisetReorder{K,M}(a, n);

    lemma MultisetParity_Combined{K,L,M}:
      \forall value_type *a, u, v, w, integer n;
        u != v                          ==>
        u != w                          ==>
        v != w                          ==>
        MultisetParity{K,L}(a, n, u, v)  ==>
        MultisetParity{L,M}(a, n, w, u)  ==>
        MultisetParity{K,M}(a, n, w, v);
  }
*/
```

Listing 9.18: The logic definition(s) `MultisetParity`

9.6.4. How do fluctuations arise?

The simplest way to creation a small fluctuation is to update an array element with a different value. Thus, similar to the predicate `ArrayUpdate` [7.2] we introduce predicate `MultisetUpdate` [9.19] which in turn relies on `MultisetParity` [9.18]. Lemma `ArrayUpdate_MultisetUpdate` [9.19] then formalizes the claim that updating an array element with a different value creates a small fluctuation.

```
/*@
  axiomatic MultisetUpdate
  {
    predicate
    MultisetUpdate{K,L}(value_type* a, integer n, integer i, value_type v) =
      \let u = At{K}(a, i);
      u != v                          &&
      0 <= i < n                      &&
      MultisetReorder{K,L}(a, 0, i)   &&
      MultisetReorder{K,L}(a, i+1, n) &&
      MultisetParity{K,L}(a, n, v, u);

    lemma ArrayUpdate_MultisetUpdate{K,L}:
      \forall value_type *a, v, integer n, i;
        ArrayUpdate{K,L}(a, n, i, v)  ==>  MultisetUpdate{K,L}(a, n, i, v);
  }
*/
```

Listing 9.19: The logic definition(s) `MultisetUpdate`

9.7. The push_heap algorithm

The push_heap algorithm assumes that the first $n-1$ elements of an array of length n form already a heap and adds to it the element a[n-1].

Whereas in the C++ Standard Library [19, §28.7.7.1] push_heap works on a range of random access iterators, our version operates on an array of value_type. We therefore use the following signature for push_heap

```
void push_heap(value_type* a, size_type n);
```

The push_heap algorithm expects that n is greater or equal than 1. It also assumes that the array a[0..n-2] forms a heap. The algorithms then *rearranges* the array a[0..n-1] such that the resulting array is a heap. In this sense the algorithm *pushes* the element a[n-1] on the given heap.

9.7.1. Formal specification of push_heap

The following listing shows our specification of push_heap. Note that the post condition reorder states that push_heap is not allowed to change the number of occurrences of an array element. Without this post condition, an implementation that assigns 0 to each array element would satisfy the post condition heap—surely not what a user of the algorithm has in mind.

```
/*@
   requires   nonempty:  0 < n;
   requires   valid:     \valid(a + (0..n-1));
   requires   heap:      Heap(a, n-1);

   terminates           \true;
   exits                \false;
   assigns              a[0..n-1];

   ensures    heap:     Heap(a, n);
   ensures    reorder:  MultisetReorder{Old,Here}(a, n);
*/
void
push_heap(value_type* a, size_type n);
```

Listing 9.20: Formal specification of push_heap

Pushing an element on a heap usually *rearranges* several elements of the array (cf. Figures 9.21 and 9.22). We therefore must be able express that push_heap only reorders the elements of the array. We re-use the predicate MultisetReorder [7.55] to formally describe this property.

9.7.2. Implementation of push_heap

The following two figures illustrate how push_heap affects an array, which is shown as a tree with blue and grey nodes, representing heap and non-heap nodes, respectively. Figure 9.21 shows the heap from Figure 9.2 together with the additional element 12 that is to be pushed on the heap. To be quite clear about it: the new element 12 is the last element of the array and not yet part of the heap.

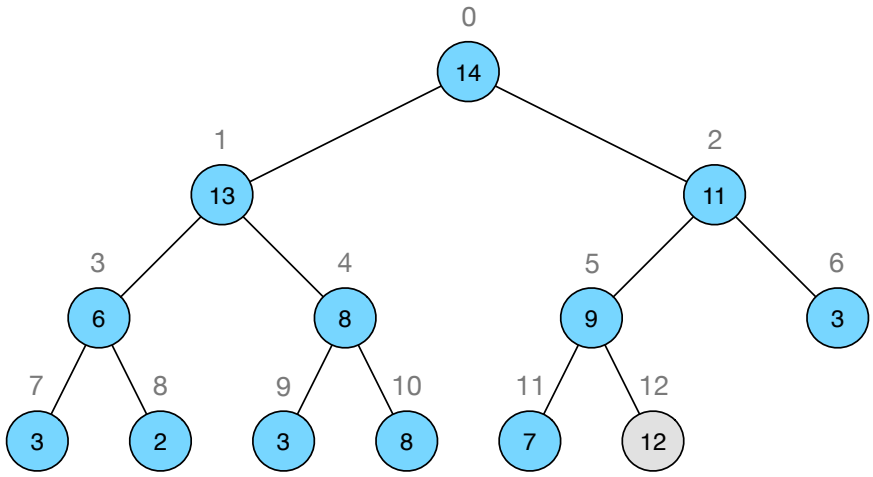

Figure 9.21.: Example heap before the call of `push_heap`

Figure 9.22 shows the array after the call of `push_heap`. We can see that now all nodes are colored in blue, i.e., they are part of the heap. The dashed nodes highlight which heap nodes have changed during the function call. The element to be pushed into the heap is now at its correct position. The arrows indicate the *cyclic reordering* of array elements to achieve the desired result.

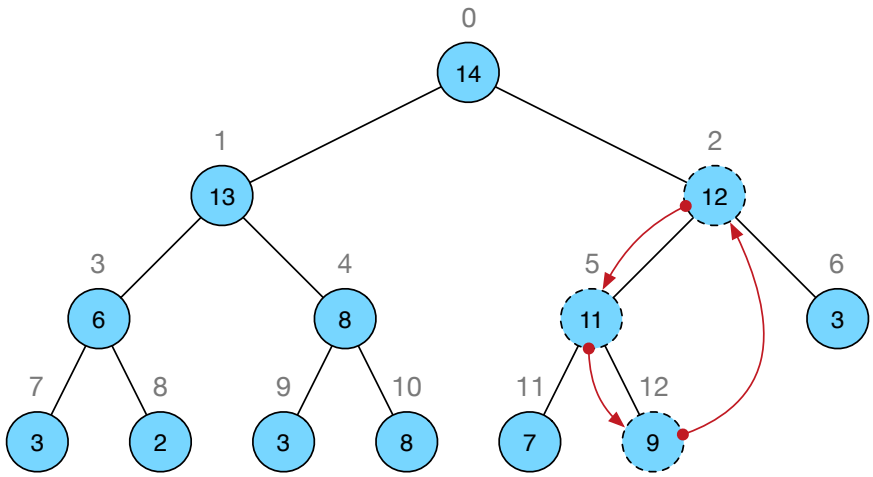

Figure 9.22.: Example heap after the call of `push_heap`

Verifying our implementation of `push_heap` [9.23] is a non-trivial undertaking. In order to better structure our discussion we refer to the central loop of the algorithm as the *main act* and the parts before and after it as *prologue* and *epilogue*.

We can establish the `heap` property of `push_heap` [9.20] already in the prologue. The `reorder` property, however, only holds at the function boundaries and is violated while `push_heap` manipulates the array. To be more precise: We loose the `reorder` property in the prologue and formally capture and maintain a slightly more general property in the main act. From this we will recover the `reorder` property in the epilogue.

We will illustrate the changes to the underlying array after each stage by figures of the array in tree form, based on the `push_heap` example from Figure 9.21.

```
void
push_heap(value_type* a, size_type n)
{
  if (1u < n) { // otherwise nothings needs to be done
    size_type c = n - 1u;
    size_type p = heap_parent(c);
    //@ assert parent: p == HeapParent(c);

    if (a[p] < a[c]) {
      const value_type v  = a[c];
      a[c] = a[p];
      //@ assert update:   ArrayUpdate{Pre,Here}(a, n, c, a[p]);
      //@ assert heap:     Heap(a, n);
      //@ assert reorder:  MultisetParity{Pre,Here}(a, n, a[p], v);

      /*@
        loop invariant bound:      0 <= c < n-1;
        loop invariant heap:       Heap(a, n);
        loop invariant less:       a[c] < v;
        loop invariant parent:     p == HeapParent(c);
        loop invariant reorder:    MultisetParity{Pre,Here}(a, n, a[c], v);
        loop invariant unchanged:  Unchanged{Pre,Here}(a, c);
        loop assigns               c, p, a[0..n-1];
        loop variant               c;
      */
      for (c = p, p = heap_parent(c); 0u < c && a[p] < v;
           c = p, p = heap_parent(c)) {
        //@ ghost value_type ac =  a[c];
        if (a[c] < a[p]) {
          a[c] = a[p];
          //@ assert update:     ArrayUpdate{LoopCurrent,Here}(a, n, c, a[p]);
          //@ assert update:     MultisetUpdate{LoopCurrent,Here}(a, n, c, a[p]);
          //@ assert bound:      0 <= c < n;
          //@ assert less:       ac < a[c] < v;
          //@ assert reorder:    MultisetParity{Pre,Here}(a, n, a[c], v);
        }
      }

      //@ ghost Epilogue: ;
      //@ assert heap: 0 == c || v <= a[HeapParent(c)];
      //@ ghost  value_type  ac = a[c];
      //@ assert update:     ac == At{Epilogue}(a, c) < v;
      //@ assert reorder:    MultisetParity{Pre,Here}(a, n, ac, v);
      a[c] = v;
      //@ assert update:   ArrayUpdate{Epilogue,Here}(a, n, c, v);
      //@ assert heap:     HeapCompatible(a, n, c, v);
      //@ assert heap:     Heap(a, n);
      //@ assert update:   MultisetUpdate{Epilogue,Here}(a, n, c, v);
      //@ assert reorder:  MultisetParity{Epilogue,Here}(a, n, v, ac);
      //@ assert reorder:  MultisetReorder{Pre,Here}(a, n);
    }
  }
}
```

Listing 9.23: Implementation of `push_heap`

Prologue

In the prologue we check whether the initial heap is nonempty, initialize some variables, *and* also check by comparing with the parent node whether `a[c]`, which is the value to be pushed on the heap and which is the last element of the array, is by chance already at the right place. If not we set aside this value in the variable `v` and assign the parent value `a[p]` to `a[c]`. Note that this assignment only occurs if the respective values differ. This allows us to formally describe the effect of the assignment using the predicate `ArrayUpdate` [7.2]. Figure 9.24 highlights the main effects of the prologue. Here and in the following figures we highlight the currently active node.

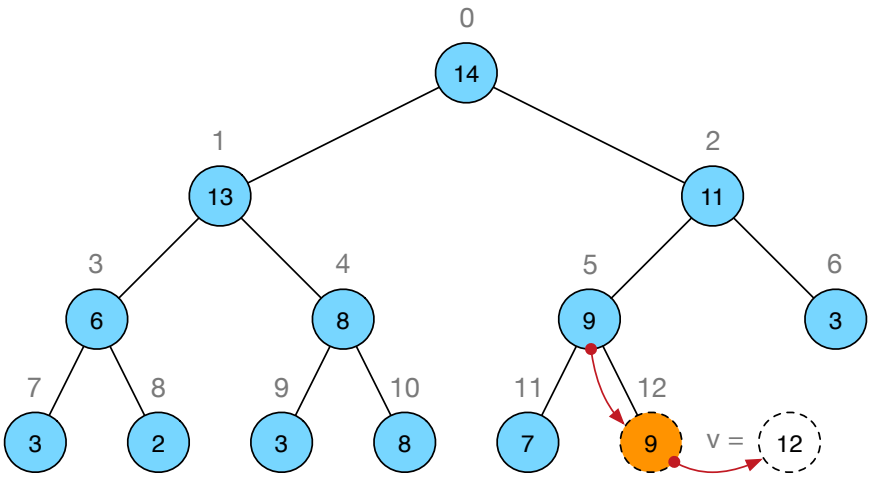

Figure 9.24.: Heap after the prologue of `push_heap`

At this point we have achieved several things.

1. The array `a[0..n-1]` is now a heap.
2. Regarding their respective number of occurences in the array `a[0..n-1]`
   - the original value `a[c]` occurs one time less
   - the original value `a[p]` occurs one time more
   - whereas all other values have not changed their number of occurences.

The first observation is expressed in the assertion `heap` whereas the small fluctuation of array elements described in the second observation is expressed by using the predicate `MultisetParity` [9.18] in the assertion `reorder`.

Main act

In the main act, we start at the parent location, which is now stored in the variable `c` (*child*). Compared to the pre-state of `push_heap` at the beginning of the main act the array `a[0..n-1]`

- contains the value `v` one time less
- contains the value `a[c]` one time more
- whereas all other values have not changed their number of occurences.

Now, as long as the index `c` is not yet the root of the heap and its consequently existing parent value `a[p]` is less than `v`, we haven't found yet an index `c` where we could insert `v` without violating the heap property.

In the loop body we proceed as follows.

- If `a[c]` is less than `a[p]` we copy the latter value on the former. Note that this assignment preserves the heap property of the array. The value `a[p]` now occurs one time more than in the pre-state whereas the now overwritten value `a[c]` occurs as often as in the pre-state. The value `v` continues to occur one time less. We then proceed to the next iteration by setting `c` to `p`.

  The verification of tracking the number of occurences happens in smaller steps than just described. It relies on the predicates `ArrayUpdate` [7.2] and `MultisetUpdate` [9.19] which we can apply in this guarded assignment. Lemma `MultisetParity_Combined` [9.18] also plays an important role here.

- Otherwise, since `c` is a child of `p`, we can conclude that `a[c]` equals `a[p]` and we continue with the next iteration after setting `c` to `p`.

This means that at the begin of the next iteration again the following conditions hold. Compared to the pre-state of `push_heap` the array `a[0..n-1]`

- contains the value `v` one time less
- contains the value `a[c]` one time more
- whereas all other values have not changed their number of occurences.

Figure 9.25 shows the our example heap after the main act. For this particular heap, only one iteration is performed until a node is reached whose parent value `a[p]` is greater or equal than `v`. Note that assignments which have previously occurred are marked with dashed arrows.

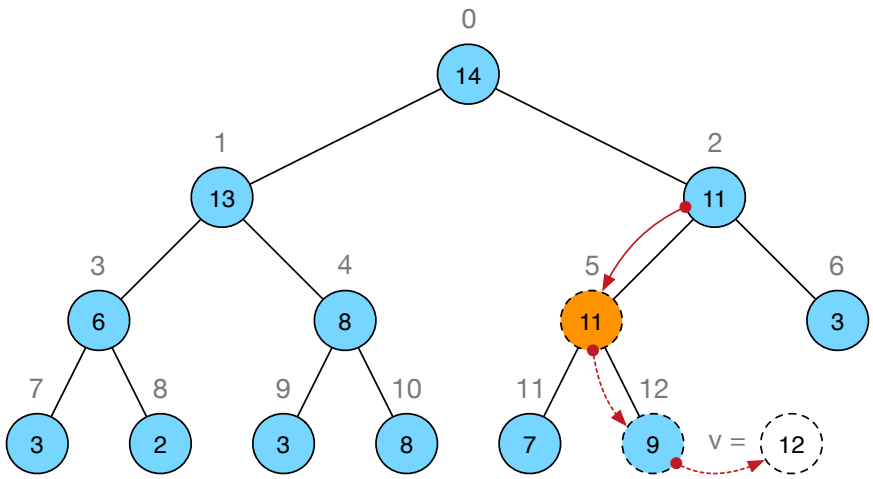

Figure 9.25.: Heap after the main act of `push_heap`

Epilogue

At this point, we have arrived at an index `c` where the assignment of the value `v` preserves the heap property. We express this formally using the predicate `HeapCompatible` [9.7].

Moreover, this assignment also corrects the imbalance in the number of occurences of the values `a[c]` and `v` and consequently establishes the desired property `reorder` of `push_heap`. The verification that this correction leads to a proper reordering relies on lemma `MultisetParity_MultisetReorder` [9.18].

Figure 9.26 shows the final assignment and highlights the completion of the cycle depicted in Figure 9.24. The figure also makes clear that the value `v` acts like an additional element in this assignment cycle.

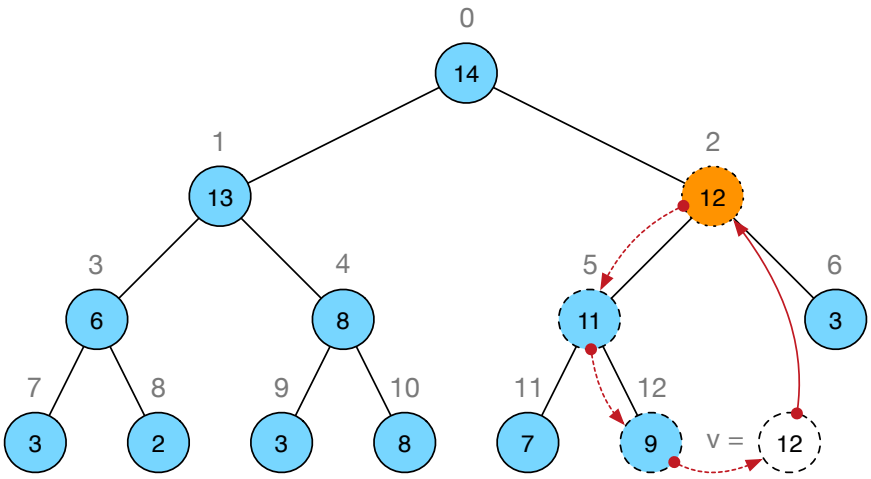

Figure 9.26.: Heap after the epilogue of `push_heap`

9.8. The pop_heap algorithm

The algorithm pop_heap moves the first element of the heap, which holds the heap's largest value, and places it at the the end of the underlying sequence. Whereas in the C++ Standard Library [19, §28.7.7.2] pop_heap works on a range of random access iterators, our version operates on an array of value_type. We therefore use the following signature for pop_heap

```
void pop_heap(value_type* a, size_type n);
```

The pop_heap algorithm expects that n is greater or equal than 1 and that the array a[0..n-1] forms a heap. The algorithms then *rearranges* the array a[0..n-1] such that the resulting array satisfies the following properties.

- a[n-1] = \old(a[0]), that is, the largest element of the original heap is transferred to the end of the array.
- the subarray a[0..n-2] is a heap

In this sense the algorithm *pops* the largest element from a heap.

9.8.1. Formal specification of pop_heap

On the basis of the preceding semi-formal description, we propose the following function contract for pop_heap [9.27].

```
/*@
   requires   bounds:   0 < n;
   requires   valid:    \valid(a + (0..n-1));
   requires   heap:     Heap(a, n);

   terminates           \true;
   exits                \false;
   assigns              a[0..n-1];

   ensures    heap:     Heap(a, n-1);
   ensures    result:   a[n-1] == \old(a[0]);
   ensures    max:      MaxElement(a, n, n-1);
   ensures    reorder:  MultisetReorder{Old,Here}(a, n);
*/
void
pop_heap(value_type* a, size_type n);
```

Listing 9.27: Formal specification of pop_heap

9.8.2. Implementation of pop_heap

In an abstract sense pop_heap is quite similar to push_heap. In push_heap we started at the last array element and climbed from there up the tree until we would find a node where to insert the new value into the heap. Every time we had reached the next parent node we moved its value down to where we had just come from.

With pop_heap its the other way round. We start at the root of the tree and descend from there by selecting an appropriate child. Every time we lift the value of the selected child to the node where just are. We repeat

this process until we find a node where we can insert the last array element into the heap. Once this is done, we can safely place the maximum element (that is the the original root node) at the last element of the array.

The following two figures illustrate how `pop_heap` affects an array, which is shown again as a tree with blue and grey nodes, representing heap and non-heap nodes, respectively. Figure 9.28 is in fact the same figure as Figure 9.2.

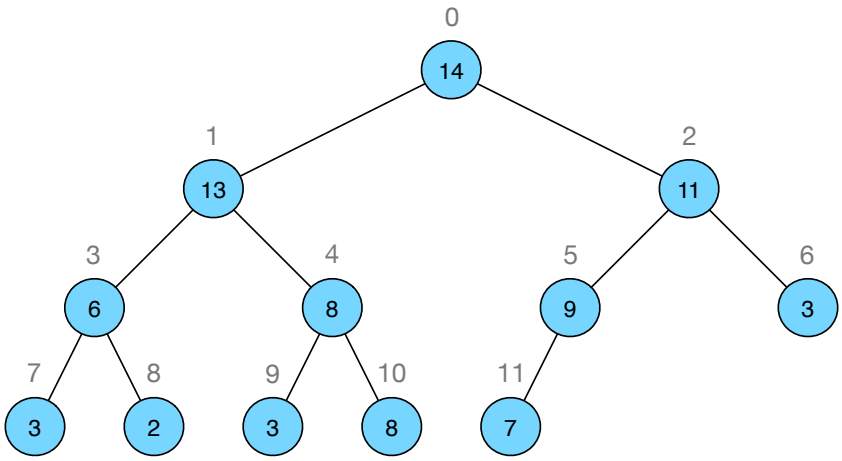

Figure 9.28.: Heap before the call of `pop_heap`

Figure 9.29, on the other hand, shows the heap after the call of `pop_heap` together with arrows that indicate how our implementation moves around elements in the underlying array. We can see that the first element of the original array, where the maximum of the heap resides, is now the last element of the array. Furthermore, the last array element is not part of the heap anymore. The dashed nodes highlight which heap nodes have changed during the call to `pop_heap`. The arrows indicate the *cyclic reordering* of array elements to achieve the desired result.

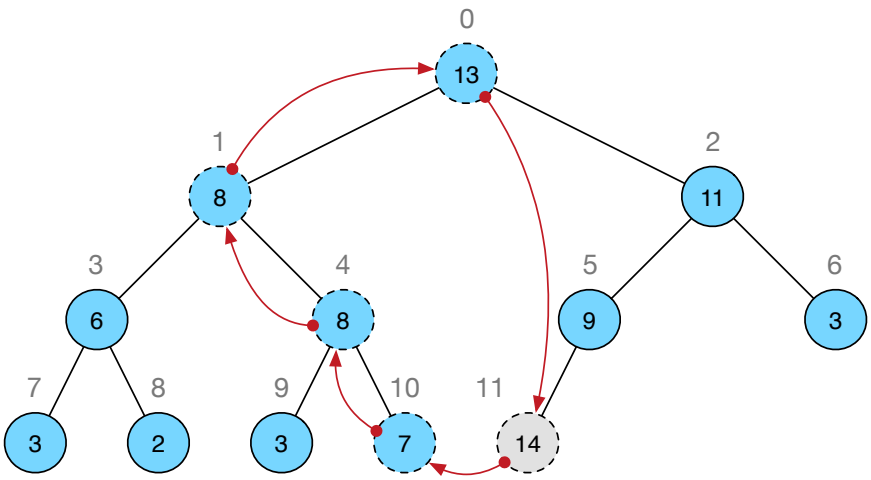

Figure 9.29.: Heap after the call of `pop_heap`

As in the case of `push_heap` [9.23] we will subdivide the discussion of the implementation of `pop_heap` [9.30] into a prologue, main act, and epilogue.

```
void
pop_heap(value_type* a, size_type n)
{
  if (1u < n) {
    //@ assert max:  MaxElement(a, n, 0);
    if (a[n - 1u] < a[0u]) { // otherwise a[0] == a[n-1] and nothing to be done
      size_type p = 0u;
      const value_type v = a[n - 1u];
      a[n - 1u] = a[p];
      //@ assert max:       a[n-1] == a[p];
      //@ assert update:    ArrayUpdate{Pre,Here}(a, n, n-1, a[p]);
      //@ assert heap:      Heap(a, n-1);
      //@ assert reorder:   MultisetParity{Pre,Here}(a, n, a[p], v);

      size_type c = heap_child(a, n - 1u, p);

      /*@
        loop invariant bounds:     0 <= p < c <= n-1;
        loop invariant parent:     c <  n-1 ==> p == HeapParent(c);
        loop invariant child:      c <  n-1 ==> HeapLeft(p) < n-1;
        loop invariant child:      c == n-1 ==> n-1 <= HeapLeft(p);
        loop invariant child:      HeapLeft(p)  < n-1 ==> a[HeapLeft(p)]  <= a[c];
        loop invariant child:      HeapRight(p) < n-1 ==> a[HeapRight(p)] <= a[c];
        loop invariant unchanged:  Unchanged{LoopEntry,Here}(a, p, n);
        loop invariant update:     a[p] == a[HeapParent(p)];
        loop invariant max:        UpperBound(a, n, a[n-1]);
        loop invariant reorder:    MultisetParity{Pre,Here}(a, n, a[p], v);
        loop invariant heap:       v < a[p];
        loop invariant heap:       Heap(a, n-1);
        loop assigns               p, c, a[0..n-2];
        loop variant               n - p;
       */
      for (; c < n - 1u && v < a[c];  p = c, c = heap_child(a, n - 1u, p)) {
        //@ assert max:         a[p] <= a[n-1];
        //@ assert heap:        a[c] <= a[p];
        if (a[c] < a[p]) {
          a[p] = a[c];
          //@ assert update:     ArrayUpdate{LoopCurrent,Here}(a, n, p, a[c]);
          //@ assert update:     ArrayUpdate{LoopCurrent,Here}(a, n-1, p, a[c]);
          //@ assert update:     MultisetUpdate{LoopCurrent,Here}(a, n, p, a[c]);
          //@ assert update:     a[c] == At{LoopCurrent}(a, c);
          //@ assert unchanged:  Unchanged{LoopEntry,Here}(a, p+1, n);
          //@ assert compatible: HeapCompatible(a, n-1, p, a[p]);
          //@ assert reorder:    MultisetParity{Pre,Here}(a, n, a[c], v);
        }
      }

      //@ ghost Epilogue: ;
      //@ assert child:      c == n-1  ||  a[c] <= v;
      //@ assert parent:     p <  n-1  &&  v < a[p];
      //@ assert compatible: HeapCompatible(a, n-1, p, v);
      a[p] = v;
      //@ assert update:     ArrayUpdate{Epilogue,Here}(a, n, p, v);
      //@ assert update:     MultisetUpdate{Epilogue,Here}(a, n, p, v);
      //@ assert reorder:    MultisetReorder{Pre,Here}(a, n);
      //@ assert heap:       Heap(a, n-1);
      //@ assert max:        UpperBound(a, n, a[n-1]);
    }
  }
}
```

Listing 9.30: Implementation of `pop_heap`

9.8.3. Prologue

In the prologue we check whether the initial heap contains at least two elements, initialize some variables, and also check whether the last array element is by chance equal to the maximum element of the heap, which resides at the index `p == 0` of the array. If this is not the case, then we set aside for future reference the last array element in the variable `v`. Finally we copy the value `a[p]` to its final destination at the end of the array. Note that this assignment only occurs if the respective values differ. This allows us, as in the case of `push_heap` [9.23], to formally describe the effect of the assignment using the predicate `ArrayUpdate` [7.2].

Figure 9.31 highlights the main effects of the prologue at the hand of our exemplary heap. Note that we have highlighted the root of the heap as the currently active node.

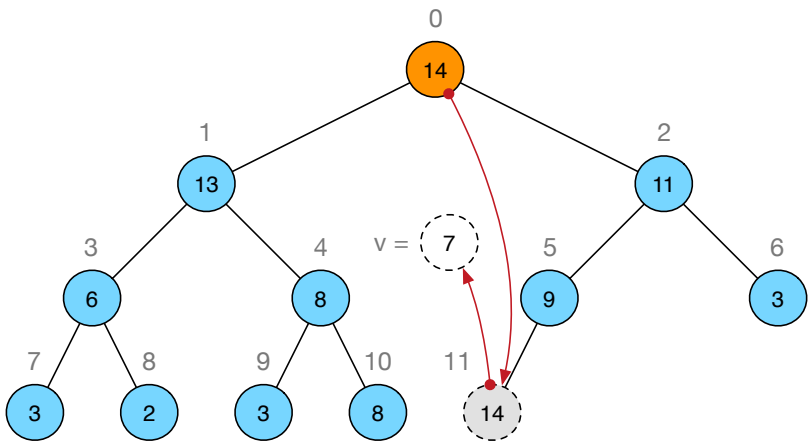

Figure 9.31.: Heap after the prologue of `pop_heap`

9.8.4. Main act

In the main act, we start at a child node `c` of the prologue's index `p`. This means that compared to the pre-state of `pop_heap` at the beginning of the main act the array `a[0..n-1]`

- contains the value `v` one time less
- contains the value `a[p]` one time more
- whereas all other values have not change their number of occurences.

Moreover, the maximum element of the original heap is now at the end of the array and we can only guarantee that the first $n - 1$ array elements got a heap. These observations are necessary reason for our loop invariants.

To be more precise, when we talk in the context of `pop_heap` of a *child node* we usually mean one of the possibly two children where the maximum of the values resides. We do this because copying that larger value to its parent node guarantees that the resulting tree is still a heap. We compute the maximum child of a node using the function `heap_child` [9.10].

Now, as long as the index `c` is not yet the index of the last array element of the heap and its value `a[c]` is less than `v`, we haven't found yet an index where we could insert `v` without violating the heap property.

In the loop body we proceed as follows.

- If `a[c]` is less than `a[p]` we copy the former value on the latter. As mentioned above, using the index `c` of the maximum child maintains heap property of the array. We use here the predicate `HeapCompatible` [9.7] to express that the insertion of the new value `a[p]` maintains the heap property of the array.

  The value `a[c]` now occurs one time more than in the pre-state whereas the now overwritten value `a[p]` occurs as often as in the pre-state of `pop_heap`. The value `v` continues to occur one time less than in the pre-state. We then proceed to the next iteration by setting `p` to `c` and computing the next maximum child node.

  As in the case of `push_heap` [9.23] the verification of the correct number of occurences of the involved values relies on the predicates `ArrayUpdate` [7.2] and `MultisetUpdate` [9.19] and on lemma `MultisetParity_Combined` [9.18].

- Otherwise, the array being a heap, we can conclude that `a[c]` equals `a[p]` and we continue with the next iteration after setting `p` to `c` and computing the corresponding new maximum child node.

The following three figures depict how the main act of `pop_heap` modifies step by step our example heap. In each step we highlight the currently active node `c`.

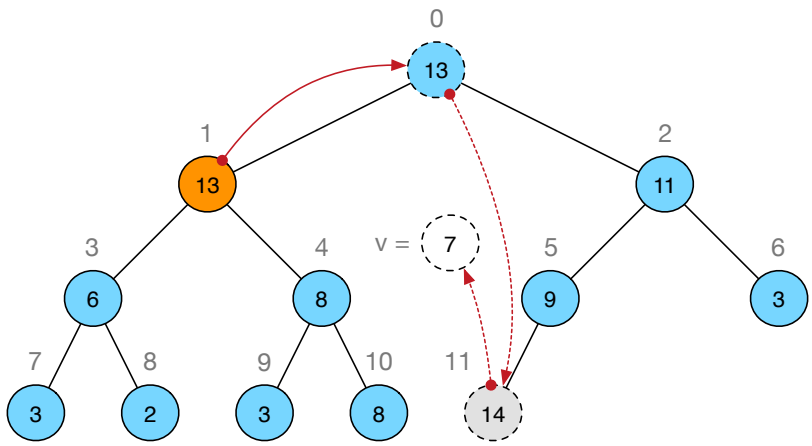

Figure 9.32.: Heap after the first iteration of of `pop_heap`

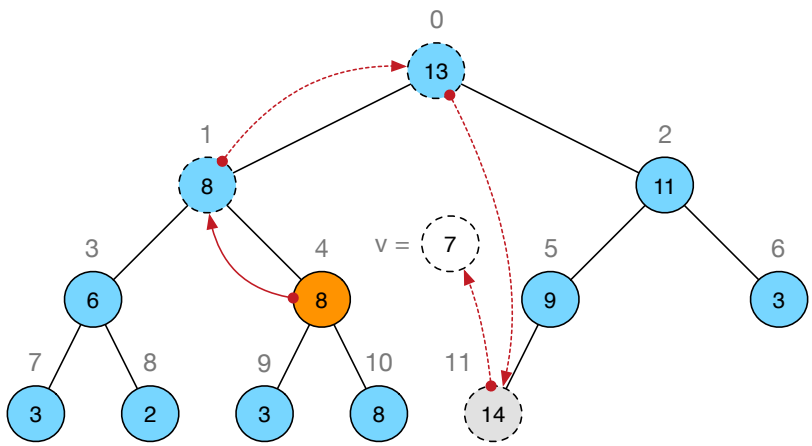

Figure 9.33.: Heap after the second iteration of `pop_heap`

Note that in the final step no value is actually copied as the involved nodes hold the same value.

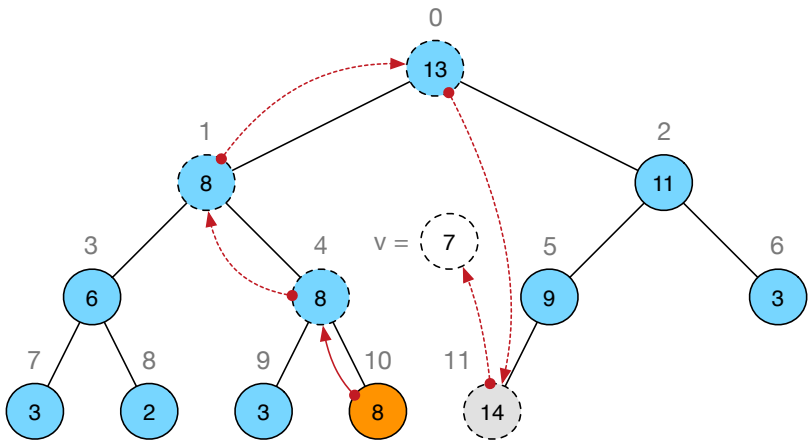

Figure 9.34.: Heap after the third iteration of `pop_heap`

We finally remark that in the main act the the last array element is never modified. Thus, the root element of the original element is still safely stored there.

9.8.5. Epilogue

After leaving the loop, we know that value `v` can be the inserted in the array at the index `p` without violating the heap property of the first $n - 1$ elements. Moreover, compared to the pre-state of `pop_heap` the array `a[0..n-1]` still

- contains the value `v` one time less
- contains the value `a[p]` one time more
- whereas all other values have not change their number of occurences.

In other words, assigning the value `v` to `a[p]` cancels this imbalance and establishes that `pop_heap` only reorders the array elements.

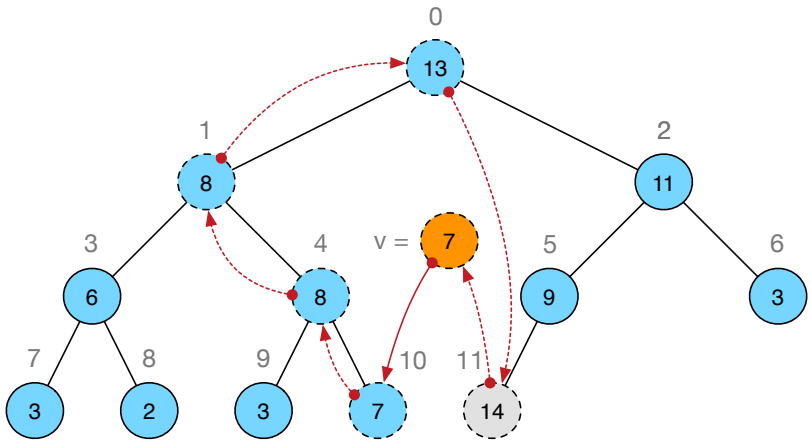

Figure 9.35.: Heap after the epilogue of `pop_heap`

In Figure 9.35 we have marked the value `v` as the currently active node despite not being an array element.

9.9. The `make_heap` algorithm

Whereas in the C++ Standard Library [19, §28.7.7.3] `make_heap` works on a pair of generic random access iterators, our version operators on a range of `value_type`. Thus the signature of `make_heap` reads

```
void make_heap(value_type* a, size_type n);
```

The function `make_heap` rearranges the elements of the given array `a[0..n-1]` such that they form a heap.

As an examples we look at the array in Figure 9.36. The elements of this array do not form a heap, as indicated by the grey colouring. Executing the `make_heap` algorithm on this array rearranges its elements so that they form a heap as shown in Figure 9.3.

8	8	7	3	14	11	3	6	2	3	13	9

Figure 9.36.: Array before the call of `make_heap`

9.9.1. Formal specification of `make_heap`

The following listing shows the specification of `make_heap`.

```
/*@
   requires   valid:    \valid(a + (0..n-1));

   terminates           \true;
   exits                \false;
   assigns              a[0..n-1];

   ensures    heap:     Heap(a, n);
   ensures    reorder:  MultisetReorder{Old,Here}(a, n);
*/
void
make_heap(value_type* a, size_type n);
```

Listing 9.37: Formal specification of `make_heap`

Like with `push_heap` the formal specification of `make_heap` must ensure that the resulting array is a heap of size `n` and contains the same multiset of elements as in the pre-state of the function. These properties are expressed by the `heap` and `reorder` postconditions respectively. The `reorder` postcondition uses the predicate `MultisetReorder` [7.55] to ensure that `make_heap` only rearranges the array elements.

9.9.2. Implementation of `make_heap`

The implementation of `make_heap`, shown in the next listing, is straightforward. From low to high the array's elements are pushed to the growing heap. We used `i < n` as loop condition, rather than the more tempting `i <= n`, in order to admit also `n == SIZE_TYPE_MAX`; as a consequence, we had to call `push_heap` [9.20] with `i+1`. The iteration starts at `i+1 == 2`, because an array with length one is a heap already.

```
void
make_heap(value_type* a, size_type n)
{
  if (0u < n) {
    /*@
       loop invariant bounds:     1 <= i <= n;
       loop invariant heap:       Heap(a, i);
       loop invariant reorder:    MultisetReorder{Pre,Here}(a, n);
       loop invariant unchanged:  Unchanged{Pre,Here}(a, i+1, n);
       loop assigns   i, a[0..n-1];
       loop   variant n - i;
    */
    for (size_type i = 1u; i < n; ++i) {
      push_heap(a, i + 1u);
      //@ assert reorder:    MultisetReorder{LoopCurrent,Here}(a, i+1);
      //@ assert unchanged:  Unchanged{LoopCurrent,Here}(a, i+1, n);
      //@ assert reorder:    MultisetReorder{LoopCurrent,Here}(a, n);
    }

    //@ assert reorder:    MultisetReorder{Pre,Here}(a, n);
  }

  //@ assert  heap: Heap(a, n);
}
```

Listing 9.38: Implementation of `make_heap`

Since the loop statement consists just of a call to `push_heap` [9.20] we obtain the both loop invariants `heap` and `reorder` by simply lifting them from the contract of `push_heap`.

The postcondition of `push_heap` only specifies the multiset of elements from index 0 to `i`. We therefore also have to specify that the elements from index `i+1` to `n-1` are only reordered. This property can be derived from the `unchanged` property of `push_heap`.

9.10. The `sort_heap` algorithm

Whereas in the C++ Standard Library [19, §28.7.7.4] `sort_heap` works on a range of random access iterators, our version operates on an array of `value_type`. We therefore use the following signature for `sort_heap`

```
void sort_heap(value_type* a, size_type n);
```

The function `sort_heap` rearranges the elements of a given heap `a[0..n-1]` in increasing order. Thus, applying `sort_heap` to the heap in Figure 9.3 produces the increasing array in Figure 9.39.

2	3	3	3	6	7	8	8	9	11	13	14

Figure 9.39.: Array after the call of `sort_heap`

9.10.1. Formal specification of `sort_heap`

The following listing shows our specification of `sort_heap`. The formal specification of `sort_heap` must ensure that the resulting array is increasing. Furthermore the multiset contained by the array must be the same as in the pre-state of the function. The postconditions `increasing` and `reorder` express these properties, respectively. The specification effort is relatively simple because we can reuse

```
/*@
   requires   valid:       \valid(a + (0..n-1));
   requires   heap:        Heap(a, n);

   terminates              \true;
   exits                   \false;
   assigns                 a[0..n-1];

   ensures    reorder:     MultisetReorder{Old,Here}(a, n);
   ensures    increasing:  Increasing(a, n);
*/
void
sort_heap(value_type* a, size_type n);
```

Listing 9.40: Formal specification of `sort_heap`

9.10.2. Implementation of `sort_heap`

The implementation of `sort_heap` is relatively simple because it relies on `pop_heap` [9.27] performing essential work. Our implementation of `sort_heap` repeatedly calls `pop_heap` to extract the maximum of the shrinking heap and adding it to the part of the array that is already in increasing order. The loop invariants of `sort_heap` describe the content of the array in two parts. The first `i` elements form a heap and are described by the `heap` invariant. The last `n-i` elements are already arranged in increasing order.

As already mentioned in the introduction of Chapter 6, we use the predicate `WeaklyIncreasing` [6.2] for the loop annotation `increasing`. Thus, after leaving the loop we have in fact "only" shown that `WeaklyIncreasing(a, n)` holds. In order to derive from this fact the final assertion `increasing` that uses the predicate `Increasing` [6.1] we rely on lemma `WeaklyIncreasing_Increasing` [6.3].

```
void
sort_heap(value_type* a, size_type n)
{
  /*@
     loop invariant bound:       0 <= i <= n;
     loop invariant heap:        Heap(a, i);
     loop invariant lower:       LowerBound(a, i, n, a[0]);
     loop invariant reorder:     MultisetReorder{Pre,Here}(a, 0, n);
     loop invariant increasing:  WeaklyIncreasing(a, i, n);
     loop assigns i, a[0..n-1];
     loop variant i;
  */
  for (size_type i = n; i > 1u; --i) {
    //@ ghost Before: ;
    pop_heap(a, i);
    //@ assert lower:    LowerBound(a, i, n, a[i-1]);
    //@ assert reorder:  MultisetReorder{Before,Here}(a, 0, i);
    //@ assert reorder:  Unchanged{Before,Here}(a, i, n);
  }

  //@ assert increasing: Increasing(a, n);
}
```

Listing 9.41: Implementation of `sort_heap`

To verify the property `reorder` we rely on the lemmas `MultisetReorder` [7.55], applied to the assertions introduced after the call to `pop_heap`. Using the ghost label `Before`, these assertions establish

- `MultisetReorder{Before,Here}(a, 0, i)` together with
- `Unchanged{Before,Here}(a, i, n)`,

which imply the desired loop invariant `MultisetReorder{Pre,Here}(a, 0, n)`.

10. Sorting Algorithms

Many issues in computer science can be exemplified in the field of sorting algorithms; see e.g. [24] for a famous textbook. Therefore we arrange some of the most common classic sorting algorithms. In this chapter, we present algorithms of the C++ Standard Library [19, §28.7.1] that are related to the task of sorting a linear array.

Following [25], we have also used (C rephrasings of) functions from the C++ Standard Library as far as possible to implement the different algorithmic approaches.

- `is_sorted` in §10.1 is an algorithm that checks if a given array is already in increasing order.
- `partial_sort` in §10.2 rearranges a given array into two parts. All elements in the first part are less or equal than those of the second part. Moreover, while the first part is sorted, the order of elements in the second part is unspecified.
- `bubble_sort` in §10.3 describes a simple, well-known and sorting algorithm.[19]
- `selection_sort` in §10.4 presents the classic *selection sort* algorithm.[20]
- `insertion_sort` in §10.5 the also well-known *insertion sort* algorithm.[21]
- `heap_sort` in §10.6 describes the quite efficient *heap sort*, which relies on the algorithms presented in Chapter 9.[22]
- `merge` in §10.7 the *merge* algorithm from *merge sort*.[23]

While `heap_sort` achieves a run-time complexity upper bound of $O(n \cdot \log(n))$ due to the efficiency of the heap data structure, both `selection_sort` and `insertion_sort` need $O(n^2)$ in the average case, and also in the worst case.

Note that the `sort` algorithm from the C++ Standard Library is not handled here because it typically relies on *introspection sort* which is sophisticated mix of various classic algorithms.[24] In future releases we plan to handle the more algorithms related sorting.

[19] See `https://en.wikipedia.org/wiki/Bubble_sort`
[20] See `https://en.wikipedia.org/wiki/Selection_sort`
[21] See `https://en.wikipedia.org/wiki/Insertion_sort`
[22] See `https://en.wikipedia.org/wiki/Heapsort`
[23] See `https://en.wikipedia.org/wiki/Merge_sort`
[24] See `https://en.wikipedia.org/wiki/Introsort`

The sorting algorithms in this chapter essentially share the following contract; it is their implementations that differ fundamentally.

```
/*@
   requires valid: \valid(a + (0..n-1));

   assigns  a[0..n-1];

   ensures increasing:  Increasing(a, n);
   ensures reorder:     MultisetReorder{Old, Here}(a, n);
*/
void xxx_sort(value_type* a, size_type n);
```

As mentioned in the introduction of Chapter 6, we use the predicate `Increasing` [6.1] in the contracts of our sorting algorithms but often resort to the simpler predicate `WeaklyIncreasing` [6.2] in the loop invariants and assertions. In order to conclude that the desired postcondition `Increasing(a, n)` holds, we rely on lemma `WeaklyIncreasing_Increasing` [6.3].

10.1. The `is_sorted` algorithm

Our version of the `is_sorted` algorithm compared to the C++ Standard Library [19, §28.7.1.5] has the signature

```
bool is_sorted(const value_type* a, size_type n);
```

It returns **`true`** if the given array is in increasing order, and **`false`** otherwise.

10.1.1. Formal specification of `is_sorted`

The following listing shows the acsl specification of `is_sorted`. In the contract, we use the predicate `Increasing` [6.1], which states that any array element is always less or equal to any other element right of it. We'll use an easier-to-handle predicate in the implementation of `is_sorted` [10.2].

```
/*@
  requires   valid:   \valid_read(a + (0..n-1));

  terminates          \true;
  exits               \false;
  assigns             \nothing;

  ensures    result:  \result <==> Increasing(a, n);
*/
bool
is_sorted(const value_type* a, size_type n);
```

Listing 10.1: Formal specification of `is_sorted`

10.1.2. Implementation of `is_sorted`

The implementation of `is_sorted` is shown in the next Listing. As usual, `is_sorted` doesn't compare every array element to all that are right to it, but only to the immediately adjacent one, which is of course more efficient. For this, we use the predicate `WeaklyIncreasing` [6.2] in the loop invariant of the implementation.

```
bool
is_sorted(const value_type* a, size_type n)
{
  if (0u < n) {
    /*@
        loop invariant increasing: WeaklyIncreasing(a, i+1);
        loop assigns i;
        loop variant n - i;
    */
    for (size_type i = 0u; i < n - 1u; ++i) {
      if (a[i] > a[i + 1u]) {
        return false;
      }
    }
  }

  return true;
}
```

Listing 10.2: Implementation of `is_sorted`

Since our implementation uses `WeaklyIncreasing` in its loop invariant, and follows the same principle in its code, its verification is straight-forward—except for the final reasoning that `WeaklyIncreasing (a,n)` implies `Increasing(a,n)`.

We have the lemma `WeaklyIncreasing_Increasing` [6.3] for that step, which needs to be proven manually with Coq. The converse lemma `Increasing_WeaklyIncreasing` [6.3] is proven automatically, but isn't actually needed to verify our `is_sorted` implementation. Alternatively, we could have dragged the predicate `Increasing` along the loop, which happens to cause no particular problems in this case.

10.2. The `partial_sort` algorithm

Our version of the `partial_sort` algorithm compared to the C++ Standard Library [19, §28.7.1.3] has the signature

```
void partial_sort(value_type* a, size_type m, size_type n);
```

The algorithm *reorders* the given array `a` in such a way that it represents a *partition*: each member of the left part `a[0..m-1]` is less or equal to each member of the right part `a[m..n-1]`. Moreover, the algorithm *sorts* the left part in increasing order. The order of elements in the right part, however, is *unspecified*. Figure 10.3 uses a bar chart to depict a typical result of a call `partial_sort(a, m, n)`. In the post-state, the left and the right part is colored in green and orange, respectively.

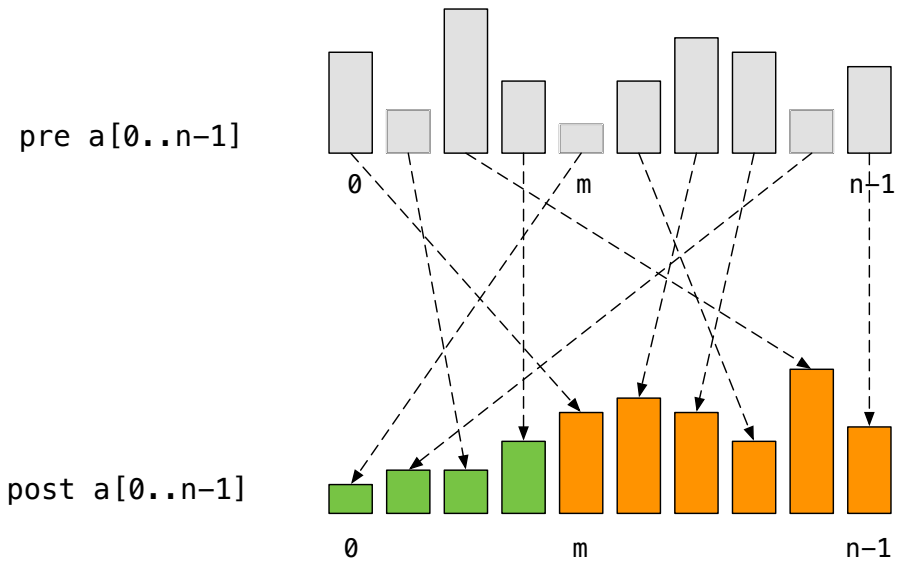

Figure 10.3.: Effects of `partial_sort`

10.2.1. The predicate `Partition`

We start by introducing the new predicate `Partition` [10.4] which formalizes the partitioning property.

```
/*@
  axiomatic Partition
  {
    predicate
    Partition{L}(value_type* a, integer m, integer n) =
      \forall integer i, k; 0 <= i < m <= k < n  ==>  a[i] <= a[k];
  }
*/
```

Listing 10.4: The logic definition(s) `Partition`

The lemmas in the following listing are used in proofs of properties and annotations related to the loop invariants `upper`, `lower`, and `partition` of `partial_sort`.

```
/*@
  axiomatic PartitionLemmas
  {
    lemma MultisetReorder_SomeEqual{K,L}:
      \forall value_type *a, integer n, i;
        0 < n                          ==>
        0 <= i < n                     ==>
        MultisetReorder{K,L}(a, n)     ==>
        SomeEqual{K}(a, n, \at(a[i],L));

    lemma MultisetReorder_LowerBound{K,L}:
      \forall value_type* a, integer n, value_type v;
        0 <= n                         ==>
        MultisetReorder{K,L}(a, n)     ==>
        LowerBound{K}(a, n, v)         ==>
        LowerBound{L}(a, n, v);

    lemma MultisetReorder_UpperBound{K,L}:
      \forall value_type* a, integer n, value_type v;
        0 <= n                          ==>
        MultisetReorder{K,L}(a, n)      ==>
        UpperBound{K}(a, n, v)          ==>
        UpperBound{L}(a, n, v);

    lemma MultisetReorder_PartitionLowerBound{K,L}:
      \forall value_type* a, integer m, n;
        0 < m <= n                       ==>
        MultisetReorder{K,L}(a, 0, m)    ==>
        Partition{K}(a, m, n)            ==>
        Unchanged{K,L}(a, m, n)          ==>
        LowerBound{L}(a, m, n, \at(a[0],L));
  }
*/
```

Listing 10.5: The logic definition(s) `PartitionLemmas`

- Lemma `MultisetReorder_SomeEqual` states that a value `a[i]` taken from a range `a[0..n-1]` after some reordering must have been in that range already before reordering. It is used to prove the subsequent lemmas.

- Lemma `MultisetReorder_LowerBound` informally says that a lower bound `v` of a range `a[0..n-1]` keeps its property even after the range is reordered.

- Dually, lemma `MultisetReorder_UpperBound` says that reordering a range doesn't affect any of its upper bounds.

- Lemma `MultisetReorder_PartitionLowerBound` describes a more particular situation: if each element in `a[0..m-1]` is known to be a less or equal than element `a[m..n-1]` and the former range is reordered while the latter is kept untouched, then `a[0]` will still be a lower bound of `a[m..n-1]`. We employ this lemma to infer that, after `push_heap` [9.20] was called, the new heap maximum `a[0]`, is a lower bound of `a[m..i]`,

The proof of `MultisetReorder_SomeEqual` [10.5] relies on the lemma `Count_SomeEqual` [4.47]. We also rely on the lemma `MultisetSwap_Middle` [7.59] in order to verify that the loop invariant `reorder` is preserved.

10.2.2. Formal specification of `partial_sort`

The formal specification of the `partial_sort` function is shown in the following listing. It uses the just introduced predicate `Partition` and reuses the previously defined predicates `Increasing` [6.1] and `MultisetReorder` [7.55].

```
/*@
  requires   valid:      \valid(a + (0..n-1));
  requires   split:      0 <= m <= n;

  terminates              \true;
  exits                   \false;
  assigns                 a[0..n-1];

  ensures    reorder:     MultisetReorder{Old,Here}(a, n);
  ensures    partition:   Partition(a, m, n);
  ensures    increasing:  Increasing(a, m);
*/
void
partial_sort(value_type* a, size_type m, size_type n);
```

Listing 10.6: Formal specification of `partial_sort`

10.2.3. Implementation of `partial_sort`

Overview and structure

The implementation of `partial_sort` is shown in Listings 10.7 and 10.9. For readability, the code is split into two parts. The first listing contains the overall structure of the algorithm and the scan loop, while the second listing provides the detailed reasoning inside the loop body, including assertions.

The algorithm initially calls `make_heap` [9.37] to rearrange the left part `a[0..m-1]` into a heap. After that, it scans the right part, from left to right, for elements that are too small; each such element is exchanged for the left part's maximum by applying `pop_heap` [9.27] and `push_heap` [9.20] appropriately. When the scan is done, the smallest elements are collected in the left part. Finally, the heap is converted into an increasingly ordered range by `sort_heap` (9.10).

Before analyzing the preservation of the loop invariants in detail, we note that the body of the scan loop makes several procedure calls whose correctness depends on properties that are not all explicit in the surrounding control structure. These properties are documented locally by means of `assert` clauses and a small number of *ghost labels*, which mark relevant program points for later reference.

```
void
partial_sort(value_type* a, size_type m, size_type n)
{
  if (m > 0u) {
    make_heap(a, m);

    //@ assert reorder: Unchanged{Pre,Here}(a, m, n);
    /*@
      loop invariant bound:     m <= i <= n;
      loop invariant heap:      Heap(a, m);
      loop invariant upper:     UpperBound(a, 0, m, a[0]);
      loop invariant lower:     LowerBound(a, m, i, a[0]);
      loop invariant reorder:   MultisetReorder{Pre,Here}(a, i);
      loop invariant unchanged: Unchanged{Pre,Here}(a, i, n);
      loop assigns              i, a[0..n-1];
      loop variant              n-i;
    */
    for (size_type i = m; i < n; ++i) {
      if (a[i] < a[0u]) {
        //@ ghost BeforePop: ;
        pop_heap(a, m);
        //@ assert lower:      a[0] <= a[m-1];
        //@ assert lower:      a[i] <  a[m-1];
        //@ assert lower:      LowerBound(a, m, i, a[m-1]);
        //@ assert upper:      UpperBound(a, 0, m-1, a[0]);
        //@ assert upper:      UpperBound(a, 0, m,   a[m-1]);
        //@ assert partition:  Partition(a, m, i);
        //@ assert unchanged:  Unchanged{BeforePop,Here}(a, m, i);
        //@ assert reorder:    MultisetReorder{BeforePop,Here}(a, m);
        //@ assert reorder:    MultisetReorder{Pre,Here}(a, i);
        //@ assert unchanged:  Unchanged{Pre,Here}(a, i, n);
```

Listing 10.7: Implementation of `partial_sort` (1): overall structure and scan loop

Loop invariants and their preservation

In the scan loop, we maintain the following invariants:

- the left part is a heap (invariant `heap`);
- its maximal element, `a[0]`, is a separating element between the left part `a[0..m-1]` and the right part `a[m..i-1]`, that is, an upper bound of the left part (invariant `upper`) and a lower bound of the right part (invariant `lower`);
- the range `a[i..n-1]` is yet unchanged (invariant `unchanged`); and
- only permutation operations have been applied to `a[0..i-1]` (invariant `reorder`).

In order to preserve these invariants after incrementing `i`, nothing has to be done if `a[0]` is already a lower bound for `a[i]`. Otherwise, execution enters the `then` branch of the loop. The intermediate states of this case are depicted in Figure 10.8.

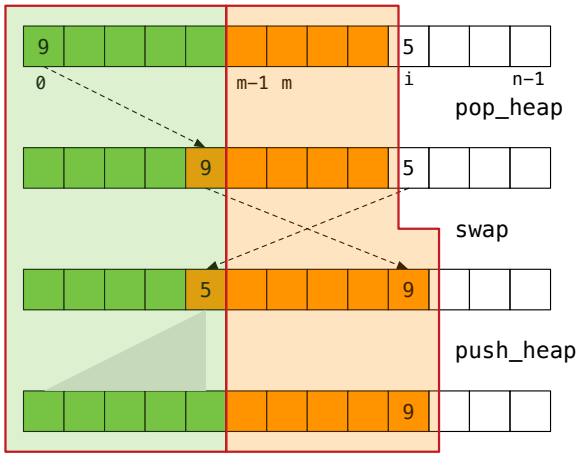

Figure 10.8.: An iteration of `partial_sort`

The elements considered so far are shown using the same coloring convention as in Figure 10.3; in particular, the heap part is shown in green. The overlaid transparent red shape indicates the ranges to which `Partition` applies in each state. The figure assumes the initial contents of `a[0]` and `a[i]` to be 9 and 5; for generality, we denote them by p and q, respectively.

After `pop_heap` and `swap`, we have p at `a[i]` and q at `a[m-1]`. At that point we know:

1. $q < p \leq$ `a[k]` for each $m \leq k < i$;
2. $q < p =$ `a[i]`;
3. `a[j]` $\leq p \leq$ `a[k]` for each $0 \leq j < m - 1$ and each $m \leq k < i$; and
4. `a[j]` $\leq p =$ `a[i]` for each $0 \leq j < m - 1$.

Altogether, we obtain `a[j]` $\leq p \leq$ `a[k]` for each $0 \leq j < m$ and each $m \leq k < i + 1$. Thus, `Partition (a,m,i+1)` holds, although no explicit separating element can be named at this point.

After calling `push_heap`, which only reorders elements inside the left part, this property is preserved. Moreover, `a[0]` again becomes an upper bound of the left part and hence a separating element between `a[0..m-1]` and `a[m..i]`. The loop invariants `upper` and `lower` are therefore re-established. These two invariants are eventually used to prove the contract property `partition`.

```
        //@ ghost BeforeSwap: ;
        //@ assert reorder:    MultisetReorder{Pre,Here}(a, i+1);
        swap(a + m - 1u, a + i);
        //@ assert swapped:    ArraySwap{BeforeSwap,Here}(a, m-1, i, n);
        //@ assert unchanged:  Unchanged{BeforeSwap,Here}(a, m-1);
        //@ assert unchanged:  Unchanged{Pre,Here}(a, i+1, n);
        //@ assert reorder:    MultisetReorder{BeforeSwap,Here}(a, m-1, i+1);
        //@ assert reorder:    MultisetReorder{BeforeSwap,Here}(a, i+1);
        //@ assert reorder:    MultisetReorder{Pre,Here}(a, i+1);
        //@ assert lower:      a[m-1] < a[i];
        //@ assert partition:  Partition(a, m, i+1);
        //@ assert upper:      UpperBound(a, 0, m-1, a[0]);

        //@ ghost BeforePush: ;
        push_heap(a, m);
        //@ assert heap:       Heap(a, m);
        //@ assert unchanged:  Unchanged{BeforePush,Here}(a, m, i+1);
        //@ assert unchanged:  Unchanged{BeforePush,Here}(a, i+1, n);
        //@ assert reorder:    MultisetReorder{Pre,Here}(a, i+1);
        //@ assert upper:      UpperBound(a, 0, m,   a[0]);
        //@ assert lower:      LowerBound(a, m, i+1, a[0]);
      }
    }

    //@ assert partition: Partition(a, m, n);
    //@ ghost BeforeSort: ;
    sort_heap(a, m);
    //@ assert unchanged:  Unchanged{BeforeSort,Here}(a, m, n);
    //@ assert reorder:    MultisetReorder{BeforeSort,Here}(a, m);
    //@ assert reorder:    MultisetReorder{Pre,Here}(a, n);
    //@ assert partition:  Partition(a, m, n);
  }
}
```

Listing 10.9: Implementation of `partial_sort` (2):

Remarks on specification style and complexity

Compared to its size, the algorithm performs many procedure calls. In this respect, it is closer to real-world software than most of the algorithms presented in this tutorial. For almost every procedure call, the callee's contract is instantiated to the actual parameters and recorded as assertions that may refer to the pre-state of a procedure call by means of *ghost labels*. In contrast, `assert` clauses are used to document intermediate reasoning steps needed for subsequent arguments.

The worst-case time complexity of the implementation is $O((n+m)\cdot\log m)$. An implementation that ignores `m` and simply sorts `a[0..n-1]` also satisfies the contract of `partial_sort` [10.6], but has $O(n\cdot\log n)$ complexity. Elementary arithmetic shows that `partial_sort` is asymptotically better if and only if

$$\log m < \frac{n}{m}\cdot\log\left(\frac{n}{m}\right),$$

that is, when n is sufficiently larger than m.

10.3. The bubble_sort algorithm

The bubble_sort algorithm traverses the given array a[0..n-1] from left to right, maintaining a right-adjusted, constantly growing range a[n-i..n-1] that is already in increasing order. We achieve this range by iterating through the array and swapping two adjacent elements, if their respective value are in the wrong order.

10.3.1. Formal specification of bubble_sort

The following listing shows our (generic sorting) contract for bubble_sort.

```
/*@
  requires   valid:       \valid(a + (0..n-1));

  terminates              \true;
  exits                   \false;
  assigns                 a[0..n-1];

  ensures    reorder:     MultisetReorder{Old,Here}(a, n);
  ensures    increasing:  Increasing(a, n);
 */
void
bubble_sort(value_type* a, size_type n);
```

Listing 10.10: Formal specification of bubble_sort

10.3.2. Implementation of bubble_sort

Our implementation of bubble_sort is shown in the next listing. As it is typical for bubble_sort, the implementation uses two nested loops.

We first discuss the verification of the fact that bubble_sort produces an increasing array. For this we introduce for the *outer loop* the invariant increasing. This loop annotation states that the subrange a[n-i+1..n-1] is in increasing order. An important ingredient on the verification of the increasing property is the claim that the first element a[n-i+1] of the already sorted subrange is an upper bound of *all* elements left of it. This claim is encoded in the loop invariant upper of the outer bound. In order to support this claim up we exploit the fact that the index j of the inner loop points to the maximum element of the subrange a[0..j]. We formalize this last property in the loop invariant max.

Note that the loop invariants increasing and upper occur also in the inner loop. This shall "assure" the outer loop that the inner loop really preserves these properties.

```
void
bubble_sort(value_type* a, size_type n)
{
  if (0 < n) {
    /*@
      loop invariant bound:          1 <= i <= n;
      loop invariant increasing:     WeaklyIncreasing(a, n-i+1, n);
      loop invariant upper:          1 < i ==> UpperBound(a, n-i+1, a[n-i+1]);
      loop invariant reorder:        MultisetReorder{Pre,Here}(a, n);
      loop assigns i, a[0..n-1];
      loop variant n-i;
    */
    for (size_type i = 1u; i < n; ++i) {
      /*@
        loop invariant bound:        0 <= j <= n-i;
        loop invariant increasing:   WeaklyIncreasing(a, n-i+1, n);
        loop invariant upper:        1 < i ==> UpperBound(a, n-i+1, a[n-i+1]);
        loop invariant max:          MaxElement(a, j+1, j);
        loop invariant reorder:      MultisetReorder{LoopEntry,Here}(a, j+1);
        loop invariant reorder:      Unchanged{LoopEntry,Here}(a, j+1, n);
        loop assigns                 j, a[0..n-1];
        loop variant n-j;
      */
      for (size_type j = 0u; j < n - i; ++j) {
        if (a[j] > a[j + 1u]) {
          //@ assert max:       MaxElement(a, j+1, j);
          //@ assert reorder:   MultisetReorder{LoopEntry,Here}(a, 0, j+1);
          //@ assert reorder:   MultisetReorder{LoopEntry,Here}(a, 0, j+2);
          swap(&a[j], &a[j + 1u]);
          //@ assert max:       MaxElement(a, j+2, j+1);
          //@ assert swap:      ArraySwap{LoopCurrent,Here}(a, j, j+1, n);
          //@ assert unchanged: Unchanged{LoopCurrent,Here}(a, j);
          //@ assert reorder:   a[j+1] == At{LoopCurrent}(a, j);
          //@ assert reorder:   a[j]   == At{LoopCurrent}(a, j+1);
          //@ assert reorder:   MultisetReorder{LoopCurrent,Here}(a, j, j+2);
        }
      }
    }
  }

  //@ assert increasing: Increasing(a, n);
}
```

Listing 10.11: Implementation of `bubble_sort`

We now discuss briefly the verification of the postcondition `reorder`. In each iteration of the outer loop various elements of the not yet sorted subrange `a[0..n-1]` are swapped with their respective neighbour. More specifically, we know for the iteration `j` of the *inner loop* that while subrange `a[0..j]` has been rearranged, the subrange `a[j+1..n-1]` has not been modified yet. Together this ensures that the loop invariant `reorder` holds for the *outer loop*.

10.4. The `selection_sort` algorithm

Our version of the `selection_sort` algorithm has the signature

```
void selection_sort(value_type* a, size_type n);
```

The `selection_sort` algorithm sorts an array in increasing order, left to right, by selecting in each step the minimum element of the remaining segment and *swaps* it with its first element. This implies that each member of the increasingly ordered initial segment is less or equal than each member of the remaining segment.

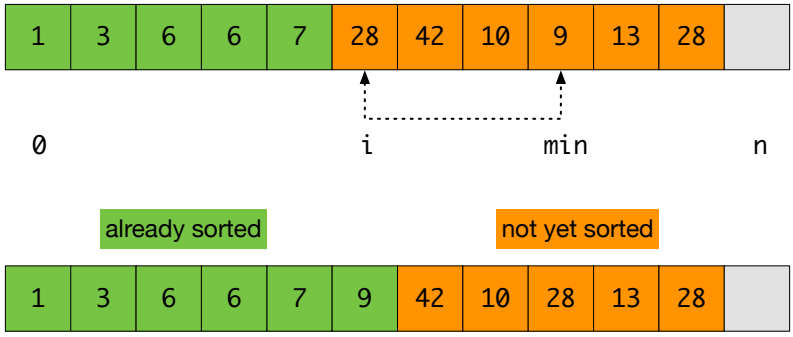

Figure 10.12.: An iteration of `selection_sort`

Figure 10.12 shows a typical situation in an example run. The algorithm will swap the `28` at position `i` with the `9` at position `min` to extend the increasingly ordered initial segment one field to the right.

10.4.1. Formal specification of `selection_sort`

The following listing shows the specification of `selection_sort`.

```
/*@
  requires   valid:       \valid(a + (0..n-1));

  terminates              \true;
  exits                   \false;
  assigns                 a[0..n-1];

  ensures    reorder:     MultisetReorder{Old,Here}(a, n);
  ensures    increasing:  Increasing(a, n);
*/
void
selection_sort(value_type* a, size_type n);
```

Listing 10.13: Formal specification of `selection_sort`

10.4.2. Implementation of `selection_sort`

The implementation of `selection_sort` is shown in the next listing. We use `min_element` [5.15] to find the minimum element of the remaining array segment.

```
void
selection_sort(value_type* a, size_type n)
{
  /*@
    loop invariant bound:       0 <= i <= n;
    loop invariant reorder:     MultisetReorder{Pre,Here}(a, n);
    loop invariant increasing:  WeaklyIncreasing(a, i);
    loop invariant increasing:  0 < i  ==> LowerBound(a, i, n, a[i-1]);
    loop assigns   i, a[0..n-1];
    loop variant   n - i;
  */
  for (size_type i = 0u; i < n; ++i) {
    const size_type sel = i + min_element(a + i, n - i);

    if (i < sel) {
      //@ ghost Before: ;
      swap(a + sel, a + i);
      //@ assert swapped: ArraySwap{Before,Here}(a, i, sel, n);
    }

    //@ assert reorder: MultisetReorder{LoopCurrent,Here}(a, n);
    //@ assert reorder: MultisetReorder{Pre,Here}(a, n);
  }

  //@ assert increasing: Increasing(a, n);
}
```

Listing 10.14: Implementation of `selection_sort`

The loop invariants `increasing` and `lower` establish that the initial segment `a[0..i-1]` is in increasing order and, respectively, state that `a[i-1]` is a lower bound of the remaining segment `a[i..n-1]`. Since the `min_element` call uses an address offset, we had to employ again the *shift lemmas* from the collection `ArrayBoundsShift` [6.14].

The loop invariant `reorder`, on the other hand, states that the multiset of values in the array `a` are only *rearranged* during the algorithm. While this is intuitively most obvious (as the call to `swap` [7.6] routine is the only code that modifies `a`), it took considerable effort to prove it formally; in particular, we need to capture the effect of calling `swap` on the array content.

In the current code, this effect is no longer expressed by a statement contract. Instead, we place a ghost label `Before` immediately before the call to `swap` [7.6] and then assert `swapped` afterwards. Together, these annotations relate the array state at `Before` to the state `Here` via `ArraySwap{Before,Here}(a, i, sel, n)`, which provides the required hypotheses for applying `MultisetSwap_Middle` [7.59]. This lemma, in turn, relies on the lemmas about `MultisetReorder` [7.55] and formalizes the fact that swapping two elements of an array is a reordering.

10.5. The `insertion_sort` algorithm

Like `selection_sort`, the algorithm `insertion_sort` traverses the given array `a[0..n-1]` left to right, maintaining a left-adjusted, constantly increasing range `a[0..i-1]` that is already in increasing order.

Unlike `selection_sort`, however, `insertion_sort` adds `a[i]` to the initial segment in the `i`th step (see Figure 10.15). It determines the (rightmost) appropriate position to insert `a[i]` by a call to `upper_bound` [6.8] and then uses `rotate` [7.27] to perform a *circular shift* to establish the insertion.

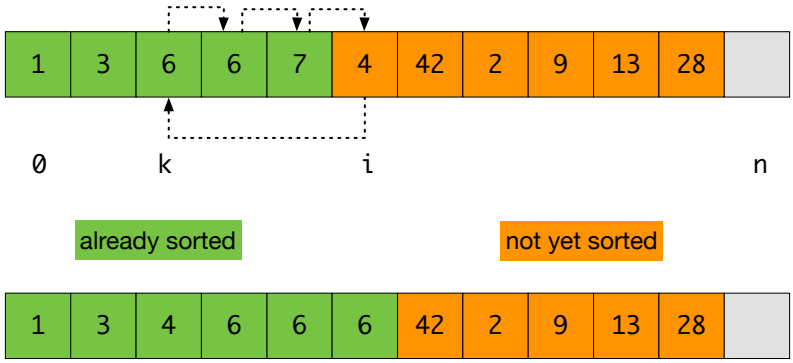

Figure 10.15.: An iteration of `insertion_sort`

10.5.1. Formal specification of `insertion_sort`

The following listing shows our (generic sorting) contract for `insertion_sort`.

```
/*@
  requires   valid:       \valid(a + (0..n-1));

  terminates              \true;
  exits                   \false;
  assigns                 a[0..n-1];

  ensures    reorder:     MultisetReorder{Old,Here}(a, n);
  ensures    increasing:  Increasing(a, n);
*/
void
insertion_sort(value_type* a, size_type n);
```

Listing 10.16: Formal specification of `insertion_sort`

10.5.2. Implementation of `insertion_sort`

The implementation of `insertion_sort` is shown in the next listing. We used an ACSL statement contract to specify those aspects of the `rotate` contract that are needed here. Properties related to the result of `insertion_sort` being in increasing order are labelled `increasing`. Properties related to the rearrangement of elements are labelled `reorder` and, whenever their order isn't changed, `unchanged`.

```
void
insertion_sort(value_type* a, size_type n)
{
  /*@
    loop invariant bound:         0 <= i <= n;
    loop invariant reorder:       MultisetReorder{Pre,Here}(a, 0, i);
    loop invariant unchanged:     Unchanged{Pre,Here}(a, i, n);
    loop invariant increasing:    WeaklyIncreasing(a, i);
    loop assigns   i, a[0..n-1];
    loop variant   n - i;
  */
  for (size_type i = 0u; i < n; ++i) {
    const size_type k = upper_bound(a, i, a[i]);
    //@ assert bound:   0 <= k <= i;
    //@ ghost Before: ;
    //@ assert lower:       StrictLowerBound(a, k, i, a[i]);
    //@ assert upper:       UpperBound(a, k, a[i]);
    rotate(a + k, i - k, i - k + 1u);
    //@ assert rotate:      Equal{Before,Here}(a, k, i, k+1);
    //@ assert rotate:      Equal{Before,Here}(a, i, i+1, k);
    //@ assert increasing:  WeaklyIncreasing(a, k+1, i+1);
    //@ assert increasing:  WeaklyIncreasing(a, i+1);
    //@ assert reorder:     MultisetReorder{LoopCurrent,Here}(a, 0, k);
    //@ assert reorder:     MultisetReorder{LoopCurrent,Here}(a, k, i+1);
    //@ assert reorder:     MultisetReorder{LoopCurrent,Here}(a, 0, i+1);
    //@ assert reorder:     MultisetReorder{Pre,Here}(a, i+1);
  }

  //@ assert increasing: Increasing(a, n);
}
```

Listing 10.17: Implementation of `insertion_sort`

When we originally implemented and verified `rotate`, we had not yet in mind to use that function inside `insertion_sort`. Consequently, the properties needed for the latter are not directly provided by the former. One possible solution would be to extend the contract of `rotate` [7.27] with the additional properties required by `insertion_sort` and to redo its verification. However, if `rotate` is assumed to be part of a pre-verified library, this approach is not feasible, since its implementation may not be available for re-verification.

Instead, we follow a different approach, namely to show that the original specification of `rotate` *implies* all properties required in `insertion_sort`. This is a further application of the Hoare calculus' implication rule (§3.3).

At the call site of `rotate` inside the loop of `insertion_sort`, we introduce the ghost label `Before` immediately before the call. This label captures the state of the array just prior to rotation and allows us to relate the post-state to the pre-state by means of assertions of the form `Equal{Before,Here}` on suitable subranges. In this way, the effects of `rotate` are expressed locally, without modifying its contract.

We use several lemmas, listed below, to make the necessary implications explicit and to assist the provers in establishing them. Some of these lemmas require manual proofs by induction.

Lemma `Increasing_Equal` [6.3] assumes that a range `a[m..n-1]` is ordered and states that any elementwise equal range `a[m+p..n+p-1]` is ordered as well. Together with the assertions relating `Before` and `Here`, this lemma is used to show that the call to `rotate` [7.27] preserves the order of those elements that are shifted upwards (cf. Figure 10.15).

Lemma `CircularShift_StrictLowerBound` [10.18] is employed to show that if the range `a[k..i-1]` has `a[i]` as a strict lower bound in the state labelled `Before`, then after the call to `rotate` it has `a[k]` as such a bound. This precisely reflects the fact that `rotate` is used as a *circular shift* at the call site.

Finally, lemma `CircularShift_MultisetReorder` [10.18] establishes that a circular shift merely reorders the range to which it is applied. Combined with the assertions relating `Before` and `Here`, this lemma allows us to conclude that the multiset of array elements is preserved across the call to `rotate`, as required by the loop invariant of `insertion_sort`.

```
/*@
   axiomatic CircularShiftLemmas
   {
     lemma CircularShift_StrictLowerBound{K,L}:
       \forall value_type* a, integer m, n;
         StrictLowerBound{K}(a, m, n, \at(a[n],K))  ==>
         Equal{K,L}(a, m, n, m+1)                   ==>
         Equal{K,L}(a, n, n+1, m)                   ==>
         StrictLowerBound{L}(a, m+1, n+1, \at(a[m],L));

     lemma CircularShift_MultisetReorder{K,L}:
       \forall value_type* a, integer m, n;
         0 <= m <= n                ==>
         Equal{K,L}(a, m, n, m+1)   ==>
         Equal{K,L}(a, n, n+1, m)   ==>
         MultisetReorder{K,L}(a, m, n+1);
   }
*/
```

Listing 10.18: The logic definition(s) `CircularShiftLemmas`

10.6. The heap_sort algorithm

The heap_sort algorithm has the signature

```
void heap_sort(value_type* a, size_type n);
```

It relies upon the heap algorithms discussed in Chapter 9 to efficiently transform the array into increasing order.

10.6.1. Formal specification of heap_sort

The following Listing shows the specification of heap_sort.

```
/*@
  requires   valid:       \valid(a + (0..n-1));

  terminates              \true;
  exits                   \false;
  assigns                 a[0..n-1];

  ensures    reorder:     MultisetReorder{Old,Here}(a, n);
  ensures    increasing:  Increasing(a, n);
*/
void
heap_sort(value_type* a, size_type n);
```

Listing 10.19: Formal specification of heap_sort

10.6.2. Implementation of heap_sort

The implementation of heap_sort, shown in the next listing is straightforward. Given the input array a[0..n-1], we use make_heap [9.37] to arrange it into a heap; after that, we call sort_heap [9.40] to sort this heap into increasing order.

```
void
heap_sort(value_type* a, size_type n)
{
  make_heap(a, n);
  sort_heap(a, n);
}
```

Listing 10.20: Implementation of heap_sort

10.7. The `merge` algorithm

Our version of the `merge` algorithm from the C++ standard library [19, 28.7.5] has the following signature.

```
void
merge(const value_type* a, size_type n,
      const value_type* b, size_type m, value_type* result);
```

The function `merge` combines two ordered input arrays `a[0..n-1]` and `b[0..m-1]` into a single ordered output array. Both input arrays are required to be ordered with respect to the ordering of the integer type `value_type`. The array starting at `result` must designate a memory region that is disjoint from the input arrays and large enough to store exactly $m + n$ elements.

Upon termination, the array `result[0..n+m-1]` contains all elements of `a` and `b`, arranged in ordered form. All elements of the input arrays appear exactly once in the output. If two elements from different input arrays are equal, the element originating from `a` precedes the element originating from `b`; moreover, the relative order of equal elements within each input array is preserved.

10.7.1. Formal specification of `merge`

The following listing 10.21 shows the specification of `merge`. The specification expects the input arrays of the proper size and in increasing order and the output array of enough size to contain all input elements. The input arrays are required to not overlap with the output array.

```
/*@
  requires   bound:        m + n <= SIZE_TYPE_MAX;
  requires   valid:        \valid_read(a + (0..m-1));
  requires   valid:        \valid_read(b + (0..n-1));
  requires   valid:        \valid(c + (0..m+n-1));
  requires   sep:          \separated(a + (0..m-1), c + (0..m+n-1));
  requires   sep:          \separated(b + (0..n-1), c + (0..m+n-1));
  requires   increasing:   Increasing(a, m);
  requires   increasing:   Increasing(b, n);

  terminates               \true;
  exits                    \false;
  assigns                  c[0 .. m+n-1];

  ensures    increasing:   Increasing(c, m + n);
  ensures    unchanged:    Unchanged{Old,Here}(a, m);
  ensures    unchanged:    Unchanged{Old,Here}(b, n);
 */
void
merge(const value_type* a, size_type m,
      const value_type* b, size_type n, value_type* c);
```

Listing 10.21: Formal specification of `merge`

Open Issue 10.7.1 (Completeness of the postcondition of merge).
The current formal specification characterizes only the size and the ordering of the output array `result` *. The following additional properties, which are stated informally, are not yet captured by the formal postconditions:*

- *all elements of the input arrays appear exactly once in the output;*
- *if two elements from different input arrays are equal, the element originating from* `a` *precedes the element originating from* `b`*;*
- *the relative order of equal elements within each input array is preserved.*

These aspects will be addressed in subsequent versions of this document.

10.7.2. More Lemmas on `WeaklyIncreasing`

We introduce in the following listing several lemmas about `WeaklyIncreasing` [6.2] that are instrumental for the formal verification of `merge`. Each lemma captures a basic reasoning step about weakly increasing array segments that repeatedly occurs in the proof.

- Lemma `WeaklyIncreasing_Shrink` [10.22] expresses that the *weakly increasing* property is stable under restriction: any sub-range of a weakly increasing array is itself weakly increasing.
- Lemma `WeaklyIncreasing_AddElement` [10.22] characterizes the extension of a weakly increasing range by one element. It states that appending a new element preserves the weakly increasing property, provided this element is greater than or equal to the last element of the existing range.
- Lemma `WeaklyIncreasing_Shift` [10.22] captures the invariance of `WeaklyIncreasing` under pointer arithmetic. It allows rewriting the property when array segments are accessed through shifted pointers.
- Lemma `WeaklyIncreasing_Unchanged` [10.22] states that the weakly increasing property is preserved between two program points if the corresponding array segment is unchanged.
- Lemma `WeaklyIncreasing_Equal` [10.22] formalizes preservation under element-wise equality: if two array segments are in one-to-one correspondence and contain equal elements at each index, then `WeaklyIncreasing` holds for one segment if and only if it holds for the other.
- Lemma `WeaklyIncreasing_Join` [10.22] gives sufficient conditions for concatenating two consecutive weakly increasing ranges. It states that the combined range is weakly increasing if both sub-ranges are weakly increasing and the last element of the first range does not exceed the first element of the second.

```
/*@
  axiomatic WeaklyIncreasingLemmas
  {
    lemma WeaklyIncreasing_Shrink{L}:
      \forall value_type *a, integer m, n, p, q;
        m <= p <= q <= n           ==>
        WeaklyIncreasing(a, m, n)  ==>
        WeaklyIncreasing(a, p, q);

    lemma WeaklyIncreasing_AddElement{L}:
      \forall value_type *a, integer n;
        1 < n                      ==>
        a[n-2] <= a[n-1]           ==>
        WeaklyIncreasing(a, n-1)   ==>
        WeaklyIncreasing(a, n);

    lemma WeaklyIncreasing_Shift{L}:
      \forall value_type *a, integer m, n;
        WeaklyIncreasing(a + m, 0, n)  <==>
        WeaklyIncreasing(a, m, n + m);

    lemma WeaklyIncreasing_Equal{K,L}:
      \forall value_type *a, *b, integer m, n;
        Equal{K,L}(a, m, n, b)         ==>
        WeaklyIncreasing{K}(a, m, n)   ==>
        WeaklyIncreasing{L}(b, m, n);

    lemma WeaklyIncreasing_Unchanged{K,L}:
      \forall value_type *a, integer m, n;
        Unchanged{K,L}(a, m, n)        ==>
        WeaklyIncreasing{K}(a, m, n)   ==>
        WeaklyIncreasing{L}(a, m, n);

    lemma WeaklyIncreasing_Join{L}:
      \forall value_type *a, integer m, n;
        0 < m < n                  ==>
        WeaklyIncreasing(a, m)     ==>
        WeaklyIncreasing(a, m, n)  ==>
        a[m-1] <= a[m]             ==>
        WeaklyIncreasing(a, n);
  }
*/
```

Listing 10.22: The logic definition(s) `WeaklyIncreasingLemmas`

10.7.3. Implementation of `merge`

The implementation of `merge`, shown in the next listing, follows the standard merge procedure. The algorithm traverses both input arrays and, at each iteration, writes the smaller of the two current elements to the result array. In this way, the result array is constructed incrementally in (weakly) increasing order. When the end of one input array is reached, the remaining elements of the other array are copied unchanged to the result array. The listing contains several assertions whose sole purpose is to guide the automatic provers.

The **`while`** loop is responsible for the incremental construction of the increasing prefix of the result array; after the loop, the algorithm completes the array by copying the remaining elements.

Note that Lemma `WeaklyIncreasing_Increasing` [6.3] is used to show that the output array is in increasing order.

```
void
merge(const value_type* a, size_type m,
      const value_type* b, size_type n, value_type* c)
{
  //@ assert increasing:  WeaklyIncreasing(a, 0, m);
  size_type i = 0;
  size_type j = 0;
  size_type x = 0;

  if (0 < m || 0 < n) {
    /*@ loop invariant index:       0 <= i <= m;
        loop invariant index:       0 <= j <= n;
        loop invariant index:       x == i+j;
        loop invariant index:       0 <= x <= m+n-1;
        loop invariant upper:       i < m  ==>  UpperBound(c, 0, x, a[i]);
        loop invariant upper:       j < n  ==>  UpperBound(c, 0, x, b[j]);
        loop invariant increasing:  WeaklyIncreasing(c, x);
        loop assigns i, j, x, c[0 .. m+n-1];
        loop variant (m+n) - (i+j);
     */
    while (i < m && j < n) {
      if (a[i] < b[j]) {
        c[x++] = a[i++];
        //@ assert upper:       i < m  ==>  UpperBound(c, 0, x, a[i]);
        //@ assert increasing:  WeaklyIncreasing(c, 0, x);
      }
      else {
        c[x++] = b[j++];
        //@ assert upper:       j < n  ==>  UpperBound(c, 0, x, b[j]);
        //@ assert increasing:  WeaklyIncreasing(c, 0, x);
      }

      //@ assert increasing:  WeaklyIncreasing(c, 0, x);
    }

    //@ ghost  Epilogue: ;
    //@ assert index:       x == i+j;
    //@ assert index:       i == m  ^^  j == n;
    //@ assert index:       i <  m  ^^  j <  n;
    //@ assert increasing:  WeaklyIncreasing(c, 0, x);
    //@ assert unchanged:   Unchanged{Pre,Here}(a, 0, m);
    //@ assert increasing:  WeaklyIncreasing(a, 0, m);
```

Listing 10.23: Implementation of `merge` (1)

```
    if (i < m) {
      //@ assert upper:       0 < x  ==>  c[x-1] <= a[i];
      //@ assert increasing:  WeaklyIncreasing(a, i, m);
      //@ assert increasing:  WeaklyIncreasing(a+i, 0, m-i);

      copy(a + i, m - i, c + x);
      //@ assert equal:       c[x] == At{Epilogue}(a, i);
      //@ assert equal:       a[i] == At{Epilogue}(a, i);
      //@ assert equal:       c[x] == a[i];
      //@ assert equal:       Equal{Epilogue,Here}(a+i, m-i, c+x);
      //@ assert index:       m-i+x == m+n;
      //@ assert unchanged:   Unchanged{Pre,Here}(a, 0, m);
      //@ assert unchanged:   Unchanged{Pre,Here}(b, 0, n);
      //@ assert increasing:  WeaklyIncreasing(c+x, 0, m-i);
      //@ assert increasing:  WeaklyIncreasing(c, x, m+n);
    }
    else {
      //@ assert upper:       0 < x  ==>  c[x-1] <= b[j];
      //@ assert unchanged:   Unchanged{Pre,Here}(a, 0, m);
      //@ assert unchanged:   Unchanged{Pre,Here}(b, 0, n);
      //@ assert increasing:  WeaklyIncreasing(b, 0, n);
      //@ assert increasing:  WeaklyIncreasing(b+j, 0, n-j);

      copy(b + j, n - j, c + x);
      //@ assert equal:       c[x] == At{Epilogue}(b, j);
      //@ assert equal:       b[j] == At{Epilogue}(b, j);
      //@ assert equal:       c[x] == b[j];
      //@ assert equal:       Equal{Epilogue,Here}(b+j, n-j, c+x);
      //@ assert index:       n-j+x == m+n;
      //@ assert unchanged:   Unchanged{Epilogue,Here}(c, 0, x);
      //@ assert increasing:  WeaklyIncreasing(c+x, 0, n-j);
      //@ assert increasing:  WeaklyIncreasing(c, x, m+n);
    }

    //@ assert unchanged:   Unchanged{Epilogue,Here}(c, 0, x);
    //@ assert increasing:  WeaklyIncreasing(c, 0, x);
    //@ assert increasing:  WeaklyIncreasing(c, x, m+n);
    //@ assert increasing:  0 < x  ==>  c[x-1] <= c[x];
    //@ assert increasing:  0 < x  ==>  WeaklyIncreasing(c, x-1, m+n);
    //@ assert increasing:  WeaklyIncreasing(c, 0, m+n);
  }
}
```

Listing 10.24: The Implementation of `merge` (2)

Part V.

Verification of data structures

11. The `stack` data type

So far we have used the ACSL specification language for the task of specifying and verifying one single C function at a time. However, in practice we are also faced with the task to implement a family of functions, usually around some sophisticated data structure, which have to obey certain rules of interdependence. In this kind of task, we are not only interested in the properties of a single function but also in properties describing how several function play together.

The C++ Standard Library provides a generic container adaptor `stack` [19, §26.6.6] whose signature and behavior we try to follow as far as our C implementation it allows. For a more detailed discussion of our approach to the formal verification of `stack` we refer to Kim Völlinger's thesis [26].

A *stack* is a data type that can hold objects and has the property that, if an object *a* is *pushed* on a stack *before* object *b*, then *a* can only be removed (*popped*) after *b*. A stack is, in other words, a *first-in, last-out* data type (see Figure 11.1). The *top* function of a stack returns the last element that has been pushed on a stack.

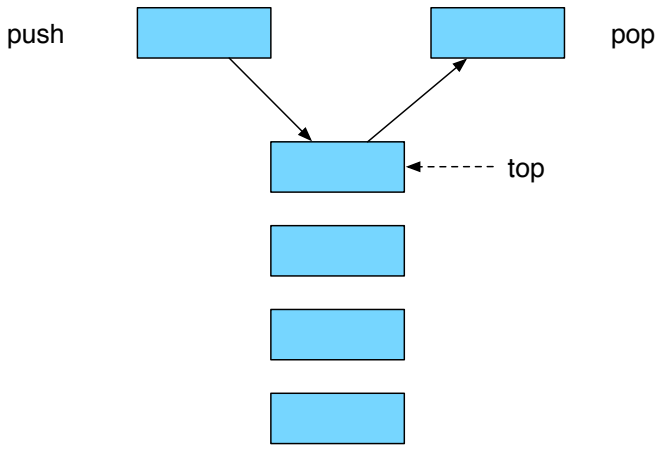

Figure 11.1.: Push and pop on a stack

We consider only stacks that have a finite *capacity*, that is, that can only hold a maximum number *c* of elements that is constant throughout their lifetime. This restriction allows us to define a stack without relying on dynamic memory allocation. When a stack is *created* or *initialized*, it contains no elements, i.e., its *size* is 0. The function *push* and *pop* increases and decreases the size of a stack by at most one, respectively.

11.1. Methodology overview

Figure 11.2 gives an overview of our methodology to specify and verify abstract data types (verification of one axiom shown only).

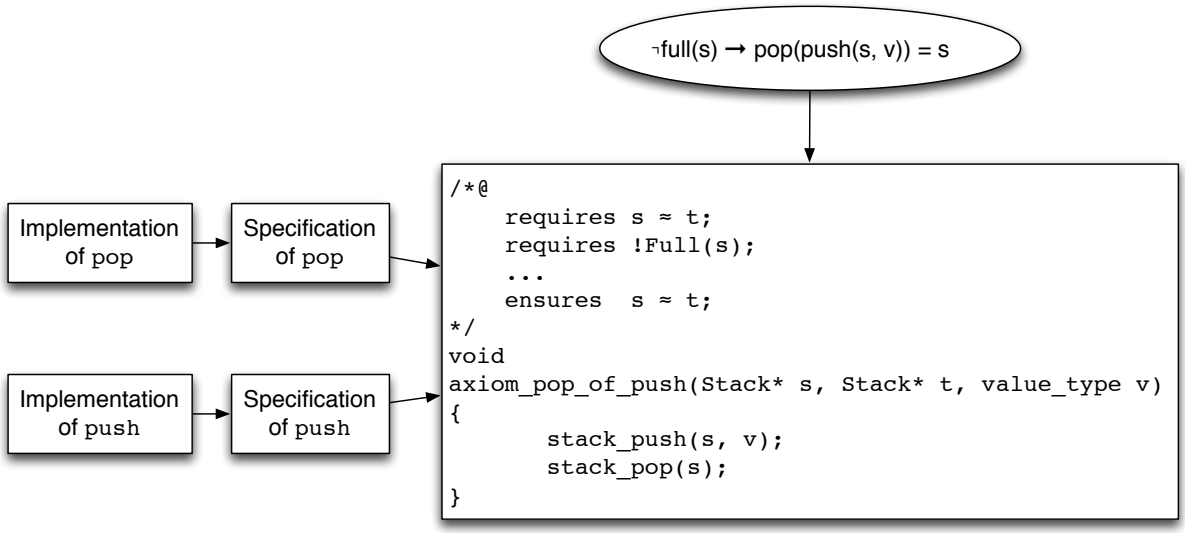

Figure 11.2.: Methodology Overview

What we will basically do is:

1. specify axioms about how the stack functions should interact with each other (§11.2),
2. define a basic implementation of C data structures (only one in our example, viz. `struct Stack`; see §11.3) and some invariants the instances of them have to obey (§11.4),
3. provide for each stack function an ACSL contract and a C implementation (§11.6),
4. verify each function against its contract (§11.6),
5. transform the axioms into ACSL-annotated C code (§11.7), and
6. verify that code, using access function contracts and data-type invariants as necessary (§11.7).

§11.5 provides an ACSL-predicate deciding whether two instances of a `struct Stack` are considered to be equal (indication by "≈" in Figure 11.2), while §11.6.1 gives a corresponding C implementation. The issue of an appropriate definition of equality of data instances is familiar to any C programmer who had to replace a faulty comparison `if(s1 == s2)` by the correct `if(strcmp(s1,s2)== 0)` to compare two strings `char *s1,*s2` for equality.

11.2. Stack axioms

To specify the interplay of the stack access functions, we use a set of axioms[25], all but one of them having the form of a conditional equation.

Let V denote an arbitrary type. We denote by S_c the type of stacks with capacity $c > 0$ of elements of type V. The aforementioned functions then have the following signatures.

$$\begin{aligned} \text{init} &: S_c \to S_c, \\ \text{push} &: S_c \times V \to S_c, \\ \text{pop} &: S_c \to S_c, \\ \text{top} &: S_c \to V, \\ \text{size} &: S_c \to \mathbb{N}. \end{aligned}$$

With $\mathbb{B}$ denoting the *boolean* type we will also define two auxiliary functions

$$\begin{aligned} \text{empty} &: S_c \to \mathbb{B}, \\ \text{full} &: S_c \to \mathbb{B}. \end{aligned}$$

To qualify as a stack these functions must satisfy the following rules which are also referred to as *stack axioms*.

11.2.1. Stack initialization

After a stack has been initialized its size is 0.

$$\text{size}(\text{init}(s)) = 0. \tag{11.1}$$

The auxiliary functions empty and full are defined as follows

$$\text{empty}(s), \quad \text{iff} \quad \text{size(s)} = 0, \tag{11.2}$$

$$\text{full}(s), \quad \text{iff} \quad \text{size(s)} = c. \tag{11.3}$$

We expect that for every stack s the following condition holds

$$0 \leq \text{size}(s) \leq c. \tag{11.4}$$

11.2.2. Adding an element to a stack

To push an element v on a stack the stack must not be full. If an element has been pushed on an eligible stack, its size increases by 1

$$\text{size}(\text{push}(s, v)) = \text{size}(s) + 1, \qquad \text{if} \quad \neg\text{full}(s). \tag{11.5}$$

Moreover, the element pushed on a stack is the top element of the resulting stack

$$\text{top}(\text{push}(s, v)) = v, \qquad \text{if} \quad \neg\text{full}(s). \tag{11.6}$$

[25] There is an analogy in geometry: Euclid (e.g. [27]) invented the use of axioms there, but still kept definitions of *point*, *line*, *plane*, etc. Hilbert [28] recognized that the latter are not only unformalizable, but also unnecessary, and dropped them, keeping only the formal descriptions of relations between them.

11.2.3. Removing an element from a stack

An element can only be removed from a non-empty stack. If an element has been removed from an eligible stack the stack size decreases by 1

$$\text{size}(\text{pop}(s)) = \text{size}(s) - 1, \qquad \text{if} \quad \neg\text{empty}(s). \tag{11.7}$$

If an element is pushed on a stack and immediately afterwards an element is removed from the resulting stack then the final stack is equal to the original stack

$$\text{pop}(\text{push}(s, v)) = s, \qquad \text{if} \quad \neg\text{full}(s). \tag{11.8}$$

Conversely, if an element is removed from a non-empty stack and if afterwards the top element of the original stack is pushed on the new stack then the resulting stack is equal to the original stack.

$$\text{push}(\text{pop}(s), \text{top}(s)) = s, \qquad \text{if} \quad \neg\text{empty}(s). \tag{11.9}$$

11.2.4. A note on exception handling

We don't impose a requirement on `push(s, v)` if `s` is a full stack, nor on `pop(s)` or `top(s)` if `s` is an empty stack. Specifying the behavior in such *exceptional* situations is a problem by its own; a variety of approaches is discussed in the literature. We won't elaborate further on this issue, but only give an example to warn about "innocent-looking" exception specifications that may lead to undesired results.

If we'd introduce an additional error value `err` in the element type V and require `top(s)= err` if `s` is empty, we'd be faced with the problem of specifying the behavior of `push(s, err)`. At first glance, it would seem a good idea to have `err` just been ignored by `push`, i.e. to require

$$\text{push}(s, \text{err}) = s. \tag{11.10}$$

However, we then could derive for any non-full and non-empty stack `s`, that

$$\begin{aligned}
\text{size}(s) &= \text{size}(\text{pop}(\text{push}(s, \text{err}))) && \text{by 11.8} \\
&= \text{size}(\text{pop}(s)) && \text{as assumed in 11.10} \\
&= \text{size}(s) - 1 && \text{by 11.7}
\end{aligned}$$

i.e. no such stacks could exist, or all `int` values would be equal.

11.3. The structure `stack` and its associated functions

We now introduce one possible C implementation of the above axioms. It is centred around the C structure `stack` shown in the following listing.

```
struct Stack
{
  value_type* obj;

  size_type   capacity;

  size_type   size;
};

typedef struct Stack Stack;
```

Listing 11.3: Definition of type `stack`

This struct holds an array `obj` of positive length called `capacity`. The capacity of a stack is the maximum number of elements this stack can hold. The field `size` indicates the number elements that are currently in the stack. See also Figure 11.4 which attempts to interpret this definition according to Figure 11.1.

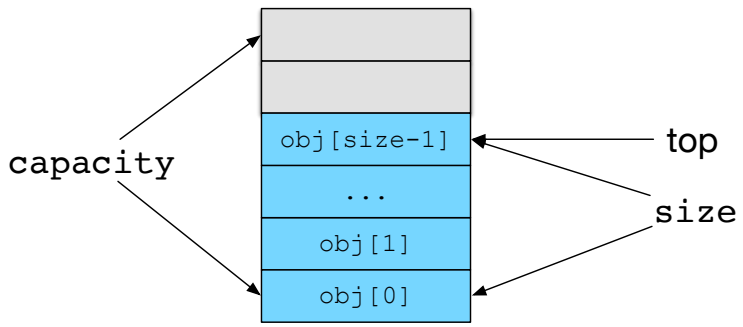

Figure 11.4.: Interpreting the data structure `stack`

Based on the stack functions from §11.2, we declare in the next listing the following functions as part of our `stack` data type.

```
void        stack_init(Stack* s, value_type* a, size_type n);

bool        stack_equal(const Stack* s, const Stack* t);

size_type   stack_size(const Stack* s);

bool        stack_empty(const Stack* s);

bool        stack_full(const Stack* s);

value_type  stack_top(const Stack* s);

void        stack_push(Stack* s, value_type v);

void        stack_pop(Stack* s);
```

Listing 11.5: Declaration of functions of type `stack`

Most of these functions directly correspond to methods of the C++ `std::stack` template class [19, §26.6.6.1]. The function `stack_equal` corresponds to the comparison operator ==, whereas one use of `stack_init` is to bring a stack into a well-defined initial state. The function `stack_full` has no counterpart in `std::stack`. This reflects the fact that we avoid dynamic memory allocation, while `std::stack` does not.

11.4. Stack invariants

Not every possible instance of type `stack` is considered a valid one, e.g., with our definition of `stack` in Listing 11.3, `Stack s = {{0,0,0,0},4,5}` is not. In the following listing, we present basic logic functions and predicates that we will use throughout this chapter In particular, we define the predicate `StackInvariant` [11.6] that discriminates valid and invalid instances.

```
/*@
  axiomatic StackInvariant
  {
    logic integer
    StackCapacity{L}(Stack* s) = s->capacity;

    logic integer
    StackSize{L}(Stack* s) = s->size;

    logic value_type*
    StackStorage{L}(Stack* s) = s->obj;

    logic integer
    StackTop{L}(Stack* s) = s->obj[s->size-1];

    predicate
    StackEmpty{L}(Stack* s) =  StackSize(s) == 0;

    predicate
    StackFull{L}(Stack* s)  =  StackSize(s) == StackCapacity(s);

    predicate
    StackInvariant{L}(Stack* s) =
      0 < StackCapacity(s) &&
      0 <= StackSize(s) <= StackCapacity(s) &&
      \valid(StackStorage(s) + (0..StackCapacity(s)-1)) &&
      \separated(s, StackStorage(s) + (0..StackCapacity(s)-1));
  }
*/
```

Listing 11.6: The logic definition(s) `StackInvariant`

We start, with the auxiliary logic function `StackCapacity`, `StackSize` and `StackStorage` which we can use in specifications to refer to the fields `capacity`, `size` and `obj` of `stack`, respectively. This listing also contains the logic function `StackTop` which defines the array element with index `size - 1` as the top place of a stack.

The reader can consider this as an attempt to hide implementation details from the specification. We intentionally use here integer as a return value of these logic functions. Inaccurate use of logic functions with bounded types in axioms with arithmetic operations may lead to inconsistencies.

We also introduce the predicates `StackEmpty` [11.6] and `StackFull` [11.6] that express the concepts of empty and full stacks by referring to a stack's size and capacity (see Equations (11.2) and (11.3)).

There are some obvious invariants that must be fulfilled by every valid object of type `stack`:

- The stack capacity shall be strictly greater than zero (an empty stack is ok but a stack that cannot hold anything is not useful).
- The pointer `obj` shall refer to an array of length `capacity`.
- The number of elements `size` of a stack the must be non-negative and not greater than its capacity.

These invariants are all formalized in the predicate `StackInvariant` [11.6].

Note how the use of the previously defined logic functions and predicates allows us to define the stack invariant without directly referring to the fields of `stack`.

We sometimes wish to express that there is no *memory aliasing* between two stacks. If there were aliasing, then modifying one stack could modify the other stack in unexpected ways. In order to express that there is no aliasing between two stacks, we define the predicate `StackSeparated` in the next listing.

```
/*@
  axiomatic StackUtility
  {
    predicate
    StackSeparated(Stack* s, Stack* t) =
      \separated(s, s->obj + (0..s->capacity-1),
                 t, t->obj + (0..t->capacity-1));

    predicate
    StackUnchanged{K,L}(Stack* s) =
      StackSize{K}(s)     == StackSize{L}(s)      &&
      StackStorage{K}(s)  == StackStorage{L}(s)   &&
      StackCapacity{K}(s) == StackCapacity{L}(s)  &&
      Unchanged{K,L}(StackStorage{K}(s), StackSize{K}(s));
  }
*/
```

Listing 11.7: The logic definition(s) `StackUtility`

This listing also contains the predicate `StackUnchanged` [11.7] that we will use to describe cases that the contents of a stack hasn't changed.

11.5. Equality of stacks

Defining equality of instances of non-trivial data types, in particular in object-oriented languages, is not an easy task. The book *Programming in Scala* [29, Chapter 28] devotes to this topic a whole chapter of more than twenty pages. In the following two sections we give a few hints how ACSL and Frama-C can help to correctly define equality for a simple data type.

We consider two stacks as equal if they have the same size and if they contain the same objects. To be more precise, let `s` and `t` two pointers of type `stack`, then we define the predicate `StackEqual` as in the following listing.

```
/*@
  axiomatic StackEquality
  {
    predicate
    StackEqual{S,T}(Stack* s, Stack* t) =
      StackSize{S}(s) == StackSize{T}(t) &&
      Equal{S,T}(StackStorage{S}(s), StackSize{S}(s), StackStorage{T}(t));

    lemma StackEqual_Reflexive{S} :
      \forall Stack* s; StackEqual{S,S}(s, s);

    lemma StackEqual_Symmetric{S,T} :
      \forall Stack *s, *t;
        StackEqual{S,T}(s, t)  ==>  StackEqual{T,S}(t, s);

    lemma StackEqual_Transitive{S,T,U}:
      \forall Stack *s, *t, *u;
        StackEqual{S,T}(s, t)  ==>
        StackEqual{T,U}(t, u)  ==>
        StackEqual{S,U}(s, u);
  }
*/
```

Listing 11.8: The logic definition(s) `StackEquality`

Our use of labels in this listing makes the specification somewhat hard to read (in particular in the last line where we reuse the predicate `Equal` [4.28]. However, this definition of `StackEqual` will allow us later to compare the same stack object at different points of a program. The logical expression `StackEqual {A,B}(s,t)` reads informally as: The stack object `*s` at program point `A` equals the stack object `*t` at program point `B`.

The reader might wonder why we exclude the capacity of a stack into the definition of stack equality. This approach can be motivated with the behavior of the method `capacity` of the class `std::vector<T>`. There, equal instances of type `std::vector<T>` may very well have different capacities.[26]

If equal stacks can have different capacities then, according to our definition of the predicate `StackFull` [11.6], we can have to equal stacks where one is full and the other one is not.

A finer, but very important point in our specification of equality of stacks is that the elements of the arrays `s->obj` and `t->obj` are compared only up to `s->size` and *not* up to `s->capacity`. Thus the two stacks `s` and `t` in Figure 11.9 are considered equal although there is are obvious differences in their internal arrays.

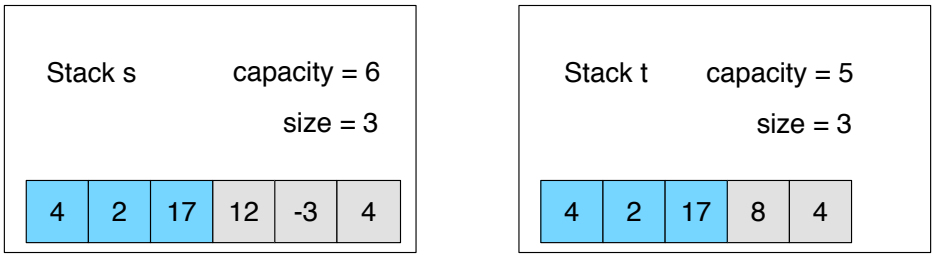

Figure 11.9.: Example of two equal stacks

If we define an equality relation (=) of objects for a data type such as `stack`, we have to make sure that the following rules hold.

[26] See `http://www.cplusplus.com/reference/vector/vector/capacity`

$$\text{reflexivity} \qquad \forall s \in S : s = s, \tag{11.11a}$$

$$\text{symmetry} \qquad \forall s, t \in S : s = t \implies t = s, \tag{11.11b}$$

$$\text{transitivity} \qquad \forall s, t, u \in S : s = t \wedge t = u \implies s = u. \tag{11.11c}$$

Any relation that satisfies the conditions (11.11) is referred to as an *equivalence relation*. The mathematical set of all instances that are considered equal to some given instance `s` is called the equivalence class of `s` with respect to that relation.

Our formalization of `StackEquality` [11.8] shows these three rules for the relation `StackEqual`; it can be automatically verified that they are a consequence of the definition of `StackEqual`.

The two stacks in Figure 11.9 show that an equivalence class of `StackEqual` can contain more than one element.[27] The stacks `s` and `t` in Figure 11.9 are also referred to as two *representatives* of the same equivalence class. In such a situation, the question arises whether a function that is defined on a set with an equivalence relation can be defined in such a way that its definition is *independent of the chosen representatives*.[28] We ask, in other words, whether the function is *well-defined* on the set of all equivalence classes of the relation `StackEqual`.[29] The question of well-definition will play an important role when verifying the functions of the `stack` (see §11.6).

[27] This is a common situation in mathematics. For example, the equivalence class of the rational number $\frac{1}{2}$ contains infinitely many elements, viz. $\frac{1}{2}, \frac{2}{4}, \frac{7}{14}, \ldots$.

[28] This is why mathematicians know that $\frac{1}{2} + \frac{3}{5}$ equals $\frac{7}{14} + \frac{3}{5}$.

[29] See `http://en.wikipedia.org/wiki/Well-definition`.

11.6. Verification of stack functions

In this section we verify the functions

- `stack_equal` (§11.6.1)
- `stack_init` (§11.6.2)
- `stack_size` (§11.6.3)
- `stack_full` (§11.6.4)
- `stack_empty` (§11.6.5)
- `stack_top` (§11.6.6)
- `stack_push` (§11.6.7)
- `stack_pop` (§11.6.8)

of the data type `stack`. To be more precise, we provide for each of function `stack_foo`:

- an ACSL specification of `stack_foo`
- a C implementation of `stack_foo`
- a C function `stack_foo_wd`[30] accompanied by a an ACSL contract that expresses that the implementation of `stack_foo` is well-defined. Figure 11.10 shows our methodology for the verification of well-definition in the `pop` example, (≈) again indicating the user-defined `stack` equality.

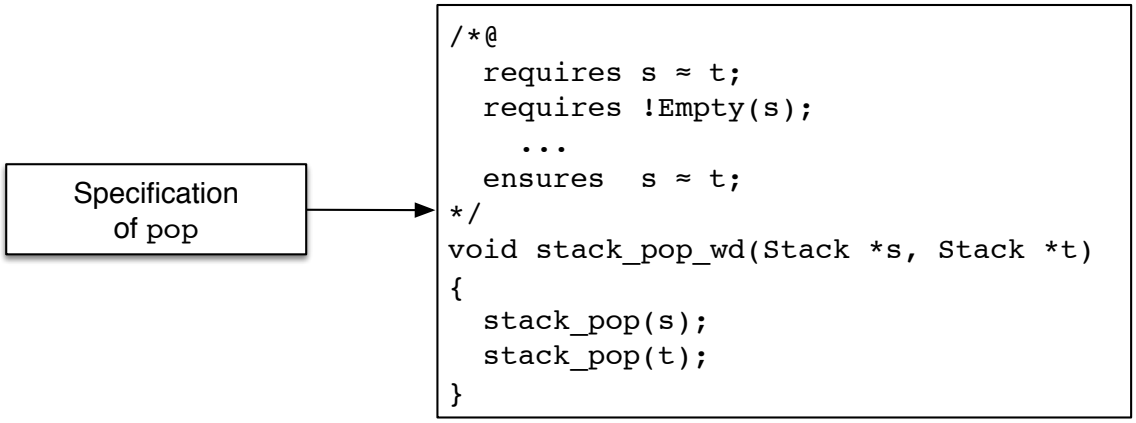

Figure 11.10.: Methodology for the verification of well-definition

Note that the specifications of the various functions will explicitly refer to the *internal state* of `stack`. In §11.7 we will show that the *interplay* of these functions satisfy the stack axioms from §11.2.

[30]The suffix `_wd` stands for *well definition*

11.6.1. The function `stack_equal`

The function `stack_equal` in the following listing is the runtime counterpart for the `StackEqual` [11.8] predicate. Note that this specifications explicitly refers to valid stacks.

```
/*@
  requires   valid:      \valid(s) && StackInvariant(s);
  requires   valid:      \valid(t) && StackInvariant(t);

  terminates             \true;
  exits                  \false;
  assigns                \nothing;

  ensures    equal:      \result == 1  <==>  StackEqual{Here,Here}(s, t);
  ensures    not_equal:  \result == 0  <==> !StackEqual{Here,Here}(s, t);
*/
bool
stack_equal(const Stack* s, const Stack* t);
```

Listing 11.11: Formal specification of `stack_equal`

The implementation of `stack_equal` in the next listing compares two stacks according to the same rules of predicate `StackEqual`.

```
bool
stack_equal(const Stack* s, const Stack* t)
{
  return (s->size == t->size) && equal(s->obj, s->size, t->obj);
}
```

Listing 11.12: Implementation of `stack_equal`

11.6.2. The function `stack_init`

The following listing presents the contract of `stack_init`. The preconditions require a valid stack descriptor, a positive capacity, and a contiguous storage area of sufficient size that is separated from the stack descriptor itself.

```
/*@
  requires   valid:      \valid(s);
  requires   capacity:   0 < capacity;
  requires   storage:    \valid(storage + (0..capacity-1));
  requires   sep:        \separated(s, storage + (0..capacity-1));

  terminates             \true;
  exits                  \false;
  assigns                s->obj      \from storage;
  assigns                s->capacity \from capacity;
  assigns                s->size     \from \nothing;

  ensures    valid:      \valid(s);
  ensures    capacity:   StackCapacity(s) == capacity;
  ensures    storage:    StackStorage(s) == storage;
  ensures    invariant:  StackInvariant(s);
  ensures    empty:      StackEmpty(s);
*/
void
stack_init(Stack* s, value_type* storage, size_type capacity);
```

Listing 11.13: Formal specification of `stack_init`

The `assigns` clauses make the data dependencies explicit: the storage pointer `s->obj` is initialized from the parameter `storage`, the capacity field from `capacity`, and the size field is set to a constant value, independent of any inputs.

The postconditions establish that the stack is properly initialized, its capacity and storage are set as requested, and the stack invariant holds. For clarity, both `StackEmpty(s)` and the corresponding size condition are stated, even though emptiness is equivalent to a size of zero.

The next listing shows the implementation of `stack_init`. It simply initializes `obj` and `capacity` with the respective value of the array and sets the field `size` to zero.

```
void
stack_init(Stack* s, value_type* storage, size_type capacity)
{
  s->obj      = storage;
  s->capacity = capacity;
  s->size     = 0u;
}
```

Listing 11.14: Implementation of `stack_init`

11.6.3. The function `stack_size`

The function `stack_size` is the runtime version of the logic function `StackSize` [11.6]. The specification of `stack_size` in the following listing simply states that `stack_size` produces the same result as `StackSize`.

```
/*@
  requires   valid: \valid(s) && StackInvariant(s);

  terminates        \true;
  exits             \false;
  assigns           \nothing;

  ensures    size:  \result == StackSize(s);
*/
size_type
stack_size(const Stack* s);
```

Listing 11.15: Formal specification of `stack_size`

As in the definition of the logic function `StackSize` the implementation of `stack_size` in the next listing simply returns the field `size`.

```
size_type
stack_size(const Stack* s)
{
  return s->size;
}
```

Listing 11.16: Implementation of `stack_size`

The next listing shows our check whether `stack_size` is well-defined. Since `stack_size` neither modifies the state of its `stack` argument nor that of any global variable we only check whether it produces the same result for equal stacks. Note that we simply may use operator == to compare integers since we didn't introduce a nontrivial equivalence relation on that data type.

```
/*@
  requires   valid:  \valid(s) && StackInvariant(s);
  requires   valid:  \valid(t) && StackInvariant(t);
  requires   equal:  StackEqual{Here,Here}(s, t);

  terminates         \true;
  exits              \false;
  assigns            \nothing;

  ensures    equal:  \result;
*/
bool
stack_size_wd(const Stack* s, const Stack* t)
{
  return stack_size(s) == stack_size(t);
}
```

Listing 11.17: Implementation of `stack_size_wd`

11.6.4. The function `stack_full`

The function `stack_full` is the runtime version of the predicate `StackFull` [11.6].

```
/*@
  requires   valid:     \valid(s) && StackInvariant(s);

  terminates            \true;
  exits                 \false;
  assigns               \nothing;

  ensures    full:      \result == 1  <==>  StackFull(s);
  ensures    not_full:  \result == 0  <==> !StackFull(s);
*/
bool
stack_full(const Stack* s);
```

Listing 11.18: Formal specification of `stack_full`

As in the definition of the predicate `StackFull` the implementation of `stack_full` in the next listing simply checks whether the size of the stack equals its capacity.

```
bool
stack_full(const Stack* s)
{
  return stack_size(s) == s->capacity;
}
```

Listing 11.19: Implementation of `stack_full`

Note that with our definition of stack equality (§11.5) there can be equal stack with different capacities. As a consequence, there can are equal stacks where one is full while the other is not. In other words, `stack_full` is not well-defined!

11.6.5. The function `stack_empty`

The function `stack_empty` is the runtime version of the predicate `StackEmpty` [11.6].

```
/*@
  requires   valid:      \valid(s) && StackInvariant(s);

  terminates             \true;
  exits                  \false;
  assigns                \nothing;

  ensures    empty:      \result == 1  <==>  StackEmpty(s);
  ensures    not_empty:  \result == 0  <==> !StackEmpty(s);
*/
bool
stack_empty(const Stack* s);
```

Listing 11.20: Formal specification of `stack_empty`

As in the definition of the predicate `StackEmpty` the implementation of `stack_empty` in the next listing simply checks whether the size of the stack is zero.

```
bool
stack_empty(const Stack* s)
{
  return stack_size(s) == 0u;
}
```

Listing 11.21: Implementation of `stack_empty`

The following listing shows our check whether `stack_empty` is well-defined.

```
/*@
  requires   valid:  \valid(s) && StackInvariant(s);
  requires   valid:  \valid(t) && StackInvariant(t);
  requires   equal:  StackEqual{Here,Here}(s, t);

  terminates         \true;
  exits              \false;
  assigns            \nothing;

  ensures    equal:  \result;
*/
bool
stack_empty_wd(const Stack* s, const Stack* t)
{
  return stack_empty(s) == stack_empty(t);
}
```

Listing 11.22: Implementation of `stack_empty_wd`

11.6.6. The function stack_top

The function stack_top is the runtime version of the logic function StackTop [11.6]. The specification of stack_top in the following listing simply states that for non-empty stacks stack_top produces the same result as StackTop which in turn just returns the element obj[size-1] of stack.

```
/*@
  requires   valid:  \valid(s) && StackInvariant(s);

  terminates         \true;
  exits              \false;
  assigns            \nothing;

  ensures    top:    !StackEmpty(s) ==> \result == StackTop(s);
*/
value_type
stack_top(const Stack* s);
```

Listing 11.23: Formal specification of stack_top

For a non-empty stack the implementation of stack_top in the next listing simply returns the element obj[size-1]. Note that our implementation of stack_top does not crash when it is applied to an empty stack. In this case we return the first element of the internal, non-empty array obj. This is consistent with our specification of stack_top which only refers to non-empty stacks.

```
value_type
stack_top(const Stack* s)
{
  if (!stack_empty(s)) {
    return s->obj[s->size - 1u];
  }
  else {
    return s->obj[0u];
  }
}
```

Listing 11.24: Implementation of stack_top

The next listing shows our check whether `stack_top` is well-defined. Since our axioms in §11.2 did not impose any behavior on the behavior of `stack_top` for empty stacks, we prove the well-definition of `stack_top` only for nonempty stacks.

```
/*@
  requires   valid:  \valid(s) && StackInvariant(s) && !StackEmpty(s);
  requires   valid:  \valid(t) && StackInvariant(t) && !StackEmpty(t);
  requires   equal:  StackEqual{Here,Here}(s, t);

  terminates         \true;
  exits              \false;
  assigns            \nothing;

  ensures    equal:  \result;
*/
bool
stack_top_wd(const Stack* s, const Stack* t)
{
  return stack_top(s) == stack_top(t);
}
```

Listing 11.25: Implementation of `stack_top_wd`

11.6.7. The function `stack_push`

The following listing shows the specification of the function `stack_push`. In accordance with Axiom (11.5), `stack_push` is supposed to increase the number of elements of a non-full stack by one. The specification also demands that the value that is pushed on a non-full stack becomes the top element of the resulting stack (see Axiom (11.6)).

```
/*@
  requires valid:       \valid(s) && StackInvariant(s);

  terminates            \true;
  exits                 \false;
  assigns               s->size, s->obj[s->size];

  behavior full:
    assumes             StackFull(s);
    assigns             \nothing;
    ensures valid:      \valid(s) && StackInvariant(s);
    ensures full:       StackFull(s);
    ensures unchanged:  StackUnchanged{Old,Here}(s);

  behavior not_full:
    assumes             !StackFull(s);
    assigns             s->size;
    assigns             s ->obj[s->size];
    ensures valid:      \valid(s) && StackInvariant(s);
    ensures size:       StackSize(s) == StackSize{Old}(s) + 1;
    ensures top:        StackTop(s) == v;
    ensures storage:    StackStorage(s) == StackStorage{Old}(s);
    ensures capacity:   StackCapacity(s) == StackCapacity{Old}(s);
    ensures not_empty:  !StackEmpty(s);
    ensures unchanged:  Unchanged{Old,Here}(StackStorage(s), StackSize{Old}(s));

  complete behaviors;
  disjoint behaviors;
*/
void
stack_push(Stack* s, value_type v);
```

Listing 11.26: Formal specification of `stack_push`

The implementation of `stack_push` is shown in the next listing. It checks whether its argument is a non-full stack in which case it increases the field `size` by one but only after it has assigned the function argument to the element `obj[size]`.

```
void
stack_push(Stack* s, value_type v)
{
  if (!stack_full(s)) {
    //@ assert not_full: s->size < s->capacity;
    s->obj[s->size++] = v;
  }
}
```

Listing 11.27: Implementation of `stack_push`

The following listing shows our formalization of the well-definition for `stack_push`. The function

stack_push does not return a value but rather modifies its argument. For the well-definition of we therefore check whether it turns equal stacks into equal stacks.

```
/*@
  requires   valid:     \valid(s) && StackInvariant(s);
  requires   valid:     \valid(t) && StackInvariant(t);
  requires   equal:     StackEqual{Here,Here}(s, t);
  requires   not_full:  !StackFull(s) && !StackFull(t);
  requires   sep:       StackSeparated(s, t);

  terminates            \true;
  exits                 \false;
  assigns               s->size, s->obj[s->size];
  assigns               t->size, t->obj[t->size];

  ensures    valid:     StackInvariant(s) && StackInvariant(t);
  ensures    equal:     StackEqual{Here,Here}(s, t);
*/
void
stack_push_wd(Stack* s, Stack* t, value_type v)
{
  stack_push(s, v);
  stack_push(t, v);
  //@ assert top:   StackTop(s) == v;
  //@ assert top:   StackTop(t) == v;
  //@ assert equal: Equal{Here,Here}(StackStorage(s), StackSize{Pre}(s),
      StackStorage(t));
}
```

Listing 11.28: Implementation of stack_push_wd

However, equality of the stack arguments is not sufficient for a proof that stack_push is well-defined. We must also ensure that there is no *aliasing* between the two stacks. Otherwise modifying one stack could modify the other stack in unexpected ways. In order to express that there is no aliasing between two stacks, we use the predicate StackSeparated [11.7].

In order to achieve an automatic verification of stack_push_wd [11.28] we have added the assertions top and equal and introduced the lemma StackPush_Equal [11.29] in the following listing.

```
/*@
  axiomatic StackLemmas
  {
    lemma StackPush_Equal{K,L}:
      \forall Stack *s, *t;
        StackEqual{K,K}(s,t)                      ==>
        StackSize{L}(s) == StackSize{K}(s) + 1    ==>
        StackSize{L}(s) == StackSize{L}(t)        ==>
        StackTop{L}(s)  == StackTop{L}(t)         ==>
        Equal{L,L}(StackStorage{L}(s),
                   StackSize{K}(s),
                   StackStorage{L}(t))            ==>
        StackEqual{L,L}(s,t);
  }
*/
```

Listing 11.29: The logic definition(s) StackLemmas

11.6.8. The function `stack_pop`

The following listing shows the specification of the function `stack_pop`. In accordance with Axiom (11.7), `stack_pop` is supposed to reduce the number of elements in a non-empty stack by one. In addition to the requirements imposed by the axioms, our specification demands that `stack_pop` changes no memory location if it is applied to an empty stack.

```
/*@
  requires   valid:     \valid(s) && StackInvariant(s);

  terminates            \true;
  exits                 \false;
  assigns               s->size;

  ensures    valid:     \valid(s) && StackInvariant(s);

  behavior empty:
    assumes             StackEmpty(s);
    assigns             \nothing;
    ensures empty:      StackEmpty(s);
    ensures unchanged:  StackUnchanged{Old,Here}(s);

  behavior not_empty:
    assumes             !StackEmpty(s);
    assigns             s->size;
    ensures size:       StackSize(s) == StackSize{Old}(s) - 1;
    ensures full:       !StackFull(s);
    ensures storage:    StackStorage(s) == StackStorage{Old}(s);
    ensures capacity:   StackCapacity(s) == StackCapacity{Old}(s);
    ensures unchanged:  Unchanged{Old,Here}(StackStorage(s), StackSize(s));

  complete behaviors;
  disjoint behaviors;
*/
void
stack_pop(Stack* s);
```

Listing 11.30: Formal specification of `stack_pop`

The implementation of `stack_pop` is shown in the next listing. It checks whether its argument is a non-empty stack in which case it decreases the field `size` by one.

```
void
stack_pop(Stack* s)
{
  if (!stack_empty(s)) {
    --s->size;
  }
}
```

Listing 11.31: Implementation of `stack_pop`

The next listing shows our check whether `stack_pop` is well-defined. As in the case of `stack_push` we use the predicate `StackSeparated` [11.7] in order to express that there is no aliasing between the two stack arguments.

```
/*@
  requires   valid:  \valid(s) && StackInvariant(s);
  requires   valid:  \valid(t) && StackInvariant(t);
  requires   equal:  StackEqual{Here,Here}(s, t);
  requires   sep:    StackSeparated(s, t);

  terminates         \true;
  exits              \false;
  assigns            s->size;
  assigns            t->size;

  ensures    valid:  StackInvariant(s);
  ensures    valid:  StackInvariant(t);
  ensures    equal:  StackEqual{Here,Here}(s, t);
*/
void
stack_pop_wd(Stack* s, Stack* t)
{
  stack_pop(s);
  stack_pop(t);
}
```

Listing 11.32: Implementation of `stack_pop_wd`

11.7. Verification of stack axioms

In this section we show that the stack functions defined in §11.6 satisfy the stack Axioms of §11.2.

The annotated code has been obtained from the axioms in a fully systematical way. In order to transform a condition equation $p \rightarrow s = t$:

- Generate a clause `requires p`.
- Generate a clause `requires x1 == ... == xn` for each variable `x` with n occurrences in s and t.
- Change the i-th occurrence of `x` to `xi` in s and t.
- Translate both terms s and t to reversed polish notation.
- Generate a clause `ensures y1 == y2`, where `y1` and `y2` denote the value corresponding to the translated s and t, respectively.

This makes it easy to implement a tool that does the translation automatically, but yields a slightly longer contract in our example.

11.7.1. Resetting a stack

Our formulation in ACSL /C of the axiom in Equation (11.1) is shown in the following listing.

```
/*@
  requires   valid:  \valid(s);
  requires   valid:  \valid(a + (0..n-1));
  requires   sep:    \separated(s, a + (0..n-1));
  requires   pos:    0 < n;

  terminates         \true;
  exits              \false;
  assigns            s->obj      \from a;
  assigns            s->capacity \from n;
  assigns            s->size     \from \nothing;

  ensures    size:   \result == 0;
  ensures    valid:  StackInvariant(s);
*/
size_type
axiom_size_of_init(Stack* s, value_type* a, size_type n)
{
  stack_init(s, a, n);
  return stack_size(s);
}
```

Listing 11.33: Implementation of `axiom_size_of_init`

11.7.2. Adding an element to a stack

Axioms (11.5) and (11.6) describe the behavior of a stack when an element is added.

```
/*@
  requires   valid:     \valid(s) && StackInvariant(s);
  requires   not_full:  !StackFull(s);

  terminates            \true;
  exits                 \false;
  assigns               s->size, s->obj[s->size];

  ensures    size:      \result == StackSize{Old}(s) + 1;
  ensures    valid:     StackInvariant(s);
*/
size_type
axiom_size_of_push(Stack* s, value_type v)
{
  stack_push(s, v);
  return stack_size(s);
}
```

Listing 11.34: Implementation of `axiom_size_of_push`

Except for the `assigns` clauses, the ACSL specification refers only to encapsulating logic functions and predicates defined in §11.4. If ACSL would provide a means to define encapsulating logic functions returning also sets of memory locations, the expressions in `assigns` clauses would not need to refer to the details of our `stack` implementation.[31] As an alternative, `assigns` clauses could be omitted, as long as the proofs are only used to convince a human reader.

```
/*@
  requires   valid:     \valid(s) && StackInvariant(s);
  requires   not_full:  !StackFull(s);

  terminates            \true;
  exits                 \false;
  assigns               s->size, s->obj[s->size];

  ensures    top:       \result == v;
*/
value_type
axiom_top_of_push(Stack* s, value_type v)
{
  stack_push(s, v);
  return stack_top(s);
}
```

Listing 11.35: Implementation of `axiom_top_of_push`

[31] In [14, §2.3.4], a powerful sublanguage to build memory location set expressions is defined. We will explore its capabilities in a later version.

11.7.3. Removing an element from a stack

This section shows the Listings for Axioms 11.7, 11.8 and 11.9 which describe the behavior of a stack when an element is removed.

```
/*@
  requires   valid:  \valid(s) && StackInvariant(s);
  requires   empty:  !StackEmpty(s);

  terminates         \true;
  exits              \false;
  assigns            s->size;

  ensures    size:   \result == StackSize{Old}(s) - 1;
*/
size_type
axiom_size_of_pop(Stack* s)
{
  stack_pop(s);
  return stack_size(s);
}
```

Listing 11.36: Implementation of `axiom_size_of_pop`

```
/*@
  requires   valid:     \valid(s) && StackInvariant(s);
  requires   not_full:  !StackFull(s);

  terminates            \true;
  exits                 \false;
  assigns                s->size, s->obj[s->size];

  ensures    equal:    StackEqual{Old,Here}(s, s);
*/
void
axiom_pop_of_push(Stack* s, value_type v)
{
  stack_push(s, v);
  stack_pop(s);
}
```

Listing 11.37: Implementation of `axiom_pop_of_push`

```
/*@
  requires   valid:      \valid(s) && StackInvariant(s);
  requires   not_empty:  !StackEmpty(s);

  terminates             \true;
  exits                  \false;
  assigns                 s->size, s->obj[s->size-1];

  ensures    equal:      StackEqual{Old,Here}(s, s);
*/
void
axiom_push_of_pop_top(Stack* s)
{
  const value_type v = stack_top(s);
  stack_pop(s);
  stack_push(s, v);
}
```

Listing 11.38: Implementation of `axiom_push_of_pop_top`

Part VI.

Appendices

A. Results of formal verification with Frama-C

In this chapter we introduce the formal verification tools used in this tutorial. We will afterwards present to what extent the examples from Chapters 4–11 could be deductively verified.

Within Frama-C, the Frama-C/WP plug-in [1] enables deductive verification of C programs that have been annotated with the ANSI/ISO-C Specification Language (ACSL) [8]. The Frama-C/WP plug-in uses weakest precondition computations to generate proof obligations. To formally prove the ACSL properties, these proof obligations can be submitted to external automatic theorem provers or interactive proof assistants. For the precise settings for Frama-C/WP we employed in this release we refer to Chapter 1.

A.1. Verification settings

Here are the most important options of Frama-C that we used in for almost all functions.[32]

```
-pp-annot
-no-unicode
-wp
-wp-rte
-wp-model Typed
-warn-unsigned-overflow
-warn-unsigned-downcast
-wp-timeout 1
-wp-coq-timeout 5
```

Note that we use a relative small timeout value for the provers. For a couple of algorithms, however, we had to use a larger timeout.

For the precise versions of the employed provers we refer to Table 1.1 on Page 3.

[32]For the `my_lrand48()` function in `shuffle`, the option `-warn-unsigned-overflow` is disabled as explained in §7.18.

A.2. Verification results (sequential)

In the *sequential verification scenario* each proof obligation is processed by a set of automatic and interactive theorem provers that are arranged as a *pipe*.[33] This means that each prover passes on to the next prover only those proof obligations that it could not verify. This *verification pipeline* is shown in Figure A.1.

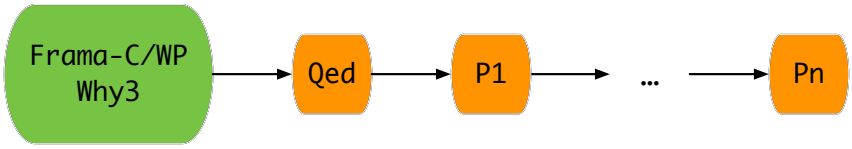

Figure A.1.: Verification pipeline of automatic and interactive theorem provers

For each algorithm we list in the following tables the number of generated verification conditions (VC), the percentage of proven verification conditions, and the number of VC proven by each prover. The value zero is indicated by an empty cell. The tables show that all verification conditions could be verified. Please note that the number of proven verification conditions do *not* reflect on the quality/strength of the individual provers. The reason for that is that we "pipe" each verification condition sequentially through a list of provers (see Figure A.1).

Algorithm		**Verification Conditions**		**Individual Provers**						
				T	U	QD	AE	Z3	C5	CQ
`find`	§4.1	27/27	(100%)	1	1	19	6	·	·	·
`find2`	§4.2	29/29	(100%)	1	1	17	10	·	·	·
`find3`	§4.3	32/32	(100%)	1	1	11	16	·	·	3
`find4`	§4.3.4	34/34	(100%)	·	·	13	18	·	·	3
`find5`	§4.3.4	23/23	(100%)	·	·	7	13	·	·	3
`find_if_not`	§4.4	38/38	(100%)	1	1	11	20	·	·	5
`find_first_of`	§4.5	43/43	(100%)	·	·	35	8	·	·	·
`adjacent_find`	§4.6	30/30	(100%)	1	1	19	9	·	·	·
`mismatch`	§4.7	28/28	(100%)	1	1	19	7	·	·	·
`equal`	§4.7	9/ 9	(100%)	·	·	8	1	·	·	·
`search`	§4.8	46/46	(100%)	·	·	35	11	·	·	·
`search_n`	§4.9	81/81	(100%)	1	1	55	24	·	·	·
`find_end`	§4.10	36/36	(100%)	·	·	23	13	·	·	·
`count`	§4.11	36/36	(100%)	1	1	10	17	·	·	7

Table A.2.: Results for non-mutating algorithms

[33] Sequential processing is achieved by passing the option `-wp-par 1` to Frama-C/WP.

Algorithm		Verification Conditions		Individual Provers						
				T	U	QD	AE	Z3	C5	CQ
`clamp`	§5.4	30/30	(100%)	1	1	23	5	·	·	·
`make_pair`	§5.3	6/ 6	(100%)	1	1	4	·	·	·	·
`max_element`	§5.5	32/32	(100%)	1	1	22	8	·	·	·
`max_element2`	§5.6	38/38	(100%)	1	1	26	10	·	·	·
`max_seq`	§5.7	16/16	(100%)	·	·	12	4	·	·	·
`min_element`	§5.8	38/38	(100%)	1	1	26	10	·	·	·
`minmax_element`	§5.9	64/64	(100%)	·	·	48	16	·	·	·

Table A.3.: Results for maximum and minimum algorithms

Algorithm		Verification Conditions		Individual Provers						
				T	U	QD	AE	Z3	C5	CQ
`lower_bound`	§6.3	27/27	(100%)	1	1	11	14	·	·	·
`upper_bound`	§6.4	27/27	(100%)	1	1	13	12	·	·	·
`equal_range`	§6.5	26/26	(100%)	·	·	20	6	·	·	·
`equal_range2`	§6.5	74/74	(100%)	·	·	28	39	·	·	7
`binary_search`	§6.6	20/20	(100%)	·	·	15	5	·	·	·

Table A.4.: Results for binary search algorithms

Algorithm		Verification Conditions		Individual Provers						
				T	U	QD	AE	Z3	C5	CQ
`fill`	§7.2	17/ 17	(100%)	1	1	9	6	·	·	·
`swap`	§7.3	10/ 10	(100%)	1	1	5	3	·	·	·
`swap_ranges`	§7.4	25/ 25	(100%)	·	·	14	11	·	·	·
`copy`	§7.5	18/ 18	(100%)	1	1	9	7	·	·	·
`copy_backward`	§7.6	19/ 19	(100%)	1	1	8	9	·	·	·
`reverse_copy`	§7.7	20/ 20	(100%)	1	1	9	9	·	·	·
`reverse`	§7.8	26/ 26	(100%)	·	·	9	17	·	·	·
`rotate_copy`	§7.9	19/ 19	(100%)	·	·	7	12	·	·	·
`rotate`	§7.10	21/ 21	(100%)	·	·	11	10	·	·	·
`replace_copy`	§7.11	21/ 21	(100%)	1	1	10	9	·	·	·
`replace`	§7.12	18/ 18	(100%)	1	1	9	7	·	·	·
`remove_copy`	§7.13	26/ 26	(100%)	1	1	11	13	·	·	·
`remove_copy2`	§7.14	76/ 76	(100%)	1	1	12	45	1	·	16
`remove_copy3`	§7.15	110/110	(100%)	1	1	14	72	1	1	20
`remove`	§7.16	104/104	(100%)	1	1	12	68	1	1	20
`shuffle`	§7.17	58/ 58	(100%)	·	·	19	30	·	·	9
`random_number`	§7.18	40/ 40	(100%)	2	2	21	15	·	·	·

Table A.5.: Results for mutating algorithms

Algorithm		Verification Conditions		Individual Provers						
				T	U	QD	AE	Z3	C5	CQ
`iota`	§8.1	19/19	(100%)	1	1	10	7	·	·	·
`accumulate`	§8.2	26/26	(100%)	1	1	9	13	·	·	2
`inner_product`	§8.3	29/29	(100%)	1	1	9	16	·	·	2
`partial_sum`	§8.4	59/59	(100%)	1	1	17	38	·	·	2
`adjacent_difference`	§8.5	34/34	(100%)	1	1	11	21	·	·	·
`partial_sum_inv`	§8.6	42/42	(100%)	·	·	10	29	·	·	3
`adjacent_difference_inv`	§8.6	42/42	(100%)	·	·	10	29	·	·	3

Table A.6.: Results for numeric algorithms

Algorithm		Verification Conditions		Individual Provers						
				T	U	QD	AE	Z3	C5	CQ
`heap_parent`	§9.3	13/ 13	(100%)	1	1	3	8	·	·	·
`heap_child`	§9.3	46/ 46	(100%)	1	1	13	30	·	·	1
`is_heap_until`	§9.4	42/ 42	(100%)	1	1	14	25	·	·	1
`is_heap`	§9.5	27/ 27	(100%)	·	·	12	14	·	·	1
`push_heap`	§9.7	121/121	(100%)	·	·	35	71	·	4	11
`pop_heap`	§9.8	142/142	(100%)	·	·	44	82	·	5	11
`make_heap`	§9.9	72/ 72	(100%)	·	·	25	38	·	·	9
`sort_heap`	§9.10	76/ 76	(100%)	·	·	22	42	·	·	12

Table A.7.: Results for heap algorithms

Algorithm		Verification Conditions		Individual Provers						
				T	U	QD	AE	Z3	C5	CQ
`is_sorted`	§10.1	26/ 26	(100%)	1	1	12	9	·	·	3
`partial_sort`	§10.2	143/143	(100%)	·	·	38	86	·	1	18
`bubble_sort`	§10.3	93/ 93	(100%)	·	·	32	47	·	2	12
`selection_sort`	§10.4	74/ 74	(100%)	·	·	24	34	·	·	16
`insertion_sort`	§10.5	81/ 81	(100%)	·	·	28	38	2	·	13
`heap_sort`	§10.6	53/ 53	(100%)	·	·	15	29	·	·	9
`merge`	§10.7	119/119	(100%)	·	·	41	65	7	1	5

Table A.8.: Results for algorithms related to sorting

Algorithm		Verification Conditions		Individual Provers						
				T	U	QD	AE	Z3	C5	CQ
`stack_equal`	§11.6.1	20/20	(100%)	·	·	9	11	·	·	·
`stack_init`	§11.6.2	16/16	(100%)	1	1	4	10	·	·	·
`stack_size`	§11.6.3	8/ 8	(100%)	1	1	1	5	·	·	·
`stack_full`	§11.6.4	13/13	(100%)	·	·	7	6	·	·	·
`stack_empty`	§11.6.5	12/12	(100%)	·	·	7	5	·	·	·
`stack_top`	§11.6.6	18/18	(100%)	·	·	8	10	·	·	·
`stack_push`	§11.6.7	43/43	(100%)	·	·	28	15	·	·	·
`stack_pop`	§11.6.8	31/31	(100%)	·	·	19	12	·	·	·

Table A.9.: Results for stack functions

Algorithm		Verification Conditions		Individual Provers						
				T	U	QD	AE	Z3	C5	CQ
`stack_size_wd`	§11.6.3	14/14	(100%)	·	·	10	4	·	·	·
`stack_empty_wd`	§11.6.5	14/14	(100%)	·	·	10	4	·	·	·
`stack_top_wd`	§11.6.6	14/14	(100%)	·	·	10	4	·	·	·
`stack_push_wd`	§11.6.7	17/17	(100%)	·	·	6	11	·	·	·
`stack_pop_wd`	§11.6.8	14/14	(100%)	·	·	8	6	·	·	·

Table A.10.: Results for the well-definition of stack functions

Algorithm		Verification Conditions		Individual Provers						
				T	U	QD	AE	Z3	C5	CQ
`axiom_size_of_init`	§11.7.1	17/17	(100%)	·	·	13	4	·	·	·
`axiom_size_of_push`	§11.7.2	14/14	(100%)	·	·	11	3	·	·	·
`axiom_top_of_push`	§11.7.2	13/13	(100%)	·	·	10	3	·	·	·
`axiom_size_of_pop`	§11.7.3	13/13	(100%)	·	·	10	3	·	·	·
`axiom_pop_of_push`	§11.7.3	12/12	(100%)	·	·	8	4	·	·	·
`axiom_push_of_pop_top`	§11.7.3	17/17	(100%)	·	·	12	5	·	·	·

Table A.11.: Results for stack axioms

B. Changes in previous releases

This chapter describes the changes in previous versions of this document. For the most recent changes we refer to Chapter 1.

The version numbers of this document are related to the versioning of Frama-C [2]. The versions of Frama-C are named consecutively after the elements of the periodic table. Therefore, our version numbering (X.Y.Z) are constructed as follows:

X the major number of our tutorial is the atomic number[34] of the chemical element after which Frama-C is named.

Y the Frama-C subrelease number

Z the subrelease number of this tutorial

B.1. New in Version 32.0.0 (Germanium, December 2025)

This release is intended for Frama-C [2, v32.0] issued in December 2025. We are also using for this release the Why3 platform [3, v1.8.2] and the provers listed in the following table.

Prover	Type	Version	Reference
Alt-Ergo	automatic	2.6.2	[4]
CVC5	automatic	1.3.2	[5]
Z3	automatic	4.15.2	[6]
Coq	interactive	8.20.1	[7]

Information on automatic and interactive theorem provers

The main changes in this release concern the rewriting of Coq proofs to reflect changes in Frama-C/WP.

New examples

None.

Improvements

- Logic and proofs: rename `RemovePartition` to `IndexOfNotEqual` and adjust `remove` and `remove_copy3` implementations and tutorial text accordingly.
- Add new lemmas to improve verification rate: `Accumulate_Next`, `InnerProduct_Init` and `InnerProduct_Next`.
- Cleanup: remove references to `count2` from tutorial.
- Build system: fix and clean up Makefile targets.

[34] See `http://en.wikipedia.org/wiki/Atomic_number`

Open issues

These following warnings are mainly due to changes in the behavior and diagnostics of Frama-C/WP and will be addressed in a later revision of ACSL by Example.

- **Missing explicit `terminates` / `exits` clauses**
  Function contracts are present, but lack explicit `terminates` and/or `exits` clauses for functions without visible bodies; WP therefore generates default clauses.
- **Pedantic assigns warning**
  An `assigns` clause is missing or too weak compared to the implementation, reported under `wp:pedantic-assigns`.
- **Unsupported statement-level specification**
  A statement contract is present but currently ignored by WP due to lack of support.

B.2. New in Version 22.0.0 (Titanium, November 2020)

This release is intended for Frama-C [2, v22.1] issued in November 2020. We are also using for this release the Why3 platform [3, v1.3.3] and the provers listed in the following table.

Prover	Type	Version	Reference
Alt-Ergo	automatic	2.3.3	[4]
CVC4	automatic	1.7	[30]
Z3	automatic	4.8.6	[6]
Coq	interactive	8.12.1	[7]

Information on automatic and interactive theorem provers

Note that all automatic provers use the Why3 interface. However, the interactive prover Coq still relies on the native interface provided by Frama-C/WP.

New examples

None.

Improvements

Updated to Coq 8.12.1.

Open issues

- The contract of algorithm `merge` does not handle the reordering of the involved arrays.

B.3. New in Version 21.1.1 (Scandium, September 2020)

This release is intended for Frama-C [2, v21.1] issued in June 2020. We are also using for this release the Why3 platform [3, v1.3.3] and the provers listed in the following table.

Prover	Type	Version	Reference
Alt-Ergo	automatic	2.3.3	[4]
CVC4	automatic	1.7	[30]
Z3	automatic	4.8.6	[6]
Coq	interactive	8.9.1	[7]

Information on automatic and interactive theorem provers

Note that all automatic provers use the Why3 interface. However, the interactive prover Coq still relies on the native interface provided by Frama-C/WP.

New examples

None.

Improvements

- general changes
  - disable CVC3 and switch back to Z3 `4.8.6`
  - refactor Coq proofs
  - reduce general timeout to 1s (Coq timeout 5s)
- improve loop invariants of `remove_copy3` and `remove` to reduce timeout
- add lemma `AdjacentDifference_InverseBounds` to reduce timeout of `adjacent_difference_inv` robust
- add lemmas `Count_Single`, `Count_Single_Bounds`, `Count_Single_Shift` and `Count_Cut`
- heap algorithms
  - rework `push_heap` and add more assertions to reduce timeout
  - rework `pop_heap` and finally verify property `reorder`
  - add predicates `ArrayUpdate`, `MultisetParity`, `MultisetUpdate` and supporting lemmas
  - remove predicate `PushHeapAdjust` and accompanying lemmas
  - rename `heap_child_max` to `heap_child` and improve both contract and implementation
  - add lemma `HeapParent_Zero`
- sorting and reordering
  - rework contract and annotations of merge
  - no need more for option `-wp-split`
  - add lemma `WeaklyIncreasing_Shrink`
  - add lemma `WeaklyIncreasing_Unchanged`
  - add more annotations to `bubble_sort` to reduce timeout
  - add lemma `MultisetSwap_FrontMiddle`

Open issues

- The contract of algorithm `merge` does not handle the reordering of the involved arrays.

Renaming of ACSL definitions

We sometimes rename predicates, logic functions or lemmas in order to make them more precise or make the naming more consistent.

- rename suffix `_Read` to `_Unchanged` in names of lemmas

Old name	New name
`EqualRanges`	`Equal`
`Equal_Increasing`	`Increasing_Equal`
`Equal_Count`	`Count_Equal`
`MultisetUnchanged`	`MultisetReorder`
`MultisetUnchanged_Union`	`MultisetReorder_DisjointUnion`
`Reorder_Match`	`MultisetReorder_SomeEqual`
`Reorder_LowerBound`	`MultisetReorder_LowerBound`
`Reorder_LowerBounds`	`MultisetReorder_PartitionLowerBound`
`Reorder_UpperBound`	`MultisetReorder_UpperBound`
`SwappedInside`	`ArraySwap`
`SwappedInside_Reorder`	`MultisetSwap_Middle`

B.4. New in Version 21.1.0 (Scandium, July 2020)

This release is intended for Frama-C [2, v21.1] issued in June 2020. We are also using for this release the Why3 platform [3, v1.3.1] and the provers listed in the following table.

Prover	Type	Version	Reference
Alt-Ergo	automatic	2.3.2	[4]
CVC4	automatic	1.7	[30]
CVC3	automatic	2.4.1	[31]
Z3	automatic	4.8.8	[6]
Coq	interactive	8.9.1	[7]

Information on automatic and interactive theorem provers

Note that all automatic provers use the Why3 interface. However, the interactive prover Coq still relies on the native interface provided by Frama-C/WP.

New examples

None.

Improvements

- Improve many code annotations in order to maintain the verification rate and to reduce timeout values.
- Use predicate `WeaklyIncreasing` instead of `Increasing` in the assertions and invariants of sorting algorithms. This allows the removal of lemma `IncreasingUpperBound`.
- Add Coq to parallel verification

- Replace an ACSL lemma on integer division by a Coq lemma in driver.
- `remove_copy` and `remove`
  - Remove logic helper function `NextNotEqual` for `RemovePartition` which served as a workaround in Frama-C 20.
  - Remove lemma `RemovePartition_StrictlyIncreasing`.
  - Make the definition of predicate `Remove` more flexible.
  - Remove lemma `Remove_Update`.
- heap algorithms
  - Simplify definition of predicates `MultisetRetainRest` and `MultisetMinus`.
  - Add predicate `PushHeapAdjust`.
  - Add lemmas `PushHeapAdjust_Init` and `PushHeapAdjust_Finish`.
  - Rename lemma `MultisetPushHeapRetain` to `PushHeapAdjust_Retain`.
  - Add predicate `HeapCompatible` and lemmas `Heap_Shrink`, `Heap_Unchanged` and `Heap_Update`.
  - Improve annotations of `push_heap` and `pop_heap`.
  - Remove predicate `HeapChildMax` and simplify the contract of `heap_child_max`.
- Remove lemma `SwappedInside_Preserve` which was of limited usefulness.
- Add lemmas `Accumulate_Init`, `AccumulateBounds_Read` and `Accumulate_Read_Shrink` , which were suggested by Allan Blanchard, in to simplify the verification of `partial_sum` and `adjacent_difference_inv`.
- Add lemmas `Unchanged_Symmetric` and `MultisetUnchanged_Symmetric`.

Open issues The following algorithms and/or lemmas are not completely verified

- `pop_heap`
- `merge`

B.5. New in Version 20.0.2 (Calcium, April 2020)

This release is intended for Frama-C [2, v20.0] issued in December 2019. We are also using for this release the Why3 platform [3, v1.2.1] and the provers listed in the following table.

Prover	Type	Version	Reference
Alt-Ergo	automatic	2.3.2	[4]
CVC4	automatic	1.6	[30]
CVC3	automatic	2.4.1	[31]
Z3	automatic	4.8.6	[6]
Coq	interactive	8.9.1	[7]

Information on automatic and interactive theorem provers

Note that all automatic provers use the Why3 interface. However, the interactive prover Coq still relies on the native interface provided by Frama-C/WP.

New examples

- Add examples `find4` and `find5` that verify the equivalence of the contracts of `find2` and `find3`.
- Add example `find_if_not`.

Improvements

- Add indices for examples and logic definitions.
- Re-add results of running all provers in parallel. Thanks to Allan Blanchard for explaining how Frama-C/WP's *session* mechanism can be used in the implementation.
- Fix a ghost label in `partial_sort`. Thanks to Virgile Prevosto for pointing out stricter checks in upcoming releases of Frama-C.
- Reduce very long verification times of several examples.
  - Add assertion `unchanged` to empty else branch of `remove_copy3`.
  - Add assertion `reorder` to empty else branch of `shuffle`.
  - Rewrite assertion `update` of `remove`.
  - Add another assertion `heap` to `push_heap`.
- Remove chapter on `unique_copy` because on its reliance on axioms. Moreover, the main ideas are already extensively discussed in the sections on `remove_copy` and `remove`.
- Verify properties of operator < within example `clamp`.
- Improve admitted Coq proof of `Reorder_Match`.
- Fix misplaced arrow in figure of `equal_range` algorithm

Open issues The following algorithms and/or lemmas are not completely verified

- `pop_heap` (property `reorder`)
- `merge` (property `reorder`)
- `Reorder_Match`

B.6. New in Version 20.0.1 (Calcium, March 2020)

This release is intended for Frama-C [2, v20.0] issued in December 2019. We are also using for this release the Why3 platform [3, v1.2.1] and the provers listed in the following table.

Prover	Type	Version	Reference
Alt-Ergo	automatic	2.3.1	[4]
CVC4	automatic	1.6	[30]
CVC3	automatic	2.4.1	[31]
Z3	automatic	4.8.6	[6]
Coq	interactive	8.9.1	[7]

Information on automatic and interactive theorem provers

Note that all automatic provers use the Why3 interface. However, the interactive prover Coq still relies on the native interface provided by Frama-C/WP.

New examples

- add a third version of `find` that is specified using the new logic function `Find`

Improvements

- improve text in many places
- improve specification of `remove_copy` and `remove`
  - provide an explicit definition of `RemovePartition` that allows to replace axioms by lemmas
  - rename predicate `ConstantRange` to `AllEqual` and add its negation `SomeNotEqual`
  - add logic functions `CountNotEqual` and `FindNotEqual`
- place all logic definitions in `axiomatic` blocks to better control generated names
- make names of ACSL predicates, functions and lemmas more uniform and place them together in files where appropriate
- among the renamed ACSL entities are
  - rename predicate `HasValue` to `SomeEqual` and add its negation `NoneEqual`
  - rename lemma `HasValueImpliesPositiveCount` to `SomeEqualCount`
  - rename lemma `PositiveCountImpliesHasValue` to `Count_SomeEqual`
  - rename `RotatePreservesStrictLowerBound` to `CircularShift_StrictLowerBound`
  - rename `RotateImpliesMultisetUnchanged` to `CircularShiftMultisetUnchanged`

Open issues

The following algorithms and/or lemmas are not completely verified

- `pop_heap`
- `Reorder_Match`

B.7. New in Version 20.0.0 (Calcium, December 2019)

Aside from the above-mentioned version of Frama-C we are using for this release the Why3 platform [3, v1.2.1] and the provers listed in the following table. Note that all automatic provers are use the Why3 interface. In other words, we do not use anymore the native interface for Alt-Ergo.

Prover	Type	Version	Reference
Alt-Ergo	automatic	2.3.0	[4]
CVC4	automatic	1.6	[30]
CVC3	automatic	2.4.1	[31]
Z3	automatic	4.8.6	[6]
Coq	interactive	8.9.1	[7]

Information on automatic and interactive theorem provers

New examples

- add `bubble_sort`

Improvements

- remove Why3 and Alt-Ergo lemmas from driver
- switch from memory model 'Typed+Ref' to 'Typed'
- the E theorem prover is not yet supported by this version of Frama-C
- no results on parallel verification are reported in this release
- rewrite `random_shuffle` to `shuffle`
  - adapt signature of `random_number`
  - add auxiliary function `random_init`
- replace, where applicable, ghost labels by loop labels or statement labels
- remove lemma `SwapImpliesMultisetUnchanged` by using predicate `SwappedInside` and its related lemmas
- improve specification and verification rate of numeric algorithms
  - resolve overloaded version of `Accumulate` into `AccumulateDefault`
  - resolve overloaded version of `AccumulateBounds` into `AccumulateDefaultBounds`
  - improve definition of predicate `PartialSum`
  - add lemmas `Difference_Zero` and `Difference_Next`
  - add predicate `DefaultBounds`
- add assigns in behaviors of maxmin and non-mutating algorithms
  - `find`, `find2`, `find_first_of`, `adjacent_find`, `mismatch`, `search`, `find_end`
  - `max_element`, `max_element2`, `min_element`, `minmax_element`
- rename predicate `Sorted` to `Increasing`; also rename related logic names

- rename `EqualRangesPreservesSorted` ↦ `EqualRangesPreservesIncreasing`
- rename `SortedUpperBound` ↦ `IncreasingUpperBound`
- rename `WeaklySortedAddElement` ↦ `WeaklyIncreasingAddElement`
- rename `WeaklySortedShift` ↦ `WeaklyIncreasingShift`
- rename `EqualRangesWeaklySorted` ↦ `EqualRangesWeaklyIncreasing`
- rename `WeaklySortedJoin` ↦ `WeaklyIncreasingJoin`
- rename `WeaklySortedLemmas` ↦ `WeaklyIncreasingLemmas`
- rename `SortedIFFWeaklySorted` ↦ `IncreasingIFFWeaklyIncreasing`
- rename `SortedImpliesWeaklySorted` ↦ `IncreasingImpliesWeaklyIncreasing`
- rename `WeaklySortedImpliesSorted` ↦ `WeaklyIncreasingImpliesIncreasing`
- rename `WeaklySorted` ↦ `WeaklyIncreasing`
- rename `SortedShift` ↦ `Increasing_Shift`

- remove lemma `SortedDownIsHeap`

Open issues

The following algorithms and/or lemmas are not completely verified

- `adjacent_difference_inv`
- `pop_heap`
- `random_number`
- `ReorderImpliesMatch`

B.8. New in Version 19.1.0 (Potassium, October 2019)

This release is intended for Frama-C 19.1 (*Potassium*), issued in September 2019. [2]

Aside from the above-mentioned version of Frama-C we are using for this release the Why3 platform [3, v1.2.0] and the provers listed in the following table. Note that all automatic provers are use the Why3 interface. In other words, we do not use anymore the native interface for Alt-Ergo.

Prover	Type	Version	Reference
Alt-Ergo	automatic	2.3.0	[4]
CVC4	automatic	1.6	[30]
CVC3	automatic	2.4.1	[31]
Z3	automatic	4.8.6	[6]
E	automatic	2.3	[32]
Coq	interactive	8.9.1	[7]

Information on automatic and interactive theorem provers

Improvements

- Rename arguments of `search` and `find_end` and improve also the description of these algorithms.
- Rename and reorder arguments of `search_n`, make the verification more robust and improve its description.
- Make verification of property `size` of `remove_copy2` more robust.
- Explain role of lemma `RemoveImpliesNotHasValue` in `remove_copy3` and `remove`.
- Simplify definition of `RemoveSize` and `RemovePartition`.
- Make verification of property `reorder` of `partial_sort` more robust.
- Strengthen precondition of `replace_copy`.
- Rename lemma `random_number_modulo` into `RandomNumberModulo`.
- Differentiate between properties `unique` and `solitary` for `unique_copy` examples.
- Simplify the implementation of `is_heap` by calling the new function `is_heap_until`.
- Replace remaining instances of label `Pre` in contracts by `Old`.
- Unify use of `Unchanged` predicate for mutating algorithms.

New examples

- Add the algorithm `clamp` which "clips" a value between a pair of boundary values.
- Add the algorithm `minmax_element` and improve description of other algorithms related to finding minimum and maximum values.
- Add new example `is_heap_until` that generalizes `is_heap`.
- The following examples are not new since they were implicitly used as helper functions for other examples. They are now explicitly listed as examples.
  - `make_pair`
  - `random_number`
  - `heap_parent`
  - `heap_child_max` (formerly known as `heap_maximum_child`

Open issues

The following algorithms and/or lemmas are not completely verified

- `adjacent_difference_inv`
- `partial_sum_inv`
- `pop_heap`
- `ReorderImpliesMatch`

B.9. New in Version 19.0.0 (Potassium, June 2019)

- Structure of document
  - The document is now structured into several parts.
  - The chapter on classic sorting algorithms has been merged into the chapter on sorting.
  - The various variants of `unique_copy` are now grouped into a separate chapter.
- Fix various inconsistencies
  - Change the return types of the logic functions `Accumulate`, `Difference`, `Capacity`, `Size`, `Top` from bounded one (e.g., `value_type`, `size_type`) to integer. A combination of bounded type for a logic function with an arithmetic operations in the logical definitions may lead to inconsistency. This fixes the inconsistencies in the `accumulate`, `stack` and `stack_wd` examples.
  - Fix an inconsistency in `DifferenceRead` axiom: restriction on the array size added to premises.
- Various improvements
  - An important change is the rewriting of the implicit, *axiomatic* definitions of `Accumulate`, `Count`, `Difference`, `InnerProduct` and `UniqueSize` logic functions to explicit, *recursive* ones. Accordingly, all axioms in the respective examples have been rewritten as lemmas.
  - Generalize `CountSectionMonotonic`, `UnchangedSection` lemmas: remove restriction on lower bound for the range.
  - Fix typo in postcondition of `find`.
  - Rewrite specifications of `remove_copy` and `remove` examples.
  - Rename predicate `RemoveCount` to `RemoveSize`.
  - Gather all versions of `MultisetRetainRest` in section on `push_heap`.
  - Add another figure to highlight simple contract for `unique_copy`.
  - Adapt Coq proofs to the fact that the `Z` scope is not available by default.
- New examples
  - Add `count2` example with an inductive predicate instead of a logic function in `count`.
  - Add `merge` example.
- Infrastructure
  - Travis-CI configuration for the GitHub repository added as an illustrative example of how the verification results could be reproduced.
  - Add support for Frama-C/AstraVer plugin.

B.10. New in Version 18.0.0 (Argon, December 2018)

- Replace the links to the (now abandoned) original site of *Standard Template Library* (STL) by references to the C++ standard.

- Add new algorithm `unique_copy` (two versions).
- Add another assertion `half` for `reverse`.
- Add two overloaded versions of predicate `ConstantRange` and use them for the algorithms `fill` and `unique_copy`, respectively.

B.11. New in Version 17.1.0 (Chlorine, July 2018)

The exact version number of Frama-C originally was Chlorine-20180502. This version number was changed in October 2018 to 17.1

- Slightly change the definition of predicate `HasEqualNeighbors` and its use in the specification of `adjacent_find`.
- Remove the algorithm `remove` and the more elaborate version of `remove_copy`. We are currently working on new specifications of these algorithms.
- Adapt some Coq proofs related to the logic function `Count` in order to reflect changes in output of Frama-C/WP.
- Remove table on ACSL lemmas that had to be proved by Coq.

B.12. New in Version 16.1.1 (Sulfur, March 2018)

- fix several errors reported by Aaron Rocha, including,
  - fix an error in figure for `upper_bound` algorithms
- fix merging of contracts in second version of `binary_search`
- improve and justify the `retain` annotations of in the implementation of `remove`
- Alt-Ergo is now directly called in the parallel setting (instead of going through Why3) to be compatible with the sequential setting
- add a third assertion `reorder` in the `random_shuffle` body to keep verification rate at 100% after prover upgrade

B.13. New in Version 16.1.0 (Sulfur, December 2017)

- special thanks to Aaron Rocha who provided various improvements for Chapters 4, 5, and 6
- improve some mutating algorithms
  - add more assertions to `reverse` to reduce reliance on CVC3
  - improve structure and ACSL annotations of `remove_copy` and `remove`
    - add overloaded version of predicate `MultisetRetainRest`
    - add lemma `HasValueImpliesPositiveCount`
    - add lemma `PositiveCountImpliesHasValue`

    - remove lemma `HasValueShiftInversion`
    - remove lemma `HasValueCountInversion`
  - add custom lemma `random_number_modulo` for `random_shuffle`
- add new Chapter 10 with more algorithms related to sorting
  - add algorithm `is_sorted` including predicate `WeaklyIncreasing`
    - add lemma `IncreasingImpliesWeaklyIncreasing`
    - add lemma `WeaklyIncreasingImpliesIncreasing`
  - add algorithm `partial_sort` including predicate `Partition`
    - add lemma `ReorderImpliesMatch`
    - add lemma `ReorderPreservesUpperBound`
    - add lemma `ReorderPreservesLowerBound`
    - add lemma `PartialReorderPreservesLowerBounds`
    - add lemma `SwappedInside`
    - add lemma `SwappedInsideMultisetUnchanged`
    - add lemma `SwappedInsidePreservesMultisetUnchanged`
- improve various lemmas
  - rename lemma `SortedUp` to `IncreasingUpperBound`
  - generalize lemma `UnchangedSection`
  - refactor lemma `HeapBounds` into `C_Division_Two`

B.14. New in Version 15.1.2 (Phosphorus, October 2017)

- fix several typos reported by `seniorlackey@github` (thanks a lot!)
- add a new chapter on classic sorting algorithms which comprises
  - `selection_sort` including lemma `SwapImpliesMultisetUnchanged`
  - `insertion_sort` including lemmas
    - `RotatePreservesStrictLowerBound`
    - `RotateImpliesMultisetUnchanged`
    - `EqualRangesPreservesIncreasing`
    - `EqualRangesPreservesCount`
  - `heap_sort`
- heap algorithms
  - remove length requirements in `pop_heap`, `sort_heap`, `make_heap`, and `heap_sort`
    - introduce `SIZE_TYPE_MAX` to catch border cases in ACSL and C

- improve description of `pop_heap`
  - add predicate `HeapChildMax`
  - provide the auxiliary function `heap_child_max`
  - the postcondition `reorder` is still not verified
- improve description of `push_heap`
- other, minor improvements
  - add auxiliary function `heap_parent`
  - add predicate `SortedDown` and lemma `SortedDownIsHeap`
  - add lemmas `HeapParentChild` and `HeapChilds`
  - add lemmas `HeapParentBounds` and `HeapChildBounds`

B.15. New in Version 15.1.1 (Phosphorus, September 2017)

- add ensures clause to default behavior of the following algorithms
  - `find`, `find_first_of`, `adjacent_find`, `mismatch`, `search`, `search_n`, `find_end`
  - `max_element`, `min_element`
- rewrite axiomatic definitions to ensure disjoint guards which is better suited for E-ACSL
  - concerns the axiomatic definitions of `Count`, `Accumulate`, `InnerProduct` and `Difference`
  - some Coq proofs related to `Count` had to be adapted as well
- shorten names of some auxiliary algorithms
  - `adjacent_difference_inverse` ↦ `adjacent_difference_inv`
  - `partial_sum_inverse` ↦ `partial_sum_inv`
- heap algorithms
  - fix a typo in Figure 9.3
  - fix a typo in Figure 9.39
  - explain that there can be multiple representations of an array as a heap
  - add a version of `pop_heap` that is, however, not completely verified

B.16. New in Version 15.1.0 (Phosphorus, June 2017)

- The verification results are now part of the appendix.
- Fix an error in the specification of the well-definition of `stack_size`.
- This release of Frama-C/WP could not discharge some of our assertions of `push_heap`. We therefore have completely rewritten the annotations and also tweaked the implementation of `push_heap`.

We also added some new predicates and lemmas to maintain a concise specification that can easily be verified by automatic provers.

  - add predicate `MultisetAdd` and lemma `MultisetAdd_Distinct`
  - add predicate `MultisetMinus` and lemma `MultisetMinus_Distinct`
  - add predicate `MultisetRetain` and lemma `MultisetPushHeapRetain`
  - provide an additional version of predicate `MultisetRetainRest`
  - and lemma `MultisetPushHeapClosure`

B.17. New in Version 14.1.1 (Silicon, April 2017)

- changes in verification infrastructure
  - add verification results for the case where each proof obligation is submitted to all automatic theorem provers
- changes in algorithms
  - simplify loop invariants of `search_n` and improve description
  - rename predicate `CountOneHit` to `CountHit`
  - rename predicate `CountOneMiss` to `CountMiss`
  - rewrite predicates `EqualRanges` and `Reverse` in order to simplify the task for automatic theorem provers
  - remove lemmas on `Reverse` that were necessary for `rotate` but are not needed anymore
  - rename predicate `Valid(Stack*)` to `Invariant(Stack*)` and remove `\valid` from `Invariant(Stack*)`
  - add a simple random number generator to `random_shuffle` and verify it
- fix an inconsistency in the axioms for `Count` (thanks to Denis Efremov for reporting this issue)
  - add more guards to axioms `CountSectionHit` and `CountSectionMiss`
  - add corresponding guards to lemmas
    * `CountSectionOne`, `CountHit`, `CountMiss` and `CountOne`
    * `RemoveCountHit` and `RemoveCountMiss`
  - add lemma `Unchanged_Shift` and add more assertions to `remove` in order to simplify the task for automatic theorem provers

B.18. New in Version 14.1.0 (Silicon, January 2017)

- use label `Old` instead of `Pre` in function contracts
- add algorithm `rotate`
- rewrite definition of predicates `EqualRanges` and `Reverse` and provide more overloaded versions

- add figures for algorithms `rotate` and `replace_copy`
- update figure for predicate `Reverse`
- update Coq proofs and add a table with more information on the ACSL lemmas that had to be verified with Coq

B.19. New in Version 13.1.1 (Aluminium, November 2016)

- improve layout of tables of verification results
- use two additional automatic theorem provers (CVC3 and E)
- non-mutating algorithms
  - add algorithm `find_end`
  - add definition of predicate `HasSubRange` on subranges
  - add definition of predicate `EqualRanges` on subranges
  - rename lemma `HasSubRange_fit_size` to `HasSubRangeSize`
  - rename lemma `HasConstantSubRange_fit_size` to `HasSubRangeSize`
  - rename logic function `CountSection` to `Count` (using overloading in ACSL)
  - add lemma `HasValueCountInversion`
  - add lemma `HasValueShiftInversion`
  - add lemma `Count_Shift`
- mutating algorithms
  - add algorithm `copy_backward`
  - relax precondition on separation of `copy`, `replace_copy` and `remove_copy`
  - provide a more sophisticated implementation of `remove`
  - re-introduce a second version of `remove_copy` that also specifies the *stability* of the algorithm
  - add algorithm `random_shuffle`

B.20. New in Version 13.1.0 (Aluminium, August 2016)

The most notable changes of this version are the re-introduction of heap algorithms in Chapter 9. This new description of heap algorithms is based to a large extend on the bachelor thesis of one of the authors [23].

- provide names ("labels") for more ACSL annotations
- non-mutating algorithms
  - reorder and improve description in chapter on non-mutating algorithms
  - add more figures to describe algorithms
  - add non-mutating algorithm `search_n`

  - rewrite logic function `Count` with new logic function `CountSection`
  - move lemmas `Count_Bounds` and `CountMonotonic` to separate files
  - use `integer` instead of `size_type` in `HasSubRange`
  - change index computation in `HasEqualNeighbors`
- maximum and minimum algorithms
  - isolate predicate `ConstantRange` from predicates on lower and upper bounds
  - fix typo in precondition of first version of `max_element`
- binary search algorithms
  - add version `Sorted` for subranges
  - add second (more efficient) version of `equal_range`
    * add lemmas `SortedShift`, `LowerBound_Shift`, `StrictLowerBound_Shift`, `UpperBound_Shift` and `StrictUpperBound_Shift` to support the automatic verification of this version of `equal_range`
  - add figures to binary search algorithms and improve description
- mutating algorithms
  - greatly reduce the number of assertions needed to verify the first version `remove_copy`
  - temporarily remove the second version of `remove_copy` which also specified the *stability* of the algorithm
  - add `remove`, an in-place variant of `remove_copy`
  - rename predicate `RetainAllButOne` to `MultisetRetainRest`
- re-introduce chapter on heap algorithms
  - includes the heap algorithms `is_heap`, `push_heap`, `make_heap` and `sort_heap`
  - for `pop_heap` only a function contract is provided in this version
  - add lemma `SortedUp` to support verification of `sort_heap`
  - add several lemmas to combine the predicates `Unchanged` and `MultisetUnchanged`

B.21. New in Version 12.1.0 (Magnesium, February 2016)

A main goal of this release is to reduce the number of proof obligations that cannot be verified automatically and therefore must be tackled by an interactive theorem prover such as Coq. To this end, we analyzed the proof obligations (often using Coq) and devised additional assertions or ACSL lemmas to guide the automatic provers. Often we succeeded in enabling automatic provers to discharge the concerned obligations. Specifically, whereas the previous version 11.1.1 of *ACSL by Example* listed *nine* proof obligations that could only be discharged with Coq, the document at hand (version 12.1.0) only counts *five* such obligations. Moreover, all these remaining proof obligations are associated to ACSL lemmas, which are usually easier to tackle with Coq than proof obligations directly related to the C code. The reason for this is that ACSL lemmas usually have a much smaller set of hypotheses.

Adding assertions and lemmas also helps to alleviate a problem in Frama-C/WP Magnesium and Sodium where prover processes are not properly terminated.[35] Left-over "zombie processes" lead to a deterioration of machine performance which sometimes results in unpredictable verification results.

- mutating algorithms
  - simplify annotations of `replace_copy` and add new algorithm `replace`
    - add predicate `Replace` to write more compact post conditions and loops invariants
  - add several lemmas for predicate `Unchanged` and use predicate `Unchanged` in postconditions of mutating and numeric algorithms
  - simplify annotations of `reverse`
    - rename `Reversed` to `Reverse` (again) and provide another overloaded version
    - add figure to support description of the `Reverse` predicate
  - changes regarding `remove_copy`
    - rename `PreserveCount` to `RetainAllButOne`
    - rename `StableRemove` to `RemoveMapping`
    - add statement contracts for both versions of `remove_copy` such that only ACSL lemmas require Coq proofs
- numeric algorithms
  - define limits `VALUE_TYPE_MIN` and `VALUE_TYPE_MAX`
  - simplify specification of `iota` by using new logic function `Iota`
  - simplify implementation of `accumulate`
    - add overloaded predicates `AccumulateBounds`
    - add lemmas `AccumulateDefault0`, `AccumulateDefault1`, `AccumulateDefaultNext`, and `AccumulateDefault_Read`
  - simplify implementation of `inner_product`
    - add predicates `ProductBounds` and `InnerProductBounds`
  - enable automatic verification of `partial_sum`
    - add lemmas `PartialSumSection`, `PartialSumUnchanged`, `PartialSum_Step` ,
      and `PartialSumStep2` to automatically discharge loop invariants
  - enable automatic verification of `adjacent_difference`
    - add logic function `Difference` and predicate `AdjacentDifference`
    - add predicate `AdjacentDifferenceBounds`
    - add lemmas `AdjacentDifference_Step` and `AdjacentDifference_Section` to automatically discharge proof obligation

[35] See `https://bts.frama-c.com/view.php?id=2154`

  - add two auxiliary functions `partial_sum_inverse` and `adjacent_difference_inverse` in order to verify that `partial_sum` and `adjacent_difference` are inverse to each other
    * add lemmas `PartialSumInverse` and `AdjacentDifferenceInverse` to support the automatic verification of the auxiliary functions

- stack functions
  - add lemma `StackPush_Equal` to enable the automatic verification of the well-definition of `stack_push`

B.22. New in Version 11.1.1 (Sodium, June 2015)

- add Chapter on numeric algorithms
  - move `iota` algorithm to numeric algorithms (§8.1)
  - add `accumulate` algorithm (§8.2)
  - add `inner_product` algorithm (§8.3)
  - add `partial_sum` algorithm (§8.4)
  - add `adjacent_difference` algorithm (§8.5)

B.23. New in Version 11.1.0 (Sodium, March 2015)

- Use built-in predicates `\valid` and `\valid_read` instead of `valid_range`.
- Simplify loop invariants of `find_first_of`.
- Replace two loop invariants of `remove_copy` by ACSL lemmas.
- Rename several predicates
  - `IsEqual` ↦ `EqualRanges`.
  - `IsMaximum` ↦ `MaxElement`.
  - `IsMinimum` ↦ `MinElement`.
  - `Reverse` ↦ `Reversed`.
  - `IsSorted` ↦ `Sorted`.
- Several changes for `stack`:
  - Rename `stack` functions from *foo*`_stack` to `stack_`*foo*.
  - Equality of stacks now ignores the `capacity` field. This is similar to how equality for objects of type `std::vector<T>` is defined. As a consequence `stack_full` is not well-defined any more. Other stack functions are not effected.
  - Remove all assertions from stack functions (including in axioms).
  - Describe predicate `Separated` in text.

B.24. New in Version 10.1.1 (Neon, January 2015)

- use option `-wp-split` to create simpler (but more) proof obligations
- simplify definition of predicate `Count`
- add new predicates for lower and upper bounds of ranges and use it in
  - `max_element`
  - `min_element`
  - `lower_bound`
  - `upper_bound`
  - `equal_range`
  - `fill`
- use a new auxiliary assertion in `equal_range` to enable the complete *automatic* verification of this algorithm
- add predicate `Unchanged` and use it to simplify the specification of several algorithms
  - `swap_ranges`
  - `reverse`
  - `remove_copy`
  - `stack_push` and `stack_push_wd`
  - `stack_pop` and `stack_pop_wd`
- add predicate `Reverse` and use it for more concise specifications of
  - `reverse_copy`
  - `reverse`
- several changes in the two versions of `remove_copy`
  - use predicate `HasValue` instead of logic function `Count`
  - add predicate `PreserveCount`
  - reformulate logic function `RemoveCount`
  - add predicate `StableRemove`
  - add predicate `RemoveCountMonotonic`
  - add predicate `RemoveCountJump`
- use overloading in ACSL to create shorter logic names for `stack`
- remove unnecessary labels in several `stack` functions

B.25. New in Version 10.1.0 (Neon, September 2014)

- remove additional labels in the `assumes` clauses of some stack function that were necessary due to an error in Oxygen
- provide a second version of `remove_copy` in order to explain the specification of the *stability* of the algorithms
- coarsen loop assigns of mutating algorithms
- temporarily remove the `unique_copy` algorithm

B.26. New in Version 9.3.1 (Fluorine, not published)

- specify bounds of the return value of `count` and fix reads clause of `Count` predicate
- use an auxiliary function `make_pair` in the implementation of `equal_range`
- provide more precise loop assigns clauses for the mutating algorithms
  - simplify implementation of `fill`
  - removed the `ensures \valid(p)` clause in specification of `swap`
  - simplify implementation of `swap_ranges`
  - simplify implementation of `copy`
  - fix implementation of `reverse_copy` after discovering an undefined behavior
  - new implementation of `reverse` that uses a simple **`for`**-loop
  - simplify implementation of `replace_copy`
  - refactor specification and simplify implementation of `remove_copy`
- remove work-around with `Pre`-label in `assumes` clauses of `stack_push` and `stack_pop`

B.27. New in Version 9.3.0 (Fluorine, December 2013)

- adjustments for *Fluorine* release of Frama-C
- `swap` now ensures that its pointer arguments are valid after the function has been called
- change definition of `size_type` to **`unsigned int`**
- change implementation of the `iota` algorithm . The content of the field `a` is calculated by increasing the value `val` instead of sum `val+i`.
- change implementation of `fill`.
- The specification/implementation of `stack` has been revised by Kim Völlinger [26] and now has a much better verification rate.

B.28. New in Version 8.1.0 (Oxygen, not published)

- simplified specification and loop annotations of `replace_copy`
- add binary search variant `equal_range`
- greatly simplified specification of `remove_copy` by using the logic function `Count`
- remove chapter on heap operations

B.29. New in Version 7.1.1 (Nitrogen, August 2012)

- improvements with respect to several suggestions and comments of Yannick Moy, e.g., specification refinements of `remove_copy`, `reverse_copy` and `iota`
- restricted verification of algorithms to Frama-C/WP with Alt-Ergo
- replaced deprecated `\valid_range` by `\valid`
- fixed inconsistencies in the description of the `stack` data type
- binary search algorithms can now be proven without additional axioms for integer division
- changed axioms into lemmas to document that provability is expected, even if not currently granted
- adopted new Fraunhofer logo and contact email

B.30. New in Version 7.1.0 (Nitrogen, December 2011)

- changed to Frama-C Nitrogen
- changed to Why 2.30
- discussed both plug-ins Frama-C/WP and Jessie
- removed `swap_values` algorithm

B.31. New in Version 6.1.0 (Carbon, not published)

- changed definition of `stack`
- renamed `reset_stack` to `init_stack`

B.32. New in Version 5.1.1 (Boron, February 2011)

- prepared algorithms for checking by the new Frama-C/WP plug-in of Frama-C
- changed to Alt-Ergo Version 0.92, Z3 Version 2.11 and Why 2.27
- added List of user-defined predicates and logic functions
- added remarks on the relation of logical values in C and ACSL

- rewrote section on `equal` and `mismatch`
- used a simpler logical function to count elements in an array
- added `search` algorithm
- added chapter to unite the maximum/minimum algorithms
- added chapter for the new `lower_bound`, `upper_bound` and `binary_search` algorithms
- added `swap_values` algorithm
- used `IsEqual` predicate for `swap_ranges` and `copy`
- added `reverse_copy` and `reverse` algorithms
- added `rotate_copy` algorithm
- added `unique_copy` algorithm
- added chapter on specification of the data type `stack`

B.33. New in Version 5.1.0 (Boron, May 2010)

- adaption to Frama-C Boron and Why 2.26 releases
- changed from the `-jessie-no-regions` command-line option to using the pragma `SeparationPolicy(`*`value`*`)`

B.34. New in Version 4.2.2 (Beryllium, May 2010)

- changed to latest version of CVC3 2.2
- added additional remarks to our implementation of `find_first_of`
- changed `size_type` (**`int`**) to `integer` in all specifications
- removed casts in `fill` and `iota`
- renamed `is_valid_range` as `IsValidRange`
- renamed `has_value` as `HasValue`
- renamed predicate `all_equal` as `IsEqual`
- extended timeout to 30 sec.

B.35. New in Version 4.2.1 (Beryllium, April 2010)

- added alternative specification of `remove_copy` algorithm that uses `ghost` variables
- added Chapter on heap operations
- added `mismatch` algorithm
- moved algorithms `adjacent_find` and `min_element` from the appendix to chapter on non-mutating algorithms

- added typedefs `size_type` and `value_type` and used them in all algorithms
- renamed `is_valid_int_range` as `is_valid_range`

B.36. New in Version 4.2.0 (Beryllium, January 2010)

- complete rewrite of pre-release
- adaption to Frama-C Beryllium 2 release

Bibliography

[1] WP Plug-in. http://frama-c.com/wp.html.

[2] Frama-C Software Analyzers. http://frama-c.com, 2018.

[3] Why – Where Programs Meet Provers. https://www.why3.org, 2025.

[4] Sylvain Conchon, Evelyne Contejean, and Johannes Kanig. The Alt-Ergo SMT Solver. http://alt-ergo.lri.fr, 2018.

[5] Clark Barrett and Cesare Tinelli. Homepage of CVC5. https://cvc5.github.io, 2025.

[6] Microsoft Research. The Z3 Theorem Prover. https://github.com/Z3Prover/z3, 2025.

[7] The ROCQ Consortium. https://rocq-prover.org. https://rocq-prover.org, 2025.

[8] ANSI/ISO C Specification Language. http://frama-c.com/acsl.html, 2025.

[9] CEA LIST, Laboratory of Applied Research on Software-Intensive Technologies. http://www-list.cea.fr/gb/index_gb.htm.

[10] INRIA-Saclay, French National Institute for Research in Computer Science and Control . http://www.inria.fr/saclay/.

[11] LRI, Laboratory for Computer Science at Université Paris-Sud. http://www.lri.fr/.

[12] Fraunhofer-Institut für Offene Kommunikationssysteme (FOKUS). http://www.fokus.fraunhofer.de.

[13] Virgile Prevosto. ACSL Mini-Tutorial. http://frama-c.com/download/acsl-tutorial.pdf.

[14] Patrick Baudin, Pascal Cuoq, Jean-Christophe Filliâtre, Claude Marché, Benjamin Monate, Yannick Moy, and Virgile Prevosto. ACSL 1.13 Implementation in Argon 18.0. https://frama-c.com/download/acsl-implementation-18.0-Argon.pdf, 2018.

[15] Allan Blanchard. Introduction to C Program Proof using Frama-C and its wp plugin. http://allan-blanchard.fr/publis/frama-c-wp-tutorial-en.pdf, December 2017.

[16] Programming languages – C, Committee Draft. http://www.open-std.org/JTC1/SC22/WG14/www/docs/n1362.pdf, 2009.

[17] C.A.R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12:576–583, 1969.

[18] Robert W. Floyd. Assigning meanings to programs. In J. T. Schwartz, editor, *Proc. Symposium on Applied Mathematics*, volume 19 of *Mathematical Aspects of Computer Science*, pages 19–32, Providence, RI, 1967. American Mathematical Society.

[19] Richard Smith. Working Draft, Standard for Programming Language C++. http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/n4659.pdf, 2017. publicly available draft of C++ 17 standard.

[20] Donald E. Knuth, James H. Morris, and Vaughan R. Pratt. Fast pattern matching in strings. *SIAM J Comput*, 6(2):323–350, Jun 1977.

[21] Lincoln E. Moses and Robert V. Oakford. *Tables of Randon Permutations*. Stanford University Press, 1963.

[22] Ming Li and Paul Vitányi. *An Introduction to Kolmogorov Complexity and Its Applications*. Graduate texts in computer science. Springer, New York, 1997.

[23] Timon Lapawczyk. Formale Verifikation von Heap-Algorithmen mit Frama-C. bachelor thesis, Humboldt-Universität zu Berlin, July 2016.

[24] D.E. Knuth. *Sorting and Searching*, volume 3 of *The Art of Computer Programming*. Addison-Wesley, 1973.

[25] Sellibitze. How to Implement Classic Sorting Algorithms in Modern C++. `https://stackoverflow.com/questions/24650626/how-to-implement-classic-sorting-algorithms-in-modern-c`, Aug 2014.

[26] Kim Völlinger. Einsatz des Beweisassistenten Coq zur deduktiven Programmverifikation. Diplomarbeit, Humboldt-Universität zu Berlin, August 2013. `https://www2.informatik.hu-berlin.de/top/_media/www/mitarbeiter/diplomarbeit-kim-voellinger.pdf`.

[27] Richard Fitzpatrick J.L. Heiberg. *Euclid's Elements of Geometry*. `http://farside.ph.utexas.edu/euclid.html`, Austin/TX, 2008.

[28] David Hilbert. *Grundlagen der Geometrie*. B.G.Teubner, Stuttgart, 1968.

[29] Martin Odersky, Lex Spoon, and Bill Venners. *Programming in Scala*. Artima, 2008.

[30] Clark Barrett and Cesare Tinelli. Homepage of CVC4. `http://cvc4.cs.stanford.edu/web/`, 2018.

[31] Clark Barrett and Cesare Tinelli. Homepage of CVC3. `http://www.cs.nyu.edu/acsys/cvc3/`, 2010.

[32] Stephan Schulz. The E Theorem Prover. `https://wwwlehre.dhbw-stuttgart.de/~sschulz/E/E.html`, 2018.

Index of ACSL definitions

Caveat: This index has been automatically generated from the ACSL /C sources. For the time being it mentions only the page where an ACSL definition is first included in the text. Moreover, if a listing had to be split, then the page number refers to the first part of the listing even if the specific ACSL definition appears in the second part.

A

Accumulate 148–150
Accumulate_Init 149
Accumulate_Next 149
Accumulate_Unchanged 149
Accumulate_Unchanged_Shrink 149
AccumulateBounds 149
AccumulateBounds_Unchanged 149
AccumulateDefault 150, 156
AccumulateDefault_Next 150
AccumulateDefault_One 150
AccumulateDefault_Unchanged 150
AccumulateDefault_Zero 150
AccumulateDefaultBounds 150
AccumulateDefaultBounds_Shrink 150
AdjacentDifference 160, 161
AdjacentDifference_Inverse 164, 165
AdjacentDifference_InverseBounds 164, 165
AdjacentDifference_Section 160, 162
AdjacentDifference_Step 160, 162
AdjacentDifferenceBounds 160, 161
AllEqual 41, 58, 93, 104
AllSomeNot 41
ArrayBounds 71
ArrayBoundsShift 95, 96, 213
ArrayExtrema 72
ArraySwap 141
ArrayUpdate 102, 183, 187, 188, 193, 194
ArrayUpdate_MultisetUpdate 183
ArrayUpdate_Shrink 102
ArrayUpdate_UpperBound 102
At 102, 134, 136, 181

C

C_Bit 143
CircularShift_MultisetReorder 216
CircularShift_StrictLowerBound 216
CircularShiftLemmas 216

Count 63, 64, 66, 125, 126, 138
Count_Bounds 64
Count_Cut 64
Count_Empty 63, 64
Count_Equal 64
Count_Hit 63, 64
Count_Increasing 64
Count_Miss 64
Count_One 63, 64
Count_Shift 64
Count_Single 64
Count_Single_Bounds 64
Count_Single_Shift 64
Count_SomeEqual 66, 205
Count_Unchanged 64
Count_Union 64
CountFind 66
CountFindNotEqual 129
CountNotEqual 125, 126, 129
CountNotEqual_AllEqual 129
CountNotEqual_Count 125, 126
CountNotEqual_Decrement 129
CountNotEqual_Empty 125
CountNotEqual_FindNotEqual 129
CountNotEqual_Hit 125
CountNotEqual_Increasing 125
CountNotEqual_Lower 125
CountNotEqual_Miss 125
CountNotEqual_SomeNotEqual 129
CountNotEqual_Unchanged 125
CountNotEqual_Union 125
CountNotEqual_Upper 125
CountNotEqual_WeaklyIncreasing 125
CountNotEqual_Zero 129

D

DefaultBounds 163
Difference 159
Difference_Next 159
Difference_Unchanged 159
Difference_Zero 159

E

Equal 51–53, 55, 107, 110, 112, 232

F

Find 36, 38, 42, 44
Find_Empty 36
Find_Hit 36
Find_Increasing 36
Find_Limit 36
Find_Lower 36

Find_MissHit 36
Find_MissMiss 36
Find_NoneEqual 36
Find_ResultEqual 36
Find_ResultNoneEqual 36
Find_SomeEqual 36
Find_Upper 36
Find_WeaklyIncreasing 36
FindNotEqual 41, 42, 44, 129
FindNotEqual_AllEqual 42
FindNotEqual_Empty 42
FindNotEqual_Hit 42
FindNotEqual_Increasing 42
FindNotEqual_Limit 42
FindNotEqual_Lower 42
FindNotEqual_MissHit 42
FindNotEqual_MissMiss 42
FindNotEqual_ResultAllEqual 42
FindNotEqual_ResultNotEqual 42
FindNotEqual_SomeNotEqual 42
FindNotEqual_Unchanged 42
FindNotEqual_Upper 42
FindNotEqual_WeaklyIncreasing 42

G

Greater_Less 70
GreaterOrEqual_Less 70

H

HasConstantSubRange 57, 58
HasConstantSubRange_Sizes 57, 58
HasEqualNeighbors 48–50
HasSubRange 54, 55, 58, 61
HasSubRange_Sizes 54
HasValueOf 45–47
Heap 175, 177, 179
Heap_ChildBounds 174
Heap_Childs 174
Heap_Maximum 175
Heap_ParentBounds 174
Heap_ParentChild 174
Heap_ParentLeft 174
Heap_ParentRight 174
Heap_Shrink 175
Heap_Unchanged 175
HeapCompatible 175, 189, 194
HeapCompatible_Update 175
HeapLeft 174, 176
HeapNodes 174
HeapParent 174, 176
HeapParent_Zero 174

HeapRight 174, 176

I

Increasing 85–87, 199, 202, 206
Increasing_Equal 86, 87, 216
Increasing_Shift 86, 87
Increasing_WeaklyIncreasing 86, 87, 203
IncreasingLemmas 87
IndexOfNotEqual 130
IndexOfNotEqual_Core 130
IndexOfNotEqual_Count 130
IndexOfNotEqual_Empty 130
IndexOfNotEqual_Left 130
IndexOfNotEqual_Lower 130
IndexOfNotEqual_Next 130
IndexOfNotEqual_NotEqual 130
IndexOfNotEqual_Right 130
IndexOfNotEqual_Segment 130
IndexOfNotEqual_StrictlyWeakIncreasing 130
IndexOfNotEqual_Unchanged 130
IndexOfNotEqual_Upper 130
InnerProduct 154
InnerProduct_Init 154
InnerProduct_Next 154
InnerProduct_Unchanged 152, 154
InnerProductBounds 152, 154
IotaGenerate 146

L

Less 70
Less_Antisymmetry 70
Less_Irreflexivity 70
Less_Transitivity 70
LessOrEqual_Less 70
LowerBound 71, 81, 82, 88
LowerBound_Shift 96

M

MaxElement 71, 72, 78, 83, 175
MinElement 71, 72, 81, 83
Multiset_AddMinusRetain 181
Multiset_AddMinusRetainReorder 181
Multiset_MinusAddRetain 181
MultisetAdd 180, 181
MultisetAdd_Distinct 180
MultisetMinus 180, 181
MultisetMinus_Distinct 180
MultisetOperations 180, 181
MultisetParity 182, 183, 187
MultisetParity_Combined 182, 188, 194
MultisetParity_MultisetReorder 182, 189
MultisetParity_UnchangedFirst 182

MultisetParity_UnchangedSecond 182
MultisetReorder 138, 139, 181, 184, 196, 199, 206, 213
MultisetReorder_DisjointUnion 138
MultisetReorder_LowerBound 205
MultisetReorder_PartitionLowerBound 205
MultisetReorder_SomeEqual 205
MultisetReorder_Symmetric 138
MultisetReorder_Transitive 138
MultisetReorder_UpperBound 205
MultisetRetain 180, 181
MultisetRetain_Distinct 180
MultisetRetainRest 181
MultisetSwap 141
MultisetSwap_FrontMiddle 141
MultisetSwap_Middle 141, 205, 213
MultisetUpdate 183, 188, 194

N

NoneEqual 33–35, 37, 42, 123
NoneEqual_NotSomeEqual 33
NotAllEqual_SomeNotEqual 41
NotSomeEqual_NoneEqual 33
NumericInverse 164

P

PartialSum 156
PartialSum_Inverse 164
PartialSum_One 156
PartialSum_Section 156
PartialSum_Step 156
PartialSum_Unchanged 156
Partition 204
PartitionLemmas 205
ProductBounds 152, 154

R

RandomNumberModulo 143
Remove 132, 133, 135
Replace 120, 122
Reverse 113–115

S

SomeEqual 33, 34, 37, 42, 46, 66, 97
SomeNone 33, 34
SomeNotEqual 41
SomeNotEqual_NotAllEqual 41
StackCapacity 230
StackEmpty 230, 239
StackEqual 232, 235
StackEqual_Reflexive 232
StackEqual_Symmetric 232
StackEqual_Transitive 232

StackEquality 232, 233
StackFull 230, 232, 238
StackInvariant 230, 231
StackLemmas 243
StackPush_Equal 243
StackSeparated 231, 243, 245
StackSize 230, 237
StackStorage 230
StackTop 230, 240
StackUnchanged 231
StackUtility 231
StrictLowerBound 71, 81–83, 90
StrictLowerBound_Shift 96
StrictUpperBound 71, 78, 84, 88, 93
StrictUpperBound_Shift 96

U

Unchanged 43, 102, 103, 108, 115
Unchanged_Extend 103
Unchanged_MultisetReorder 138
Unchanged_Shift 103
Unchanged_Shrink 103
Unchanged_Symmetric 103
Unchanged_Transitive 103, 165
UnchangedLemmas 103
UpperBound 71, 78, 90
UpperBound_Shift 96

W

WeaklyIncreasing 86, 87, 199, 202, 203, 219
WeaklyIncreasing_AddElement 219, 221
WeaklyIncreasing_Equal 219, 221
WeaklyIncreasing_Increasing 86, 87, 199, 202, 203, 221
WeaklyIncreasing_Join 219, 221
WeaklyIncreasing_Shift 219, 221
WeaklyIncreasing_Shrink 219, 221
WeaklyIncreasing_Unchanged 219, 221
WeaklyIncreasingLemmas 221

Index of examples

Caveat: This index has been automatically generated from the ACSL /C sources. For the time being it mentions only a few of the pages where a C function occurs in the text. Moreover, if a listing had to be split, then a page number might refer only to the first part of the listing.

A

accumulate 151, 152
adjacent_difference 161, 162, 164, 165
adjacent_difference_inv 160, 165
adjacent_find 49, 50
axiom_pop_of_push 248
axiom_push_of_pop_top 250
axiom_size_of_init 246
axiom_size_of_pop 248
axiom_size_of_push 247
axiom_top_of_push 247

B

binary_search 97, 98
bubble_sort 210, 211

C

clamp 74, 75
copy 110, 111
copy_backward 112
count 66, 67, 148

E

equal 52, 53
equal_range 92–94, 96
equal_range2 ..... 92, 94, 95

F

fill 104, 105
find 31–34, 55
find2 ..... 34, 35, 38–40, 46, 47, 52, 58, 61, 98
find3 ..... 38–40, 44, 45
find4 39
find5 40
find_end 61, 62
find_first_of 46, 47
find_if_not 44

H

heap_child 176, 177, 193

heap_parent 176
heap_sort 217

I

inner_product 152, 154, 155
insertion_sort 214, 215
iota 146, 147
is_heap 179
is_heap_until 178, 179
is_sorted 202, 203

L

lower_bound 88–90, 92, 94, 98

M

make_heap 196, 197, 206, 217
make_pair 73, 84
max_element 76, 77, 79, 82
max_element2 ..... 78–80, 83
max_seq 80
merge 218, 221
min_element 81–83, 213
minmax_element 69, 83, 84
mismatch 52, 53

P

partial_sort 206–209
partial_sum 150, 157, 158, 161, 164, 165
partial_sum_inv 164
pop_heap 175, 176, 182, 190, 191, 193, 199, 206
push_heap 175, 182, 184–186, 191, 193, 194, 197, 205, 206

R

random_number 140, 142, 143
remove 132, 135, 136
remove_copy 124, 125, 128, 133
remove_copy2 ..... 124, 126–128
remove_copy3 ..... 41, 124, 127, 132–136
replace 121, 122
replace_copy 121
reverse 115, 119
reverse_copy 114
rotate 118, 119, 214–216
rotate_copy 117, 118

S

search 55, 56, 58, 62
search_n 58, 60
selection_sort 212, 213
shuffle 139, 140, 142
size_pair_type 73
sort_heap 86, 198, 199, 217

stack_empty 239
stack_empty_wd 239
stack_equal 235
stack_full 238
stack_init 236
stack_pop 244
stack_pop_wd 245
stack_push 242
stack_push_wd 243
stack_size 237
stack_size_wd 237
stack_top 240
stack_top_wd 241
swap 105, 106, 108, 115, 140, 141, 213
swap_ranges 107, 108

U

upper_bound 90–92, 94, 214